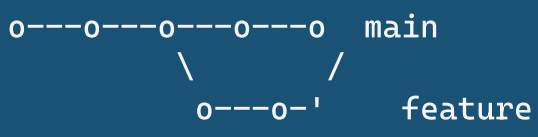
GIT FOR NEPAL

Teaching Version Control

A Foundation Book for Grades 8–12



नेपालका लागि Git — संस्करण नियन्त्रण, कक्षा ८–१२

Ram Chandra Giri and contributors

Edition 0.2 · October 2026

Text CC BY 4.0 · Code and templates MIT

Teaching Version Control: A Foundation Book for Grades 8–12

Edition 0.2 · October 2026. Grades 9–12 are unpiloted drafts.

© Ram Chandra Giri and contributors. Text licensed under Creative Commons Attribution 4.0 International (CC BY 4.0); code and templates under the MIT License. You may copy, print, adapt and share this book, including commercially, with credit:

"Git for Nepal" by Ram Chandra Giri and contributors, licensed under CC BY 4.0. Source: https://github.com/rcgiri-physics/git-for-nepal

Contents · Website: https://rcgiri-physics.github.io/git-for-nepal/. Third-party resources keep their own licences and are linked, not copied, unless their licence allows it.

Contents

Teaching Version Control: A Foundation Book for Grades 8–12 5

Part I. Teaching Version Control as a Foundation 7
Part I. Teaching version control as a foundation 8
Teacher hub 16
Teacher training: 3-hour introduction 17
Setup guide 18

Part II. Grade 8 — Time Machine for My Work 19
Grade 8 — Time Machine for My Work 20
1. The final_final_v3 problem 22
Teacher guide — Module 1 (1 period) 26
2. Snapshots = commits 28
Teacher guide — Module 2 (2 periods) 32
3. My first repo 34
Teacher guide — Module 3 (2 periods) 39
4. Reading history and comparing versions 41
Teacher guide — Module 4 (2 periods) 46
5. Undo and restore 48
Teacher guide — Module 5 (2 periods) 52
6. Statistics: chart my commits 54
Teacher guide — Module 6 (1 period) 58
7. Mini-project: Class Magazine 59
Teacher guide — Module 7 (2 periods + home) 62
Printable cards (Grade 8) 64

Part III. Grade 9 — Branch and Share 65
Grade 9 — Branch and Share 66
1. Terminal basics bridge 68
2. status, add, commit, log on the command line 72
3. Branches 77
4. Merge and the merge-conflict game 82
5. Remotes: clone, push, pull 87
6. Python scripts under version control 92
7. SQL under version control 96
8. Pair project: Ward Facts 100

Part IV. Grade 10 — Team Workflows 104
Grade 10 — Team Workflows 105
1. Issues as a to-do list 107
2. Fork vs clone vs branch 111
3. Pull requests 115
4. Code review with a checklist 119
5. Real merge conflicts, resolved in pairs 123
6. Tags and releases 127
7. Publishing a static website from the repo 131
8. Automatic checks (CI) — intro 135
9. Team project: Our School Website 139

Part V. Grade 11 — Open-Source Contributor 143
Grade 11 — Open-Source Contributor 144

1. Open-source licences and attribution 146
2. Reading a project: README, CONTRIBUTING, issues 151
3. Triage: finding a good first issue 155
4. Your first PR — to this curriculum 158
5. rebase, stash, revert, reset — when to use each 162
6. Commit-message and branching conventions 168
7. Software process models and Git workflows 172
8. Data project: version, clean and document open data 176

Part VI. Grade 12 — Engineering with Git 180

Grade 12 — Engineering with Git 181
1. Git from the inside 183
2. Branching strategies 188
3. Automation: CI with GitHub Actions / Forgejo Actions 192
4. Releases and semantic versioning 196
5. Security: secrets and safe habits 200
6. Reviewing AI-assisted changes 204
7. Debugging history: blame and bisect 208
8. Team capstone 212
9. Where Git goes next 216

Part VII. After Grade 12 219

Part VII. After Grade 12: the foundation, consolidated 220
Pathway map: where Git goes next 224

Part VIII. Version Control in Any Project 225

Part VIII. Version control in any project 226

Part IX. Books, Reading and Free Resources 234

Part IX. Books and further reading 235
Free resource library 238

Appendices 245

Appendix A. Command cheat sheet 246
Appendix B. Troubleshooting: "Oops, Git!" 250
Appendix C. Glossary 254
Appendix D. Module template (for contributors) 259

Teaching Version Control: A Foundation Book for Grades 8–12

Git for Nepal — an open, offline-first book for students, teachers and anyone starting a project.

Preface

Every piece of work has a history. A student's essay grows over a week; a team's website changes every day; a dataset is corrected, extended and re-published; a government policy is revised. **Version control** is the practice — and **Git** the tool — that records that history: what changed, when, who changed it and why, with the power to go back, to work in parallel and to combine everyone's work safely.

Most people learn version control late, in a hurry, from scattered tutorials. This book does the opposite. It builds the idea **slowly and deliberately over five school years**, starting with paper cards in Grade 8 and finishing with a professional team project in Grade 12, so that it becomes a **foundation**: a correct mental model, automatic good habits, and the judgement to use them in any project.

It was written for Nepal — every lesson works in a classroom **with no computer**, with **offline computers**, on a **school network**, or with the **internet** — but nothing in it is specific to one country.

Who this book is for

You are…	Start with	Then
a student following the book on your own	Grade 8, Module 1	one module at a time; do the PRIMM lab and at least one "Practise more" exercise; check yourself with the exit ticket
a teacher	Part I: Teaching version control as a foundation and the Teacher hub	teach one module ahead of your class; use the teacher guide/notes and answer keys
a learner joining late (+2, university, adult)	Part VII: the bridge course	the competency checklist, then Part VIII
someone starting a project	Part VIII: Version control in any project	the cheat sheet and troubleshooting appendices
a curriculum planner	Part I and the CDC brief	the alignment files in the repository

How the book is organised

Part	Content
I	Teaching version control as a foundation — the concept spine, threshold concepts, misconceptions, assessment, labs
II	Grade 8 — Time Machine for My Work: snapshots, history, undo, statistics, a class magazine
III	Grade 9 — Branch and Share: terminal, branches, merges, remotes, Python and SQL in Git
IV	Grade 10 — Team Workflows: issues, pull requests, review, releases, websites, CI
V	Grade 11 — Open-Source Contributor: licences, first contributions, rebase/reset/reflog, conventions, open data
VI	Grade 12 — Engineering with Git: internals, strategies, CI/CD, SemVer, security, AI review, bisect, capstone
VII	After Grade 12: the foundation, consolidated — checklist, practical test, bridge course
VIII	Version control in any project — playbook, recipes, templates
IX	Books and further reading

Part	Content
Library	Free resource library — every exercise source, with licences
App. A	Command cheat sheet
App. B	Troubleshooting: "Oops, Git!"
App. C	Glossary (English and Nepali)

How every lesson is built

Each module follows the same pattern, so students and teachers always know where they are:

1. **At a glance** — time, materials, delivery tiers, outcomes.
2. **Hook** — a real problem.
3. **Understand it** — the explanation, readable on its own.
4. **See it** — a worked example with real Git output.
5. **Unplugged / concept activity** — paper or visual model.
6. **PRIMM lab** — Predict, Run, Investigate, Modify, Make.
7. **Parsons practice** — put the steps in order.
8. **Common mistakes** — symptom, cause, fix.
9. **Practise more with free resources** — exercises from the best free materials worldwide.
10. **Exit ticket**, **Key points**, **New words**, **Extension quest**, and **Teacher notes** with answer keys.

Delivery tiers

Every module lists the tiers it supports: **0** no computer · **1** offline computers (USB kit) · **2** school LAN server (Forgejo) · **3** internet. See the Teacher hub.

Status of this edition

This is **edition 0.2** (October 2026). The Grade 8 chapter is the most complete; Grades 9–12 are full drafts that **have not yet been piloted in classrooms**. Nepali translations exist as summaries; the expanded English text is the new source for full translation. Corrections and improvements are welcome — Grade 11 Module 4 shows how to contribute.

Licence and credit

Text: **CC BY 4.0**; code and templates: **MIT**. You may print, adapt and share this book — including commercially — if you credit it:

> "Git for Nepal" by Ram Chandra Giri and contributors, licensed under CC BY 4.0. Source: https://github.com/rcgiri-physics/git-for-nepal

External resources referred to in this book keep their own licences; several are linked rather than copied for that reason (see the resource library).

Part I. Teaching Version Control as a Foundation

Part I. Teaching version control as a foundation

This part is for **teachers, teacher trainers and curriculum planners**. It explains *why* the lessons in Parts II–VI are ordered and designed the way they are, so that you can teach them well, adapt them to your school, and judge whether your students have built a real foundation — not just memorised commands.

> **In one paragraph**
>
> Teach **one mental model** early and keep returning to it: *a commit is a snapshot of the whole project, with a message, pointing to its parent; branches and tags are labels on commits; every copy of a repository is complete.* Teach it **on paper before on screen**, **with buttons before commands**, **with safe tools before sharp ones**, **alone, then in pairs, teams and communities**. Assess the **process** (the history) and not only the product. Everything else in this book is built on that.

1. What "foundation" means

After five years a student should leave school with three things:

1. **A correct mental model** of how version control works — robust enough that they can reason about a command they have never seen ("this rewrites history, so I must not use it on a shared branch").
2. **Habits** that have become automatic — small commits with clear messages, reading the diff before committing, pulling before pushing, reviewing kindly, never committing secrets or private data.
3. **Judgement** — knowing which tool fits a situation, when to ask, how to work in someone else's project, and how to bring version control into *any* new project (Part VIII).

Commands change; hosts come and go; a solid model, good habits and judgement last a career.

2. The concept spine: eight strands over five grades

Each strand starts simply and deepens every year. When you teach a lesson, know which strand it belongs to and what came before.

Strand	Grade 8	Grade 9	Grade 10	Grade 11	Grade 12
A. History and snapshots	commit = snapshot + message + parent; reading `log` and `diff`	CLI cycle; `diff --staged`	history as team evidence	message conventions; reading others' history	objects, hashes, refs
B. Undo and safety	restore, revert	switching protects work	revert on shared branches	stash, reset, rebase, reflog — the golden rule	secret leaks; history rewriting as incident response
C. Branching and integration	(one line only)	branch = label; merge; conflicts	conflicts on PRs; merge `main` in	rebase vs merge	strategies; protection; bisect
D. Sharing and remotes	(one computer)	clone, push, pull, bundles	forks, `upstream`	contributing to other projects	CI/CD, releases as distribution
E. Collaboration and communication	messages for others	pair agreement	issues, PRs, review, charter	triage, etiquette, conventions	review of AI-assisted work; capstone process

Strand	Grade 8	Grade 9	Grade 10	Grade 11	Grade 12
F. Automation and quality	—	`.gitignore`	check scripts, CI intro	data reproducibility	CI workflows, SemVer, scanners
G. Ethics, licences, privacy, security	no real names; pseudonyms	no secrets in repos	credit on the public web	licences and attribution	secrets, signing, supply chain, AI policy
H. Version control beyond code	essays, magazine	data and SQL scripts	websites	open data	pathways; any project (Part VIII)

Using the spine: when a student struggles in Grade 11 with rebase, the problem is often in strand A or C (they do not see branches as labels on commits) — go back to the Grade 9 sticky-note model before going forward.

3. Threshold concepts

Some ideas are *thresholds*: once understood, everything after makes sense; until then, students memorise and get stuck. Plan extra time, paper models and repeated checks for these.

Threshold concept	Introduced	Diagnostic question	If they don't have it
A commit is a **whole snapshot**, not a list of changes	G8 M2	"If you delete commit 3's card, can you still rebuild commit 4?" (yes — each card is complete)	repeat the paper-commit game
Changes move through **three places** (working copy, staging area, history)	G8 M3	"You saved but `git status` shows *modified*. Where is your change?"	three trays on the desk
Saving ≠ committing	G8 M3	"The power went off. Which work is safe?"	demonstrate after a real power cut
A **branch is a label**, not a copy	G9 M3	"How much disk space does a new branch use?" (almost none)	sticky notes on cards
Every clone is a **complete, equal copy**	G9 M5	"The server breaks. What is lost?" (nothing that anyone has pulled)	acting out with card rows
Merging compares with a **common ancestor**	G9 M4	"Why is there no conflict when we edit different lines?"	paper merge game
Adding vs rewriting history	G8 M5 → G11 M5	"Your teammate already pulled. Revert or reset?"	card table with a second group
Objects never change; refs move	G12 M1	"Why does rebase change commit ids?"	object graph on paper

4. Mental models to teach — and to avoid

Teach	Avoid	Why
Commit = photo of the whole project + caption	"Git saves your changes"	leads to "a commit is a diff" and to confusion with Ctrl+S
History = chain of commits pointing to parents	history = a list of file versions	branches and merges need the graph
Branch = sticky note on a commit	branch = a folder / a copy	makes switching "disappearing files" incomprehensible
HEAD = "you are here" pointer	HEAD = "the newest commit"	detached HEAD then makes sense

Teach	Avoid	Why
Remote = another full copy with a name	remote = "the cloud" / "GitHub"	Git ≠ GitHub; offline and LAN work become obvious
Pull = fetch + merge	pull = download	explains why pull can conflict
Conflict = Git asking a human	conflict = error / failure	reduces panic
Revert = new opposite commit; reset = move a label	"undo" as one thing	students choose the wrong undo
PR = request + discussion pointing at a branch	PR = a copy of the code	updates by pushing to the branch make sense

5. Misconception catalogue

Use these as exit-ticket items, discussion prompts and test distractors. Each lesson's exit ticket already includes one.

#	Misconception	Correct idea	Addressed in
1	Git saves every keystroke	only commits are saved snapshots	G8 M2
2	Saving the file puts it in history	only a commit does	G8 M3
3	A diff shows the whole file	only changed lines plus context	G8 M4
4	Once committed, mistakes can't be fixed	restore / revert	G8 M5
5	Revert deletes the bad commit	it adds an opposite commit	G8 M5
6	More commits = better work	activity ≠ quality	G8 M6
7	The terminal does things by itself	it only runs what you type	G9 M1
8	`git add` saves to history	it stages	G9 M2
9	A branch is a full copy	it is a label	G9 M3
10	After a conflict my work is lost	both sides are kept until you decide	G9 M4
11	Git = GitHub	GitHub is one host for Git repositories	G9 M5
12	`commit` sends work to the server	only push does	G9 M5
13	If code works, history doesn't matter	history explains, debugs, undoes	G9 M6
14	Git can merge live databases	version the scripts, not the database	G9 M7
15	A fork is a branch	a fork is a whole server-side repository	G10 M2
16	A PR is a copy of the code	it points at a branch	G10 M3
17	Reviewing is for experts	anyone can spot clarity and checks	G10 M4
18	Conflicts mean someone did wrong	conflicts are normal in parallel work	G10 M5
19	A tag moves with new commits	tags stay	G10 M6
20	A passing check means a perfect page	checks cover only their rules	G10 M8
21	If it's online, it's free to use	no licence = no permission	G11 M1
22	Rebase and merge give identical history	rebase rewrites into a line	G11 M5
23	Messages are only for me	they are for every future reader and tool	G11 M6
24	Agile means no planning	planning is frequent and small	G11 M7
25	If I can download data, I can redistribute it	check the licence	G11 M8

#	Misconception	Correct idea	Addressed in
26	Git stores differences	the model stores snapshots	G12 M1
27	Gitflow is always professional	choose by context	G12 M2
28	Green CI means no bugs	only the written checks passed	G12 M3
29	Private repos keep secrets safe	a leaked secret is burned	G12 M5
30	If tests pass, the AI change is fine	tests may be weakened or incomplete	G12 M6
31	Blame is for punishing people	it finds context and who to ask	G12 M7

Research on learners' confusions with Git (e.g. Isomöttönen and Cochez, 2014) found many of the same problems — especially the staging area, branches and remote synchronisation — which is why this book spends paper-model time on exactly those.

6. Sequencing principles

These rules decided the order of every lesson.

1. **Concept before commands.** Paper and visual models come first; commands name an idea students already have (Wagner and Thurner, 2025, recommend separating concepts from tool skills).
2. **Low floor → high ceiling: GUI → CLI → internals.** Grade 8 uses buttons (VS Code, GitHub Desktop); Grade 9 the terminal; Grade 12 the plumbing. Every student can pass Grade 8; fast learners always have an extension quest.
3. **Safe tools before sharp tools.** Restore and revert in Grade 8; reset and rebase only in Grade 11, with the golden rule.
4. **Local before remote.** One computer (G8) → two computers (G9) → a server with a team (G10) → strangers' projects (G11) → automation (G12).
5. **Solo → pair → team → community.** Social complexity grows with technical complexity, not before it.
6. **Delay accidental complexity.** SSH keys, credential helpers and configuration details are postponed; pseudonymous HTTP accounts and bundles keep early lessons about ideas.
7. **Real, Nepali, open artefacts.** A class magazine, a ward-facts script, a school website, a contribution, a capstone — each grade ends with something real (the "Nepal data thread").
8. **Spiral back.** Every grade revisits earlier strands at a deeper level (e.g. undo: restore → revert → reset/reflog → incident response).

7. Lesson design in practice

PRIMM with Git

Predict → **R**un → **I**nvestigate → **M**odify → **M**ake (Sentance, Waite and Kallia, 2019). For Git, the "program" is a command and the "output" is the repository state:

Stage	Git example
Predict	"Write what `git status` will say after `git add`."
Run	run it; compare with the prediction
Investigate	"Why is README listed twice?" (staged + modified)
Modify	change the scenario: edit after staging, then predict again
Make	create your own scenario and challenge a partner

Insist on *written* predictions; that is where learning happens.

Parsons problems

Jumbled steps to put in order (Parsons and Haden, 2006). They test understanding of *sequence and dependency* without requiring recall of syntax — ideal for mixed-ability rooms and for Tier 0. Every lesson has a set; printable strips are on the Grade 8 cards page.

Unplugged models

Cards, string, sticky notes, trays, floor maps and acting out make invisible structures visible. They also make every lesson possible with **no computer at all** (Tier 0).

Live demonstration ("think aloud")

When you demonstrate on the projector, *type slowly and say your thinking*: "Before I commit, I always check the diff — let me read it… oh, I left a debug line." Make a mistake on purpose and fix it. Students copy habits more than commands.

Pair programming

Driver (keyboard) and **navigator** (reads, predicts, checks), swapping every 10 minutes. It halves the number of computers needed and doubles the talking about ideas.

Worked examples and fading

Each lesson's *See it* section is a worked example with real output. In later lessons, show less and ask more ("fade" the support).

8. Assessment progression

Grade	Formative	Practical	Summative
8	exit tickets; reading the card rows	3 commits + restore + revert	commit-history rubric (Class Magazine), oral "show me yesterday's version"
9	predict-the-status worksheets	create → branch → merge → push	pair project rubric; Parsons card test
10	review-comment quality	PR with review; fix a conflicted PR	team rubric with individual rows; viva
11	claim comments; licence attributions	merged contribution	`CONTRIBUTIONS.md` portfolio; data project rubric
12	CI logs, security checklist	bisect a planted bug	capstone (100-point rubric); viva

Assess the process: read the history

The repository **is** the evidence. Before marking a product, read `git log` (or the card row): Are commits small and meaningful? Do messages say what and why? Did the student restore, revert, resolve, review? Is there work across several sessions? `git shortlog -s -n` shows contribution, but **read messages and reviews, not only counts** — counts are easy to game and unfair to students who make fewer, larger, well-explained commits.

Fitting a marks-based exam system

Every grade has a board-style item bank in `assessments/` (definitions, ordering, "what will this command do?", misconception true/false, short explanations). Practical rubrics give marks too, so internal assessment can combine both.

9. Running labs in Nepali schools

Delivery tiers

Tier	Setting	What to do
0	no computer	cards, string, sticky notes, printed diffs and logs; every lesson has an unplugged path
1	offline computers	USB kit: portable Git + VS Code, self-hosted Learn Git Branching, practice repos as `.bundle` files, *Pro Git* PDF
2	school LAN	Forgejo on a Raspberry Pi or old PC; issues, PRs and (with a runner) CI work exactly like GitHub
3	internet	GitHub / Codeberg / GitLab; GitHub Skills; real open-source contributions (learners 13+)

40 students, 10 computers

- **Pairs at PCs, the rest at paper:** half the class runs the unplugged version while the other half is at computers; swap after 20 minutes.
- **Stations:** PCs for the lab, a table for Parsons strips, a table for the predict worksheet.
- **One projector:** the class predicts aloud; volunteers drive.

Shared lab PCs

- Set a **pseudonymous identity per PC** (`Student 07`, `s07@example.invalid`) and `init.defaultBranch main`, `pull.rebase false`, and an easy editor (`code --wait`).
- Keep each student's repo on a named USB folder or a per-student folder; back up the whole folder **including** `.git`.

Power cuts

Teach "commit often" as the defence. A commit is safe on disk even if the PC then loses power. After the power returns, run `git status` with the class — it is a perfect lesson.

Windows quirks

Use **Git Bash** for the same commands as Linux; ignore `LF will be replaced by CRLF` warnings in Grade 8; in Grade 10–12, add a `.gitattributes` (`* text=auto`, `*.sh text eol=lf`) so scripts run in CI.

10. Differentiation and inclusion

- **Language:** explain in Nepali (or the students' home language); keep commands, code and branch names in English. The glossary gives Nepali terms.
- **Colour:** diffs use colour *and* `+` / `−` signs; teach both. Printed materials are black-and-white friendly.
- **Support:** sentence starters for messages, printed step cards with screenshots, pre-filled tables, buddy pairing.
- **Stretch:** every lesson has an extension quest; GitHub Skills and Learn Git Branching levels for fast finishers.
- **Participation:** rotate driver/navigator and team roles so every student types, reviews and presents — including students who are quiet or new to computers.

- **No shame:** badges in each student's own `BADGES.md`, never a public leaderboard.

11. Safety, privacy and ethics

- **Under 13:** no online accounts (GitHub's terms require 13+). Grade 8 is entirely local.
- **Pseudonyms** for Git identities; never real emails.
- **No personal data** (names with marks, phone numbers, photos, addresses) in any repository, public or private.
- **Vetted projects only** for under-18 contributions; agree with maintainers in advance.
- **AI use:** a student may use an AI helper only if they can explain every line of the diff they commit; follow school and CDC policy (G12 M6).
- **Fake secrets only** in security lessons.

12. Growing as a teacher

You do **not** need to know Git before you start: teach Grade 8 one lesson ahead of your students.

1. Do the 3-hour training outline.
2. Work through Grade 8 yourself on paper and on a PC.
3. Read Software Carpentry's git-novice ‹https://swcarpentry.github.io/git-novice/› lesson and the short paper by Blischak, Davenport and Wilson (2016) — see the reading list.
4. Play Learn Git Branching ‹https://learngitbranching.js.org/› up to the Remote tab.
5. For pedagogy, read Greg Wilson's *Teaching Tech Together* ‹https://teachtogether.tech/›.
6. Join or start a local community of practice; contribute your improvements to this book (Grade 11 Module 4 shows how).

13. A year plan template

Each grade is a "chapter" of 12–20 periods. A typical school-year layout (adjust to CDC hours and your timetable):

Term	Grade 8	Grade 9	Grade 10	Grade 11	Grade 12
1	M1–M3	M1–M3	M1–M4	M1–M3	M1–M3
2	M4–M6	M4–M6	M5–M8	M4–M6	M4–M7
3	M7 project + exhibition	M7–M8 project	M9 team project	M7–M8 data project	M8 capstone, M9

14. Research base

- Wagner, A. and Thurner, V. (2025). *Rethinking How We Teach Git: Pedagogical Recommendations and Practical Strategies for the Information Systems Curriculum.* Journal of Information Systems Education, 36(1). https://jise.org/Volume36/n1/JISE2025v36n1pp1-12.html
- Sentance, S., Waite, J. and Kallia, M. (2019). *Teaching computer programming with PRIMM: a sociocultural perspective.* Computer Science Education, 29(2–3). doi:10.1080/08993408.2019.1608781
- Isomöttönen, V. and Cochez, M. (2014). *Challenges and Confusions in Learning Version Control with Git.* In ICTERI 2014, Communications in Computer and Information Science, Springer. doi:10.1007/978-3-319-13206-8_9
- Parsons, D. and Haden, P. (2006). *Parson's programming puzzles: a fun and effective learning tool for first programming courses.* Proceedings of the 8th Australasian Computing Education Conference.

- Blischak, J. D., Davenport, E. R. and Wilson, G. (2016). *A Quick Introduction to Version Control with Git and GitHub.* PLOS Computational Biology. https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004668
- Raspberry Pi Foundation, *PRIMM* quick read (see `research/sources.md`, S8).

The full list of sources used to build this book is in `research/sources.md` in the repository.

Teacher hub

You do **not** need to know Git before you start. Read the 3-hour training outline, then run Module 1 of Grade 8 from the printed guide.

Delivery tiers

Tier	Setting	How lessons run
0	No computer	paper commits, Parsons cards, printed handouts
1	Offline computers	portable Git + VS Code from USB; practice repos as `.bundle` files
2	School LAN	Forgejo on a Raspberry Pi / old PC (see `offline/README.md` in the repo)
3	Internet	GitHub / Codeberg / GitLab; real open-source contribution

Classroom practices

- Work in **pairs** (driver and navigator, swap every 10 minutes). This also covers "fewer computers than students".
- Read the **commit history**, not just the final file — it shows the student's process.
- Keep a **no-shame** progress chart: badges in each student's own `BADGES.md`, never a public leaderboard.
- Never put student names, photos or marks into a public repo (see `SECURITY.md` in the repo).
- **AI helpers:** a student may use an AI helper only if they can explain every line of the diff they commit.

Assessment

Each grade offers a board-style written item bank (`assessments/`) **and** a practical rubric.

Start here

Read Part I. Teaching version control as a foundation: the concept spine, threshold concepts, misconception catalogue and assessment progression behind every lesson. Every lesson now has a full explanation, a worked example, a common-mistakes table, free-resource exercises and an answer key.

Pages

- Training outline (3 hours)
- Setup guide
- Nepali text status ‹https://rcgiri-physics.github.io/git-for-nepal/teachers/ne-status/›

Teacher training: 3-hour introduction

Audience: computer teachers with no Git background. **Needs:** one computer per two teachers (or none: Part 1 is paper).

Time	Part	Activity
0:00–0:15	Why	Show a folder with `report_final_final_v3_real.doc`. Ask: "which one is the right one?" Discuss.
0:15–0:45	Paper commits	Run Grade 8 Module 2 exactly as students would: index cards = commits, string = history.
0:45–1:00	Break	
1:00–1:45	First repo	Install Git (USB kit, tier 1). Make a repo, commit three times, read `git log`.
1:45–2:15	Undo and compare	`git diff`, `git restore`, `git revert`. Explain the three most common mistakes.
2:15–2:40	Your classroom	Plan: which tier you have, how many pairs, which module first. Read one teacher guide together.
2:40–3:00	Exit	Each teacher writes one thing they will try and one worry. Collect (anonymously) for the maintainers.

Trainer notes

- Let teachers be confused with the *model* first; commands come after.
- Show that mistakes are recoverable: deliberately delete a file and bring it back.
- Hand out the printed Module 1 teacher guide and a Parsons card set.

Follow-up (2-day workshop kit — outline)

Day 1: Grade 8–9 labs end to end. Day 2: set up a LAN Forgejo or GitHub class organisation; plan the pilot; practise giving feedback on commit messages.

Setup guide

Tier 0 — no computer

Print the unplugged pages of each lesson and the card sets from `docs/assets/cards/`. Nothing to install.

Tier 1 — offline computers

1. Copy the **USB kit** (`offline/usb-kit/`, see its README) to each lab PC: Git for Windows (portable), VS Code (portable) and the self-hosted learnGitBranching build.
2. Open a terminal and run `git --version`.
3. Set a name and email **that is not the student's real identity**: `git config --global user.name "Student 07"` and `git config --global user.email "s07@example.invalid"`.
4. Copy a practice repo from its `.bundle` file: `git clone g08-class-magazine.bundle my-magazine`.

Tier 2 — school LAN

Follow `offline/forgejo-lan/` in the repo. A Raspberry Pi-class machine is enough.

Tier 3 — internet

Teacher creates a **private** class organisation. Students under 13 do not create accounts (GitHub terms). From Grade 9 learners aged 13+ may have accounts managed with the teacher.

Power cuts

Git saves nothing until you commit. Commit often; a commit is safe on disk even if the PC then loses power.

Part II. Grade 8 — Time Machine for My Work

Grade 8 — Time Machine for My Work

Big idea: every file has a history, and you can go back. **Length:** ≈ 12 periods (calibrate to CDC hours — unverified). **Accounts:** none; everything stays on the school computer (students may be under 13). **Tools:** paper and cards (Tier 0) · Git + VS Code or GitHub Desktop, and the self-hosted Learn Git Branching visualiser (Tier 1+). **Hooks:** file management, digital literacy, basic statistics.

How this chapter is built

Grade 8 is the **foundation** of the whole book. Students meet one mental model — *a commit is a snapshot of the whole project, with a message, that points to its parent* — first on paper, then on a computer. Everything in Grades 9–12 (branches, remotes, pull requests, CI, internals) is built on that picture, so take time here.

```
M1  why?              (paper)
M2  snapshots         (paper cards)
M3  first repo        (GUI)
M4  read history      (diff)
M5  undo              (safe undo)
M6  statistics        (maths link)
M7  project           (own work)
```

Modules

#	Module	Periods	Tiers	Key commands / actions
1	The `final_final_v3` problem · teacher guide	1	0–3	— (unplugged)
2	Snapshots = commits · teacher guide	2	0–3	paper commits
3	My first repo · teacher guide	2	1–3	`status`, `add`, `commit`
4	Reading history and comparing versions · teacher guide	2	0–3	`log`, `show`, `diff`
5	Undo and restore · teacher guide	2	0–3	`restore`, `revert`
6	Statistics: chart my commits · teacher guide	1	0–3	`log --format`
7	Mini-project: Class Magazine · teacher guide	2	0–3	everything

Printable materials: cards, Parsons strips and badges.

Chapter outcomes

By the end, a student can:

1. explain why keeping many copies of a file by renaming is risky;
2. explain a commit as a saved snapshot with a message;
3. create a repo, make commits, and read the history;
4. compare two versions and say what changed;
5. restore an earlier version of a file and undo a commit safely;
6. present a bar chart of their own commit counts.

Free resources used in this chapter

All Grade 8 exercises work **without an account**. The main ones: Learn Git Branching ‹https://learngitbranching.js.org/› (self-hosted copy for offline labs), Software Carpentry's git-novice ‹https://swcarpentry.github.io/git-novice/› episodes 1, 4 and 5 (CC BY, may be printed), *Pro Git* ‹https://git-scm.com/book/en/v2› chapters 1–2 (link only), and the VS Code Source Control docs ‹https://code.visualstudio.com/docs/sourcecontrol/overview›. The full list with licences is in the free resource library.

Assessment

- Exit tickets in every lesson.
- **Commit-history rubric:** ≥ 5 meaningful commits, each message says *what and why*, no commit larger than one idea.
- Oral explanation (2 minutes): "show me how you would get yesterday's version back."
- Board-style item bank: `assessments/g08-item-bank.md`.

Capstone

Class Magazine: each student keeps one article's history; the class exhibits the "evolution" of an article. Practice repo: `practice-repos/g08-class-magazine/`.

Ready for Grade 9?

A student is ready when they can, without help: make three commits with good messages; find who/when/why for a change; and choose between *restore* and *revert*. Students who cannot yet should repeat Modules 3–5 with a buddy at the start of Grade 9 (the bridge course also covers this).

GRADE 8 · MODULE 1 · 1 PERIOD · TIERS 0, 1, 2, 3

1. The `final_final_v3` problem

> **At a glance**
>
> **Time:** 1 period (40 min) · **Tiers:** 0–3 (fully unplugged) · **You need:** a printed paragraph per group, blank A4 sheets, pens
>
> **Before this:** nothing — this is the first lesson of the whole book.
>
> **By the end you can:** explain why renaming files is a poor way to keep versions; list what a good history must record; say what a version control system is.

Hook (5 min)

Your teacher shows this folder on the board. It is a real kind of folder; most people have one.

```
essay.doc
essay_new.doc
essay_final.doc
essay_final_final.doc
essay_final_v3_REAL.doc
essay_final_v3_REAL_sir_checked.doc
```

Answer without opening any file:

- Which one should I hand in?
- Which one did Sir correct?
- What changed between `essay_new` and `essay_final`, and *why*?

You cannot answer. That is the problem this whole book solves.

Understand it

What is a "version"?

A **version** is the state of your work at one moment. When you write an essay, you make many versions: the first rough draft, the draft after a friend's comments, the draft after the teacher's corrections. Every version is a little different from the one before.

Keeping versions is useful. You might:

- want an **old paragraph back** that you deleted last week;
- need to **show your teacher what changed** after their comments;
- want to know **who** changed something in a group project, and **why**;
- want to **try a risky idea** without losing the work that already works.

The renaming method and why it fails

Most people keep versions by saving a copy under a new name: `essay_final.doc`, `essay_final_final.doc`, and so on. This feels safe, but it breaks in at least six ways:

Problem	What happens
No order	Is `final_v3` newer than `final_final`? Names do not say. You guess from the "date modified", which changes if you just open and save.
No reason	Nothing records *why* a copy was made. "REAL" is not a reason.
No difference	To see what changed, you must open two files side by side and read every line.
Easy to overwrite	One wrong "Save" over the wrong copy and a version is gone for ever.
No "who"	In a group, you cannot tell who wrote which part.
Wasted space and confusion	Six full copies of the same essay. Multiply by every subject and every student.

When two people work on the same file, it becomes worse: Sita has `report_sita.doc`, Ram has `report_ram_edited.doc`, and someone must combine them by hand, line by line.

What a good history records

Think of a careful shopkeeper's **khata** (account book). Every line records *when* something happened, *what* happened, *who* did it, and often *why* ("credit to Hari dai until Friday"). Nobody tears out old pages; mistakes are corrected with a new line. Because the old lines stay, the shopkeeper can always answer "how did we get here?".

A good history of your work needs the same four things:

1. **What** changed — exactly which words or lines.
2. **When** — the date and time.
3. **Who** — the person who made the change.
4. **Why** — a short message in plain words.

…and one power that a khata does not have: **go back** — bring back any earlier version in seconds.

Version control systems

A **version control system** is a program that keeps this history for you. Instead of copies with strange names, you keep **one** folder. Whenever you reach a good point, you tell the program "remember this, and here is why". Later it can show you the whole history, the exact difference between any two moments, and give you back any old version.

The version control system used by almost every software team, many writers and many scientists in the world is called **Git**. It is free and open source, works without the internet, and runs on old computers too. Over the next five years you will learn it step by step: first as a time machine for your own work (Grade 8), then for sharing work (Grade 9), for teams (Grade 10), for open-source projects (Grade 11) and for professional engineering (Grade 12).

```
Without version control              With version control (Git)
-----------------------              --------------------------
essay.doc                            essay.doc   <- one file
essay_new.doc                          history:
essay_final.doc                          5 Oct  Fix spelling after Sir's comments   (Sita)
essay_final_final.doc                    3 Oct  Add conclusion                       (Sita)
essay_final_v3_REAL.doc                  1 Oct  First draft of introduction        (Sita)
```

Unplugged: the folder game (15 min)

Groups of 4. Each group gets a printed paragraph (6 lines, for example a short school notice).

1. **Round 1–4 (2 min each):** the group makes one edit (change a word, add a line, remove a line). After each edit, copy the *whole* paragraph onto a **new** sheet and give it a file name the way people usually do (`notice_new`, `notice_final`, …). Never change an old sheet.
2. Put all sheets in a pile and shuffle them.
3. The teacher secretly asks one group at a time: "*Go back to the version before the third edit.*" Time how long it takes.
4. Then: "*What changed in the second edit, and why did you make it?*"

Discuss as a class:

- How long did it take to find the right sheet? Did anyone pick the wrong one?
- Could you tell *what* changed without reading both sheets word by word?
- Did any sheet say *why* the edit was made? *When*? *Who* made it?
- What would you add to each sheet to make the game easy? (Students usually invent: a number, a date, a name and a reason — exactly what Git records.)

Think (PRIMM — Predict) (10 min)

1. **Predict:** 30 students each keep 6 copies of 5 assignments. How many files is that? *(30 × 5 × 6 = 900.)*
2. **Predict:** two people both edit `report_final.doc` on different computers and both save. What happens when they bring their files together? *(Two different files that must be combined by hand; one person's work is easily lost.)*
3. **Investigate:** look at your own computer, phone or Google Drive at home. Do you have a `final_final`? (Describe it tomorrow — don't bring the file.)

Key idea (5 min)

A good history records **what** changed, **when**, **who** changed it and **why**, and it lets you **go back**. File names cannot do that. A version control system such as **Git** does.

Common mistakes

What students say	Better thinking
"I just press Ctrl+Z."	Undo disappears when you close the file or the power goes; a history must survive for months.
"I keep everything in Google Drive, so it's safe."	Drive keeps *copies*; it does not say why something changed, and in a group it cannot combine two people's edits line by line.
"Version control is only for programmers."	Writers, scientists, lawyers, designers and governments use it; you will use it for an essay first.
"A longer, more careful file name fixes it."	Names cannot show differences or reasons, and can still be confused.

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Read the short page **"About Version Control"** in the free book *Pro Git*, §1.1 ‹https://git-scm.com/book/en/v2/Getting-Started-About-Version-Control› (teacher can read it aloud or print one page for class use). Find the three kinds of version control it describes and draw a picture of one.	0 (printed) / 3	15 min	drawing + 2 sentences

#	Exercise	Tier	Time	Hand in
2	Look at the Jorge Cham comic "FINAL.doc" in Software Carpentry's **Automated Version Control** ‹https://swcarpentry.github.io/git-novice/01-basics.html› episode. Write your own Nepali-context version of the comic in 4 boxes.	3 (or teacher shows it on one screen)	20 min	4-box comic
3	Open Learn Git Branching ‹https://learngitbranching.js.org/?locale=en_US&level=intro1› (self-hosted copy in the lab, Tier 1). Don't type anything yet — just watch the demo of the first level. What do you think the circles are?	1 / 3	5 min	one guess, written down
4	**Family interview:** ask an adult how they keep old versions of important papers (land records, shop accounts, letters). What do they record? Compare with *what, when, who, why*.	0	homework	5 lines

Exit ticket (5 min)

1. Name one problem with `final_final_v3`.
2. What four things should a history record?
3. *Misconception check:* "A bigger file name like `essay_final_v3_REAL` is a good version system." True or false? Why?

Key points

- Renaming copies gives no order, no reason, no difference, no "who", and is easy to overwrite.
- A good history records **what, when, who, why**, and lets you go back.
- A **version control system** keeps that history for you in one folder.
- **Git** is the most widely used, free and open-source version control system, and works offline.

New words

version, **history**, **version control system**, **Git** — see the glossary.

Extension quest

Design a better "khata" for your essay on paper: a table with columns you think a history needs. Keep it — in Module 2 you will compare it with what Git really stores.

Teacher guide — Module 1 (1 period)

Goal of the period: students *feel* the pain of renaming copies before they ever see Git. Do not show Git commands today.

Timing (40 min): hook 5 · folder game 15 · predict 10 · key idea 5 · exit 5. Read "Understand it" aloud in pieces during the discussion, or set it as reading for home. **Materials:** one printed paragraph per group (6 lines of any Nepali or English text, e.g. a short school notice), 6 blank A4 sheets per group, pens, a clock or phone timer. No computer needed.

Before class (10 min)

- Write the folder listing from the hook on the board (or a chart paper you can reuse every year).
- Prepare one "secret question" card per group: "*Go back to the version before the third edit.*"
- Optional: think of your own real `final_final` story — teachers' stories hook students best.

Running the folder game

1. Give each group the printed paragraph. Stress the rule: **never change an old sheet; always copy to a new one.**
2. Call out the four rounds with a timer. Walk around; make sure each group names its sheets in the "usual" messy way.
3. Collect each group's pile, shuffle it in front of them, give it back, and ask the secret question. Time it aloud.
4. Ask the follow-up: *what changed in edit 2, and why?* Most groups cannot say why.
5. Ask: *what would you write on each sheet to make this easy?* Write their answers on the board; circle **number, date, name, reason**. Tell them: "This is exactly what Git records. You just invented version control."

Common errors and misconceptions

You hear / see	Respond with
Students edit the original sheet	"Never change an old sheet — that is like overwriting a file."
"Just use Ctrl+Z."	Undo disappears when you close the file or the power goes; history must survive for months.
"Google Drive keeps versions."	Some tools keep copies, but they do not record *why*, and cannot combine two people's edits line by line. Git can (Grade 9–10).
"This is for programmers only."	Mention writers, scientists, lawyers, governments (Nepal's own open-data projects keep their files in Git).

Fallbacks

Same in all tiers: this lesson is fully unplugged. Large class (50+): groups of 6, fewer rounds (3).

Differentiation

- **Support:** give a printed table with columns *what / when / who / why* to fill during the discussion.
- **Stretch:** ask "What would a history need if 20 people edit the same notice?" (leads to Grade 9–10).

Evidence to collect

Exit tickets. Look for: the four things (what, when, who, why) and the idea of *going back*.

Answer key

- **Hook:** no way to know without opening every file — and even then, no reasons are recorded.
- **Predict 1:** 30 × 5 × 6 = 900 files. **Predict 2:** two different files that must be combined by hand; one person's work is easily lost.
- **Exit 1:** any of: unclear order, no reason recorded, cannot see the difference, easy to overwrite, no "who", wastes space. **2:** what, when, who, why. **3:** False — names give no reason and no difference, and can still be confused or overwritten.
- **Free-resource 1:** local, centralised, distributed version control. Git is *distributed* (every copy has the full history) — a good word to praise but not required.
- **Free-resource 3:** circles are commits (snapshots). Accept any guess; it is revisited in Module 2.

Extension quest

Students design their own "khata" table for an essay's history. Keep the best ones on the wall; in Module 2 compare them with what a commit stores.

GRADE 8 · MODULE 2 · 2 PERIODS · TIERS 0, 1, 2, 3

2. Snapshots = commits

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0–3 (unplugged core; computer optional) · **You need:** 8 index cards per pair, string or a pen to draw lines, pens
>
> **Before this:** Module 1 (why renaming fails).
>
> **By the end you can:** explain what a commit is; draw a history where each commit points to its parent; write a clear commit message.

Hook (5 min)

A builder in your town takes a photo after every stage of building a house: foundation, first floor walls, roof, paint. Months later the owner asks, "Where exactly are the water pipes in the wall?" The builder opens the photo from the "walls" stage. Nothing was lost, because every stage was **photographed and labelled**.

A **commit** is a photo of your whole project plus a note.

Understand it

A commit is a snapshot

When you **commit**, Git takes a **snapshot**: a complete picture of every file in your project *at that moment*. It also writes down:

Part of a commit	Example	Answers the question
a **message**	`Add second verse about the river`	*why / what*
the **author**	`Student 07`	*who*
the **date and time**	`5 Oct 2026, 10:00`	*when*
the **parent**	"the commit before me is c2"	*what came before*
a **commit id**	`78a873e`	a unique name for this snapshot

Do you remember the four things from Module 1 — what, when, who, why? A commit records all four, plus a link to the commit before it.

Snapshots, not just changes

It is natural to think a commit stores only "the line I changed". It does not: each commit is a **whole picture** of the project. Git is clever about space — if a file did not change, the new snapshot simply *points to* the old copy instead of storing it again — but in your head you should think "*each commit = the complete project at that moment*". That is why you can jump back to *any* commit and get everything exactly as it was.

History is a chain

Every commit (except the very first) remembers its **parent**, the commit that came just before it. Following the parents backwards gives you the **history**, oldest to newest:

```
  c1  <---  c2  <---  c3  <---  c4
 "Write    "Add      "Fix      "Add
  title"    verse 1"  spelling" verse 2"
                                  ^
                                 newest
```

The arrows point *backwards* because each commit knows its parent, but a parent does not know its future children. You will see this again in Grade 9 when we learn branches.

Commit ids

Git names each commit with a long code such as `78a873e8c1…`. It is calculated from the commit's content, so two different commits can never have the same id. People usually use the first 7 characters (`78a873e`). You never need to remember them — you copy them when you need one.

Writing good messages

The message is the part a future reader (often *you*, three months later) relies on. A good message:

- starts with a **verb** in the present tense: *Add*, *Fix*, *Remove*, *Change*, *Move*;
- says **what** changed and, if it is not obvious, **why** — in one short line (about 50 characters);
- describes **one idea**. If you need the word "and" twice, it was probably two commits.

Weak message	Why it is weak	Better message
`stuff`	says nothing	`Add paragraph about food stalls`
`fix`	fix what?	`Fix spelling of Bhaktapur`
`final`	every commit is "final" for a moment	`Add closing sentence`
`changed lots of things and fixed typo and new title`	three ideas in one	three commits: `Fix typo…`, `Change title…`, `Add…`

When should you commit?

Commit when you reach a **small, complete step** you might want to return to: a paragraph is finished, a mistake is fixed, a new section is added. Not after every word (too many), and not once at the end of a week (too few — you cannot go back to the middle).

See it

On a computer, after three commits, the history looks like this (from the practice repo; Module 3 shows you how to make it). Newest is at the top:

```
$ git log --oneline
b16c8b6 Say where the festival is and what happens
78a873e Add better title and opening line
d38a483 Add working title
```

Each line is one card from today's game: an id, then the message. Your ids will be different — that is normal.

Unplugged: the paper-commit game (25 min)

Materials: index cards, string, pens. Pairs.

1. Each pair writes a 4-line poem about their village or school on a sheet. This sheet is the **working copy** — the thing you are editing.
2. **Make a commit:** copy the *whole sheet* onto an index card. At the top write a short message (*what + why*), e.g. `Add first two lines`. Number it `c1`. Write your pair name as author and today's date.
3. Edit the sheet (add a line). Make card `c2` with its message. On `c2` write **parent: c1**, and connect the cards with string (or draw an arrow from `c2` back to `c1`).
4. Keep going until you have four commits. Lay the cards in a row. This row is the **history** — what `git log` shows.
5. **Swap** cards with another pair. Answer *only from their cards*: What changed between `c2` and `c3`? Why? Who? When?
6. Give the other pair one piece of feedback on their messages.

Rules of a good message

- Starts with a verb: "Add", "Fix", "Remove".
- Says *what and why* in one line.
- One idea per commit.

Predict (PRIMM) (10 min)

1. **Predict:** your teacher shows 3 cards with messages `stuff`, `fix`, `final`. Can you tell what changed in each? What would you need to do to find out?
2. **Run:** now look at 3 cards: `Add title`, `Fix spelling of Bhaktapur`, `Add third verse`. Can you tell now?
3. **Investigate:** which set would help a teacher, a partner or *you in three months*? Why?
4. **Modify:** rewrite the three weak messages so that they are useful (look at the cards' contents).

Parsons practice (10 min)

Put these commits in order to make a sensible history, using the messages as clues: *Write title* · *Add first verse* · *Fix spelling* · *Add second verse*. Then draw the parent arrows.

Common mistakes

Mistake	Why it is a problem	Fix
Copying only the changed line onto the card	A commit is the *whole* project; you could not go back to it alone	copy the whole sheet
Forgetting the parent	the history cannot be followed	always write "parent: c…" and draw the arrow
Rubbing out an old card to fix it	history is changed; the record is no longer true	make a **new** card that fixes it
One huge commit at the end	cannot go back to the middle	commit after each small step

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Learn Git Branching, level `intro1` "Introduction to Git Commits" ‹https://learngitbranching.js.org/?locale=en_US&level=intro1›. Type `git commit` twice to finish the level. Then type `undo` and try again slowly, watching each new circle appear with an arrow to its parent.	1 (self-hosted) / 3	10 min	a drawing of the final graph

#	Exercise	Tier	Time	Hand in
2	In the same tool type `sandbox`, then `git commit` five times. Draw what you see. Which circle is the newest? Which way do the arrows point, and why?	1 / 3	10 min	drawing + 1 sentence
3	**Message makeover:** your teacher gives 10 weak messages (see the teacher guide). Rewrite each as a good one-line message.	0	15 min	list of 10
4	Read *Pro Git* §1.3 "What is Git?" ‹https://git-scm.com/book/en/v2/Getting-Started-What-is-Git%3F›, the part about "Snapshots, Not Differences" (teacher may read aloud). Explain the house-photo idea to a family member.	3 (or read aloud)	15 min	3 sentences

Exit ticket (5 min)

1. What is a commit?
2. Write a good message for adding a heading to an essay.
3. *Misconception check:* "Git automatically saves every keystroke." True or false? (False — only when you commit.)

Key points

- A **commit** is a snapshot of the whole project plus a message, author, date, parent and id.
- The **history** is a chain of commits, each pointing back to its parent.
- Good messages start with a verb, say what and why, and describe one idea.
- Commit after each small, complete step.
- Never change an old commit to fix a mistake; add a new one.

New words

commit, **snapshot**, **working copy**, **message**, **parent**, **commit id**, **history / log** — see the glossary.

Extension quest

Make the cards for a *wrong* edit (for example a line in the wrong place) and add a new commit that fixes it, instead of rubbing out an old card. History is never rewritten here — this prepares Module 5.

Teacher guide — Module 2 (2 periods)

Goal: a firm mental model — *commit = whole snapshot + message + parent*. Everything later in the book builds on this picture.

Timing: Period 1: hook 5, "Understand it" (teacher explains with the board drawing) 10, paper commits 25. Period 2: swap activity 10, predict 10, Parsons 10, free-resource exercise 1 or 3 (5–10), exit 5. **Materials:** 8 index cards per pair, string or lines drawn with pen, pens. Optional: one computer + projector for Learn Git Branching.

Board drawing (copy this exactly)

```
c1  <---  c2  <---  c3  <---  c4
```

Point at the arrows: "each commit remembers the one before it." Write a message under each card.

Common errors

- **"A commit is a copy of only the changed line."** Correct it: in the game each card holds the *whole* sheet. Git stores a snapshot of the project and cleverly shares unchanged parts.
- **Vague messages** ("done", "ok"). Use the swap activity to show why they hurt: the other pair cannot answer the questions.
- **Forgetting the parent.** Insist on the string or arrow.
- **Rubbing out a card.** Say: "a khata never tears out a page." Make a new card.

Fallbacks

Fully unplugged. With no cards, use half sheets of paper. With one computer, run free-resource exercise 1 on the projector and let pairs call out the commands.

Differentiation

- **Support:** give sentence starters for messages: *Add…*, *Fix…*, *Remove…*, *Change … to …*.
- **Stretch:** "If a commit stores the whole project, why doesn't the disk fill up?" (Unchanged files are shared, not copied — Grade 12 internals.)

Evidence to collect

The card rows: check that each card has a message starting with a verb, a parent, and the whole text.

Message makeover set (free-resource exercise 3)

Give these ten weak messages with a short description of the change; students rewrite them: `update` (added a date to the notice) · `asdf` (new paragraph about the library) · `final` (closing sentence) · `fix` (spelling of *Janakpur*) · `changes` (moved the title to the top) · `ok now` (removed a repeated line) · `done!!` (added the author's role) · `new` (added a second verse) · `edit 3` (changed "big" to "huge") · `misc` (fixed two typos in paragraph 2).

Sample answers: `Add date to the notice`, `Add paragraph about the library`, `Add closing sentence`, `Fix spelling of Janakpur`, `Move title to the top`, `Remove repeated line`, `Add author's role`, `Add second verse`, `Change "big" to "huge" in line 3`, `Fix two typos in paragraph 2`.

Answer key

- **Parsons:** Write title → Add first verse → Fix spelling → Add second verse (any order consistent with the card messages is fine if the student justifies it). Arrows point from each card back to the one before.
- **Exit 1:** a saved snapshot of the whole project with a message (and author, date, parent). **2:** e.g. `Add heading to the essay`. **3:** False.
- **Free-resource 1–2:** circles = commits; the newest is the one the `main*` label points at; arrows point to the parent because a commit records where it came from.
- **Free-resource 4:** Git stores snapshots of the whole project, not lists of differences.

Extension quest

Make the cards for a *wrong* edit and add a new commit that fixes it, instead of rubbing out an old card. (History is never rewritten here — prepares Module 5.)

GRADE 8 · MODULE 3 · 2 PERIODS · TIERS 1, 2, 3

3. My first repo

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 1–3 (Tier 0 version below) · **You need:** a lab PC with Git and VS Code (or GitHub Desktop) from the USB kit; the practice repo `g08-class-magazine`; your Module 2 cards
>
> **Before this:** Module 2 (commit = snapshot + message + parent).
>
> **By the end you can:** open a repository, change a file, stage it, commit it with a good message, and explain what `git status` is telling you.

> **No computer?**
>
> Tier 0: the teacher does the steps on one computer while pairs run the *same* steps on their paper cards. The "three places" picture below works on paper.

Hook (5 min)

In Module 2 you made commit cards by hand: copy the whole sheet, write a message, link the parent. It was slow. Today the computer makes the cards for you — in one click, perfectly, with the date, author and parent filled in automatically.

Understand it

A repository is a folder with a memory

A **repository** (**repo** for short) is an ordinary folder that Git is watching. It looks like any folder: you see your files (`article.md`, pictures, notes). The difference is a hidden sub-folder called `.git`. That is where Git keeps every commit — the whole history. If you delete `.git`, the folder forgets its history and becomes an ordinary folder again. So: **never touch `.git` by hand.**

You make a folder into a repository once, with `git init` (or "Initialize Repository" in VS Code). Or you get a ready-made one, as we do today, by **cloning** the practice repo.

Three places

Your work moves through three places on its way into history:

```
my files              the "next snapshot" box        the history
(working copy)  --->  (staging area)          --->   (commits)
     edit                  add / stage                  commit
```

- **Working copy** — the files you see and edit. Saving in your editor (Ctrl+S) only changes this.
- **Staging area** — a box where you put the changes you want in your *next* snapshot. "Add" or "stage" = *choose what goes in the photo*.
- **History** — the commits. "Commit" = *take the photo and write the message*.

Why a box in the middle? Imagine you fixed a spelling mistake *and* started a new paragraph that is only half done. You can stage only the spelling fix, commit it with the message `Fix spelling of Pokhara`, and leave the half paragraph for later. Each commit stays **one idea**.

Saving is not committing

This is the most important sentence of the lesson:

> **Ctrl+S saves the file. Only a commit puts it into history.**

If you edit and save but do not commit, Git knows the file *changed* but has no snapshot of it. If you then make a mess, there is nothing to go back to.

Reading `git status`

`git status` answers "where are my changes right now?". In plain words:

Git says	It means
`nothing to commit, working tree clean`	every change is safely in a commit
`Changes not staged for commit: modified: article.md`	you changed and saved the file, but it is *not* in the box yet
`Changes to be committed: modified: article.md`	the change is in the box; the next commit will include it
`Untracked files: my-notes.md`	a new file Git has never seen; add it if you want it in history

In VS Code the same information appears in the **Source Control** panel: **M** = modified, **U** = untracked, **A** = added (staged new file). GitHub Desktop shows a list of changed files with ticks.

Who am I?

Every commit records an author. Before the first commit on a computer, Git needs a name and an email. In school we use a **pseudonym**, never your real email:

```
git config --global user.name "Student 07"
git config --global user.email "s07@example.invalid"
```

Your teacher may already have done this on the lab PCs.

See it

Here is the whole cycle on the command line, with the real output (your commit id will be different). The teacher can show this on the projector while students use the buttons in VS Code.

```
$ git status
On branch main
nothing to commit, working tree clean          <- everything is committed

  (edit article.md: change the title, press Ctrl+S)

$ git status
On branch main
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   article.md                 <- changed, not in the box

$ git add article.md                           <- put it in the box

$ git status
On branch main
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   article.md                 <- in the box, ready

$ git commit -m "Change title to Dashain Festival at Our School"
[main 366579e] Change title to Dashain Festival at Our School
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git log --oneline -2
366579e Change title to Dashain Festival at Our School
4ebb164 Add closing sentence about the tastiest stall
```

Notice that `git status` even tells you which command to use next — read its hints.

PRIMM lab (40 min) — pairs, driver/navigator

Use the practice repo `practice-repos/g08-class-magazine`. Copy it from the USB kit, or make your own copy with:

```
git clone g08-class-magazine.bundle my-magazine
```

The **driver** uses the mouse and keyboard; the **navigator** reads the steps, predicts, and writes in the notebook. Swap roles every 10 minutes.

Predict — before every step, the navigator writes what they think will happen.

1. Open the folder `my-magazine` in VS Code (*File → Open Folder*) and open the **Source Control** panel (the branch-shaped icon on the left), or open it in GitHub Desktop. *Checkpoint:* the panel shows no changes. That matches `nothing to commit`.
2. **Predict & run:** change the title in `article.md` and save. What shows up in the panel? *(It is listed with an M — modified.)*
3. **Investigate:** click the file name. A **diff** opens: the old line on the left/red, the new line on the right/green. Module 4 explains this fully.
4. **Stage** the file (the **+** button next to it). Predict: where does it move? *(To "Staged Changes".)* Write a message in the box — `Change title to …` — and press **Commit** (✓). *Checkpoint:* the panel is empty again; your commit is in history.
5. **Modify:** add a sentence, stage, commit with a good message. Repeat until you have **3 new commits**. Each one = one idea.
6. **Investigate:** make two changes at once (fix a word in paragraph 1, add a line at the end). Stage only… can you? *(In VS Code, open the diff, select the lines, right-click → "Stage Selected Ranges". It is fine if this is hard — the idea matters more than the click.)*

7. **Make:** create a new file `my-notes.md` with two lines about your article idea. Predict what the panel shows *(U — untracked)*. Stage and commit it.
8. Open the history (VS Code: *Timeline* at the bottom of the Explorer, or the *Graph* in Source Control; GitHub Desktop: *History* tab). Count your commits. Compare with your Module 2 cards: what did the computer fill in for you?

Optional command-line peek (teacher demonstrates on the projector): repeat steps 2–4 with `git status`, `git add article.md`, `git commit -m "…"`.

Parsons practice (10 min)

Order the steps: *edit the file* · *git add article.md* · *git commit -m "…"* · *git status says 'nothing to commit'*. Then add where **Ctrl+S** goes.

Common mistakes

Symptom	Cause	Fix
`Please tell me who you are`	no name/email set on this PC	run the two `git config` lines above
Panel says "no repository"	folder opened one level too high or too low	open the folder that contains `article.md` (and the hidden `.git`)
Commit button greyed out / "no changes added to commit"	nothing staged	stage a file first (+)
Commit fails with an empty message	message box empty	write a message
"I saved it, why isn't it in history?"	saving ≠ committing	stage and commit
A strange `LF will be replaced by CRLF` warning	Windows and Linux use different line endings	harmless; ignore it in Grade 8

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Software Carpentry **Tracking Changes** ‹https://swcarpentry.github.io/git-novice/04-changes.html› — read the diagram of the staging area (the "git add → git commit" picture). Redraw it with our three places and labels in Nepali and English.	0 (printed, CC BY) / 3	15 min	drawing
2	VS Code docs, **Source Control overview** ‹https://code.visualstudio.com/docs/sourcecontrol/overview›: find the section on committing. Which keyboard shortcut commits? Try it in the lab.	1 / 3	10 min	the shortcut
3	Learn Git Branching `intro1` ‹https://learngitbranching.js.org/?locale=en_US&level=intro1› again, then in `sandbox` make 4 commits. Compare: what does the real VS Code history show that the game does not? *(messages, files, dates)*	1 / 3	10 min	2 differences
4	**Repo from scratch:** make a new empty folder `poem`, initialise it (VS Code: *Initialize Repository*), write a 4-line poem in `poem.md` over 4 commits — exactly like the Module 2 cards.	1	20 min	`git log --oneline` written in notebook
5	(Teacher or 13+ at home with internet) Read GitHub's **Hello World** ‹https://docs.github.com/en/get-started/start-your-journey/hello-world› page *without* doing it — just spot the words *repository*, *commit* and *branch*. We do the online version in Grade 9.	3	10 min	3 words found

Exit ticket (5 min)

1. What is the "box for the next snapshot" called?
2. Why can a file be changed but not yet committed?
3. *Misconception check:* "If I saved the file in my editor, it is in Git's history." True or false? (False.)

Key points

- A **repository** is a folder whose history lives in a hidden `.git` folder.
- Work flows **working copy** → **staging area** → **history** (edit → add → commit).
- **Saving is not committing.** Only a commit makes a snapshot.
- `git status` (or the Source Control panel) tells you where every change is.
- Commits need an author; in school use a pseudonym.

New words

repository (repo), `.git` **folder**, **working copy**, **staging area**, **stage / add**, **untracked**, **modified**, `git status` — see the glossary.

Extension quest

Use `git log --oneline` and match each line to a card from Module 2. Then run `git log` (without `--oneline`) and find the author and date of the oldest commit. Press **q** to leave the list.

Teacher guide — Module 3 (2 periods)

Goal: students make real commits with a GUI and can say where a change is (working copy, staging area, history).

Setup (before class, once per PC): Git and VS Code (or GitHub Desktop) installed from the Tier 1 USB kit. Then run:

```
git config --global user.name "Student 07"
git config --global user.email "s07@example.invalid"
git config --global init.defaultBranch main
```

Use a different number per PC; never students' real emails. Copy `g08-class-magazine.bundle` to each PC (see `practice-repos/g08-class-magazine/README.md`) and test one clone yourself.

Timing: P1: hook 5, concept "three places" 15, see-it demo 5, lab steps 1–4 (15). P2: lab steps 5–8 (25), Parsons 5, free-resource exercise 1 or 4 (5), exit 5.

Teaching the three places

Draw the three boxes on the board and keep them there all period. Every time a student asks "where is my change?", point to a box. Physical version: three trays on your desk labelled *working copy*, *box*, *history*; move a paper slip through them as you demonstrate.

Common errors

Symptom	Fix
"Please tell me who you are"	the `git config` commands above
Folder opened one level too high	open the folder that contains `article.md`
Commit button greyed out	nothing staged; stage a file first
Message empty, commit fails	write a message
Confusion: save vs commit	point back to the three-places picture; say "Ctrl+S is not a commit"
VS Code asks "stage all changes and commit directly?"	fine to say yes in Grade 8, but explain it skipped the box
`LF will be replaced by CRLF` warning	harmless on Windows; ignore
`git log` seems stuck	it opened a pager; press **q**

Fallbacks

- **One computer:** teacher drives with the projector; pairs sit with the paper cards and predict each result before you click.
- **No computer (Tier 0):** repeat the Module 2 cards with three-places labels: a tray for the working sheet, an envelope for "staged", the card row for history.
- **Power cut mid-lab:** files already committed are safe. Uncommitted edits may be lost if the editor had not saved them. Use it as a teaching moment after power returns: run `git status` and see what survived.

Differentiation

- **Support:** a printed step card with screenshots of the VS Code Source Control panel; pair a confident driver with an anxious navigator first, then swap.
- **Stretch:** free-resource exercise 4 (new repo from scratch) and lab step 6 (stage only part of a file).

Evidence to collect

Ask each pair to write `git log --oneline` (or the history list) in their notebook. Look for ≥ 3 new commits with verb-first messages.

Answer key

- **Exit 1:** the staging area. **2:** because it is only in the working copy until it is staged and committed. **3:** False.
- **Parsons:** edit (then Ctrl+S) → add → commit → status says nothing to commit.
- **Free-resource 2:** Ctrl+Enter in the message box commits (check your VS Code version; it may differ).
- **Free-resource 3:** VS Code shows messages, changed files, author and dates; the game shows only the graph.

Extension quest

Use `git log --oneline` and match each line to a card from Module 2.

GRADE 8 · MODULE 4 · 2 PERIODS · TIERS 0, 1, 2, 3

4. Reading history and comparing versions

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0–3 (Tier 0 uses printed versions) · **You need:** the practice repo `g08-class-magazine` (6 commits), red and green pens, printed versions for Tier 0
>
> **Before this:** Module 3 (making commits).
>
> **By the end you can:** read a history list; open a commit and say what it changed; read a diff line by line; find who, when and why for any change.

Hook (5 min)

Do you know "spot the difference" puzzles — two pictures that look the same, with seven small changes? A **diff** is a spot-the-difference that the computer solves for you, perfectly, in less than a second, even for a 100-page document.

Understand it

History: the list of commits

The **history** (also called the **log**) is the list of every commit, newest first. Each entry shows:

```
$ git log
commit 711a420055201db8271b5be81c7b6dd2243f5084      <- commit id
Author: Teacher <teacher@example.invalid>             <- who
Date:   Mon Oct 5 10:00:00 2026 +0545                 <- when (+0545 = Nepal time)

    Add Class Eight momo paragraph (typo left in on purpose)   <- why / what
```

The short form shows one line per commit:

```
$ git log --oneline
4ebb164 Add closing sentence about the tastiest stall
711a420 Add Class Eight momo paragraph (typo left in on purpose)
53bf9f6 Add paragraph about food stalls and library fund
b16c8b6 Say where the festival is and what happens
78a873e Add better title and opening line
d38a483 Add working title
```

Reading a history is like reading the contents page of a book: the messages tell the story of how the work grew. This is why good messages matter.

Diff: what exactly changed

A **diff** compares two versions and shows *only the lines that differ*:

```
--- a/article.md                    <- the old version ("a")
+++ b/article.md                    <- the new version ("b")
@@ -1,5 +1,7 @@                     <- where: old lines 1–5, new lines 1–7
 # Our School Festival               <- unchanged line (starts with a space)

-The festival is ... science models.     <- removed line (starts with -)
+The festival is ... science modals.     <- added line (starts with +)

 The best part is the food stalls. ...   <- unchanged
+
+Class Eight made momo and sold forty plates.   <- two new lines
```

The rules are short:

Line starts with	Colour (usually)	Meaning
`-`	red	this line was **removed** (or is the old form of a changed line)
`+`	green	this line was **added** (or is the new form)
a space	normal	unchanged, shown only for context

Git compares whole **lines**. If you change one word, Git shows the old line with `-` and the new line with `+`. Look carefully at the two lines to find the word — in the example above, *models* became *modals*: a typo was added!

If you cannot see colours (or print in black and white), the `+` and `-` signs carry the same information.

Three ways to ask "what changed?"

Question	Command	In VS Code / GitHub Desktop
What have I changed since my last commit (not yet committed)?	`git diff`	click a file in the Source Control panel
What did *this one commit* change?	`git show <id>`	click a commit in the history/timeline
What changed between two commits?	`git diff <old-id> <new-id>`	compare in the timeline

Detective questions

History lets you answer questions that were impossible with `final_final_v3`:

- *When* did the momo paragraph appear? → find the commit whose diff adds it, read its date.
- *Who* wrote the title? → the author of the commit that added it.
- *Why* was the opening line changed? → read that commit's message.

(In Grade 12 you will learn `git blame`, which answers "who last changed this line?" for every line at once.)

See it

Find the commit that brought the typo *modals*:

```
$ git log --oneline
...
711a420 Add Class Eight momo paragraph (typo left in on purpose)
...
$ git show 711a420
commit 711a420055201db8271b5be81c7b6dd2243f5084
Author: Teacher <teacher@example.invalid>
Date:   Mon Oct 5 10:00:00 2026 +0545

    Add Class Eight momo paragraph (typo left in on purpose)

diff --git a/article.md b/article.md
@@ -1,5 +1,7 @@
 # Our School Festival

-The festival is held every year in the school ground. Students sing, dance and show science models.
+The festival is held every year in the school ground. Students sing, dance and show science modals.

 The best part is the food stalls. Each class sells one snack and the money buys books for the library.
+
+Class Eight made momo and sold forty plates.
```

Who: Teacher. *When:* 5 October. *What:* the momo paragraph, and — sneakily — the typo. *Why:* the message (it even admits the typo). If you work on a computer where your commit ids are different, use the ones your `git log` shows.

Unplugged: spot the difference (15 min)

Pairs get two printed versions of the same paragraph (`article.md` at commit 2 and commit 5 — the files `versions/v2.md` and `versions/v5.md` in the practice repo).

1. With a **red pen**, cross out every word that was removed. With a **green pen**, underline every word that was added.
2. Now redo it *Git's way*: write the diff as lines, putting `-` before each old line and `+` before each new line.
3. Compare with another pair. Did you find the same changes? Did anyone find *modals*?
4. Discuss: what is easier — checking word by word, or the line-by-line diff?

PRIMM lab (40 min)

Repo: `g08-class-magazine` (the starter repo already has 6 commits). Pairs, driver/navigator.

1. **Predict:** look at the list of messages in `git log --oneline` (or the History view in VS Code / GitHub Desktop). Which commit added the paragraph about food stalls? Write the id.
2. **Run & investigate:** open that commit. Red lines = removed, green lines = added. Does it match your prediction?
3. **Investigate more:** for each of the 6 commits, fill this table in your notebook:

id	author	date	message	lines added	lines removed

   Tip: `git log --stat` shows how many lines each commit changed.

4. **Investigate:** the typo *modals* was fixed later. Which commit fixed it? Does its message say so? *(It does not! What does this teach about one idea per commit and honest messages?)*
5. **Modify:** make a new edit that changes one word. **Before you commit**, open the diff (`git diff` or click the file) and describe it in your notebook in your own words.

6. **Make:** write a commit message that matches *exactly* what your diff shows. Commit. Swap seats with another pair: can they tell from your message alone what the diff will show? Check with `git show`.

Command-line (optional):

```
git log --oneline
git show <commit-id>
git diff
git log --stat
```

`git diff` shows changes you have not committed; `git show` shows what one commit changed.

Parsons practice (10 min)

Match each description to a command: *list the history · what did this commit change? · what have I changed since the last commit? · how many lines did each commit change?* Commands: `git show <id>` · `git log --stat` · `git diff` · `git log --oneline`.

Common mistakes

Symptom	Cause	Fix
`git log` fills the screen and you cannot type	it opened a *pager* (a page viewer)	press **space** for the next page, **q** to quit
`git diff` shows nothing	there are no uncommitted changes, *or* your changes are already staged	use `git diff --staged` for staged changes, or `git show` for a commit
"The diff shows the whole paragraph changed for one word"	Git compares whole lines; a paragraph is one long line	look for the word inside the line; in Grade 9 you can use `git diff --word-diff`
Confusing `-` and `+`	—	`-` = old/removed, `+` = new/added; think "minus = taken away"

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Software Carpentry **Exploring History** ‹https://swcarpentry.github.io/git-novice/05-history.html›: read the first part about `git diff HEAD~1`. In our practice repo, run `git diff HEAD~1` and `git diff HEAD~2`. What does `~1` mean?	1 / 3	15 min	one sentence explanation
2	*Pro Git* §2.3 **Viewing the Commit History** ‹https://git-scm.com/book/en/v2/Git-Basics-Viewing-the-Commit-History›: try three options from its table (`--stat`, `--since`, `--author`) on the practice repo.	1 / 3	15 min	the 3 commands and what they showed
3	**Diff detective (paper):** your teacher prints `versions/v3.md` and `versions/v4.md`. Write the diff by hand with `+` and `-`, then check it on a computer with `git diff b16c8b6 53bf9f6` (ids may differ).	0 / 1	15 min	hand-written diff
4	In VS Code, open the **Timeline** for `article.md` and click two different commits. Take a screenshot (or draw) of one diff and label the old and new sides.	1	10 min	labelled picture

Exit ticket (5 min)

1. What do red and green (or `-` and `+`) lines mean in a diff?
2. Name two pieces of information a commit stores besides the file changes.

3. *Misconception check:* "A diff shows the whole file." (No — only the lines that differ, plus a few lines of context.)

Key points

- The **history/log** lists commits newest first: id, author, date, message.
- A **diff** shows only changed lines: `-` removed, `+` added, space = context.
- Git compares **lines**, not words.
- `git diff` = my uncommitted changes · `git show <id>` = one commit · `git log --stat` = size of each commit.
- Good messages make history readable; a message that hides a change (like the typo fix) misleads future readers.

New words

log, **diff**, **context line**, `git show`, `git diff`, **pager** — see the glossary.

Extension quest

Find the commit where *modals* was misspelled and the commit where it was fixed. Write a better message for the fixing commit — or explain why it should have been two separate commits.

Teacher guide — Module 4 (2 periods)

Goal: students read history and diffs fluently and use them to answer who / when / what / why.

Timing: P1: hook 5, "Understand it" with the annotated diff on the board 10, unplugged 15, lab 1–2 (10). P2: lab 3–6 (25), Parsons 5, free-resource exercise 3 (5), exit 5. **Materials:** printed `versions/v2.md` and `versions/v5.md` (one pair each), red and green pens; for Tier 0 also `log-sample.txt` and `versions/v3.md`, `versions/v4.md`.

Facts about the practice repo you need

Commit (message)	What its diff does
1 `Add working title`	creates `article.md` with a title
2 `Add better title and opening line`	new title + first sentence (3 lines added, 1 removed)
3 `Say where the festival is and what happens`	changes the opening sentence
4 `Add paragraph about food stalls and library fund`	adds 2 lines
5 `Add Class Eight momo paragraph (typo left in on purpose)`	adds momo line **and** changes *models* → *modals*
6 `Add closing sentence about the tastiest stall`	extends the momo line **and silently fixes** *modals* → *models*

Commit ids depend on the time zone of the PC that built the repo, so read them from your own `git log`.

Common errors

- `git log` opens a pager and the student is stuck: press **q** to quit.
- Colour-blind students: also teach the `+` and `-` signs at the start of lines; the unplugged activity can use "circle" and "underline" instead of colours.
- Mixing up `git diff` (uncommitted) and `git show` (a commit). Keep the three-question table on the board.
- "`git diff` shows nothing" after staging: use `git diff --staged`.
- A changed word shows as a whole changed line — expected; Git compares lines.

Fallbacks

- **Tier 0:** print the two versions of `article.md` (commit 2 and commit 5) and the log from the practice repo; run the whole lesson on paper. The lab table (step 3) can be filled from the printed table above.
- **One computer:** project it; pairs answer on paper and call out predictions.

Differentiation

- **Support:** give the lab table with ids and messages already filled; students add authors, dates, sizes.
- **Stretch:** `git log --since=2026-10-03`, `git log -p` (every diff), `git diff --word-diff`.

Evidence to collect

The completed lab table and the hand-written diff (exercise 3).

Answer key

- **Lab 1:** commit 4, `Add paragraph about food stalls and library fund`.
- **Lab 4:** commit 6 fixed the typo; its message does not mention it → messages should describe every change, or the fix should have been its own commit.
- **Exit 1:** red/`-` removed, green/`+` added. **2:** author, date/time, message, parent commit (any two). **3:** false.
- **Parsons:** `git log --oneline` list history · `git show <id>` commit changes · `git diff` uncommitted changes · `git log --stat` lines changed per commit.
- **Free-resource 1:** `HEAD~1` means "one commit before the latest"; `HEAD~2` two before.
- **Free-resource 3:** v3 → v4 adds a blank line and the food-stalls sentence (2 lines with `+`).

Extension quest

Find the commit where a word was misspelled and when it was fixed (commits 5 and 6).

GRADE 8 · MODULE 5 · 2 PERIODS · TIERS 0, 1, 2, 3

5. Undo and restore

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0–3 · **You need:** the practice repo (or your own repo from Module 3); Module 2 cards for Tier 0
>
> **Before this:** Module 4 (reading history and diffs).
>
> **By the end you can:** throw away a messy uncommitted change; bring back a file from any older commit; undo a bad commit with a new "revert" commit; choose the right undo for a situation.

Hook (5 min)

Your teacher shows a "disaster": a student deleted three paragraphs of the article and pressed Save. "*In your old way, it is gone. Today it comes back in 10 seconds.*" Then the teacher brings it back — in front of you.

Understand it

Git is a **time machine**, but a careful one. There are three different kinds of "undo", for three different situations. Choosing the right one is the skill.

Situation 1 — "I made a mess since my last commit"

You edited a file, it went wrong, you have **not committed**. You want the file exactly as it was in the last commit.

→ **Restore** the file: `git restore article.md` (VS Code: right-click the file in Source Control → *Discard Changes*; GitHub Desktop: right-click → *Discard changes*).

```
history:  c1 – c2 – c3          working copy: c3 + mess
                     \___ restore copies c3's version back ___/
```

> **The one dangerous undo**
>
> Restore throws away the uncommitted changes, and **Git cannot bring them back** — they were never in a commit. Always look at the diff first: *am I sure I want to lose all of this?*

Situation 2 — "I need an old version of one file"

Last Tuesday's version was better. You want *that* version back, but keep the history that came after.

→ **Restore from an older commit**: `git restore --source=<id> article.md`. The old version appears in your working copy as a normal change. Look at it, then **commit** it with a message such as `Bring back Tuesday's opening paragraph`.

Situation 3 — "My last commit was a mistake"

You committed something wrong. You do *not* rub out the bad commit — remember the khata: you never tear out a page. Instead you add a **new commit that does the opposite**.

→ **Revert**: `git revert <id>` (VS Code / GitHub Desktop: right-click the commit in the history → *Revert*).

```
before:  c1 – c2 – c3 – BAD
after:   c1 – c2 – c3 – BAD – "Revert BAD"     <– the article is like c3 again,
                                                   and the record shows what happened
```

Why keep the bad commit? Because history is a **true record**. A teacher or teammate can see the mistake *and* the fix. And if the "bad" commit turns out to be good after all, you can even revert the revert.

Which undo do I need?

Situation	Committed yet?	Use	History after
Messy edits I don't want	no	`git restore <file>` / *Discard*	unchanged (edits lost!)
Unstage something I added to the box by mistake	staged, not committed	`git restore --staged <file>`	unchanged (edit kept in working copy)
Bring back an old version of one file	—	`git restore --source=<id> <file>` then commit	one new commit
Undo a whole commit	yes	`git revert <id>`	one new "Revert …" commit

In Grade 11 you will meet more powerful (and more dangerous) tools such as `git reset`, which *can* rewrite history. In Grade 8 we use only the safe ones above.

The safety rule

- **Committed work is safe.** You can always get it back.
- **Uncommitted work is fragile.** Commit often — then nothing is ever really lost.

See it

A bad commit and its revert, with real output:

```
$ git commit -am "Change title to ???"
[main 28ed605] Change title to ???
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git revert --no-edit HEAD                 <– HEAD = the latest commit
[main 578a17e] Revert "Change title to ???"
 1 file changed, 1 insertion(+), 1 deletion(-)

$ git log --oneline -3
578a17e Revert "Change title to ???"         <– new commit: title is back
28ed605 Change title to ???                  <– the mistake stays in the record
366579e Change title to Dashain Festival at Our School
```

And throwing away a mess (situation 1):

```
$ git status --short
 M article.md                               <– M = modified, not committed
$ git restore article.md
$ git status
On branch main
nothing to commit, working tree clean       <– back to the last commit
```

Unplugged (15 min)

With the Module 2 cards, three situations. For each, pick the right card action and say which Git command it matches:

1. "I made a mess on my sheet since my last card." → throw the messy sheet away and re-copy the last card (**restore**).
2. "Card c4 was wrong." → make a **new** card c5 that undoes c4 (**revert**). Don't rub out c4!
3. "I need last Tuesday's paragraph." → look at card c2 and copy that paragraph back onto the sheet, then make a new card (**restore from an old commit**).

Then the teacher reads out 6 quick situations (see the teacher guide); pairs hold up a card saying **RESTORE**, **REVERT** or **OLD VERSION**.

PRIMM lab (40 min)

1. **Predict & run:** delete the middle paragraph of `article.md` and save (do not commit). Predict what Source Control shows. Look at the diff — it should be all red. Then **discard changes** for that file (in the tool) or `git restore article.md`. *Checkpoint:* the paragraph is back; `git status` says clean.
2. **Predict:** make and commit a bad edit (`Change title to ???`). Predict what `git log` shows if we **revert** it: one commit fewer, the same, or one more?
3. **Run:** revert the commit (right-click → *Revert*, or `git revert HEAD`). If an editor opens asking for a message, save and close it. Look at the log: one *new* commit, and the bad commit is still there.
4. **Investigate:** why does Git keep the bad commit? Discuss with your partner, then write one sentence. *(Because history is a record of what really happened.)*
5. **Modify:** restore only `article.md` from an older commit — for example the very first one: `git restore --source=d38a483 article.md` (use the id from your own log). Open the file: it is the first version! Run `git status` — it is a *change*, not yet committed. Decide: commit it, or restore again to go back to the latest version (`git restore article.md`).
6. **Make:** break and repair something on purpose (for example add a silly sentence and commit, then revert it). Write a revert message that explains *why* — `git revert` lets you edit the message.

Parsons practice (10 min)

Order: *notice the mistake · look at git log · choose revert · check the log again · write why*.

Common mistakes

Symptom	Cause	Fix
"I discarded changes and lost an hour's work"	restore on uncommitted work cannot be undone	always check the diff first; commit more often
A strange editor (Vim) opens during revert and you can't leave	Git asks for a revert message	type `:wq` and press Enter; or use `git revert --no-edit HEAD`; or set VS Code as the editor (teacher)
"Revert deleted my commit"	it did not — look at the log	the bad commit is still there, followed by its opposite
`restore --source` "did nothing"	the old version is in the working copy, not committed	look at the file / `git status`; commit if you want to keep it
Typed `git reset --hard` from a website	it can destroy work	stop and ask the teacher; we learn reset in Grade 11

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Software Carpentry **Exploring History** ‹https://swcarpentry.github.io/git-novice/05-history.html›, second half (restoring old versions with `git restore` / `git checkout`). Do its "Recovering Older Versions of a File" exercise on our article.	1 / 3	20 min	answer to the exercise
2	Learn Git Branching, level `rampup4` "Reversing Changes in Git" ‹https://learngitbranching.js.org/?locale=en_US&level=rampup4› — **only the `git revert` part.** Watch how revert adds a new commit. (Skip `git reset` for now.)	1 / 3	10 min	drawing of the graph after revert
3	*Pro Git* §2.4 **Undoing Things** ‹https://git-scm.com/book/en/v2/Git-Basics-Undoing-Things›: find the warning box. What does it say about discarded changes?	3	10 min	the warning in your own words
4	**Undo decision cards (paper):** make 4 cards — one per row of the "Which undo do I need?" table — with a drawing for each. Use them to test a partner.	0	15 min	the 4 cards
5	**Time-machine challenge:** in your own Module 3 `poem` repo, bring back the 2-line version of the poem, commit it, then revert that commit. Draw the history.	1	15 min	drawing + `git log --oneline`

Exit ticket (5 min)

1. Which command/tool throws away an uncommitted edit?
2. Does `revert` delete the bad commit? What does it do?
3. *Misconception check:* "Once I commit, I can never fix a mistake." (False.)

Key points

- Three undos: **restore** (throw away uncommitted mess), **restore from an old commit** (bring back an old version of a file), **revert** (undo a commit with a new commit).
- `git restore` on uncommitted work **cannot be undone** — check the diff first.
- Revert keeps the bad commit in history; history is a true record.
- Committed work is safe. Commit often.

New words

restore, **discard**, **revert**, **HEAD**, **unstage** — see the glossary.

Extension quest

After a revert, can you "revert the revert"? What happens to the article? Try it and draw the history.

Teacher guide — Module 5 (2 periods)

Goal: students choose the right, *safe* undo for a situation, and trust that committed work cannot be lost.

Timing: P1: hook 5 (live demo), "Understand it" with the decision table 10, unplugged 15, lab 1 (10). P2: lab 2–6 (30), Parsons 5, exit 5. **Before class:** on the projector PC, make the "disaster" ready: delete three paragraphs of `article.md`, save, *don't commit*. Practise `git restore article.md` once. Also set VS Code as Git's editor so revert does not open Vim: `git config --global core.editor "code --wait"` (or teach `--no-edit`).

Quick-fire situations (unplugged, round 2)

Read out; pairs hold up RESTORE / REVERT / OLD VERSION: 1. "I typed rubbish into my poem 5 minutes ago and haven't committed." → RESTORE 2. "Yesterday I committed a paragraph that is wrong." → REVERT 3. "I want the title I had on Monday." → OLD VERSION 4. "I committed a joke about my friend by mistake." → REVERT (and discuss: it is still in history! In Grade 12 we learn that secrets/private data need special handling — so never commit them.) 5. "I deleted a file but didn't commit." → RESTORE 6. "The new ending is worse than last week's." → OLD VERSION

Common errors

- Students run `restore` on work they wanted to keep. Teach "look at the diff first" as a habit; model it aloud every time.
- `revert` opens an editor asking for a message: in Vim type `:wq` + Enter; better, use `git revert --no-edit HEAD` or set VS Code as the editor (above).
- **"Revert" vs "reset":** do not teach `reset` in Grade 8. If asked: "it can rewrite history; we learn it in Grade 11."
- `restore --source` leaves a change that is not yet committed; students think nothing happened. Ask them to run `git status`.
- Revert of an older commit (not the latest) can produce a conflict. Keep to reverting `HEAD` in Grade 8.

Fallbacks

Tier 0: do the card version only and ask pairs to explain each choice. Power cuts: committed work survives — demonstrate it after the power returns.

Differentiation

- **Support:** give the "Which undo do I need?" table as a printed card to keep on the desk.
- **Stretch:** free-resource exercise 5, and "revert the revert".

Evidence to collect

The history drawing after lab step 6 (bad commit + revert) and the revert message (should say *why*).

Answer key

- **Exit 1:** `git restore <file>` / "Discard changes". **2:** no — it adds a new commit that undoes the changes. **3:** false.
- **Parsons:** notice → log → revert → log again → write why.

- **Lab 2 prediction:** one *more* commit.
- **Free-resource 3:** discarded local changes are gone for good; Git cannot recover them because they were never committed.
- **Free-resource 2:** after `git revert`, a new commit (C2') appears after the commit being reverted.

Extension quest

After a revert, can you "revert the revert"? What happens to the article? (It returns to the "bad" version, and the history shows all three steps.)

GRADE 8 · MODULE 6 · 1 PERIOD · TIERS 0, 1, 2, 3

6. Statistics: chart my commits

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 0–3 · **You need:** the practice repo or the printed `log-sample.txt`, graph paper, ruler; optional LibreOffice Calc
>
> **Before this:** Module 4 (reading history).
>
> **By the end you can:** turn a repository's history into a frequency table; draw and label a bar chart; find the mode and mean; say what the chart does and does not tell you.

Your repo's history is **data**. Today you will make a frequency table and a bar chart of it. This links to your mathematics lessons on statistics (frequency table, bar chart, mode, mean).

Hook (5 min)

"Which day did we work hardest on the magazine? Let's let the history answer."

Understand it

History is a data set

Every commit has a date and time. A list of dates is raw data, just like a list of marks or temperatures. When you **group** it (how many commits on each day?), you get a **frequency table**, and from that a **bar chart**.

Software teams do exactly this. Websites like GitHub draw a "contribution graph" from commit dates, and team leaders look at commit activity to see whether a project is alive. Now you know how it is made.

Getting the data out of Git

`git log` can print just the parts you ask for with `--format`:

You type	Git prints for each commit
`git log --date=short --format=%ad`	the date (`%ad` = author date)
`git log --format=%an`	the author's name
`git log --date=short --format="%ad %s"`	date and message (`%s` = subject)

What the chart can and cannot tell you

A commit chart measures **activity**, not **quality**. Ten tiny commits saying "fix" are not better work than two thoughtful commits. A good statistician always says what the data *does not* show.

See it

On the practice repo:

```
$ git log --date=short --format=%ad
2026-10-07
2026-10-05
2026-10-05
2026-10-02
2026-10-01
2026-10-01
```

Grouped into a frequency table:

Date	Tally	Frequency
2026-10-01	||	2
2026-10-02	|	1
2026-10-05	||	2
2026-10-07	|	1
Total		**6**

And as a bar chart (text version):

```
commits
  2 |  ##        ##
  1 |  ##   ##   ##   ##
    +----------------------
       1 Oct 2 Oct 5 Oct 7 Oct
```

There are two modes (1 Oct and 5 Oct). The mean over the 4 working days is 6 ÷ 4 = **1.5 commits per day**.

Bonus: a computer can count for you — `git log --date=short --format=%ad | sort | uniq -c` prints each date with its count. You will learn how pipes (`|`) work in Grade 9.

Collect (10 min)

Tier 1+: in your repo (the practice repo, or your own article repo), run

```
git log --date=short --format=%ad
```

Each line is the date of one commit. **Tier 0:** use the printed `log-sample.txt` from the practice repo, or the dates on your Module 2 cards.

Make a **tally table** in your notebook with columns *Date* · *Tally* · *Frequency*, as in the example above.

Chart (15 min)

Draw a bar chart on graph paper:

1. **x-axis:** dates (in order — even days with zero commits, if you want to show gaps).
2. **y-axis:** number of commits; choose a scale that fits the paper (1 square = 1 commit).
3. Draw one bar per date. Bars do not touch.
4. Give the chart a **title**, and label **both axes**.

Optional on computer: type the table into LibreOffice Calc, select it, and *Insert → Chart → Column*.

Interpret (5 min)

- Which day has the most commits? That is the **mode**.
- What is the **mean** number of commits per working day? (total commits ÷ number of days with commits)
- Was it better to commit a little every day or a lot on one day? Why might one be safer? (Think about Module 5 and power cuts.)

Common mistakes

Mistake	Fix
Counting *lines of output* as days	group the same dates first, then count
Dates out of order on the x-axis	sort them oldest → newest
Scale too big or too small	choose so the tallest bar uses most of the paper
Reading the chart as "who worked best"	commits measure activity, not quality

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Chart **commits per author** instead: `git log --format=%an` on a repo with several authors (your teacher can make one, or use the class Module 7 repos later). Which chart type fits better, bar or pie? Why?	1	15 min	chart + reason
2	*Pro Git* §2.3 **Viewing the Commit History** ‹https://git-scm.com/book/en/v2/Git-Basics-Viewing-the-Commit-History› has a table of `--format` codes. Pick two more codes (for example `%h` and `%s`) and print a custom history.	1 / 3	10 min	your command + output
3	**Hours of the day:** `git log --date=format:%H --format=%ad` prints the hour of each commit. Make a frequency table for your own repo. When do you work most?	1	15 min	table + one sentence
4	**Paper version:** using the dates on all the class's Module 2 cards, build one class frequency table and chart on chart paper for the wall.	0	20 min	wall chart

Exit ticket (5 min)

1. What is the frequency of commits on the busiest day in your chart?
2. Calculate the mean commits per day for your repo.
3. *Check:* "More commits always means better work." Agree or disagree and explain.

Key points

- A repository's history is a **data set**: dates, authors, messages.
- `git log --format=…` extracts exactly the data you want.
- Frequency table → bar chart → mode and mean.
- Commit counts measure **activity, not quality**.

New words

frequency table, **mode**, **mean**, `--format` — see the glossary.

Extension quest

Chart "commits per hour of the day" with `--format=%ad --date=format:%H`. Then compare with a classmate: who works in the morning, who in the evening?

Teacher guide — Module 6 (1 period)

Goal: apply Grade 8 statistics to a real data set the students produced themselves, and practise the critical point that activity ≠ quality.

Timing: hook 5 · collect 10 · chart 15 · interpret 5 · exit 5. "Understand it" can be read at home before the lesson.
Materials: graph paper, rulers, coloured pencils; printed `practice-repos/g08-class-magazine/log-sample.txt` for Tier 0.

Common errors

- Counting commit lines rather than commits per *date*. Model grouping on the board: circle equal dates, then tally.
- Scale choices that make bars unreadable; ask pairs to justify their scale.
- Reading the chart as a measure of quality. Use exit question 3 to discuss: commits measure activity, not quality.
- `--date=short` missing → long dates that are hard to group. Check the command.
- On Windows `sort | uniq -c` works in Git Bash, not in PowerShell — use Git Bash for the bonus.

Fallbacks

Tier 0: use the printed `log-sample.txt` (dates are included); Tier 1: LibreOffice Calc optional.

Cross-curricular link

Tell the mathematics teacher: this lesson practises frequency tables, bar charts, mode and mean with self-generated data. A joint lesson works well.

Differentiation

- **Support:** give a pre-drawn axis frame.
- **Stretch:** exercises 1 and 3; ask "what would a *histogram* of commit sizes (lines changed, from `git log --stat`) look like?"

Answer key

Answers depend on the repo. For `log-sample.txt`: 2026-10-01 → 2 commits, 10-02 → 1, 10-05 → 2, 10-07 → 1 (total 6). Busiest days: 10-01 and 10-05 (two modes). Mean over the 4 working days = 6 ÷ 4 = 1.5 commits per day. Exit 3: disagree — messages and content quality matter; many tiny meaningless commits are not better. **Free-resource 1:** bar chart — authors are categories and we compare counts; a pie works only if parts of a whole matter and there are few authors. **Free-resource 2:** e.g. `git log --format="%h %s"` → short id and subject.

Extension quest

Chart "commits per hour of the day" with `--format=%ad --date=format:%H`.

GRADE 8 · MODULE 7 · 2 PERIODS · TIERS 0, 1, 2, 3

7. Mini-project: Class Magazine

> **At a glance**
>
> **Time:** 2 periods + work at home or in free lab time over at least 3 sessions · **Tiers:** 0–3 · **You need:** a repo (Tier 1+) or a stack of commit cards (Tier 0)
>
> **Before this:** Modules 1–6.
>
> **By the end you can:** keep a real piece of writing under version control over several days, use restore or revert on purpose, and present how your work grew.

Each student keeps **one article** (a poem, a school news item, a description of a local festival, a recipe from home, a short report on a science model) under version control. At the end the class holds an **exhibition** of the "evolution" of each article: not only the final text, but *how it was made*.

Why a project?

Everything so far was practice on someone else's article. Now it is your own work, over several days — exactly how real people use version control. You will find out for yourself why small commits and good messages matter, because *you* will be the one reading your history.

The brief

- Choose a topic from the school or community. **Do not include real names of classmates**, phone numbers or addresses; use roles ("the captain", "our neighbour").
- Make a repo: copy `practice-repos/g08-class-magazine` and replace `article.md` with your own text, or create a new folder and *Initialize Repository* (Tier 1+). Tier 0: a card stack.
- Write the article in at least **5 commits** over at least **3 sessions**.
- Include at least one **restore or revert**, on purpose, with a message that says why.
- Draw the commit-count chart from Module 6.
- Prepare a 2-minute explanation for the exhibition.

Suggested shape of a good history

```
c1  Add working title and three ideas for the article
c2  Write first paragraph about where the festival happens
c3  Add paragraph about the food
c4  Change opening line after partner's feedback
c5  Revert "Add joke in the conclusion" — it was unkind
c6  Fix spelling of Janakpurdham and Mithila
c7  Add closing sentence
```

Notice: each commit is one idea; the messages tell a story; feedback and the revert are visible.

Plan

Stage	Time	What
Choose and plan	10 min	topic, 3 bullet ideas, first commit

Stage	Time	What
Draft	1 period + home	first version, commits 2–3
Peer feedback	10 min	swap with a partner; partner reads your **history first**, then the text, and writes 2 suggestions on paper
Improve	1 period	commits 4–5+, one restore or revert
Chart	10 min	commit chart from Module 6
Exhibition	20 min	show `git log` (or card stack), one diff, one chart; explain a key commit

Peer-feedback questions (for your partner)

1. Reading only the messages, can you tell how the article grew?
2. Is any commit doing two things at once?
3. One thing you liked in the text; one suggestion.

Exhibition checklist

☐ My history (`git log --oneline` printed or written, or my card row).
☐ One diff I am proud of — I can explain the red and green lines.
☐ My restore or revert, and *why* I did it.
☐ My chart, titled and labelled.
☐ The final article.

Rubric (commit-history rubric)

Criterion	3	2	1
Number of commits	≥ 5	3–4	1–2
Messages	each says what + why	most do	vague ("done")
One idea per commit	yes	mostly	no
Restore/revert used	with explanation	used	not used
Oral explanation (2 min)	clear	partly clear	unclear
Chart	correct and labelled	minor errors	missing

Maximum 18 points.

Common mistakes

Mistake	Fix
All commits in one session, minutes apart	spread work over ≥ 3 sessions
Writing everything, then committing once	commit after each paragraph
Names, phone numbers or addresses of real people	remove them; use roles
Forgetting the restore/revert	plan it: write a sentence you will revert on purpose
Messages like "update"	use the Module 2 message rules

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Before you start, play Learn Git Branching `intro1` ‹https://learngitbranching.js.org/?locale=en_US&level=intro1› once more and draw the history you *plan* for your article (5–7 circles with messages). Compare it with the real history at the end.	1 / 3	10 min	plan vs real drawing
2	Write your article in **Markdown** (headings with `#`, *italics* with `*`). Use the CommonMark 10-minute tutorial ‹https://commonmark.org/help/tutorial/› (teacher can print the reference card, or use it at home with internet).	1 / 3	15 min	article with ≥ 2 Markdown features
3	**Badges:** create `BADGES.md` in your repo and record the badges from the Grade 8 cards page you earned this chapter — commit it!	1	5 min	`BADGES.md` commit

Exit ticket (5 min)

Which commit are you proudest of, and what does its message say?

Key points

- Version control is most useful on *your own* work over *several days*.
- Small commits + clear messages = a history that tells a story.
- Undo (restore/revert) is a skill, not a failure.
- Private information never goes into a repo.

Extension quest

Compare two classmates' histories of different topics: whose is more detailed? What makes a history useful to a reader?

Chapter complete. You can now keep your own work's history. In Grade 9 you will use Git from the **terminal**, learn **branches** (try two ideas at once) and send your work to another computer with **remotes**.

Teacher guide — Module 7 (2 periods + home)

Goal: a real project over several days; process evidence (the history) is assessed, not only the final article.

Timing: P1: brief 10, plan + first commit 10, draft 20. Between periods: at least one more session (home, free lab time, or a 15-minute slot in another lesson). P2: feedback 10, improve 15, chart 5, exhibition 10–15 (extend if the class is large — run the exhibition as a **gallery walk**: half the class stands by their work, the other half visits, then swap).

Marking

Use the rubric (max 18). **Read `git log` before reading the article: process first.** Do not mark down a revert — it is a skill. Suggested grade bands: 16–18 excellent, 12–15 good, 8–11 developing, below 8 needs support (repeat Modules 3–5 with a buddy).

Making sure there are 3 sessions

Shared PCs and short periods make this hard. Options: - Keep each student's repo on a named USB folder or a per-student folder on the lab PC. - Give two 15-minute "commit slots" during other computer lessons that week. - Tier 0: cards can be taken home.

Common errors

- All commits in one session, minutes apart. Ask for ≥ 3 sessions; dates in `git log` show this.
- Writing the whole article then committing once. Teach: commit per paragraph.
- Names of classmates, phone numbers or addresses in articles. Remove before the exhibition; never publish this repo.
- Students revert the wrong commit and panic. Show that the revert itself can be reverted — nothing is lost.

Fallbacks

- **Tier 0:** students keep a stack of cards; the "exhibition" is a gallery of card rows.
- **Tier 1 with shared PCs:** each pair has a USB folder; commit to the USB. Keep a backup copy at school (copy the whole folder including `.git`).

Safety and privacy

Grade 8 repos stay on the school computer or USB. No account creation. No publishing.

Differentiation

- **Support:** give the "suggested shape" history as a template to follow; allow a shorter article.
- **Stretch:** a second file (`sources.md` listing where facts came from), Markdown formatting, the hours-of-the-day chart from Module 6.

Evidence to collect

The printed history, the chart, the rubric sheet, and a one-line teacher note from the oral explanation. Keep for the end-of-year portfolio.

Answer key

No single answer. Strong exit-ticket answers name a specific commit and explain why its message is clear or why its change mattered.

Extension quest

Compare two classmates' histories of different topics: whose is more detailed? What makes a history useful to a reader?

Printable cards (Grade 8)

Print on A4, cut along the lines. Black-and-white friendly.

Paper-commit card (print 8 per pair)

```
+-------------------------------------------+
| Commit no.: c___        parent: c___      |
| Message (what + why):                     |
| ____________________________________      |
|                                           |
| Copy of the whole sheet goes here:        |
|                                           |
|                                           |
+-------------------------------------------+
```

Parsons cards — Module 3 (cut each line into a strip, shuffle, order)

- edit the file
- git add article.md
- git commit -m "Change title"
- git status says "nothing to commit"

Parsons cards — Module 5

- notice the mistake
- look at git log
- git revert HEAD
- check the log again
- write why in the message

Badges (record in your own `BADGES.md`; no leaderboards)

Badge	Earned when
First Snapshot	made your first commit
Time Traveller	restored an old version
Spot the Difference	explained a diff aloud
Historian	five meaningful commits

Part III. Grade 9 — Branch and Share

Grade 9 — Branch and Share

> **Unpiloted draft**
> Core lessons are written but have not been tried in a classroom. Grade placement of topics is unverified against the CDC syllabus (see `research/decisions.md`).

> **Grade placement**
> Python and MySQL are confirmed in CDC's **Grade 10** textbook, not Grade 9 (see `alignment/cdc-ncf2076.yml`). Modules 6–7 can move to Grade 10; modules 1–5 and 8 do not depend on them.

Big idea: branches let people work in parallel; remotes let work travel. **Length:** ≈ 16 periods (to calibrate to CDC hours). **Accounts:** only for learners 13+, managed by the teacher; otherwise a LAN server or bundles on USB. **Tools:** Git CLI (Git Bash on Windows), Python, SQL (MySQL or SQLite), Forgejo LAN or class organisation.

How this chapter is built

Grade 8 gave students one line of history on one computer. Grade 9 adds two dimensions: **parallel lines** (branches and merges) and **other computers** (remotes). The terminal arrives first so that every later command can be written down exactly.

```
M1  terminal            (cd, ls, mkdir)
M2  CLI cycle           (Grade 8 in commands)
M3  branches            (labels on commits)
M4  merge / conflict    (two lines join)
M5  remotes             (work travels)
M6  Python              (.gitignore, debugging)
M7  SQL                 (recipe vs dish)
M8  pair project        (agreement, changelog, tag)
```

Modules

#	Module	Periods	Tiers	Key commands
1	Terminal basics bridge	1	1–3	`pwd`, `ls`, `cd`, `mkdir`
2	status/add/commit/log on the CLI	2	1–3	`init`, `status`, `add`, `commit`, `diff --staged`
3	Branches	2	0–3	`branch`, `switch -c`, `log --graph --all`
4	Merge and the conflict game	2	0–3	`merge`, `merge --abort`
5	Remotes	2	1–3	`clone`, `push`, `pull`, `fetch`, `bundle`
6	Python under version control	2	1–3	`.gitignore`, `log -p`
7	SQL under version control	2	1–3	schema/queries, dumps
8	Pair project: Ward Facts	3	1–3	`tag`, `shortlog`

Chapter outcomes

1. use the terminal to create and navigate folders
2. run the basic Git cycle on the command line
3. create, switch and merge branches

4. resolve a simple merge conflict
5. clone, push and pull to a shared remote
6. explain why scripts (not live databases) are versioned

Free resources used in this chapter

Software Carpentry's Unix Shell ‹https://swcarpentry.github.io/shell-novice/› and git-novice ‹https://swcarpentry.github.io/git-novice/› episodes 2–9 (CC BY), Learn Git Branching ‹https://learngitbranching.js.org/› levels `intro2`, `intro3`, `rampup1` and the Remote tab, *Pro Git* ‹https://git-scm.com/book/en/v2› chapters 2–3, Visualizing Git ‹https://git-school.github.io/visualizing-git/›, Oh My Git! ‹https://ohmygit.org/›, and — for learners 13+ with internet — GitHub Skills ‹https://skills.github.com/›. Full list: free resource library.

Assessment

Practical (create → branch → merge → push), Parsons card test, short written questions in board style. First-draft item bank: `assessments/g09-item-bank.md` (to be reviewed after Pilot 1).

Capstone

Ward Facts — pairs maintain a Python script and a SQL file that answer three questions about a small (synthetic) ward dataset. Practice repo: `practice-repos/g09-ward-facts/`.

Ready for Grade 10?

A student is ready when they can, without help: create a branch, commit on it and merge it; resolve a one-line conflict; and recover from a rejected push by pulling.

GRADE 9 · MODULE 1 · 1 PERIOD · TIERS 1, 2, 3

1. Terminal basics bridge

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 1–3 (Tier 0: the human-terminal game) · **You need:** a lab PC with **Git Bash** (Windows) or a terminal (Linux/macOS); chalk or tape for the floor map
>
> **Before this:** Grade 8 (or the bridge course).
>
> **By the end you can:** say where you are (`pwd`), look around (`ls`), move (`cd`), make folders (`mkdir`), and explain absolute and relative paths.

Hook (5 min)

The teacher asks a student to find `article.md` using only typed words — no mouse. It feels slow at first. Then the teacher types one line that makes 30 folders, one per student, in a second. *Why might a programmer prefer typing?*

Understand it

What is the terminal?

The **terminal** (also called the *command line* or *shell*) is a window where you give the computer instructions by typing. Each instruction is a **command**. You type it after the **prompt**, press Enter, and the computer replies with text.

```
student07@LAB-PC MINGW64 ~/school/class9        <- prompt: who @ computer, WHERE
$ ls                                            <- you type a command after the $
ward-facts  magazine  notes.txt                 <- the computer's answer
```

Why learn it when buttons exist?

- **Every Git feature** is available in the terminal; buttons cover only some.
- Commands can be **written down, shared and repeated** exactly ("type this") — clicks cannot.
- Servers, Raspberry Pis and cloud computers usually have **no mouse at all**.
- It works the same on Windows (with Git Bash), Linux and macOS.

Folders form a tree

Your files live in folders inside folders — a **tree**, drawn upside down:

```
/c/Users/student07            <- your "home" folder (shortcut: ~)
├── school
│   ├── class8
│   │   └── magazine
│   └── class9
│       └── ward-facts        <- we want to be here
└── Downloads
```

A **path** is the address of a folder or file in that tree.

- An **absolute path** starts from the top: `/c/Users/student07/school/class9/ward-facts`. It works from anywhere.

- A **relative path** starts from where you are now. From `school`, the same folder is `class9/ward-facts`.
- Two special names: `.` means "this folder" and `..` means "the folder above". `~` means your home folder.

Five commands for today

Command	Means	Example
`pwd`	**p**rint **w**orking **d**irectory — where am I?	`pwd` → `/c/Users/student07/school`
`ls`	**l**i**s**t what is here	`ls` → `class8 class9`
`cd <path>`	**c**hange **d**irectory — move	`cd class9`, `cd ..`, `cd ~`
`mkdir <name>`	**m**a**k**e **dir**ectory	`mkdir ward-facts`
`mkdir -p a/b/c`	make several levels at once	

Useful extras: `ls -a` shows hidden files (such as `.git`!), `clear` clears the screen, and the **Tab** key completes names for you — type `cd wa` then Tab. The ↑ **arrow** brings back the last command.

> **Windows labs**
> Use **Git Bash** (installed with Git for Windows), not `cmd`. Then every command in this book works the same on Windows and Linux. In `cmd` the equivalent of `ls` is `dir`.

See it

```
$ pwd
/c/Users/student07
$ mkdir -p school/class9
$ cd school/class9
$ mkdir ward-facts
$ ls
ward-facts
$ cd ward-facts
$ pwd
/c/Users/student07/school/class9/ward-facts
$ cd ..
$ pwd
/c/Users/student07/school/class9
$ cd
$ pwd
/c/Users/student07                        <- "cd" alone always takes you home
```

Unplugged: the human terminal (15 min)

Draw a floor map of folders with chalk squares: `home` → `school` → `class9` → `ward-facts`, and a side branch `Downloads`. One student is the **computer** and stands in `home`. The partner is the **user** and writes commands on paper, one at a time: `cd school`, `ls`, `cd class9`, `cd ..`, `pwd`. The computer obeys *exactly* — no guessing. If the command is impossible (`cd ward-facts` from `home`), the computer says `No such file or directory`. Swap roles. Then the class writes, on the board, the **shortest** command list to go from `Downloads` to `ward-facts` (answer: `cd ../school/class9/ward-facts`).

PRIMM lab (25 min)

1. **Predict:** what will `pwd` print when you open the terminal? Run it.
2. `ls` (or `dir` on Windows cmd; Git Bash accepts `ls`). Predict, then run. Then `ls -a`: what new names appear?

3. `mkdir ward-facts` then `cd ward-facts`. Check with `pwd`.
4. **Investigate:** what does `cd ..` do? What does `cd` alone do? What does `cd -` do?
5. **Modify:** make `a/b/c` using `mkdir -p`, go into `c`, then return home in **one** command.
6. **Make:** create a folder tree for your pair project (for example `ward-facts/data`, `ward-facts/scripts`, `ward-facts/docs`) and draw it as a tree in your notebook.

Parsons practice (5 min)

Order: *mkdir ward-facts · cd ward-facts · pwd · ls*. Then: write the commands to go from `ward-facts` back to the folder above.

Common mistakes

Symptom	Cause	Fix
`No such file or directory`	you are not where you think, or a typo	`pwd`, then `ls`; use Tab completion
Spaces in names break commands	`cd my folder` is read as two words	avoid spaces; use `my-folder`, or quotes: `cd "my folder"`
`ls` "doesn't work" on Windows	you are in `cmd` or PowerShell	open Git Bash
Case errors (`Ward-Facts` vs `ward-facts`)	Linux paths are case-sensitive	use lower-case names everywhere
Lost in the tree	—	`cd` alone goes home; start again

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Software Carpentry Shell: **Navigating Files and Directories** ‹https://swcarpentry.github.io/shell-novice/02-filedir.html› (CC BY — can be printed). Do its exercises on absolute vs relative paths.	1 / 3	30 min	exercise answers
2	Software Carpentry Shell: **Working With Files and Directories** ‹https://swcarpentry.github.io/shell-novice/03-create.html› — only the parts on `mkdir`, `touch`/creating files, and good names.	1 / 3	20 min	3 naming rules
3	explainshell ‹https://explainshell.com/›: paste `ls -la` and `mkdir -p a/b/c`. What does each part mean?	3	10 min	explanations
4	William Shotts, **Learning the Shell** ‹https://linuxcommand.org/lc3_learning_the_shell.php› — first two pages ("What is the shell?", "Navigation").	3	20 min	5 new facts
5	**Speed challenge (paper or PC):** from `home`, reach five folders the teacher names using the fewest commands.	0 / 1	10 min	command lists

Exit ticket (5 min)

1. What does `cd ..` do?
2. Which command shows where you are?
3. *Misconception:* "The terminal deletes things by itself." True or false?

Key points

- The terminal runs typed **commands**; the prompt shows where you are.

- Folders form a **tree**; a **path** is an address in it (absolute from the top, relative from here).
- `pwd` where · `ls` look · `cd` move · `mkdir` make. `..` is up, `~` is home.
- Tab completes, ↑ repeats. Use Git Bash on Windows.

New words

terminal / shell, **command**, **prompt**, **path**, **absolute / relative path**, **home folder** — see the glossary.

Extension quest

Find out what `ls -l` shows that `ls` does not. What is the date column?

Teacher notes

- **Timing:** hook 5 · understand + see it 10 · human terminal 15 · lab 10 (steps 1–4; 5–6 as homework) · exit 5. If short of time, run the human terminal for only one pair at the front.
- Use **Git Bash** on Windows labs so students see the same commands as on Linux. Pin it to the taskbar before class.
- **Avoid** `rm -r` in this lesson. If a student asks how to delete, show `rmdir` on an empty folder only.
- **Misconception:** the terminal is scary/dangerous. It only does what you type; most commands (all of today's) only look or create.
- **Fallback Tier 0:** run the human-terminal game for the whole period, including the shortest-path challenge.
- **Answer key:** Exit 1: moves to the folder above. 2: `pwd`. 3: False — it only does what you type. Lab 4: `cd` alone → home; `cd -` → back to the previous folder. Lab 5: `cd ~` or `cd`. Parsons answer as given; then `cd ..`.

GRADE 9 · MODULE 2 · 2 PERIODS · TIERS 1, 2, 3

2. status, add, commit, log on the command line

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 1–3 · **You need:** Git Bash or a terminal; a pseudonymous `user.name` / `user.email` already set
>
> **Before this:** Module 1 (terminal), and Grade 8's three places.
>
> **By the end you can:** create a repository from the terminal, run the full commit cycle, and predict `git status` before you run it.

Hook (5 min)

In Grade 8 you clicked buttons. Today you type the same actions — and every click now has a name you can write down, share in a chat, and repeat exactly. "Click the plus, then the tick" becomes `git add README.md && git commit -m "Add README"`.

Understand it

Same three places, now with names

```
  working copy  --git add-->  staging area  --git commit-->  history
       ^                                                        |
       +------------------ git restore (Grade 8) --------------+
```

Grade 8 button	Command	What it does
Initialize Repository	`git init`	makes the hidden `.git` folder
file list in Source Control	`git status`	where is every change?
click a changed file	`git diff`	changes not yet staged
+ (stage)	`git add <file>`	put the change in the box
click a staged file	`git diff --staged`	what *will* be committed
✓ **Commit** + message	`git commit -m "message"`	take the snapshot
History / Timeline	`git log`, `git log --oneline`	read the history

The shape of a Git command

```
git  commit  -m  "Add README"
 |     |      |      |
 |     |      |      +-- argument for the option (quotes keep the words together)
 |     |      +--------- option (starts with -)
 |     +---------------- subcommand: what you want Git to do
 +---------------------- the program
```

Status is your best friend

Run `git status` **before and after every other command** until you can predict its answer. It shows the branch you are on, what is staged ("to be committed"), what is changed but not staged, and what is untracked (new files Git does not know yet). It also prints hints for the next command.

A file's life:

```
 untracked --add--> staged (new file) --commit--> committed (clean)
                                                      |
                      staged (modified) <--add-- modified (edit + save)
```

`diff` vs `diff --staged`

- `git diff` = working copy compared with the staging area → *"what have I changed but not staged?"*
- `git diff --staged` = staging area compared with the last commit → *"what will my next commit contain?"*

Professionals run `git diff --staged` right before committing. It catches accidents — a debugging line, a password, a file you didn't mean to include.

Messages on the command line

`-m "…"` gives the message directly. If you forget `-m`, Git opens a text editor to ask for one. If that editor is **Vim** (a black screen with `~` signs), type `:q!` and Enter to quit without committing, then use `-m`. Your teacher can set an easier editor with `git config --global core.editor "code --wait"`.

See it

Real output from a new repository:

```
$ git init ward-facts
Initialized empty Git repository in /c/Users/student07/ward-facts/.git/
$ cd ward-facts
$ git status
On branch main

No commits yet

nothing to commit (create/copy files and use "git add" to track)
$ echo "# Ward Facts" > README.md          <- creates a file with one line
$ git status
On branch main

No commits yet

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        README.md

nothing added to commit but untracked files present (use "git add" to track)
$ git add README.md
$ git status
On branch main

No commits yet

Changes to be committed:
  (use "git rm --cached <file>..." to unstage)
        new file:   README.md
$ git commit -m "Add README"
[main (root-commit) 8b2c8b6] Add README
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
$ git log --oneline
8b2c8b6 Add README
```

`root-commit` means "the first commit — it has no parent".

Concept warm-up (10 min, unplugged)

On the board, draw the three places as boxes. The teacher reads out commands one by one (`git add notes.md`, edit `notes.md` again, `git commit` …); students move a paper slip for each file between the boxes on their desks, then say what `git status` would print.

PRIMM lab (50 min)

1. `git init ward-facts` and `cd ward-facts`. **Predict** `git status`, then run it.
2. Create `README.md` with one line (`echo "# Ward Facts" > README.md`, or use VS Code: `code README.md`). **Predict** status ("untracked"). Run.
3. `git add README.md`; predict status ("new file", staged). Run.
4. `git commit -m "Add README"`. Run `git log --oneline`.
5. **Investigate:** change README (add a sentence), run `git diff`. Then `git add README.md` and run `git diff` again — why is it empty now? Run `git diff --staged`.
6. **Investigate:** edit README *again* after staging. Run `git status`. Why is README listed **twice**, as staged and not staged? *(The box holds the first edit; the second edit is only in the working copy.)*
7. **Modify:** make 3 more commits with messages that say *what and why*. Add a second file `notes.md` in one of them.
8. **Make:** draw your history as cards (like Grade 8) and check with `git log --oneline --graph`. Then try `git log --stat` and `git show HEAD`.

Parsons practice (10 min)

Order: *git init · create a file · git add · git commit · git log*. Second set (harder): *edit file · git diff · git add · git diff --staged · git commit -m "…" · git status*.

Common mistakes

Symptom	Cause	Fix
`fatal: not a git repository`	you are in the wrong folder	`pwd`; `cd` into the repo
`Please tell me who you are`	identity not set	`git config --global user.name "Student 07"` and `user.email "s07@example.invalid"`
`error: pathspec 'Add' did not match`	message without quotes	`git commit -m "Add README"`
Vim opens	forgot `-m`	`:q!` Enter, then use `-m`
Committed in your home folder by mistake (the whole home is a repo!)	ran `git init` in `~`	ask the teacher; remove only that `.git` folder carefully
`git add .` added files you did not want	`.` means *everything here*	check with `git status` first; `git restore --staged <file>`

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Software Carpentry **Creating a Repository** ‹https://swcarpentry.github.io/git-novice/03-create.html› and **Tracking Changes** ‹https://swcarpentry.github.io/git-novice/04-changes.html›. Do the exercises "Places to Create Git Repositories" and "Choosing a Commit Message".	1 / 3	40 min	answers
2	*Pro Git* §2.2 **Recording Changes to the Repository** ‹https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository›: read the "short status" part and try `git status -s`. What do `??`, `A` and `M` mean?	1 / 3	15 min	the three codes
3	Git Immersion ‹https://gitimmersion.com/›, labs 1–10 (setup through "Changes, not files"). Use a pseudonym in the setup lab.	3	45 min	the history you made
4	**Predict-the-status worksheet** (teacher guide): 8 situations, write what `git status -s` prints for each.	0	15 min	worksheet
5	Check your setup: `git config --list --show-origin`. Where is your name stored? (*Pro Git* §1.6 **First-Time Git Setup** ‹https://git-scm.com/book/en/v2/Getting-Started-First-Time-Git-Setup›.)	1	10 min	file path

Exit ticket (5 min)

1. What does `git status` tell you?
2. Difference between `git diff` and `git diff --staged`?
3. *Misconception:* "`git add` saves my file to history." True or false?

Key points

- `init` → (edit) → `add` → `commit` → `log` is the cycle; `status` between every step.
- A file is untracked → staged → committed → modified → staged …
- `git diff` = not yet staged; `git diff --staged` = what the next commit contains.
- Always use `-m "message"` with quotes; read Git's hints.

New words

`git init`, **root commit**, **option / argument**, **short status**, `git diff --staged` — see the glossary.

Extension quest

Make an **alias**: `git config --global alias.lg "log --oneline --graph --all"`. Now `git lg` draws the graph. (*Pro Git* §2.7 "Git Aliases".)

Teacher notes

- **Timing:** P1: hook 5, understand 15, see it (live) 10, warm-up 10. P2: lab 35, Parsons 5, exit 5. Lab steps 7–8 can be homework.
- Set `user.name` / `user.email` with pseudonyms first (see teacher setup). Also set `init.defaultBranch main` and an easy editor.
- **Typical errors:** committing from the wrong folder, forgetting quotes around the message, Vim opening for the message (type `:q!`; teach `-m`).
- **Predict-the-status worksheet** (exercise 4) — situations and answers (`git status -s`): new file not added → `?? a.txt`; new file added → `A a.txt`; committed file edited → `M a.txt`; edited and added → `M a.txt`;

added then edited again → `MM a.txt`; nothing changed → (no output); file deleted with `rm` → `D a.txt`; two new files, one added → `A a.txt` and `?? b.txt`.

- **Answer key:** Exit 1: which branch, what is staged, modified, untracked. 2: unstaged changes vs staged changes. 3: False — `add` only puts it in the box; `commit` makes history. Lab 5: after `add`, the working copy equals the box, so `git diff` is empty. Exercise 2: `??` untracked, `A` added (new, staged), `M` modified.

GRADE 9 · MODULE 3 · 2 PERIODS · TIERS 0, 1, 2, 3

3. Branches

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0–3 · **You need:** Module 2 cards from Grade 8 (or new ones) and sticky notes; a terminal; Learn Git Branching (self-hosted or online)
>
> **Before this:** Module 2.
>
> **By the end you can:** explain what a branch really is, create one, switch between branches, and read a branch graph.

Hook (5 min)

Two students want to improve the same poem differently: Sita adds a verse, Hari fixes spelling. Do they have to take turns? What if Sita's verse turns out badly — can we throw it away without losing Hari's fixes?

Understand it

A branch is a label, not a copy

In Grade 8 the history was one straight line of commits. A **branch** is simply a **name stuck on one commit** — like a sticky note. The default branch is usually called `main`.

```
c1 <-- c2 <-- c3
              ^
            [main]
```

When you make a new commit on a branch, the commit is added *after* the one the label points to, and **the label moves forward** to the new commit:

```
c1 <-- c2 <-- c3 <-- c4
                     ^
                  [main]        <- the label moved
```

Creating a new branch makes a **new label on the same commit** — nothing is copied:

```
c1 <-- c2 <-- c3
              ^
       [main] [add-verse]
```

Now commit on `add-verse`, and only *that* label moves:

```
c1 <-- c2 <-- c3 <-- c4
              ^      ^
           [main]  [add-verse]
```

Switch back to `main` and commit there (Hari's spelling fix): the history **splits**. Two lines of work grow side by side from `c3`:

```
                  c4   [add-verse]
                 /
c1 <-- c2 <-- c3
                 \
                  c5   [main]
```

Because a branch is only a label (Git stores it as a tiny file containing a commit id), branches are **cheap**: you can make one for every idea, experiment or task, and delete it later.

HEAD: "you are here"

How does Git know which label to move when you commit? It keeps one more special pointer, `HEAD`, which points at the branch you are on. In `git log` you see it as `HEAD -> main`. **Switching** branches moves HEAD and changes the files in your folder to match that branch's latest commit.

Switching changes your files

This surprises everyone the first time: if `add-verse` has a file `poem.md` that `main` does not have, then after `git switch main` the file **disappears from the folder**. It is not deleted — it lives in the `add-verse` commits. Switch back and it returns. Your working folder always shows *the snapshot of the branch you are on*.

That is also why Git **refuses to switch** if you have uncommitted changes that would be overwritten — it protects your work. Commit first (or use `git stash`, Grade 11).

Commands

Task	Command	Older form (still works)
list branches (`*` = current)	`git branch`	
create a branch here	`git branch <name>`	
create and switch to it	`git switch -c <name>`	`git checkout -b <name>`
switch to a branch	`git switch <name>`	`git checkout <name>`
see all branches as a graph	`git log --oneline --graph --all`	
delete a merged branch	`git branch -d <name>`	

Good branch names are short, lower-case, with hyphens, and say what the work is: `add-verse`, `fix-spelling`, `ward-7-data`.

See it

```
$ git branch
* main
$ git switch -c add-verse
Switched to a new branch 'add-verse'
$ echo "verse" > poem.md
$ git add poem.md
$ git commit -qm "Add first verse"
$ git switch main
Switched to branch 'main'
$ ls
README.md                                  <- poem.md is not here: it lives on add-verse
$ git switch -c fix-spelling
Switched to a new branch 'fix-spelling'
$ echo "fixed" >> README.md
$ git commit -qam "Fix spelling in README"
$ git log --oneline --graph --all
* 3e7c90e (add-verse) Add first verse
| * e73e388 (HEAD -> fix-spelling) Fix spelling in README
|/
* 5e7cc8a (main) Describe the project in README
* 8b2c8b6 Add README
```

Read the graph from the bottom: two commits on `main`, then the history splits into `add-verse` and `fix-spelling`. HEAD is on `fix-spelling`.

Unplugged: the sticky-note model (15 min)

The history is a row of cards (from Grade 8, or new ones). A branch is a **sticky note** stuck on one card.

1. Put a note `main` on the newest card. Put a small coin or a note `HEAD` on the `main` note.
2. **Create a branch:** write a new note `add-verse` and stick it on the *same* card.
3. **Switch:** move the HEAD coin to `add-verse`.
4. **Commit:** add a new card after the current one, connect it to its parent, and move *only the note HEAD is on*.
5. Switch back to `main` (move the coin) and commit there: the cards now fork into two rows.
6. Ask: *when we switched, which card's sheet should be on the desk as the working copy?*

PRIMM lab (40 min)

Visual first: open Learn Git Branching level `intro2` "Branching in Git" ‹https://learngitbranching.js.org/?locale=en_US&level=intro2› (self-hosted in the lab). Complete it, then continue in your own repo.

1. In `ward-facts`: **predict** then run `git branch`.
2. `git switch -c add-verse` (older Git: `git checkout -b add-verse`). Predict what `git log --oneline --graph --all` shows. Run it.
3. Make a commit on `add-verse` that adds a new file. Switch back: `git switch main`. **Investigate:** where did your new file go? (It is not in the working folder — it is on the other branch.) Switch again and check.
4. **Investigate:** edit `README.md` without committing, then try to switch. What happens? Read Git's message. Commit (or undo the edit) and switch.
5. **Modify:** create a second branch `fix-spelling` from `main` and commit there.
6. **Make:** draw the three labels, HEAD and the commits with sticky notes; compare with `git log --graph --all`. They should match exactly.

Parsons practice (10 min)

Order to make a branch with one commit and return: *switch -c · edit · add · commit · switch main*.

Common mistakes

Symptom	Cause	Fix
"My file vanished!"	you switched to a branch that does not have it	switch back; it is in the other branch's commits
`error: Your local changes … would be overwritten by checkout`	uncommitted changes conflict with the target branch	commit them, or undo them, then switch
Commit went to the wrong branch	forgot to switch first	check the prompt/`git status` before committing; in Grade 11 you learn to move commits
`git switch` not recognised	very old Git	use `git checkout <name>`; update Git if possible
`You are in 'detached HEAD' state`	switched to a commit id, not a branch	`git switch main`; detached HEAD is explained in Grade 12

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Learn Git Branching `intro2` ‹https://learngitbranching.js.org/?locale=en_US&level=intro2› and `rampup1` "Detach yo' HEAD" ‹https://learngitbranching.js.org/?locale=en_US&level=rampup1›.	1 / 3	20 min	drawing of each goal graph
2	Visualizing Git ‹https://git-school.github.io/visualizing-git/›: type `git commit`, `git branch idea`, `git checkout idea`, `git commit`. Watch the labels move.	3	10 min	screenshot or drawing
3	*Pro Git* §3.1 **Branches in a Nutshell** ‹https://git-scm.com/book/en/v2/Git-Branching-Branches-in-a-Nutshell›: look at the figures with `HEAD` and `testing`. Redraw one with our sticky notes.	3	20 min	redrawn figure
4	**Oh My Git!** (download ‹https://ohmygit.org/›, Tier 1 after download): play the levels about branches.	1	30 min	levels completed
5	**Paper quiz:** the teacher draws 4 graphs; for each, write which branch HEAD is on and which files would be in the folder.	0	10 min	answers

Exit ticket (5 min)

1. What is a branch?
2. Why did the new file disappear when you switched back to main?
3. *Misconception:* "A branch is a full copy of the folder." True or false?

Key points

- A **branch** is a movable **label** on a commit; committing moves the label you are on.
- **HEAD** marks the branch you are on.
- **Switching** changes the files in your folder to that branch's snapshot.
- Branches are cheap: one per idea or task.
- Git refuses to switch rather than lose uncommitted work.

New words

branch, `main`, **HEAD**, **switch**, **graph**, **detached HEAD** — see the glossary.

Extension quest

Run `cat .git/HEAD` and `cat .git/refs/heads/main`. What do the files contain? (You have just seen that a branch really is a tiny file with a commit id. Grade 12 goes deeper.)

Teacher notes

- **Timing:** P1: hook 5, understand with board diagrams 15, sticky notes 15, LGB `intro2` 5. P2: lab 30, Parsons 5, exit 5.
- **The key idea is the pointer.** If students ask "is it a copy?" — no, it is a label, which is why branches are cheap. Always commit before switching; if Git refuses to switch, that protects uncommitted work.
- **Common misconception:** "the branch is the commits after the split." Branch = label on one commit; the commits belong to the graph.
- **Fallback Tier 0:** sticky notes for the whole lab; exercise 5 as the practical.
- **Answer key:** Exit 1: a movable label pointing at a commit. 2: the file exists only in the commits of the other branch; the folder shows the current branch's snapshot. 3: False. Lab 4: Git refuses (or carries the change if there is no conflict — both are possible; discuss why). Extension: `ref: refs/heads/main` and a 40-character commit id.

GRADE 9 · MODULE 4 · 2 PERIODS · TIERS 0, 1, 2, 3

4. Merge and the merge-conflict game

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0–3 · **You need:** a printed 5-line poem per pair, two pen colours; your `ward-facts` repo with branches from Module 3
>
> **Before this:** Module 3 (branches).
>
> **By the end you can:** merge a branch, say whether Git did a fast-forward or made a merge commit, and calmly resolve a conflict.

Hook (5 min)

Two edits to the same line of the poem. Sita wrote "*Rivers flow down to the Terai*"; Hari wrote "*Rivers rush to the plains*". Who wins? Should a computer decide?

Understand it

Merging joins two lines of work

In Module 3 the history split into branches. **Merging** brings the work of one branch *into* another. You always merge **into the branch you are on**:

```
git switch main          # go to the branch that should receive the work
git merge add-verse      # bring add-verse's commits into main
```

Git can merge in two ways, depending on the shape of the history.

Case 1: fast-forward

If `main` has **not moved** since the branch was made, there is nothing to combine. Git simply slides the `main` label forward to the branch's newest commit:

```
before:  c1 -- c2 [main]                after:  c1 -- c2 -- c3 [main] [add-verse]
                  \
                   c3 [add-verse]
```

No new commit is made. Git prints `Fast-forward`.

Case 2: merge commit

If **both** branches have new commits, Git makes a new **merge commit** with **two parents** — it joins the two lines:

```
before:         c3 [fix-spelling]          after:         c3 -------.
               /                                           /          \
 c1 -- c2 --+                               c1 -- c2 --+              m [main]
               \                                           \          /
                c4 [main]                                   c4 -------'
```

Git compares both branches with their **common ancestor** (`c2`, the last commit they share) and combines the changes automatically — as long as they changed *different* lines.

Conflicts: when a human must decide

If both branches changed **the same lines** of the same file, Git cannot know which version is right. It stops and marks the file with a **conflict**:

```
Mountains stand tall
<<<<<<< HEAD
Rivers flow down to the Terai          <- the version on the branch you are on (main)
=======
Rivers rush to the plains              <- the version from the branch you are merging
>>>>>>> fix-spelling
The morning is cold
```

Marker	Means
`<<<<<<< HEAD`	start of *your* side (the current branch)
`=======`	divider
`>>>>>>> fix-spelling`	end of *their* side (the branch being merged)

To resolve: edit the file so it says what it *should* say — keep one side, the other, or write a combination — and **delete all three marker lines**. Then `git add` the file (meaning "resolved") and `git commit` to finish the merge.

> **A conflict is not an error**
>
> A conflict is Git politely asking a human for a decision. Nothing is lost: both versions are in the file and in history. If you feel lost, `git merge --abort` puts everything back to before the merge.

Avoiding conflicts

Conflicts are normal in teams, but you can make them rare: **merge often** (small branches live short lives), **split work by file or section**, and **talk** before two people edit the same paragraph.

See it

A fast-forward, then a real conflict:

```
$ git switch main
$ git merge add-verse
Updating c26da4a..f60bbe5
Fast-forward
 poem.txt | 1 +
 1 file changed, 1 insertion(+)

  (now main and fix-spelling both change line 2)

$ git merge fix-spelling
Auto-merging poem.txt
CONFLICT (content): Merge conflict in poem.txt
Automatic merge failed; fix conflicts and then commit the result.
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
        both modified:   poem.txt

  (edit poem.txt: keep "Rivers rush down to the Terai", delete the markers)

$ git add poem.txt
$ git commit --no-edit
$ git log --oneline --graph --all
*   97832b0 (HEAD -> main) Merge branch 'fix-spelling'
|\
| * 3d873e6 (fix-spelling) Change line 2 to rush
* | be9221d Name the Terai in line 2
|/
* f60bbe5 (add-verse) Add sixth line about terraces
* c26da4a Add poem
```

The merge commit `97832b0` has two parents — you can see both lines joining in the graph.

Unplugged: the paper merge-conflict game (20 min)

Pairs share a printed 5-line poem — this is the **common ancestor**.

1. Each partner gets a copy and edits it with their own pen colour: partner A changes lines 1 and 3; partner B changes lines 3 and 5.
2. **Merge:** on a fresh sheet, combine both copies. Lines only one person changed: take that change (Git does this automatically). Line 3, changed by both: write the conflict with markers — `<<<<<<< A`, A's line, `=======`, B's line, `>>>>>>> B`.
3. **Resolve:** the pair discusses and writes the final line 3, crossing out the markers.
4. Write the rule on the board: *Git can combine changes to different lines, but when both change the same line, a human decides.*

PRIMM lab (40 min)

1. **Predict:** you merge `add-verse` (new lines) into `main`. Will there be a conflict? Will it be a fast-forward? Run `git switch main`, `git merge add-verse`.
2. **Investigate:** `git log --graph --oneline --all`. Was it a fast-forward or a merge commit? How can you tell?
3. **Create a conflict:** make sure you have a `poem.txt` committed on `main`. On `fix-spelling` change line 2 and commit; on `main` change line 2 differently and commit. **Predict**, then `git merge fix-spelling`.

4. Run `git status`. Open the file, find the markers, decide the final line, delete the markers, `git add poem.txt`, `git commit`.
5. **Investigate:** draw the graph. How many parents does the merge commit have? Check with `git log --oneline --graph`.
6. **Modify:** repeat with your partner changing *different* lines — no conflict. Then try `git merge --abort` in the middle of a conflict and check `git status`.
7. **Make:** write a team rule for your pair project: who edits which file, and how often you merge.

Parsons practice (10 min)

Order for resolving a conflict: *merge · open the file · edit and remove markers · git add · git commit*.

Common mistakes

Symptom	Cause	Fix
Markers `<<<<<<<` left in the file and committed	forgot to delete them	edit the file, remove markers, commit again; search with `git grep "<<<<<<<"`
Merged in the wrong direction	ran merge on the wrong branch	check `git status` first; `git merge --abort` if not finished
Vim opens for the merge message	merge commit needs a message	`:wq` to accept, or `git commit --no-edit`
`fatal: You have not concluded your merge`	a merge is still in progress	finish (`add` + `commit`) or `git merge --abort`
"Git deleted my line!"	you chose the other side	it is still in the branch's commit; `git show fix-spelling:poem.txt`

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Learn Git Branching `intro3` "Merging in Git" ‹https://learngitbranching.js.org/?locale=en_US&level=intro3›.	1 / 3	10 min	goal graph drawing
2	Software Carpentry **Conflicts** ‹https://swcarpentry.github.io/git-novice/09-conflict.html›: do the exercise "Solving Conflicts that You Create".	1 / 3	30 min	your resolved file + log
3	*Pro Git* §3.2 **Basic Branching and Merging** ‹https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging›: find the figures for fast-forward and for the three-way merge. Label the common ancestor.	3	20 min	labelled sketch
4	GitHub Skills **Resolve merge conflicts** ‹https://github.com/skills/resolve-merge-conflicts› (13+, account, internet).	3	45 min	completed course
5	**Conflict cards (paper):** the teacher hands out 6 pairs of edits; decide for each: no conflict / fast-forward / conflict.	0	10 min	answers

Exit ticket (5 min)

1. When does a conflict happen?
2. What do the markers mean?
3. *Misconception:* "After a conflict my work is lost." True or false?

Key points

- `git merge <branch>` brings that branch's work **into the current branch**.
- **Fast-forward:** the label just moves; **merge commit:** a new commit with two parents.
- Git merges changes to **different lines** automatically; the **same lines** → conflict → a human decides.
- Resolve: edit, remove markers, `git add`, `git commit`. Escape: `git merge --abort`.
- Merge often, split work, talk — fewer conflicts.

New words

merge, **fast-forward**, **merge commit**, **common ancestor**, **conflict**, **conflict markers** — see the glossary.

Extension quest

After a merge commit, run `git log --first-parent --oneline`. What does it hide? Why might a team like this view?

Teacher notes

- **Timing:** P1: hook 5, understand 15, paper game 20. P2: see it (live on projector) 5, lab 25, Parsons 5, exit 5.
- **Students panic at conflicts.** Normalise: conflicts are Git asking for a human decision. `git merge --abort` returns to before the merge. Make a conflict *on purpose* in front of the class and resolve it slowly, thinking aloud.
- In VS Code, a conflicted file shows buttons *Accept Current / Accept Incoming / Accept Both* — fine to use, but students must still understand the markers.
- **Conflict cards (exercise 5) answers:** same line both sides → conflict; different lines → merge commit, no conflict; only one side changed → fast-forward; one side adds a line at the end, other edits line 1 → no conflict; both add *different* lines at the end → conflict (same place); one renames the file, other edits it → usually no conflict (Git detects renames).
- **Fallback Tier 0:** the paper game plus exercise 5.
- **Answer key:** Exit 1: when both branches changed the same lines (or one deleted a file the other changed). 2: start of current side / divider / end of incoming side. 3: False — both versions are kept until you decide.

GRADE 9 · MODULE 5 · 2 PERIODS · TIERS 1, 2, 3

5. Remotes: clone, push, pull

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 1 (USB bundles), 2 (school Forgejo), 3 (class organisation; learners 13+) · **You need:** a remote URL from the teacher, or a USB stick
>
> **Before this:** Module 4 (merge).
>
> **By the end you can:** clone a repository, push your commits, pull your partner's, and fix "rejected — fetch first".

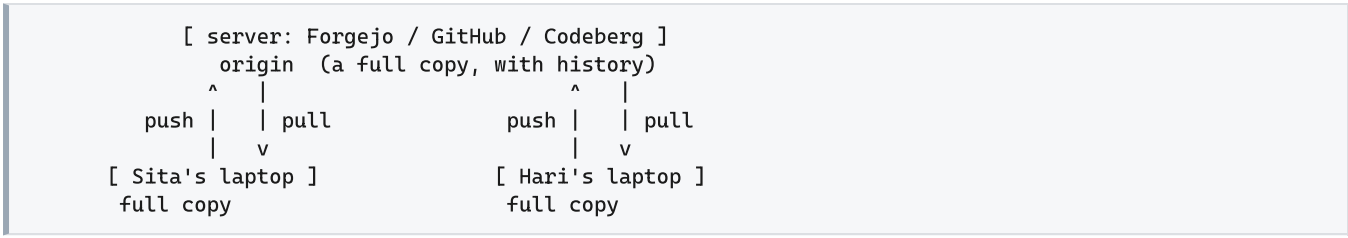
Hook (5 min)

How do two computers share work without e-mailing files back and forth, and without losing anyone's changes? Today your repository learns to travel.

Understand it

Every copy is complete

Git is **distributed**: when you copy a repository, you get **all** of its history, not just the latest files. Your laptop's copy, your partner's copy and the server's copy are equals. A **remote** is simply *another copy of the same repository* that your copy knows about, with a name — usually `origin` for the place you cloned from.



Four verbs

Command	What travels	When
`git clone <url>`	the **whole** repository, once	at the start
`git push origin main`	your **new commits** → the remote	after you commit
`git pull`	the remote's new commits → you (fetch + merge)	before you start work, and when push is rejected
`git fetch`	download new commits but **don't merge** yet	when you want to look first

Nothing moves until you say so. `git commit` never touches the network — it only writes to *your* copy. That is why Git works perfectly offline, and why a power or internet cut can never damage a commit you already made.

Remote-tracking branches

After a fetch or pull, your repository remembers where the remote's branches were, as `origin/main`. `git status` compares: `Your branch is ahead of 'origin/main' by 1 commit` means you have something to push; `behind` means you should pull.

"Rejected — fetch first"

Sita pushes. Then Hari, who has *not* got Sita's commit, tries to push. The server refuses, because accepting Hari's history would throw Sita's commit away:

```
 ! [rejected]        main -> main (fetch first)
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. ... use 'git pull' before pushing again.
```

The fix is always the same: **pull** (which merges Sita's work into Hari's), resolve any conflict, then **push** again. Never "force" a push to make the error go away — that is how teams lose work.

Hosts are interchangeable

Git is the tool; **GitHub, GitLab, Codeberg and Forgejo are places that host Git repositories** and add web pages, issues and pull requests. Everything in this lesson works the same on all of them, on a school LAN server, or with a USB stick:

Tier	Remote	URL looks like
1 — offline	a **bundle** file passed on USB	`share.bundle`
2 — school LAN	Forgejo on a Raspberry Pi or old PC	`http://192.168.1.10:3000/class9/ward-facts.git`
3 — internet	a teacher-managed class organisation	`https://github.com/our-class/ward-facts.git`

See it

Two people, one server (real output; paths shortened):

```
sita$ git clone http://school-server/class9/ward-facts.git
Cloning into 'ward-facts'...
sita$ cd ward-facts
sita$ git remote -v
origin  http://school-server/class9/ward-facts.git (fetch)
origin  http://school-server/class9/ward-facts.git (push)
sita$ git commit -qm "Add notes file"     (after adding notes.md)
sita$ git push origin main
To http://school-server/class9/ward-facts.git
   97832b0..129d106  main -> main

hari$ git commit -qm "Add data file"      (Hari has not pulled Sita's commit)
hari$ git push origin main
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'http://school-server/class9/ward-facts.git'
hari$ git pull
Merge made by the 'ort' strategy.
 notes.md | 1 +
hari$ git push origin main
   129d106..dc34351  main -> main
```

Offline with a USB stick (Tier 1):

```
sita$ git bundle create /e/share.bundle --all     <- E: is the USB stick
hari$ git pull /e/share.bundle main               <- on the other computer
```

Concept drawing (10 min, unplugged)

Draw two laptops and a server on the board, each with a full row of commit cards. Act it out with three students holding card rows: **clone** (copy the whole row), **push** (add your new card to the server's row — only if the server has nothing you don't have!), **pull** (copy the server's new cards to you). Then act out the rejected push and its fix.

PRIMM lab (50 min)

Tier 2 (Forgejo LAN) or Tier 3 (class organisation, students 13+):

1. The teacher gives each pair a remote URL and pseudonymous logins. Partner A: `git clone <url> ward-facts`.
2. **Predict**, then run `git remote -v`.
3. Commit locally, `git push origin main`. Check the server's web page: is your commit there?
4. Partner B clones (or pulls). Partner A commits and pushes again. Partner B: **predict** what `git pull` changes, then run it. **Investigate** `git log --oneline --graph`.
5. **Modify:** both edit *different* files and commit. Both push — the second push is **rejected**. Read the message aloud. Pull, then push.
6. **Modify:** now both edit the *same line*. Push, get rejected, pull → conflict → resolve (Module 4) → push.

Tier 1 (no network):

- Partner A: `git bundle create share.bundle --all`, copy to USB.
- Partner B: `git clone share.bundle ward-facts` the first time, then later `git pull share.bundle main`.
- Swap directions with a new bundle. (Bundles are exactly how this book's practice repos travel to offline schools.)
- **Make:** write a 3-step teamwork routine on a poster: **pull · work (add, commit) · push**. Add "pull again if push is rejected".

Parsons practice (10 min)

Order: *pull · edit · add · commit · push*. Second set: *push · rejected · pull · resolve conflict · add · commit · push*.

Common mistakes

Symptom	Cause	Fix
`! [rejected] … (fetch first)`	the remote has commits you don't have	`git pull`, then push
`fatal: Need to specify how to reconcile divergent branches`	new Git asks how `pull` should combine	once: `git config --global pull.rebase false` (merge), then pull again
`Authentication failed`	wrong login / token	check the class login; on GitHub passwords don't work for push — the teacher sets up a token or uses the LAN server
`fatal: repository not found`	URL typo or no access	copy the URL from the web page again
Pushed to the wrong branch	typed a different name	ask the teacher; don't force-push

Symptom	Cause	Fix
Someone suggests `git push --force`	it overwrites others' work	**never** in a shared branch

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Learn Git Branching, **Remote** tab: `remote1` Clone Intro ‹https://learngitbranching.js.org/?locale=en_US&level=remote1› to `remote6` Git Pushin'. Watch `o/main` (origin/main) move.	1 / 3	40 min	levels done + one drawing
2	Software Carpentry **Remotes in GitHub** ‹https://swcarpentry.github.io/git-novice/07-github.html› and **Collaborating** ‹https://swcarpentry.github.io/git-novice/08-collab.html› — read; do the steps on the class server instead of GitHub.	2 / 3	45 min	your log
3	*Pro Git* §2.5 **Working with Remotes** ‹https://git-scm.com/book/en/v2/Git-Basics-Working-with-Remotes›: try `git remote show origin`. What does it tell you?	2 / 3	15 min	output explained
4	GitHub Skills **Introduction to GitHub** ‹https://github.com/skills/introduction-to-github› (13+, with teacher's permission).	3	60 min	completed course
5	Software Carpentry **Hosting** ‹https://swcarpentry.github.io/git-novice/13-hosting.html›: list two pros and cons of hosting at school vs on a public service.	0 (printed) / 3	15 min	table

Exit ticket (5 min)

1. Does `git commit` send anything over the network?
2. What does `git clone` copy?
3. *Misconception:* "GitHub is Git." True or false?

Key points

- A **remote** is another full copy of the repository; `origin` is the usual name.
- `clone` once, then **pull · work · push**.
- `commit` is local; only `push` / `pull` / `fetch` use the network.
- A rejected push means "pull first"; never force-push shared work.
- Git ≠ GitHub: hosts are interchangeable; bundles work with no network at all.

New words

remote, **origin**, **clone**, **push**, **pull**, **fetch**, `origin/main`, **bundle**, **host** — see the glossary.

Extension quest

Run `git fetch` then `git log --oneline main..origin/main`. What does this list? (Commits on the remote that you have not merged yet.)

Teacher notes

- **Timing:** P1: hook 5, understand 15, acting out 10, lab steps 1–3 (10). P2: lab steps 4–7 (35), Parsons 5, exit 5.

- **Privacy:** students under 13 must not create accounts on GitHub; use the LAN server or teacher-managed accounts. Never use real emails.
- **Before class:** create one repo per pair on the server and test a clone + push yourself. Set `git config --global pull.rebase false` on every lab PC to avoid the "divergent branches" message.
- **Auth:** use the server's HTTP with a pseudonymous account; delay SSH keys until Grade 10. GitHub requires a personal access token (not a password) for HTTPS pushes — prepare this or use the LAN server.
- **Typical error:** "rejected, non-fast-forward / fetch first" → pull first.
- **Fallback:** Tier 1 bundles for the whole lab; Tier 0: the acting-out activity with card rows.
- **Answer key:** Exit 1: No. 2: the whole repository with all history (and sets up `origin`). 3: False — GitHub is a website that hosts Git repositories. Extension: commits on the remote not yet in your branch.

GRADE 9 · MODULE 6 · 2 PERIODS · TIERS 1, 2, 3

6. Python scripts under version control

Placement check

CDC's Class 10 computer textbook (2025 ed.) teaches **Python (Unit 4)**. A Grade 9 syllabus has not been located, so students may not have met Python yet. Teach this module in Grade 10 (after Python is taught), or run it read-only with the provided files: the Git skill is the goal.

At a glance

Time: 2 periods · **Tiers:** 1–3 · **You need:** Python 3, a terminal, a copy of `practice-repos/g09-ward-facts`

Before this: Modules 2–5.

By the end you can: decide which files belong in a repository, write a `.gitignore`, write a useful README, and use history to find the commit that broke a program.

Hook (5 min)

Your program worked yesterday and breaks today. You changed "only a little". Can history tell you exactly what changed — and when it broke?

Understand it

What belongs in a repository?

A repository should contain everything a person needs to **rebuild and understand** the project, and nothing that the computer can **regenerate** or that must stay **secret**.

Kind of file	Example	In Git?	Why
source code	`facts.py`	**yes**	it is the work
documentation	`README.md`, `DATA.md`	**yes**	people need it
small input data you are allowed to share	`data/wards.csv`	**yes**	the script needs it
generated / cache files	`__pycache__/`, `*.pyc`	no	Python recreates them
output you can recreate	`result.txt`, `chart.png`	usually no	run the script again
local databases	`ward.db`	no	rebuild from `.sql` (Module 7)
editor settings	`.vscode/`, `.idea/`	usually no	personal
secrets	passwords, API keys, `.env`	**never**	anyone with the repo can read them — for ever

`.gitignore`

A file named `.gitignore` in the top folder lists patterns of files Git should not track. Ignored files do not appear in `git status` and `git add .` skips them.

```
# Python cache files
__pycache__/
*.pyc

# local database built from schema.sql
*.db

# secrets — never commit
.env
```

Pattern rules: `*` matches any characters; a name ending with `/` means a folder; `#` starts a comment. `.gitignore` itself **is** committed, so the whole team ignores the same files. Ready-made templates for every language exist in github/gitignore ‹https://github.com/github/gitignore› (CC0 — copy freely).

> **`.gitignore` does not un-track**
> If a file was already committed, adding it to `.gitignore` does not remove it from history. Remove it from tracking with `git rm --cached <file>` and commit — and if it was a secret, treat it as leaked (Grade 12, Module 5).

A README that helps

Every repository needs a `README.md` that answers: **What is this? How do I run it? Where does the data come from? Who made it, and under what licence?** A stranger (or you next year) should be able to run the project using only the README.

History as a debugging tool

When a program breaks, ask *what changed since it last worked?*

- `git diff` — what have I changed since my last commit?
- `git log -p` — every commit **with its diff**; scroll to find a suspicious change (`q` quits).
- `git log -p facts.py` — the same, only for one file.
- `git show <id>` — one commit's change.
- `git restore --source=<good-id> facts.py` — bring back the last working version (Grade 8, Module 5).

This is why **small commits** matter for code: if each commit changes one thing, the commit that broke the program is short and easy to read. (In Grade 12 you will learn `git bisect`, which finds the bad commit automatically.)

See it

```
$ python facts.py
Largest ward: Test Ward Four (2310 people)
$ git status --short --ignored
!! __pycache__/                    <- !! = ignored, thanks to .gitignore
$ git log -p -1 facts.py
commit 4f1e2d7 ...
    Rename variable top to biggest
diff --git a/facts.py b/facts.py
@@ -17,5 +17,5 @@
 if __name__ == "__main__":
     wards = load()
-    top = largest_ward(wards)
+    biggest = largest_ward(wards)
     print(f"Largest ward: {top['ward_name']} ({top['population']} people)")
```

The diff shows the bug at once: the variable was renamed on one line but not on the next, so `top` no longer exists. (Your ids will differ.)

Concept sort (10 min, unplugged)

Print 12 file names on slips (`facts.py`, `__pycache__/facts.cpython-312.pyc`, `README.md`, `.env`, `ward.db`, `data/wards.csv`, `chart.png`, `.vscode/settings.json`, `schema.sql`, `passwords.txt`, `DATA.md`, `notes-for-me.txt`). Pairs sort them into **commit**, **ignore**, **never**. Discuss the hard ones (`chart.png`, `notes-for-me.txt`).

PRIMM lab (50 min)

Repo: `practice-repos/g09-ward-facts` — copy the folder, then `git init` it and commit everything (`git add . && git commit -m "Start from ward-facts starter"`), or clone the bundle your teacher gives you.

1. **Predict:** read `facts.py`. What will it print? Run `python facts.py`.
2. **Investigate:** open `.gitignore`. Run `git status --short --ignored`. Which file is ignored, and why did it appear only *after* running Python?
3. **Investigate:** temporarily rename `.gitignore` to `gitignore.txt`. What does `git status` show now? Rename it back.
4. **Modify:** change the question the script answers (for example, the ward with the *smallest* population: use `min`). Run, then `git diff`. Commit with a message that says *why*.
5. **Break it on purpose:** rename a variable on one line only (as in *See it*), commit with an innocent message. Run the script — error! Use `git log -p` to find which commit broke it. Fix it with a **new** commit whose message names the bad commit.
6. **Make:** on a branch `add-average`, add a function `average_population(wards)` and print it. Merge the branch into `main` (Module 4). Update the README with one line: *how to run*.

Parsons practice (10 min)

Order: *edit facts.py · run python · git diff · git add · git commit*. Second set: *program fails · git log -p · find bad commit · fix · commit "Fix … introduced in …"*.

Common mistakes

Symptom	Cause	Fix
`__pycache__` shows as untracked	no `.gitignore`	add the patterns above
File still tracked after adding to `.gitignore`	it was committed before	`git rm --cached <file>`, commit
`.gitignore` "doesn't work"	saved as `.gitignore.txt` by Windows Notepad	rename; turn on "show file extensions"
`python` not found	Python not installed / not on PATH	try `python3` or `py`; ask the teacher
Committed a password	—	tell the teacher; change the password; see Grade 12 Module 5

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Software Carpentry **Ignoring Things** ‹https://swcarpentry.github.io/git-novice/06-ignore.html›: do the exercises "Ignoring Nested Files" and "Including Specific Files".	1 / 3	25 min	answers
2	Open the Python template in github/gitignore ‹https://github.com/github/gitignore›. Find three patterns we did not use and explain what they ignore.	3	15 min	3 patterns
3	*Pro Git* §2.2 **Recording Changes** ‹https://git-scm.com/book/en/v2/Git-Basics-Recording-Changes-to-the-Repository›, section "Ignoring Files": write the five glob-pattern rules in your own words.	3	15 min	rules
4	GitHub Skills **Communicate using Markdown** ‹https://github.com/skills/communicate-using-markdown› (13+, account) — then improve your README.	3	40 min	improved README
5	**Debug race (Tier 1):** the teacher gives a repo with 10 commits where one broke the script. Find it with `git log -p` faster than the other pairs.	1	15 min	bad commit id

Exit ticket (5 min)

1. Why do we ignore `__pycache__/`?
2. How can `git log -p` help debugging?
3. *Misconception:* "If the code works, the history does not matter."

Key points

- Commit **source, docs and small shareable data**; ignore **generated files**; **never** commit secrets.
- `.gitignore` is itself committed; it does not remove already-tracked files.
- A README says what, how to run, where data comes from, who and licence.
- `git log -p` and small commits turn history into a debugging tool.

New words

`.gitignore`, **pattern / glob**, **generated file**, **secret**, **README**, `git log -p` — see the glossary.

Extension quest

Write a `requirements.txt` (even if empty) and a "How to run" section that a Grade 8 student could follow. Ask one to try it.

Teacher notes

- **Timing:** P1: hook 5, understand 15, concept sort 10, lab 1–3 (10). P2: lab 4–6 (35), Parsons 5, exit 5.
- Hours depend on CDC placement of Python (unverified). If students have not met Python, use only the provided scripts (read/run/modify) — the Git skill is the goal.
- **Debug race setup (exercise 5):** make 10 small commits to `facts.py`; in commit 6 introduce a typo (`ward_nmae`). Bundle it (`git bundle create debug.bundle --all`).
- **Concept sort answers:** commit — `facts.py`, `README.md`, `data/wards.csv`, `schema.sql`, `DATA.md`; ignore — `__pycache__/…`, `ward.db`, `chart.png` (if regenerated), `.vscode/…`, `notes-for-me.txt` (personal); never — `.env`, `passwords.txt`.
- **Answer key:** Exit 1: Python regenerates them; they are not source. 2: it shows each commit's changes, so you can spot when a bug appeared. 3: False — history lets you understand, debug and undo. Lab 2: `__pycache__/` appears after Python runs because Python creates it then.

GRADE 9 · MODULE 7 · 2 PERIODS · TIERS 1, 2, 3

7. SQL under version control

Placement check

CDC's Class 10 computer textbook (2025 ed.) teaches **MySQL and databases (Unit 2)**. A Grade 9 syllabus has not been located, so students may not have met databases yet. Teach this module in Grade 10 (after Unit 2), or run it read-only with the provided files: the Git skill is the goal.

At a glance

Time: 2 periods · **Tiers:** 1–3 · **You need:** `practice-repos/g09-ward-facts`; MySQL **or** SQLite (or Python's built-in `sqlite3` module)

Before this: Module 6.

By the end you can: explain the difference between a database and the scripts that build it, keep both schema and queries in Git, and rebuild the database from them at any time.

Hook (5 min)

A database is a living thing: in a real ward office, records change every minute. What exactly would you put in Git — the whole database file, every minute? What happens when two people change it?

Understand it

Recipe vs. dish

Think of a database as a **dish** and the SQL scripts as the **recipe**. You cannot keep a history of a dish — it changes as people eat it — but you can keep a history of the recipe, and cook the dish again whenever you want.

	Database file (`ward.db`, MySQL data folder)	**SQL scripts (`schema.sql`, `queries.sql`)**
What it is	the live data, in a binary format	plain text instructions
Readable diff?	no (binary)	**yes** — line by line
Merge two people's changes?	no	**yes**, like any text file
Size	can be huge	small
In Git?	**no** (add `*.db` to `.gitignore`)	**yes**

So we version:

- `schema.sql` — the structure: `CREATE TABLE …` (and, for small teaching data, `INSERT …`).
- `queries.sql` — the questions we ask: `SELECT …`.
- later, **migrations** — numbered scripts that change the structure step by step (`001_create_wards.sql`, `002_add_area.sql`). This is how professional teams evolve databases; you will meet it again in Grade 12.

Dumps

A **dump** is a database exported *as a recipe*: `mysqldump wards_db > dump.sql` (MySQL) or `sqlite3 ward.db .dump > dump.sql` writes SQL statements that recreate the tables and data. Dumps of small, shareable data can be committed; dumps containing **personal data must never be** — a ward database with names and phone numbers is private.

Rebuild any time

Because the recipe is in Git, anyone can rebuild the exact database from any commit:

```
git switch main
sqlite3 ward.db < schema.sql       # cook the dish
sqlite3 ward.db < queries.sql      # ask the questions
```

Checkout an old commit and you can even rebuild **last month's** database — very useful for "the report used to say 6,210, why does it say 6,500 now?".

Changing a table safely

When the structure changes, change **schema and queries together** in one commit, so every commit in history is consistent (the queries in commit X work with the schema in commit X).

See it

```
$ cat queries.sql
-- Q1: total population
SELECT SUM(population) AS total_population FROM wards;
-- Q2: the largest ward
SELECT ward_name, population FROM wards ORDER BY population DESC LIMIT 1;
$ sqlite3 ward.db < schema.sql
$ sqlite3 ward.db < queries.sql
6210
Test Ward Four|2310
$ git status --short
                                  <- nothing! ward.db is ignored by .gitignore
$ git diff                        (after adding a column in schema.sql)
-  households INTEGER NOT NULL
+  households INTEGER NOT NULL,
+  area_km2 REAL
```

No SQLite program? Python has one built in:

```
python -c "import sqlite3; c=sqlite3.connect('ward.db'); c.executescript(open('schema.sql').read())"
```

Unplugged: recipe or dish? (10 min)

Pairs get cards: *a photo of the school register*, *the instructions for drawing the register's columns*, *the list of questions the head teacher asks every month*, *the filled-in register for October*, *a printed copy of all students' phone numbers*. Sort into **recipe (version it)**, **dish (rebuild it)**, **private (never share)**.

PRIMM lab (50 min)

Repo: `practice-repos/g09-ward-facts` (your copy from Module 6).

1. Open `schema.sql` and `queries.sql`. **Predict** the result of each query on the data (add up the populations by hand!).

2. Run them (MySQL if available; or SQLite for offline: `sqlite3 ward.db < schema.sql`, then `sqlite3 ward.db < queries.sql`). The files use only portable SQL. Were you right?
3. **Investigate:** run `git status`. Why does `ward.db` not appear?
4. **Modify:** on a branch `query-small-wards`, add a query answering "which wards have fewer than 1,000 people?". Rebuild, run, commit, merge into `main`.
5. **Modify:** change the schema — add a column `area_km2 REAL` and give each `INSERT` a value (invent them; it is synthetic data). Add a query for population density (`population / area_km2`). Commit schema **and** queries together in one commit with a message that says why.
6. **Investigate:** `git log -p schema.sql` — read the history of the table's structure.
7. **Make:** write 3 questions about the dataset in a `QUESTIONS.md`, with the SQL answer for each.

Parsons practice (10 min)

Order to change a table safely: *edit schema.sql · rebuild the database · run queries · git add · git commit*.

Common mistakes

Symptom	Cause	Fix
`ward.db` committed	missing `*.db` in `.gitignore`	`git rm --cached ward.db`, add the pattern, commit
`table wards already exists`	ran the schema twice without `DROP TABLE`	our schema starts with `DROP TABLE IF EXISTS`; keep it
A query works in MySQL but not SQLite	non-portable SQL	stick to standard `SELECT`, `WHERE`, `ORDER BY`, `GROUP BY`
Old queries break after a schema change	schema and queries committed separately	commit them together
Real names or phone numbers in `INSERT` lines	private data	remove; use synthetic data; tell the teacher

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Software Carpentry **Exploring History** ‹https://swcarpentry.github.io/git-novice/05-history.html› — use `git diff HEAD~1 schema.sql` and `git restore --source=HEAD~1 schema.sql` on your schema.	1 / 3	15 min	commands used
2	**Rebuild last week:** use `git log` to find the first commit, then `git switch --detach <id>`, rebuild the database and run the queries. Compare the answers with today's. Return with `git switch main`.	1	20 min	old vs new results
3	Make a dump: `sqlite3 ward.db .dump > dump.sql`. Compare `dump.sql` with `schema.sql` using `git diff --no-index schema.sql dump.sql`.	1	15 min	2 differences
4	Open data: look at the Open Data Nepal ‹https://opendatanepal.com/› catalogue and find one small dataset whose licence allows sharing. Write a `DATA.md` for it (source, licence, date) — do not download yet.	3	20 min	`DATA.md` draft

Exit ticket (5 min)

1. Why do we put `.sql` files in Git and not the database file?

2. What does a dump contain?
3. *Misconception:* "Git can merge two live databases." True or false?

Key points

- Version the **recipe** (schema, queries, migrations), not the **dish** (the database file).
- SQL scripts are text: readable diffs, mergeable, small.
- Rebuild the database from any commit; change schema and queries **together**.
- Dumps are recipes too — but never commit personal data.

New words

schema, **query**, **dump**, **migration**, **binary file** — see the glossary.

Extension quest

Split `schema.sql` into numbered migrations (`001_create_wards.sql`, `002_add_area.sql`) and write a one-line script that runs them in order.

Teacher notes

- **Timing:** P1: hook 5, understand 15, sort 10, lab 1–3 (10). P2: lab 4–7 (35), Parsons 5, exit 5.
- Data is synthetic (invented wards). Do not commit the `.db` file (`*.db` is already in the starter's `.gitignore`). If MySQL is not installed, SQLite (or Python's `sqlite3`) is acceptable for the Git skill; say so to students.
- `git switch --detach` (exercise 2) gives a "detached HEAD" message — reassure students and make sure they return with `git switch main`.
- **Sort answers:** recipe — column instructions, monthly questions; dish — filled-in October register (rebuilt each month); private — phone numbers (and the photo of the register if it shows names).
- **Answer key:** Lab 1–2: 6210; Test Ward Four, 2310. Lab 4: Test Ward One (820), Test Ward Three (990), Test Ward Five (640). Exit 1: text scripts diff and merge; the database file is binary, large and changes constantly. 2: SQL statements that recreate the tables and data. 3: False.

GRADE 9 · MODULE 8 · 3 PERIODS · TIERS 1, 2, 3

8. Pair project: Ward Facts

> **At a glance**
>
> **Time:** 3 periods (+ homework) · **Tiers:** 1 (bundles), 2, 3 · **You need:** the shared remote for your pair (or USB bundles), the `g09-ward-facts` starter
>
> **Before this:** Modules 1–7.
>
> **By the end you can:** plan and deliver a small project with a partner, using branches, merges, a remote, a changelog and a tag.

Hook (5 min)

The class will present to a visiting official (or the head teacher): "*What do the ward numbers tell us?*" Each pair answers three questions — and must be able to show **how** they got the answers, step by step, from their history.

Understand it

Working as a pair

Two people, one repository, two computers. The project only goes well if you agree how to work **before** you start. A **working agreement** is a short written list of rules, kept in the repo as `AGREEMENT.md`:

```
# Our working agreement
- Branch names: `name/topic`, e.g. `sita/q2-density`.
- Sita owns `facts.py`; Hari owns `queries.sql`; we both edit README (talk first!).
- We pull at the start of every session and push at the end.
- Before merging into main, the partner reads the diff (`git log -p main..sita/q2-density`).
- Messages: verb first, say what and why.
```

The pair routine

```
start of session:  git switch main  ->  git pull
work:              git switch -c sita/q1   ->  edit  ->  add  ->  commit (repeat)
share:             git switch main  ->  git pull  ->  git merge sita/q1  ->  git push
```

Each partner works on their **own branch**, so they never block each other; `main` always holds the combined, working version.

Finishing like professionals: changelog and tag

- A `CHANGELOG.md` lists, for humans, what changed in each version: *Added 3 questions; Fixed population total.*
- A **tag** is a permanent name for one commit — like a branch label that never moves. `git tag v1.0` marks "this is the version we presented". Push it with `git push origin v1.0`. (Grade 10 Module 6 and Grade 12 Module 4 go deeper into releases.)

What the teacher looks at

The history is the evidence. `git log --oneline --graph --all` shows the branches and merges; `git shortlog -s -n` counts commits per author; the messages show your thinking.

See it

A healthy pair history at the end:

```
$ git log --oneline --graph --all
*   f7202da (HEAD -> main, tag: v1.0, origin/main) Merge branch 'hari/q3-small-wards'
|| * b4f828b (hari/q3-small-wards) Add query for wards under 1,000 people
* | 5b19654 (sita/q2-density) Print population density per ward
* | 96501d4 Add area_km2 column to schema
|/
* 0acfbde Add working agreement
* 6ca96e1 Start from ward-facts starter
$ git shortlog -s -n
     3  Hari
     3  Sita
```

Sita's branch was merged first as a **fast-forward** (main had not moved), so it looks like a straight line; Hari's branch then needed a **merge commit**.

Plan (3 periods)

Stage	When	What
Agree	P1, 15 min	write and commit `AGREEMENT.md`; choose the 3 questions
Build	P1–P2 + home	each partner answers questions on their own branch; merge via the remote
Conflict	P2	resolve at least one conflict honestly (the README is a good place)
Review	P2, 15 min	partner reads each branch's diff before merge; swap with another pair and write 2 comments on paper
Release	P3, 15 min	`CHANGELOG.md` entry, `git tag v1.0`, push the tag
Present	P3	show the graph, a diff, and the answers (3 min per pair)

Question ideas

1. Which ward has the most households? 2. What is the average household size (`population / households`) per ward? 3. Which wards are above the average population? 4. What is the population density (if you added `area_km2`)? 5. What share of the total population lives in the largest ward?

PRIMM lab

Brief: pairs maintain a Python script + SQL file answering 3 questions about the small ward dataset.

1. **Predict:** before cloning, draw the graph you *expect* at the end (two branches, two merges, a tag).
2. Clone the starter. Commit `AGREEMENT.md`. Each partner creates a branch.
3. Each partner answers questions in their files; commit small steps; merge via the shared remote.
4. Resolve at least one conflict honestly (the README is a good place — both add a "Results" section).
5. Add a `CHANGELOG.md` entry and tag the result `v1.0` (`git tag v1.0`, `git push origin v1.0`).
6. **Investigate:** compare your final graph with your prediction.
7. Present: show the graph, a diff, and the answers.

Parsons practice

Order the pair routine: *pull* · *switch to your branch* · *work* · *commit* · *switch main* · *pull* · *merge* · *push*.

Rubric

Criterion	3	2	1
Commits	≥ 8 per pair, each partner ≥ 3	5–7 / one partner < 3	< 5
Branches	each partner used a branch, merged into main	one branch	none
Conflict	resolved, markers gone, message explains	resolved	not attempted
README	explains what, how to run, data source	partly	missing
Messages	what + why	mostly	vague
Changelog + tag	both	one	neither
Answers	3 correct, explained	2	≤ 1

Maximum 21 points.

Common mistakes

Problem	Fix
One partner does everything	the agreement assigns files; `git shortlog -s -n` shows it
Working directly on `main`, overwriting each other	use your own branch; merge only after pulling
Never pulling → huge conflicts at the end	pull every session
Tag created but not on the server	`git push origin v1.0` (tags are not pushed by default)
Conflict "resolved" by deleting the partner's work	talk first; keep both ideas where possible

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Software Carpentry **Collaborating** ‹https://swcarpentry.github.io/git-novice/08-collab.html›: compare its workflow with your agreement; add one rule you like.	1 / 3	20 min	new rule
2	*Pro Git* §2.6 **Tagging** ‹https://git-scm.com/book/en/v2/Git-Basics-Tagging›: what is the difference between a lightweight and an annotated tag? Make an annotated `v1.1` (`git tag -a v1.1 -m "…"`).	1 / 3	15 min	difference + tag
3	Keep a Changelog ‹https://keepachangelog.com/en/1.1.0/›: rewrite your `CHANGELOG.md` using its headings (*Added, Changed, Fixed*).	3	15 min	changelog
4	Learn Git Branching `remote5` "Faking Teamwork" ‹https://learngitbranching.js.org/?locale=en_US&level=remote5› and `remote7` "Diverged History" (use the merge solution, not rebase).	1 / 3	20 min	levels done

Exit ticket

1. Which commit are you most proud of?
2. What did your pair agreement say?
3. *Check:* how can the teacher see who did what?

Key points

- Agree how to work first: owners, branches, when to pull and push, review.
- Each partner on their own branch; `main` is the combined, working version.

- Pull at the start, push at the end, merge often.
- Finish with a changelog and a tag; the history is your evidence.

New words

working agreement, **tag**, **changelog**, `git shortlog` — see the glossary.

Chapter complete. You can now branch, merge and share. In Grade 10 you will work in **teams** with **issues**, **pull requests** and **code review**, publish a website, and meet **automated checks (CI)**.

Teacher notes

- **Timing:** P1: hook + agreement 15, build 25. P2: build/conflict 25, review 15. P3: release 15, presentations 25.
- **Rubric:** ≥ 8 commits per pair; each partner ≥ 3; branch used and merged; ≥ 1 conflict resolved; README explains how to run; messages say what and why. **Peer review:** swap repos and write two comments on paper.
- **Fairness:** use `git shortlog -s -n` but don't mark only by counts; read the messages. One partner may legitimately have fewer, larger commits.
- **Tier 1 version:** each pair passes a bundle on USB at the end of each session (`git bundle create pair.bundle --all`; partner `git pull pair.bundle main`).
- **Answer key for question ideas (synthetic data):** Q1 Test Ward Four (480 households); Q2 sizes ≈ 4.82, 4.68, 4.83, 4.81, 4.57; Q3 average 1242 → Test Ward Two (1450) and Test Ward Four (2310); Q5 2310 / 6210 ≈ 37.2 %.

Part IV. Grade 10 — Team Workflows

Grade 10 — Team Workflows

> **Unpiloted draft**
> Core lessons are written but have not been tried in a classroom. Grade placement of topics is unverified against the CDC syllabus (see `research/decisions.md`).

> **CDC fit (verified)**
> The CDC Class 10 textbook contains Unit 2 Database Management (MySQL) and Unit 4 Programming in Python, so Grade 9 modules 6–7 (Python and SQL under version control) fit naturally here too.

Big idea: teams need rules so work does not collide. **Length:** ≈ 18 periods (to calibrate to CDC hours). **Accounts:** Accounts for learners 13+ managed with the teacher; LAN server alternative. **Tools:** Git, a Git host (GitHub, Codeberg/Forgejo, GitLab), HTML/CSS, Python (optional), check scripts.

How this chapter is built

Grade 9 was two people. Grade 10 is a **team** with a **server** in the middle: work is planned in issues, proposed in pull requests, checked by people (review) and robots (CI), and shipped as releases.

```
M1  issues               (plan)
M2  fork/clone/branch    (where copies live)
M3  pull requests        (propose)
M4  review               (check)
M5  PR conflicts         (integrate)
M6  releases             (version)
M7  publish              (ship)
M8  CI                   (robot)
M9  team project         (all together)
```

Modules

#	Module	Periods	Tiers	Key ideas
1	Issues as a to-do list	1	0, 2–3	issue anatomy, labels, `Closes #N`
2	Fork vs clone vs branch	1	2–3	`origin`, `upstream`
3	Pull requests	2	1–3	PR description, `format-patch`
4	Code review with a checklist	2	0–3	kind, specific, actionable
5	Real merge conflicts	1	1–3	merge `origin/main` into the branch
6	Tags and releases	1	1–3	annotated tags, changelog
7	Publishing a static website	2	1–3	Pages, LAN server
8	Automatic checks (CI) intro — optional	1	1–3	exit codes, CI
9	Team project	3	1–3	charter, demo, retrospective

Chapter outcomes

1. plan team work with issues
2. choose between fork, clone and branch

3. open and review pull requests
4. resolve conflicts on a PR
5. tag a release and write notes
6. publish a static site from a repo
7. run a check script

Free resources used in this chapter

GitHub Skills ‹https://skills.github.com/› courses (Introduction to GitHub, Review pull requests, Resolve merge conflicts, GitHub Pages, Hello GitHub Actions, Release-based workflow — learners 13+), *Pro Git* ‹https://git-scm.com/book/en/v2› chapters 2, 5 and 6, Keep a Changelog ‹https://keepachangelog.com/en/1.1.0/›, Open Source Guides ‹https://opensource.guide/›, Atlassian's workflow comparison ‹https://www.atlassian.com/git/tutorials/comparing-workflows›, Learn Git Branching's Remote tab and the Codeberg/Forgejo docs. Full list: free resource library.

Assessment

Team rubric (individual contribution visible in history), peer-review quality, viva. First-draft item bank: `assessments/g10-item-bank.md` (to be reviewed after Pilot 1).

Capstone

Our School Website (or a Python app) built by teams of 3–4 with issues, PRs and review. Practice repo: `practice-repos/g10-school-website/`.

Ready for Grade 11?

A student is ready when they can, without help: write a clear issue; open a PR that closes it; give a specific, kind review; and fix a conflicted PR by merging `main` into their branch.

GRADE 10 · MODULE 1 · 1 PERIOD · TIERS 0, 2, 3

1. Issues as a to-do list

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 0 (paper kanban wall), 2 (Forgejo), 3 (GitHub/Codeberg/GitLab) · **You need:** a team repository on the class server, or sticky notes and a wall
>
> **Before this:** Grade 9 (branches, remotes).
>
> **By the end you can:** write an issue that anyone can act on, label and assign it, plan with a board, and close it from a commit.

Hook (5 min)

A team of four building the school website says "*we'll just remember what to do*". Two weeks later, three people have each made an "About us" page, and nobody made the contact page. What went wrong?

Understand it

An issue is a written task

An **issue** (called a *ticket* or *work item* on some platforms) is a task, bug report, question or idea written down **where the whole team can see it** — on the Issues page of the repository's host. Each issue gets a number (`#1`, `#2`, …), a discussion thread, and a status: **open** or **closed**.

Issues turn "we should…" into a shared, numbered, checkable list. They are also history: a year later, you can read *why* a feature was built.

Anatomy of a good issue

```
Title:   Add contact page with the school's phone and address        <- verb first, specific

What:    The website has no way to contact the school.
Why:     Parents asked the head teacher for the office number.        <- reason
Done when:                                                            <- acceptance check
  - [ ] contact.html exists and is linked from the menu
  - [ ] it shows the office phone, address and hours (no personal numbers)
  - [ ] check.sh passes
Labels:  feature        Assignee: @student-12
```

Weak issue	Problem	Better
`fix it`	fix what?	`Fix broken link to festival.html in the menu`
`website bad`	no action, not kind	`Increase text size on phones — hard to read`
`do the stuff for Friday`	many tasks in one	split into 3 issues, one each

A good issue is **one task**, small enough for one person in a few days, with a **"done when"** check so everyone agrees when it is finished.

Labels, assignees, milestones, boards

- **Labels** group issues: `bug` (something is wrong), `feature` (something new), `docs` (writing), `good first issue` (easy start for newcomers — Grade 11!).

- **Assignee** — the *one* person responsible. Several people can help, but one person owns it.
- **Milestone** — a target date or version (`v1.0 — exhibition day`).
- **Board** (kanban) — columns *To do* · *Doing* · *Done*; each issue is a card that moves right.

Linking work to issues

When you fix an issue, mention its number. Most hosts (GitHub, GitLab, Forgejo/Codeberg) understand **closing keywords**: a commit message or pull request description containing `Closes #3` or `Fixes #3` will automatically close issue #3 when merged into the main branch. Just `#3` (without the keyword) creates a link without closing.

```
git commit -m "Add contact page (closes #3)"
```

See it

A team board at the start of the project:

```
| To do                              | Doing                           | Done                  |
|------------------------------------|---------------------------------|-----------------------|
| #4 Add photo gallery      feature  | #2 Write About page  @s-07      | #1 Set up repo        |
| #5 Fix menu on phones     bug      | #3 Add contact page  @s-12      |                       |
| #6 Write README for site  docs     |                                 |                       |
```

Every card has a number, a label and — once someone starts — exactly one owner.

Unplugged: the kanban wall (15 min)

Draw three columns on the board or a wall: **To do · Doing · Done**. Each team writes the tasks of the School Website project on sticky notes, one task per note, using the "anatomy" above (title + done-when on the back). Rules: a person can have at most **two** notes in *Doing*; a note moves to *Done* only when its done-when check is ticked by someone else.

PRIMM lab (25 min)

1. In your team's repo (LAN server or hosting platform), open the **Issues** page.
2. **Predict:** which of these is a good issue — `fix it`, `Add contact form to the school website`, `website bad`? Why?
3. Write **5 issues** for the School Website project, each with *what, why, done when*. **Modify** them after a neighbour team's feedback.
4. **Label** (`bug`, `feature`, `docs`) and **assign** each issue to one person. Create a milestone `v1.0`.
5. **Investigate:** comment on another team member's issue with a question. Who gets notified?
6. **Make:** put them on a board (the host's project board, or the wall); agree the first three to do. Make one commit with `closes #N` in the message and push — watch the issue close.

Parsons practice (5 min)

Order the life of an issue: *open · discuss · assign · work on a branch · close with a commit message that mentions it*.

Common mistakes

Problem	Fix
One issue holds a whole week's work	split it; one task each
Nobody assigned → nobody does it	exactly one assignee
Issue closed but the work is not finished	use "done when" checks; close only when ticked
`closes #3` in a commit on a branch didn't close it	it closes only when merged into the default branch
Personal data (phone numbers, marks) in an issue	issues are visible to everyone in the repo; edit it out

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	GitHub Docs: **Hello World** ‹https://docs.github.com/en/get-started/start-your-journey/hello-world› — read the steps; then compare with your class server: where is the Issues tab?	3	15 min	2 differences
2	Open any large open-source repository's issue list (for example First Contributions ‹https://github.com/firstcontributions/first-contributions›). Find one issue that is clearly written and one that is not. What makes the difference?	3	20 min	two examples, explained
3	Codeberg docs ‹https://docs.codeberg.org/› — find the page about issue labels / project boards.	3	10 min	link + summary
4	**Paper sort:** the teacher gives 10 issues; mark each *good / fix it* and rewrite the weak ones.	0	15 min	rewritten issues

Exit ticket (5 min)

1. What three parts make a good issue?
2. Who should an issue be assigned to?
3. *Misconception:* "Issues are only for bugs." True or false?

Key points

- An **issue** is a numbered, visible task with a discussion and a status.
- Good issue: verb-first title, what + why, **done when** checks; one task.
- Labels group, one assignee owns, milestones set targets, boards show flow.
- `Closes #N` in a merged commit or PR closes the issue automatically.

New words

issue, **label**, **assignee**, **milestone**, **kanban board**, **closing keyword** — see the glossary.

Extension quest

Add an **issue template** to your repo (`.github/ISSUE_TEMPLATE/bug.md` on GitHub, `.forgejo/issue_template/…` or `.gitea/issue_template/…` on Forgejo) so every new bug report asks *what happened / what should happen / steps*. This book's own repository has templates in `.github/ISSUE_TEMPLATE/` — look at them.

Teacher notes

- **Timing:** hook 5 · understand 10 · kanban wall 15 (or lab) · lab 20 · exit 5. With a short period, do the wall *or* the online lab.
- **Tier 0/1** teams use a paper kanban wall for the whole chapter. Keep titles in English or Nepali but code names in English.
- **Rubric for issues:** clear verb-first title, why, done-check, one task, one assignee.
- **Paper sort set:** include `help`, `Make site better`, `Add festival dates for 2083 BS to festival.html`, `ERROR!!!`, `Fix the about page and gallery and menu`, `Replace "Libary" with "Library" on index.html`, etc.
- **Answer key:** Exit 1: title, what/why, done-when. 2: one person who owns the task. 3: False — features, docs, questions and ideas too.

GRADE 10 · MODULE 2 · 1 PERIOD · TIERS 2, 3

2. Fork vs clone vs branch

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 2–3 (Tier 0 role-play below) · **You need:** the teacher's `school-website-template` repo on the class server
>
> **Before this:** Grade 9 Module 5 (remotes), Module 1.
>
> **By the end you can:** explain the three words, pick the right one for a situation, and keep a fork up to date with `upstream`.

Hook (5 min)

You want to improve a project you do **not** own — say, a Nepali dictionary app made by strangers. You cannot push to their repository. What are your options?

Understand it

Three ways to have "your own" version

	Where it lives	What it is	Who uses it	Made with
branch	inside one repository	a label for a line of work	people who can **write** to that repository	`git switch -c`
clone	**your computer**	a full local copy of a repository	everyone — you need one to work at all	`git clone <url>`
fork	**your account on the server**	a full server-side copy of someone else's repository, linked to the original	people who **cannot write** to the original	the host's *Fork* button

They are not alternatives so much as **layers**. A visitor contributing to an open-source project uses all three:

```
[ original repo ]  owner: the project          <- "upstream"
       |  Fork (on the server)
       v
[ your fork ]      owner: you                  <- "origin"
       |  git clone
       v
[ your laptop ]    branch: feature/about-page  <- where you work
```

Changes travel back up this ladder: you **push** your branch to your fork (`origin`), then open a **pull request** from your fork to the original (`upstream`) — Module 3.

Which do I need?

Situation	Use
I'm in the team and have write access	**clone** the team repo, work on a **branch**
I want to contribute to a project where I have no write access	**fork**, then clone your fork, work on a branch
I want to try an idea for an hour	a **branch** (cheap, delete later)

Situation	Use
I want my own long-term variant of a template (our school's site from the template)	**fork** (or *Use this template*, which copies without the link)

`origin` and `upstream`

When you clone your fork, Git names it `origin` automatically. Add the original project as a second remote, by convention called `upstream`:

```
git remote add upstream http://school-server/teacher/school-website-template.git
git remote -v
```

To get new work from the original into your fork:

```
git switch main
git fetch upstream
git merge upstream/main      # bring the original's new commits into your main
git push origin main         # update your fork on the server
```

Most hosts also have a **"Sync fork"** button that does the same.

See it

```
$ git clone http://school-server/s-12/school-website-template.git site
$ cd site
$ git remote add upstream http://school-server/teacher/school-website-template.git
$ git remote -v
origin    http://school-server/s-12/school-website-template.git (fetch)
origin    http://school-server/s-12/school-website-template.git (push)
upstream  http://school-server/teacher/school-website-template.git (fetch)
upstream  http://school-server/teacher/school-website-template.git (push)
$ git switch -c feature/about-page
Switched to a new branch 'feature/about-page'
```

Unplugged: owners and visitors (10 min)

A poster on the wall is the **original** repo; only two "owners" may write on it. Visitors **photocopy** the poster (fork), take a copy home (clone), and write their idea on a sticky note on *their* copy (branch). To get the idea onto the original, a visitor brings their copy to an owner and asks (pull request). Meanwhile the owners add something new to the poster: visitors must **sync** their copy (fetch + merge upstream).

PRIMM lab (25 min)

1. Teacher repo `school-website-template`. **Predict** which of fork/clone/branch you need (a) as the owner of your team repo, (b) as a visitor to the teacher's repo.
2. **Fork** the template into your account/organisation (on a LAN Forgejo server: the *Fork* button at the top right).
3. **Clone your fork.** Run `git remote -v`. Add the original as `upstream`: `git remote add upstream <url>`.
4. Create a branch `feature/about-page`. Commit one small change and push it to **origin**: `git push -u origin feature/about-page`.
5. **Investigate:** the teacher now adds a commit to the original. Fetch it (`git fetch upstream`) and look: `git log --oneline main..upstream/main`. Merge it into your `main` and push.
6. **Make:** draw the three copies (original, fork, local) and the arrows (fork, clone, push, fetch upstream, pull request).

Parsons practice (5 min)

Order: *fork* · *clone* · *branch* · *commit* · *push to fork*.

Common mistakes

Symptom	Cause	Fix
`Permission denied` / `403` on push	pushing to the original, not your fork	push to `origin` (your fork)
Cloned the original instead of the fork	—	`git remote set-url origin <your-fork-url>`, then add `upstream`
Fork is "N commits behind"	the original moved on	fetch + merge upstream, or *Sync fork*
`upstream` and `origin` mixed up	—	`git remote -v` — origin = yours, upstream = theirs
Forked the same repo many times	—	one fork per project; branches inside it

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	*Pro Git* §6.2 **Contributing to a Project** (GitHub) ‹https://git-scm.com/book/en/v2/GitHub-Contributing-to-a-Project›: read "Forking Projects" and "The GitHub Flow". List its numbered steps.	3	20 min	step list
2	*Pro Git* §5.1 **Distributed Workflows** ‹https://git-scm.com/book/en/v2/Distributed-Git-Distributed-Workflows›: which of the three workflows does our class use?	3	15 min	answer + reason
3	Learn Git Branching **Remote** tab: `remote2` Remote Branches ‹https://learngitbranching.js.org/?locale=en_US&level=remote2› and `remote3` Git Fetchin'.	1 / 3	15 min	levels done
4	Atlassian, **Comparing workflows** ‹https://www.atlassian.com/git/tutorials/comparing-workflows› — read the "Forking workflow" part. Draw it.	3	15 min	drawing

Exit ticket (5 min)

1. When do you need a fork?
2. What is `upstream`?
3. *Misconception:* "A fork is the same as a branch." True or false?

Key points

- **Branch** = label inside a repo · **clone** = copy on your computer · **fork** = your copy on the server.
- Team member with write access: clone + branch. Outsider: fork + clone + branch.
- `origin` = your fork; `upstream` = the original. Fetch and merge upstream to stay in sync.

New words

fork, **upstream**, **origin**, **sync fork**, **template repository** — see the glossary.

Extension quest

Find out what `git push -u` (set upstream tracking) changes: after it, what does `git status` say about your branch?

Teacher notes

- **Timing:** hook 5 · understand 10 · role-play 10 · lab 15 · exit 5. Prepare the template repo and an extra commit to push during lab step 5.
- Students 13+ only create accounts; teacher-managed organisation preferred. On a LAN Forgejo, create pseudonymous accounts in advance.
- **Typical confusion:** `origin` vs `upstream`; also "tracking branch" vs "remote". Keep the ladder diagram on the board.
- **Fallback Tier 1:** forks need a server. Without one, teach the role-play and simulate `upstream` with a second local repo: `git remote add upstream ../teacher-copy`.
- **Answer key:** Exit 1: when you want to contribute to a repo you cannot write to (or keep your own long-term copy on the server). 2: the remote name for the original project you forked from. 3: False — a fork is a whole server-side repository; a branch is a label inside one. Extension: it records the remote branch so `git status` says "ahead/behind 'origin/feature/about-page'" and plain `git push` / `git pull` work.

GRADE 10 · MODULE 3 · 2 PERIODS · TIERS 1, 2, 3

3. Pull requests

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 2–3 (Tier 1 version with patches below) · **You need:** your team repo with issues from Module 1
>
> **Before this:** Modules 1–2.
>
> **By the end you can:** turn an issue into a branch, the branch into a pull request with a clear description, and update the PR after feedback.

Hook (5 min)

In Grade 9 you merged straight into `main`. Imagine the whole school website team does that — on exhibition morning, someone's half-finished page breaks the menu for everyone. In a real team, **someone checks first**.

Understand it

What a pull request is

A **pull request (PR)** — GitLab calls it a *merge request (MR)* — is a request on the server that says:

> *"Please merge my branch `feature/about-page` into `main`. Here is what I changed and why. Please review it."*

It is **not** a copy of the code. It is a page that points at two branches and shows:

- **Conversation** — the description and everyone's comments;
- **Commits** — the commits on the branch;
- **Files changed** — the diff between the branch and `main`;
- **Checks** — automatic tests, if the team has them (Module 8).

```
issue #2 "Write About page"
    |  (you create a branch for it)
    v
branch feature/about-page:  c1 -- c2 -- c3      --push-->  server
    |
    v
pull request #7: feature/about-page -> main      "Closes #2"
    |   reviewer reads the diff, comments, approves
    v
merged into main  ->  issue #2 closes  ->  branch deleted
```

The PR workflow

1. Pick an issue; create a branch named after it (`feature/about-page`, `fix/menu-link`).
2. Commit in small steps; push the branch.
3. Open the PR. Write a description (template below).
4. A teammate **reviews** (Module 4). You fix things by pushing **more commits to the same branch** — the PR updates itself.
5. When approved (and checks pass), someone **merges**. The branch is deleted.

A good PR description

```
## What
Adds about.html with the school's history and a menu link.

## Why
Closes #2 — parents asked for basic information about the school.

## How to check
1. Open about.html in a browser; the menu link "About" works.
2. Run `bash check.sh` — all links OK.

## Notes for the reviewer
I was not sure about the founding year; I wrote 2045 BS from the school notice board.
```

Keep PRs small

A PR that changes 20 lines gets a careful review in 5 minutes. A PR that changes 2,000 lines gets "looks good" and hides bugs. **One issue → one branch → one small PR.**

Who merges?

Teams decide. Common rules: *the author never merges their own PR without one approval*; *the reviewer merges*; *main is protected* — the server refuses direct pushes to `main` (Grade 12 covers branch protection).

See it

```
$ git switch main && git pull
$ git switch -c feature/about-page
$ git add about.html index.html
$ git commit -m "Add About page and menu link (closes #2)"
$ git push -u origin feature/about-page
...
remote: Create a new pull request for 'feature/about-page':
remote:   http://school-server/team-3/school-website/compare/main...feature/about-page
```

Most hosts print a link to open the PR right after you push a new branch.

Unplugged: paper pull requests (15 min)

Pairs. The **author** writes a printed (or hand-written) diff of a small web page change on a sheet, with a PR description at the top. The **reviewer** writes comments in the margin using sticky notes, and either *approves* (✓) or *requests changes*. The author makes a second sheet (a new commit) answering the comments. Only then does the teacher (the "maintainer") staple it into the class's `main` folder.

PRIMM lab (50 min)

1. Take issue #1 or #2 (e.g. *Add About page*). Create `feature/about-page` and make 2–3 small commits.
2. Push the branch. Open a PR: title, description using the template above (`Closes #N`), what you changed, how to check.
3. **Predict:** what will the reviewer see on the *Files changed* tab? Check it. Is anything there you didn't mean to include?
4. Your partner reviews (Module 4 goes deeper). Meanwhile, **fix a requested change** by pushing one more commit to the same branch. **Investigate:** how did the PR page change?
5. The reviewer approves and merges. **Investigate:** did the issue close? Delete the branch (button on the PR, then `git switch main && git pull && git branch -d feature/about-page` locally).

6. **Make:** write a PR description template for your team and commit it (`.github/pull_request_template.md` on GitHub, `.forgejo/pull_request_template.md` or `.gitea/pull_request_template.md` on Forgejo).

Tier 1 version (no server): patches

```
git format-patch main          # writes 0001-Add-About-page.patch, one file per commit
# copy the .patch files to USB; the reviewer reads them (they are plain text diffs)
git am 0001-*.patch            # the maintainer applies them on their computer
```

This is how the Linux kernel still works today — by email!

Parsons practice (10 min)

Order: *create branch · commit · push · open PR · review · merge · delete branch.*

Common mistakes

Symptom	Cause	Fix
PR shows dozens of unrelated changes	branched from an old or wrong branch	branch from an up-to-date `main`
Opened a new PR for every fix	—	push more commits to the **same** branch
Issue did not close	keyword missing, or PR not merged into the default branch	`Closes #N` in the description; merge into `main`
PR description says "pls merge"	no what/why/how to check	use the template
Merged your own PR with no review	—	team rule: one approval first

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	GitHub Skills **Introduction to GitHub** ‹https://github.com/skills/introduction-to-github› (13+, account) — it ends with opening and merging a PR.	3	60 min	completed course
2	GitHub Docs, **Hello World** ‹https://docs.github.com/en/get-started/start-your-journey/hello-world›: compare its steps with ours. Which step do we do on the command line instead of the website?	3	15 min	answer
3	*Pro Git* §5.2 **Contributing to a Project** ‹https://git-scm.com/book/en/v2/Distributed-Git-Contributing-to-a-Project›: read "Commit Guidelines". Write down three rules and check your PR against them.	3	20 min	3 rules + self-check
4	Look at closed PRs in this book's own repository (or any open-source project). Find one with a great description. What makes it great?	3	15 min	link + reasons

Exit ticket (5 min)

1. What does `Closes #1` do?
2. Why open a PR instead of merging yourself?
3. *Misconception:* "A PR is a new copy of the code." True or false?

Key points

- A **PR** asks to merge one branch into another, with description, commits, diff and discussion.
- Issue → branch → small commits → push → PR → review → merge → delete branch.
- Update a PR by pushing more commits to its branch.
- Good descriptions say what, why (`Closes #N`), and how to check.
- Small PRs get good reviews.

New words

pull request (PR), **merge request (MR)**, **review**, **approve**, **request changes**, **patch** — see the glossary.

Extension quest

Open a **draft** PR early ("work in progress") to get feedback before you finish. When is that useful?

Teacher notes

- **Timing:** P1: hook 5, understand 15, paper PRs 15, lab 1–2 (5). P2: lab 3–6 (35), Parsons 5, exit 5.
- Keep PRs small (one issue). On Tier 2 Forgejo the PR flow is the same as on GitHub. Tier 1: simulate with `git format-patch` and a printed diff.
- **Before class:** make sure each team has issues from Module 1 and that `main` is protected (or at least agreed as "no direct pushes").
- **Answer key:** Exit 1: links the PR to issue #1 and closes the issue when the PR is merged into the default branch. 2: so someone reviews it first — catches mistakes, shares knowledge, protects `main`. 3: False — it points at a branch; no code is copied.

GRADE 10 · MODULE 4 · 2 PERIODS · TIERS 0, 1, 2, 3

4. Code review with a checklist

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0–3 (printed diffs work everywhere) · **You need:** an open PR from another pair (or a printed diff)
>
> **Before this:** Module 3.
>
> **By the end you can:** review a PR systematically, write comments that help, and respond to feedback without taking it personally.

Hook (5 min)

A reviewer writes "*this is bad*" on your PR. How do you feel? What do you do next? Now imagine the comment said: "*The menu link on line 12 goes to* `abuot.html` *— should it be* `about.html`*?*" What changed?

Understand it

Why review?

Code review means a teammate reads your change *before* it is merged. It:

- **catches mistakes** — broken links, typos, bugs, private data;
- **shares knowledge** — now two people understand the change;
- **keeps quality consistent** — names, style, structure;
- **teaches both sides** — reviewers often learn more than authors.

Big software companies require at least one review for *every* change. Reviewing is a skill you can practise from today — you do not have to be an expert to notice a broken link or an unclear sentence.

How to review: a routine

1. **Read the description and the issue.** What is this PR *supposed* to do?
2. **Look at the size.** Too big to understand? Ask for it to be split.
3. **Read the diff** (*Files changed*), top to bottom. Note questions.
4. **Run it.** Open the page / run the script on the branch: `git fetch origin && git switch feature/about-page`.
5. **Fill the checklist.**
6. **Write comments** on specific lines, then a summary.
7. **Decide:** *Approve*, *Request changes*, or *Comment* (just questions).

Checklist

☐ Does it do what the issue asks — no more, no less?
☐ Is it small enough to understand?
☐ Are names, messages and text clear?
☐ Does the page/program still run? Do the checks pass?
☐ Nothing private (names, phone numbers, passwords, marks) was added?
☐ Would a newcomer understand it?

Writing helpful comments

A good comment is **specific**, **kind**, **actionable**, and **about the work, not the person**.

Unhelpful	Helpful
"ugly"	"Could we use the school colour `#1a5276` for this heading, like on index.html?"
"wrong"	"Line 8 says 2045 BS but the notice board says 2043 BS — which is right?"
"you always forget the alt text"	"The image on line 15 needs `alt` text for screen readers, e.g. `alt="School gate"`."
"LGTM" on a 900-line PR	"This is too large for me to check carefully — could we split the gallery into its own PR?"

Useful patterns: **"Could we…?"**, **"What do you think about…?"**, **"I wonder if…"**, and always some **praise**: "Nice — the menu now works on phones."

Label the weight of a comment: **nit:** (tiny, optional), **question:**, **suggestion:**, **must:** (blocks merging).

Being reviewed

- Thank the reviewer; they spent time on *your* work.
- Answer every comment: fix it (push a commit and reply "Done in `a1b2c3d`") or explain why not.
- Disagree politely and with reasons. If stuck, ask a third person.
- Remember: the review is about the code, not about you.

AI-written code gets the same review

If any part of a PR was written with an AI helper, it gets the same checklist — and the author must be able to explain every line. "The AI wrote it" is never a reason to approve.

See it

A review on the *Files changed* tab:

```
  <nav>
    <a href="index.html">Home</a>
+   <a href="abuot.html">About</a>
```

> **suggestion:** The link points to `abuot.html`, but the file is `about.html`, so the link will be broken. Could you fix the spelling? `check.sh` should catch this too.

```
+ <p>Founded in 2045 BS by local families.</p>
```

> **question:** Is 2045 BS confirmed? The notice board photo in issue #2 says 2043 BS.
>
> **Summary — Request changes.** Thank you, the page reads well and the menu is consistent. Two things before merging: the link spelling, and the founding year. Everything else looks good.

Unplugged: rewrite the comments (15 min)

Pairs get 10 unhelpful review comments on cards (see teacher notes). Rewrite each to be specific, kind and actionable. Then the class votes on the best rewrite of each. Finally, review a **printed diff** of a small web page with the checklist.

PRIMM lab (50 min)

1. Swap PRs with another pair. **Predict** the riskiest line before reading in detail.
2. Read the diff, check out the branch and run the page (or `bash check.sh`). Fill the checklist.
3. Write **2 praise** comments and **2 improvement** comments on specific lines, plus a summary. Choose *Approve* or *Request changes*.
4. **Author:** reply to each comment, push fixes as new commits. **Reviewer:** check the new commits and approve.
5. **Investigate:** read the PR's conversation from top to bottom. Would a newcomer understand *why* the code looks the way it does?
6. **Make:** improve the checklist for your team and commit it to the repo as `REVIEW.md`.

Parsons practice (10 min)

Match unhelpful comments to kind rewrites: "ugly" → "Could we use the school colour here?".

Common mistakes

Problem	Fix
Approving without running anything	always open/run the change
Only negative comments	add specific praise
Comments about the person	talk about the code
Reviewing a huge PR	ask for a split
Author ignores comments	team rule: every comment gets a reply
Arguing in comments for days	meet and talk; write the decision in the PR

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	GitHub Skills **Review pull requests** ‹https://github.com/skills/review-pull-requests› (13+, account).	3	45 min	completed course
2	Open Source Guides, **How to Contribute** ‹https://opensource.guide/how-to-contribute/› — the section on communicating. Pick 3 tips that apply to reviews.	3	15 min	3 tips
3	Find a merged PR in a well-known open-source project with at least 5 review comments. Classify each comment: praise / question / suggestion / must.	3	20 min	table
4	**Printed review (Tier 0):** the teacher gives a printed diff with 5 planted problems (broken link, phone number, typo, missing alt text, unclear message). Find them.	0	15 min	list of 5

Exit ticket (5 min)

1. Name three qualities of a good comment.
2. What should a reviewer check before approving?
3. *Misconception:* "Reviewing is only for experts." True or false?

Key points

- Review catches mistakes, shares knowledge, and teaches both sides.

- Routine: description → size → diff → run → checklist → comments → decision.
- Comments: specific, kind, actionable, about the work; include praise.
- Authors answer every comment; the review is not personal.
- AI-written code gets the same review; the author must explain every line.

New words

code review, **reviewer**, **approve**, **request changes**, **nit**, **LGTM** ("looks good to me") — see the glossary.

Extension quest

Use the host's **suggestion** feature (on GitHub: the ± icon in a line comment) to propose an exact fix the author can apply with one click.

Teacher notes

- **Timing:** P1: hook 5, understand 15, rewrite cards 15, see it 5. P2: lab 35, Parsons 5, exit 5.
- **Reviewing teaches more than writing; grade the *quality of review comments*.** Suggested mini-rubric (6 points): specific (2), kind (2), actionable (2).
- Remind students that AI-generated code gets the same review as any other.
- **Unhelpful comment cards:** "ugly", "wrong", "no", "why??", "this is stupid", "fix", "LGTM" (on a huge PR), "you did it again", "I would never do it like this", "?".
- **Printed review planted problems:** `href="galery.html"`, a 10-digit phone number of a "student", "Libary", `<img src="gate.jpg">` with no alt, commit message "stuff".
- **Answer key:** Exit 1: specific, kind, actionable (about the work). 2: the checklist — does what the issue asks, small, clear, runs/checks pass, nothing private. 3: False — anyone can spot unclear text, broken links and missing checks.

GRADE 10 · MODULE 5 · 1 PERIOD · TIERS 1, 2, 3

5. Real merge conflicts, resolved in pairs

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 1–3 · **You need:** two PRs (planted by the teacher) that edit the same line of `index.html`
>
> **Before this:** Grade 9 Module 4 (conflicts), Modules 3–4.
>
> **By the end you can:** bring `main` into your branch, resolve the conflict locally, and make a blocked PR mergeable again.

Hook (5 min)

Two PRs change the same menu line. The first is merged. The second now shows a grey box: **"This branch has conflicts that must be resolved."** The merge button is disabled. Whose job is it to fix this — and how?

Understand it

Why a PR becomes conflicted

Both PRs started from the same `main`. PR A changed line 12; PR B changed line 12 differently. When A is merged, `main` moves. B's branch is now based on an *old* `main`, and its change collides with A's:

```
          A1 ---- (PR A, merged)
         /      \
main: m1 -------- m2  <- main now contains A's line 12
         \
          B1        <- PR B also changed line 12: conflict with m2
```

The server cannot ask a human in the middle of a button click, so it refuses. The fix happens on a computer.

Who fixes it?

The **author of the later PR** (B). They know what their change is meant to do, and they can check that the result works.

The fix: bring main into your branch

```
git switch feature/menu-gallery       # your PR's branch
git fetch origin                      # get the latest main from the server
git merge origin/main                 # bring main into your branch -> CONFLICT
#  ... edit the file, keep the right combination, delete the markers ...
git add index.html
git commit                            # finishes the merge (a merge commit)
git push                              # the PR updates and becomes mergeable
```

This is exactly the Grade 9 conflict, but **on your branch instead of on main**. `main` stays clean; you solve the problem where it belongs. Re-run the page and the checks after resolving — a resolution can be wrong even when the markers are gone!

Many hosts also offer a web conflict editor ("Resolve conflicts" button). It works for small text conflicts, but resolving locally lets you **test** the result.

Preventing conflicts

Conflicts are a sign of **long-lived branches** and **work that overlaps**. Habits that make them rare:

- **Small PRs, merged quickly** (hours or days, not weeks).
- **Bring `main` into your branch often** (`git fetch && git merge origin/main`), not only at the end.
- **Talk** — "I'm editing the menu today" in the team chat or the issue.
- **Split files** sensibly — a `menu.html` included everywhere instead of the menu copied into every page.

Rebase? Not yet

You may read online that people use `git rebase` instead of `merge` here. Rebase rewrites your branch's history; it is powerful and easy to misuse. We learn it in Grade 11. In Grade 10 we always **merge** `main` into the branch.

See it

```
$ git switch feature/menu-gallery
$ git fetch origin
$ git merge origin/main
Auto-merging index.html
CONFLICT (content): Merge conflict in index.html
Automatic merge failed; fix conflicts and then commit the result.
$ grep -n -A4 "<<<<<<<" index.html
12:<<<<<<< HEAD
13-    <a href="gallery.html">Gallery</a>
14-=======
15-    <a href="about.html">About</a>
16->>>>>>> origin/main
```

Both links are wanted, so the right resolution keeps **both**:

```
    <a href="about.html">About</a>
    <a href="gallery.html">Gallery</a>
```

```
$ git add index.html
$ git commit --no-edit
$ bash check.sh            # still passes?
$ git push
```

Unplugged warm-up (5 min)

Recall the Grade 9 paper game in one minute: two people change line 3 → markers → a human decides. Then ask: "*If one change is already in the class's main folder, who should redo the merge?*"

PRIMM lab (30 min)

1. Teacher plants two PRs that edit `index.html` line 12 (or each pair creates both: partner A adds an About link, partner B a Gallery link, on separate branches from the same `main`).
2. **Predict:** after PR A is merged, what will PR B's page say? Merge A and look.
3. Author of B: `git switch feature/x`, `git fetch origin`, `git merge origin/main`.
4. Open the file, resolve (here: keep both links in a sensible order), `git add`, `git commit`, run `bash check.sh`, `git push`.
5. **Investigate:** why did the PR become mergeable? Look at the PR's commit list — what new commit appeared?

6. **Modify:** repeat with a *real* disagreement (both change the page title differently). Discuss with your partner before resolving; record the decision in a PR comment.
7. **Make:** write a rule that makes this conflict less likely (e.g. one person per file per day; pull often) and add it to the team's agreement.

Parsons practice (5 min)

Order: *fetch* · *merge origin/main into your branch* · *resolve* · *add* · *commit* · *push*.

Common mistakes

Symptom	Cause	Fix
Resolved by keeping only your line, deleting a teammate's work	rushed	keep both where both are wanted; ask if unsure
Merged `main` locally but forgot to push	—	`git push`; the PR updates only after a push
Markers left in the file	—	`git grep "<<<<<<<"` before committing; checks may catch it
Merged your branch into `main` locally instead	wrong direction	`git merge --abort` (if unfinished); switch to your branch first
Used `git push --force` to "make it work"	—	never on shared branches; ask the teacher

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	GitHub Skills **Resolve merge conflicts** ‹https://github.com/skills/resolve-merge-conflicts› (13+, account) — it uses the web editor; compare with our local method.	3	45 min	one pro and one con of each method
2	Software Carpentry **Conflicts** ‹https://swcarpentry.github.io/git-novice/09-conflict.html› — exercise "Conflicts on Non-textual Files": what happens with an image?	1 / 3	15 min	answer
3	*Pro Git* §3.2 **Basic Merge Conflicts** ‹https://git-scm.com/book/en/v2/Git-Branching-Basic-Branching-and-Merging› — try `git mergetool` (or VS Code's merge editor) on your next conflict.	1	20 min	which tool you prefer
4	Learn Git Branching `remote7` "Diverged History" ‹https://learngitbranching.js.org/?locale=en_US&level=remote7› — solve it with merge.	1 / 3	15 min	level done

Exit ticket (5 min)

1. Who fixes the conflict — the first or the second PR's author?
2. Name one habit that prevents conflicts.
3. *Misconception:* "Conflicts mean someone did something wrong." True or false?

Key points

- A PR conflicts when `main` moved and changed the same lines.
- The later PR's author fixes it: fetch, **merge `origin/main` into the branch**, resolve, test, push.
- Test after resolving — no markers ≠ correct.
- Small PRs, frequent merges and talking prevent most conflicts.

New words

conflicted PR, `origin/main`, **merge editor / mergetool** — see the glossary.

Extension quest

Try `git merge --no-commit origin/main` to see the merge result *before* committing it. When could that help?

Teacher notes

- **Timing:** hook 5 · understand 10 · lab 25 · Parsons + exit 5. Plant the two PRs before class (or have each pair create both in the first 5 minutes of the lab).
- **Rebase is NOT taught here; it comes in Grade 11.** Use merge to bring `main` into the branch.
- VS Code's built-in merge editor shows *Incoming*, *Current* and *Result* panes — fine for students who struggle with raw markers.
- **Fallback Tier 1:** two local branches plus a shared bundle on USB; the "server" is the teacher's laptop.
- **Answer key:** Exit 1: the later (second) PR's author. 2: small PRs, merging main into your branch often, telling the team what you are editing, sensible file structure. 3: False — conflicts are normal when people work in parallel.

GRADE 10 · MODULE 6 · 1 PERIOD · TIERS 1, 2, 3

6. Tags and releases

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 1–3 · **You need:** your team repository with merged PRs
>
> **Before this:** Grade 9 Module 8 (first tag), Modules 3–5.
>
> **By the end you can:** mark a finished version with an annotated tag, publish a release with notes, and keep a changelog.

Hook (5 min)

The head teacher asks: "*Which version of the website is the finished one we showed at the festival?*" The team is still committing new changes every day. How do you point to *exactly* the version that was shown — today, and in a year?

Understand it

Tags: names that never move

A **branch** label moves forward with every commit. A **tag** is a label that **stays on one commit for ever**. It answers "which exact snapshot was version 1.0?".

```
  c1 -- c2 -- c3 -- c4 -- c5   [main]   <- main keeps moving
              ^
          (v1.0)                        <- the tag stays here
```

Two kinds:

	Lightweight tag	Annotated tag (use this)
Create	`git tag v1.0`	`git tag -a v1.0 -m "Festival version"`
Stores	just the name → commit	name, tagger, date, message
Use for	private bookmarks	public versions

Tags are **not pushed automatically**: `git push origin v1.0` (or `git push --tags` for all).

Releases

A **release** is a tag plus **release notes** on the hosting server's *Releases* page, often with files to download (a ZIP of the site, a program). Notes are written for **users**, not programmers:

```
## v1.0 – Festival version (2083-07-15)
### Added
- About page with the school's history (#2)
- Contact page with office phone and hours (#3)
### Fixed
- Broken "Festival" link in the menu (#5)
```

The changelog

`CHANGELOG.md` collects the notes of **every** release, newest first, in the repository itself. The free standard Keep a Changelog ‹https://keepachangelog.com/en/1.1.0/› uses the headings *Added, Changed, Deprecated, Removed, Fixed, Security* and keeps an *Unreleased* section at the top for changes not yet released.

Release notes and changelogs are **not** the same as `git log`. The log has every small commit ("Fix typo in about.html"); the changelog says what matters to users.

Version numbers, lightly

Many projects number versions **MAJOR.MINOR.PATCH** (`1.2.0`):

- **PATCH** (1.2.**1**) — a small fix;
- **MINOR** (1.**3**.0) — something new, nothing broken;
- **MAJOR** (**2**.0.0) — a big change that breaks old ways of using it.

This is called **semantic versioning**; Grade 12 learns its exact rules. For the school site: v1.0.0 at the festival, v1.1.0 when the gallery is added, v1.1.1 for a typo fix.

Going back to a release

```
git switch --detach v1.0     # look at the festival version (read-only "detached HEAD")
git switch main              # come back
git diff v1.0 main           # what changed since the festival?
git log --oneline v1.0..main # which commits came after it?
```

See it

```
$ git tag -a v1.0.0 -m "Festival version"
$ git tag
v1.0.0
$ git show v1.0.0 --no-patch
tag v1.0.0
Tagger: Student 12 <s12@example.invalid>
Date:   Thu Oct 29 11:00:00 2026 +0545

Festival version

commit 9d1e4a7 (HEAD -> main, tag: v1.0.0)
...
$ git push origin v1.0.0
To http://school-server/team-3/school-website.git
 * [new tag]         v1.0.0 -> v1.0.0
```

Unplugged (5 min)

Hold up the class magazine card row from Grade 8. Clip a **paper clip** labelled *v1.0* onto one card, and a **sticky note** *main* onto the newest card. Add two cards: the sticky note moves, the paper clip does not.

PRIMM lab (30 min)

1. On `main`, when the site is ready, **predict** what `git tag` lists. Then: `git tag -a v1.0.0 -m "First public version"`.
2. `git push origin v1.0.0`. On the server, open *Releases* (or *Tags*) and create a release from the tag with notes using the headings above.

3. **Predict:** what happens to the tag if you commit more? (It stays on the old commit.) Make a commit and check with `git log --oneline --decorate`.
4. **Investigate:** `git log --oneline v1.0.0..main` and `git diff v1.0.0 main --stat`.
5. **Make:** write `CHANGELOG.md` for v1.0.0 in three bullet points, with an empty `## [Unreleased]` section on top. Commit it via a PR.

Parsons practice (5 min)

Order: *main is ready · tag · push tag · release notes · announce*.

Common mistakes

Symptom	Cause	Fix
Tag not on the server	tags are not pushed by default	`git push origin v1.0.0`
Tagged the wrong commit	—	before pushing: `git tag -d v1.0.0`, tag again. After pushing: make a new version (v1.0.1) — don't move public tags
Release notes copied from `git log`	too detailed for users	summarise by Added/Changed/Fixed
`fatal: tag 'v1.0' already exists`	—	choose the next version number

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	*Pro Git* §2.6 **Tagging** ‹https://git-scm.com/book/en/v2/Git-Basics-Tagging›: try `git tag -l "v1.*"` and tagging an older commit.	1 / 3	15 min	commands used
2	Keep a Changelog ‹https://keepachangelog.com/en/1.1.0/›: read "Guiding principles". Improve your changelog to follow them.	3	15 min	updated changelog
3	Learn Git Branching `mixed4` "Git Tags" ‹https://learngitbranching.js.org/?locale=en_US&level=mixed4›.	1 / 3	10 min	level done
4	GitHub Skills **Release-based workflow** ‹https://github.com/skills/release-based-workflow› (13+, account).	3	45 min	completed course
5	Look at this book's own `CHANGELOG.md` (in the repository). Which headings does it use?	1	5 min	headings

Exit ticket (5 min)

1. What does a tag mark?
2. What goes in release notes?
3. *Misconception:* "The tag moves when I commit." True or false?

Key points

- A **tag** is a permanent name on one commit; use **annotated** tags for versions.
- Push tags explicitly; never move a public tag — release a new version instead.
- A **release** = tag + notes for users; a **changelog** collects them in the repo.

- MAJOR.MINOR.PATCH: break · add · fix (details in Grade 12).

New words

tag, **annotated tag**, **release**, **release notes**, **changelog**, **semantic version** — see the glossary.

Extension quest

Run `git describe` on a commit after the tag. What do the parts of its output mean? (Learn Git Branching level `mixed5` "Git Describe".)

Teacher notes

- **Timing:** hook 5 · understand 10 · unplugged 5 · lab 20 · Parsons + exit 5.
- Keep semantic versioning very light here; the full rules come in Grade 12 Module 4.
- If time is short, skip the server *Release* page and do tags + changelog only (works in Tier 1).
- **Answer key:** Exit 1: one exact commit (a version) — permanently. 2: what changed for users: added, changed, fixed (with issue numbers). 3: False. Extension: `v1.0.0-2-g9d1e4a7` = tag, commits since the tag, `g` + short id.

GRADE 10 · MODULE 7 · 2 PERIODS · TIERS 1, 2, 3

7. Publishing a static website from the repo

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 1 (open in browser), 2 (serve on LAN), 3 (Pages host) · **You need:** `practice-repos/g10-school-website`, a browser, your team repo
>
> **Before this:** Modules 3–6.
>
> **By the end you can:** edit HTML/CSS in a repo, preview it, review it through a PR, and publish it — for free — from the repository.

Hook (5 min)

Can the school have a web page without buying a server or paying a company every month? Yes — if the site is **static**, the repository itself can be the website.

Understand it

Static vs dynamic sites

	Static site	Dynamic site
Made of	files: `index.html`, `style.css`, images	a program + a database (e.g. PHP/Python + MySQL)
The server	just sends the files	runs code for every visitor
Examples	school notice pages, documentation, portfolios, **this book's website**	online shops, exam-result search, social media
Cost & safety	free hosting, very little can break or be hacked	needs a real server and maintenance

For a school's information site, static is usually enough — and every page lives in Git, with full history and review.

Repository → website

```
your repo (main branch)            Pages host                    visitors
index.html, style.css ...  --->  copies the files  --->  https://our-school.codeberg.page/
      ^                          after each push
      |  PRs with review
```

Free "Pages" services publish a branch as a website: **GitHub Pages**, **Codeberg Pages**, **GitLab Pages**. Forgejo can do the same with Actions. With no internet, any computer on the school LAN can serve the folder:

```
python -m http.server 8000      # then other PCs open http://<this-PC's-IP>:8000
```

Every change goes through a branch and a PR, so the live site only changes after review — and if something breaks, `git revert` brings the old page back.

Site generators (a glimpse)

Writing every page in HTML gets repetitive. **Static site generators** turn Markdown into HTML pages with a shared menu and style. This book uses **MkDocs**: each lesson is a `.md` file in Git; one command (`mkdocs build`) makes the website, and the GitHub workflow `.github/workflows/pages.yml` publishes it automatically on every push to `main`. Others: Jekyll, Hugo, Eleventy.

Credit and privacy on the public web

- Credit every image and text you did not create: "*Photo: Name, CC BY 4.0*". Use only what you are allowed to.
- **Never publish students' names, photos, marks or phone numbers.** A public site is public for ever (and copied by search engines).
- Contact details: the school office only.

See it

```
$ cd school-website
$ bash check.sh
g10 ok
$ git switch -c feature/about-page
  (create about.html, add <a href="about.html">About</a> to the menu)
$ bash check.sh
g10 ok
$ git add about.html index.html festival.html
$ git commit -m "Add About page and link it from every menu (closes #2)"
$ git push -u origin feature/about-page
  (open PR -> review -> merge -> the Pages host republishes main)
```

Unplugged (10 min)

Draw the journey of a page change on the board: *edit → commit → push → PR → review → merge → Pages publishes → visitor sees it*. Students act as each step holding a printed page; ask "where could a mistake be caught?" (review, check script, preview).

PRIMM lab (60 min)

Practice repo: `practice-repos/g10-school-website` (copy it into your team repo if you don't have one).

1. Open `index.html` in a browser (Tier 1 works offline).
2. **Predict**, then change the headline and the colour in `style.css`; refresh. Run `git diff` — which file did the colour change touch?
3. On a branch, add an `about.html`; link it from the menu of **every** page; run `bash check.sh`; open a PR; get it reviewed and merged.
4. **Publish:**
   - **Tier 3:** enable Pages for `main` (GitHub: *Settings → Pages → Deploy from a branch → main / root*; Codeberg: push to a repository named `pages`, or a `pages` branch — follow Codeberg's docs).
   - **Tier 2:** on the server PC, `python -m http.server 8000` in the site folder; classmates open `http://<IP>:8000`.
   - **Tier 1:** open the files directly in the browser.
5. **Investigate:** make a small change, merge it, and time how long until the live site updates.
6. **Make:** add one page that shows a small open-data table (for example 5 rows from a Census 2021 or Open Data Nepal table you are allowed to use) and **credit the source** at the bottom.

Parsons practice (5 min)

Order: *edit HTML · open in browser · commit · push · publish*.

Common mistakes

Symptom	Cause	Fix
Page works on my PC, broken online	links with `C:\Users\…` paths or wrong case (`About.html` vs `about.html`)	relative links, lower-case names; `check.sh`
Changes not visible online	Pages not enabled, wrong branch, or browser cache	check settings; wait a minute; Ctrl+F5
Images missing	image not committed, or in `.gitignore`	`git status`; add the image
Site shows the README instead	no `index.html` in the published folder	add `index.html` at the root
Personal photo or phone number published	—	remove, commit, and ask the teacher about removing it from history (Grade 12 M5)

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	GitHub Skills **GitHub Pages** ‹https://github.com/skills/github-pages› (13+, account).	3	45 min	your published page URL
2	Codeberg docs ‹https://docs.codeberg.org/› — find the *Codeberg Pages* section and list its steps.	3	15 min	steps
3	Markdown Guide ‹https://www.markdownguide.org/›: rewrite your About page in Markdown (as this book's lessons are).	1 / 3	20 min	`about.md`
4	Look at this book's `mkdocs.yml` and one lesson `.md` file. How does MkDocs know the page title?	1	10 min	answer
5	The Odin Project, **Git Basics** ‹https://www.theodinproject.com/lessons/foundations-git-basics› — its project of putting a page on GitHub.	3	60 min	URL

Exit ticket (5 min)

1. What makes a site "static"?
2. Where would you publish with no internet?
3. *Misconception:* "A website needs a database." True or false?

Key points

- A **static site** is only files; a host sends them as they are.
- **Pages** services publish a branch for free; a LAN PC can serve it with `python -m http.server`.
- Every change goes branch → PR → review → merge → publish; revert fixes a broken live page.
- Credit sources; never publish private data.

New words

static site, **dynamic site**, **Pages**, **site generator**, **relative link** — see the glossary.

Extension quest

Add a `404.html` page ("page not found") and a simple CSS rule that makes the menu readable on phones (`@media (max-width: 600px) { … }`).

Teacher notes

- **Timing:** P1: hook 5, understand 15, unplugged 10, lab 1–2 (10). P2: lab 3–6 (35), Parsons 5, exit 5.
- **Credit images and text (CC BY). Never publish student names or photos.** Use the check script `check.sh` to validate that every internal link exists.
- Pages settings change over time — check the host's current docs before class.
- If the school has no internet, the LAN server is the "live" site for the exhibition.
- **Answer key:** Exit 1: it is only files sent unchanged; no program runs per visitor. 2: on the school LAN (`python -m http.server`, or the Forgejo server). 3: False — static sites need none. Exercise 4: from the first `#` heading in the file (or the `title` in front-matter/nav).

GRADE 10 · MODULE 8 · 1 PERIOD · TIERS 1, 2, 3

8. Automatic checks (CI) — intro

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 1 (by hand), 2–3 (automatic) · **You need:** `practice-repos/g10-school-website` with `check.sh`; Git Bash
>
> **Before this:** Module 7. *Optional module* — skip to Module 9 if time is short.
>
> **By the end you can:** run a check, read its message and exit code, add a check, and explain how CI runs checks automatically on every push or PR.

Hook (5 min)

What if a robot could tell you — before a human reviewer even opens your PR — that a link on your page is broken? It never gets tired, never forgets, and checks *every* change.

Understand it

A check is a program that says pass or fail

A **check script** looks at the project and answers one question: *is this OK?* It prints messages for humans and ends with an **exit code** for computers: **0 = pass**, anything else = **fail**.

Our `check.sh` checks two rules for every `.html` file: it has a `<title>`, and every internal link points to a file that exists.

```
for f in *.html; do
  grep -q "<title>" "$f" || { echo "ERROR: $f has no <title>"; fail=1; }
  ...                                  # for each href: does the file exist?
done
exit $fail                             # 0 if nothing failed
```

CI = run the checks automatically

Continuous Integration (CI) means: every time someone pushes or opens a PR, a server automatically gets the code and runs the checks. The result appears on the PR as a green ✓ or a red ✗.

```
 push / PR  --->  CI server: clone the branch, run check.sh  --->  ✓ pass / ✗ fail on the PR
                                                                          |
                                      reviewers see it; teams can block merging on ✗
```

Why it matters:

- Humans forget to run checks; robots don't.
- Problems are caught **minutes** after they are made, by the person who made them.
- Reviewers can spend their attention on ideas, not broken links.

CI services: GitHub Actions, GitLab CI, Forgejo Actions (on our school server), Woodpecker. They all read a small configuration file in the repository that says "*on push, run these commands*". This book itself has one: `.github/workflows/ci.yml` checks every lesson, link and practice repo on every PR. Grade 12 Module 3 teaches how to write these files.

What checks cannot do

A passing check means "*these rules are met*" — not "*the page is good*". `check.sh` cannot tell you that the text is wrong, the colours are ugly, or the founding year is false. Checks **support** review; they don't replace it.

See it

```
$ bash check.sh
g10 ok
$ echo $?
0                                          <- exit code 0 = pass
  (break the Festival link: href="festivl.html")
$ bash check.sh
ERROR: index.html links to missing festivl.html
$ echo $?
1                                          <- non-zero = fail
```

Unplugged: the robot reviewer (10 min)

One student is the **robot**. They get a written rule card ("*every page must have a title box; every arrow must point to a page that exists*") and check paper "web pages" drawn by the class, saying only PASS or FAIL + the rule broken. Then discuss: what can the robot **not** notice? (bad writing, wrong facts, unkind words).

PRIMM lab (30 min)

1. `bash check.sh` in `g10-school-website`. **Predict** the result. Then `echo $?`.
2. Break a link on purpose (`href="aboot.html"`). **Predict** the message. Run again. Read the message and the exit code.
3. Fix it and commit. On Tier 2/3, the platform can run the script automatically (see Grade 12 for workflows) — if your teacher has enabled CI, push and watch the ✓ or ✗ appear on your PR.
4. **Investigate:** delete the `<title>` from one page. Which rule catches it?
5. **Make:** add one more check — for example, *every* `<img>` *has an* `alt` *attribute*:

   ```
   if grep -o '<img[^>]*>' "$f" | grep -vq 'alt='; then
     echo "ERROR: $f has an image without alt text"; fail=1
   fi
   ```

   Put it inside the loop, test it with a page that breaks the rule, then commit it via a PR.

Parsons practice (5 min)

Order: *change* · *run check* · *read message* · *fix* · *commit*.

Common mistakes

Symptom	Cause	Fix
`bash: check.sh: No such file`	wrong folder	`cd` into the site folder
`$'\r': command not found`	Windows line endings in the script	save as LF in VS Code (status bar *CRLF → LF*); the repo's `.gitattributes` can enforce it
Check passes but the page is broken	the rule doesn't cover it	add a check, *and* review

Symptom	Cause	Fix
CI is red but works on my PC	different OS, missing file not committed	read the CI log; `git status` for uncommitted files

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	GitHub Skills **Hello GitHub Actions** ‹https://github.com/skills/hello-github-actions› (13+, account).	3	45 min	completed course
2	Read this book's `.github/workflows/ci.yml`. List every check it runs.	1	15 min	list
3	Forgejo Actions docs ‹https://forgejo.org/docs/latest/user/actions/›: how is a workflow file for Forgejo similar to GitHub's?	3	15 min	2 similarities
4	Software Carpentry Shell: **Shell Scripts** ‹https://swcarpentry.github.io/shell-novice/06-script.html› — write a tiny script of your own.	1 / 3	30 min	script

Exit ticket (5 min)

1. What does a check script return?
2. Why is a robot reviewer useful?
3. *Misconception:* "If the check passes, the page is perfect." True or false?

Key points

- A check prints messages and returns an **exit code**: 0 pass, non-zero fail.
- **CI** runs checks automatically on every push/PR and shows ✓/✗.
- Checks catch rule-breaking early; humans still review meaning and quality.

New words

check script, **exit code**, **continuous integration (CI)**, **workflow**, **CI log** — see the glossary.

Extension quest

Make `check.sh` also check that every page links back to `index.html`.

Teacher notes

- **Optional unit.** If time is short, skip to the team project.
- **Timing:** hook 5 · understand 10 · robot game 5 · lab 20 · Parsons + exit 5.
- If your Forgejo or GitHub organisation has Actions enabled, add this minimal workflow to the team repos before class so students see real ✓/✗ (GitHub: `.github/workflows/check.yml`; Forgejo: `.forgejo/workflows/check.yml`):

```
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest      # Forgejo: the label of your runner
    steps:
      - uses: actions/checkout@v4
      - run: bash check.sh
```

- **Answer key:** Exit 1: pass/fail as an exit code (0 = pass) plus messages. 2: it checks every change automatically and quickly, so humans can focus on meaning. 3: False — it only checks its rules. Lab 4: the `<title>` rule.

GRADE 10 · MODULE 9 · 3 PERIODS · TIERS 1, 2, 3

9. Team project: Our School Website

> **At a glance**
>
> **Time:** 3 periods over about 3 weeks (+ work between) · **Tiers:** 1–3 · **You need:** a team repository on the class server, the `g10-school-website` starter
>
> **Before this:** Modules 1–8.
>
> **By the end you can:** run a small real project as a team — plan with issues, build with branches and PRs, review every change, release, publish, and reflect.

Hook (5 min)

The principal wants a school website for the festival. You have **3 weeks** and **4 team members**. Last year's team ended up with three "About" pages and a broken menu on the big day. This year you will use everything from this chapter.

Understand it

Team charter

Before writing code, write down how you will work. A **team charter** (`CHARTER.md`, committed in week 1) is the Grade 9 working agreement grown up:

```
# Team charter — Team 3
## Roles (rotate each week)
- Lead: keeps the board up to date, runs the weekly demo
- Reviewer: makes sure every PR gets a review within one day
- Tester: runs check.sh and opens every page before a release
- Writer: README, CHANGELOG, release notes
## Rules
- Every change goes through a PR with one approval from someone else.
- Branches: feature/<issue-number>-short-name, e.g. feature/7-gallery
- PRs say "Closes #N".
- We pull main at the start of every session.
- Meetings: Sunday and Wednesday, 10 minutes after class.
## Privacy
- No student names, photos, marks or phone numbers on the site.
```

Roles are hats, not jobs: *everyone* writes code and reviews; the role says who is responsible for making sure something happens.

The team cycle

```
issue -> branch -> commits -> PR -> CI ✓ -> review -> merge -> (weekly) demo -> (end) release
  ^                                                                              |
  +-------------------- new issues from feedback ------------------------------+
```

Weekly demo

Once a week, each team shows the class (3 minutes): the **board** (what is done, doing, to do), the **graph** (`git log --oneline --graph --all`), and the live site. Others give one compliment and one suggestion — which become new issues.

Retrospective

At the end, the team answers three questions, honestly and without blame: 1. **What went well?** (keep doing) 2. **What was hard?** (change next time) 3. **What will we try next time?** (one concrete experiment) Professional teams do this after every project; it is how teams get better.

Evidence of your contribution

The history is the evidence: commits per person (`git shortlog -s -n`), PRs opened and reviewed, issues closed, review comments written. Every team member must be **visible** in it.

Plan

Week	Goals	Evidence
1	charter; ≥ 10 issues with labels and owners; skeleton pages	`CHARTER.md`, board
2	each member: ≥ 2 PRs merged with review; CI/check passing	PR list
3	release `v1.0.0`, publish, release notes, retrospective, viva	tag, live URL, `RETRO.md`

See it

A healthy team repository at the end of week 2 (summary):

```
$ git shortlog -s -n
     9  Student 12
     8  Student 07
     7  Student 03
     6  Student 21
$ git log --oneline --merges | head -4
c1d0e9a Merge pull request #14 from team-3/feature/12-contact-hours
9ab33f2 Merge pull request #13 from team-3/feature/9-gallery
5e71b08 Merge pull request #11 from team-3/feature/8-menu-phones
2f4c6d1 Merge pull request #10 from team-3/feature/2-about
```

Lab (3 periods)

1. **Issues → plan** (≥ 10 issues), charter, board.
2. Each member works on branches; **every change goes through a PR with one review**.
3. **Weekly demo:** show the board and the graph.
4. **Tag v1.0.0 and publish.** Write release notes.
5. **Retrospective:** `RETRO.md` with the three questions.

Optional enrichment (Django): teams that finish early may build a Django starter app in Git — Django is not confirmed in the CDC syllabus, so it is never required. **Alternative project:** a small Python app (for example a quiz or a unit converter) with the same process.

Parsons practice

Order the team cycle: *issue* · *branch* · *commits* · *PR* · *review* · *merge* · *demo*.

Rubric

Criterion	4	3	2	1
Individual commits (history)	≥ 8, clear messages	5–7	3–4	< 3
PRs authored, reviewed by others	≥ 3	2	1	0
Reviews given (quality of comments)	specific, kind, actionable	mostly	vague	none
Issues linked to PRs	all	most	some	none
Release	tag + notes + changelog	two of these	one	none
Published site works, `check.sh` passes	yes	minor issues	major issues	not published
Viva (3 min): explain one design decision and one review	clear	mostly	partly	unclear

Maximum 28 points; the first three rows are **individual**, the rest are shared by the team.

Common mistakes

Problem	Fix
One person does everything, others "help"	issues assigned to everyone; rubric rows are individual
Huge PR the night before the deadline	small PRs from week 1
PRs wait days for review	reviewer role: review within one day
Merged with failing checks	team rule: no merge on ✗
Blame in the retrospective	talk about the process, not people

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Atlassian, **Comparing workflows** ‹https://www.atlassian.com/git/tutorials/comparing-workflows›: which workflow did your team use?	3	15 min	answer
2	Open Source Guides, **How to Contribute** ‹https://opensource.guide/how-to-contribute/›: which habits would have helped your team?	3	20 min	3 habits
3	*Pro Git* §5.3 **Maintaining a Project** ‹https://git-scm.com/book/en/v2/Distributed-Git-Maintaining-a-Project›: read "Generating a Build" and "The Shortlog". Try `git shortlog v1.0.0..main`.	1 / 3	15 min	output
4	Read another team's `RETRO.md`; write one suggestion for them.	1	10 min	suggestion

Exit ticket

1. What did you learn about working in a team?
2. How can the teacher see your contribution?
3. *Check:* who reviewed your PRs?

Key points

- A **charter** sets roles, rules and privacy before work starts.

- Issue → branch → PR → checks → review → merge; demo weekly; release at the end.
- The history is the evidence of each person's contribution.
- **Retrospectives** turn experience into improvement.

New words

team charter, **role**, **demo**, **retrospective**, **viva** — see the glossary.

Chapter complete. Your team can now ship a real project together. In Grade 11 you will step outside your class: **licences**, reading and contributing to **real open-source projects**, and safer history tools (**rebase, stash, revert**).

Teacher notes

- **Rubric:** individual contribution visible in history (≥ 5 commits each); ≥ 2 PRs reviewed; issues linked; v1.0 tagged; viva (3 min) on one design decision. Ties into the school project work (verify with CDC).
- **Timing:** P1 (week 1): charter + issues. P2 (week 2): demo + review clinic. P3 (week 3): release, retrospective, vivas. Between periods teams need lab access — book it in advance.
- **Viva questions:** "Show me a PR you reviewed — what did you ask for and why?", "Why is this branch named like this?", "What would you do if your PR had a conflict now?"
- **Fairness:** read messages and reviews, not only counts; students with fewer but larger, well-reviewed commits can still score well.
- **Fallback Tier 1:** one shared USB "server" bundle per team, and a teacher-run integration session each week.

Part V. Grade 11 — Open-Source Contributor

Grade 11 — Open-Source Contributor

> **Unpiloted draft**
> Core lessons are written but have not been tried in a classroom. Grade placement of topics is unverified against the CDC syllabus (see `research/decisions.md`).

> **CDC fit (verified)**
> The CDC Grade 11–12 curriculum lists, for Unit 4 Web Development, a group web-app activity "using GitHub for version control". Modules 4–6 and 8 support that activity directly.

Big idea: you can improve software and knowledge that others also use. **Length:** ≈ 20 periods (to calibrate to CDC hours). **Accounts:** Learners 13+ with teacher oversight; contribute only to vetted projects. **Tools:** Git, GitHub/Codeberg/Forgejo, the curriculum repo itself, partner projects with consent.

How this chapter is built

Grades 8–10 happened inside the classroom. Grade 11 steps **outside**: other people's projects, other people's rules, and real open data. It also adds the "sharp tools" (rebase, reset, reflog) once students have the judgement to use them safely.

```
M1  licences               (may I?)
M2  read a project         (how do they work?)
M3  triage                 (what can I do?)
M4  first PR               (contribute)
M5  rebase/stash/reset     (sharp tools, safely)
M6  conventions            (shared habits)
M7  process models         (why teams work so)
M8  data project           (real data)
```

Modules

#	Module	Periods	Tiers	Key ideas
1	Licences and attribution	2	0–3	CC BY, MIT, GPL, TASL credit
2	Reading a project	1	1–3	README, CONTRIBUTING, `git grep`
3	Triage: a good first issue	1	0, 2–3	size, clarity, claiming
4	Your first PR	2	2–3	fork → PR → review
5	rebase, stash, revert, reset	2	0–3	golden rule, reflog
6	Commit and branch conventions	1	0–3	message body, Conventional Commits
7	Process models and Git workflows	2	0–3	waterfall, Scrum, XP
8	Data project	3	1–3	raw/clean, DATA.md

Chapter outcomes

1. explain common open licences and credit sources correctly
2. find an appropriate first issue
3. open a pull request to an open project

4. use stash, rebase, revert and reset with care
5. write clear conventional messages
6. version and document a dataset

Free resources used in this chapter

First Contributions ‹https://firstcontributions.github.io/›, Open Source Guides ‹https://opensource.guide/›, choosealicense.com ‹https://choosealicense.com/›, Creative Commons ‹https://creativecommons.org/licenses/by/4.0/›, SPDX ‹https://spdx.org/licenses/›, Conventional Commits ‹https://www.conventionalcommits.org/en/v1.0.0/›, *Pro Git* ‹https://git-scm.com/book/en/v2› chapters 3, 5–7, Learn Git Branching's rebase and cherry-pick levels, The Turing Way ‹https://book.the-turing-way.org/reproducible-research/vcs› and Software Carpentry's licensing, open-science and citation episodes. Full list: free resource library.

Assessment

Portfolio (`CONTRIBUTIONS.md`), reflection, viva on a PR the student reviewed. First-draft item bank: `assessments/g11-item-bank.md` (to be reviewed after Pilot 1).

Capstone

A merged contribution and a `CONTRIBUTIONS.md` portfolio. Contribution targets: this curriculum repo first; **3–5 partner projects are still to be agreed with maintainers' consent** (FOSS Nepal, NOSK, Open Knowledge Nepal, Code for Nepal are candidates — not yet contacted).

Ready for Grade 12?

A student is ready when they can, without help: credit a CC BY source correctly; open a PR to a project that follows its CONTRIBUTING rules; choose revert vs reset by asking "is it pushed?"; and recover a commit with `reflog`.

GRADE 11 · MODULE 1 · 2 PERIODS · TIERS 0, 1, 2, 3

1. Open-source licences and attribution

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0–3 · **You need:** licence and permission cards (Tier 0), this repository's `LICENSE`, `LICENSE-CONTENT.md` and `ATTRIBUTION.md`
>
> **Before this:** Grades 8–10.
>
> **By the end you can:** say what common open licences allow and require, credit others' work correctly, choose a licence for your own project, and spot licences that cannot be mixed.

> **Not legal advice**
>
> This lesson gives general information so you can read licences sensibly. For a real legal question, ask a qualified person.

Hook (5 min)

You find a great photo of Rara Lake, a poem about Dashain and a Python program that draws charts — all online. Can you use them in your school project? Can you put them on the school website? Can the school *sell* a printed booklet that contains them?

Understand it

Copyright is the default

When someone creates a photo, text or program, they automatically own the **copyright**. Nobody else may copy, change or share it **unless the owner gives permission**. A **licence** is that permission, written down. So:

> **No licence = no permission.** "It's on the internet" does not make it free.

Open licences

An **open licence** gives everyone permission in advance, under some conditions. There are two families:

- **Open-source software licences** — for code: **MIT**, **Apache-2.0**, **GPL-3.0**, BSD, …
- **Creative Commons (CC) licences** — for writing, images, data, lessons: **CC BY**, **CC BY-SA**, **CC BY-NC**, **CC0** …

What each one says

Licence	Copy & share	Change	Use commercially	Conditions
CC0 / public domain	✓	✓	✓	none (credit is polite)
MIT (code)	✓	✓	✓	keep the copyright and licence notice
Apache-2.0 (code)	✓	✓	✓	keep notices, state changes; includes a patent grant
CC BY 4.0	✓	✓	✓	**credit** the author, link the licence, say if you changed it

Licence	Copy & share	Change	Use commercially	Conditions
CC BY-SA 4.0	✓	✓	✓	credit **and share alike**: your changed version must use the same licence
GPL-3.0 (code)	✓	✓	✓	share alike for code: if you distribute a changed program, share its source under GPL
CC BY-NC	✓	✓	✗	credit; **non-commercial only**
CC BY-ND	✓	✗	✓	credit; **no derivatives** (no changes)
"All rights reserved"	✗	✗	✗	ask the owner

The letters combine: **BY** = attribution, **SA** = share alike, **NC** = non-commercial, **ND** = no derivatives.

This book's choice

This book is **CC BY 4.0** for content and **MIT** for code (see `LICENSE-CONTENT.md` and `LICENSE`). Why? CC BY lets the CDC, any school or any publisher print and adapt it — even sell printed copies to cover costs — as long as they credit the authors. That is exactly what a national curriculum resource needs.

Writing an attribution: TASL

A good credit line has **T**itle, **A**uthor, **S**ource (link), **L**icence — and says if you changed it:

> "*Rara Lake at dawn*" by Sita Sharma, from Wikimedia Commons (link), licensed CC BY 4.0. Cropped.

For this book, `ATTRIBUTION.md` gives a ready sentence to copy.

Mixing licences

When you combine works, every licence's conditions apply at once. Some combinations are **impossible**:

- CC BY-**NC** text cannot go into this CC BY book — CC BY allows commercial use, NC forbids it.
- CC BY-**SA** text cannot go in either — SA would force the whole book to become CC BY-SA.
- MIT code can go into a GPL program (GPL's conditions then apply to the whole), but GPL code cannot go into an MIT project.

That is why this book only **links** to excellent resources like *Pro Git* (CC BY-NC-SA) instead of copying them — see the resource library.

Licences in a repository

Put the licence text in a file called `LICENSE` at the top of the repo. Hosts detect it and show it on the project page. For files with different licences, projects add notes, or use the **REUSE** standard with **SPDX identifiers** — short official names like `MIT`, `CC-BY-4.0` (this book's lessons say `license: CC-BY-4.0` in their front-matter).

See it

```
$ head -3 LICENSE
MIT License

Copyright (c) 2026 The git-for-nepal contributors
$ grep -m1 "Attribution 4.0" LICENSE-CONTENT.md
**Creative Commons Attribution 4.0 International (CC BY 4.0)**.
$ grep license: docs/en/grade-11/m01-licences/lesson.md
license: CC-BY-4.0
```

Unplugged: licence card sort (20 min)

Pairs get **licence cards** (`CC0`, `MIT`, `CC BY 4.0`, `CC BY-SA 4.0`, `CC BY-NC-SA`, `GPL-3.0`, "all rights reserved") and **permission cards** (copy, change, sell, must credit, must share alike, must keep notice). Place permission cards under each licence. Then three **scenarios**: (1) put a CC BY photo on the school website, (2) print a CC BY-NC poem in a booklet sold at the school fair, (3) use an MIT chart library in your Grade 12 project. For each: allowed? what must you do?

PRIMM lab (40 min)

1. Read this repo's `LICENSE-CONTENT.md` and `ATTRIBUTION.md`. **Predict** what you must write if you reuse Grade 8 Module 1 in your own worksheet. Write it.
2. Write attribution lines (TASL) for three given items: a photo under CC BY, a MIT library, a poem under CC BY-SA — what changes for SA?
3. **Investigate:** why can't a CC BY-NC-SA text be copied into this repo? (Conflicting terms.) Find two resources in the library marked "link only" and say why.
4. **Investigate:** on choosealicense.com ‹https://choosealicense.com/›, compare MIT, Apache-2.0 and GPL-3.0. What is the main difference?
5. **Make:** choose a licence for your own Grade 11 project (code: MIT or GPL; data/writing: CC BY or CC BY-SA). Add the `LICENSE` file, commit with a message explaining the choice.

Parsons practice (10 min)

Match each licence to what it requires (credit / share alike / no commercial use / keep notice / nothing).

Common mistakes

Mistake	Fix
"No licence shown, so it's free"	no licence = all rights reserved; ask or find another source
Credit without a link or licence name	use TASL
Copying NC or SA text into a CC BY project	link to it instead
Changing the licence of someone else's work	you can't; only the owner can
Inventing a licence ("free for students")	use a standard licence so everyone understands it

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Software Carpentry **Licensing** ‹https://swcarpentry.github.io/git-novice/11-licensing.html› — do its exercise "Can I Use Open License?".	1 / 3	20 min	answer
2	choosealicense.com ‹https://choosealicense.com/›: choose a licence for three imaginary projects (a game, a dataset, a school notes site).	3	15 min	3 choices + reasons
3	Read the human summary of CC BY 4.0 ‹https://creativecommons.org/licenses/by/4.0/›. Explain "no additional restrictions" in your own words.	3	10 min	2 sentences
4	SPDX licence list ‹https://spdx.org/licenses/›: find the identifiers for five licences in this lesson.	3	10 min	5 ids
5	REUSE ‹https://reuse.software/›: read the three steps. How would they help a project with mixed licences?	3	15 min	summary

Exit ticket (5 min)

1. What must you do when reusing CC BY work?
2. What does "share alike" require?
3. *Misconception:* "If it is on the internet, it is free to use." True or false?

Key points

- Copyright is automatic; a **licence** gives permission. No licence = no permission.
- Code: MIT, Apache-2.0, GPL. Content: CC0, CC BY, CC BY-SA, CC BY-NC, CC BY-ND.
- Credit with **TASL**; say if you changed it.
- Some licences cannot be combined; link instead of copying.
- Put a `LICENSE` file in every repo; use SPDX identifiers.

New words

copyright, **licence**, **open licence**, **attribution (TASL)**, **share alike**, **non-commercial**, **SPDX identifier**, **public domain** — see the glossary.

Extension quest

Look up a Nepali government open-data portal or OKN's `localboundaries` repository. What licence does it use? What would you need to write to use it in a school project?

Teacher notes

- **Timing:** P1: hook 5, understand 20, card sort 15. P2: lab 30, Parsons 5, exit 5.
- This is general information, not legal advice; avoid giving legal opinions. Link to the official summaries at creativecommons.org and opensource.org.
- **Card-sort answers:** CC0 — copy, change, sell; MIT — copy, change, sell, keep notice; CC BY — copy, change, sell, credit; CC BY-SA — plus share alike; CC BY-NC-SA — copy, change, credit, share alike, **not** sell; GPL-3.0 — copy, change, sell, keep notice, share source alike; all rights reserved — nothing without asking.
- **Scenarios:** (1) yes, with TASL credit; (2) no — selling is commercial; (3) yes, keep the MIT notice.
- **Answer key:** Exit 1: credit (TASL), link the licence, say if changed. 2: derivatives must use the same (or compatible) licence. 3: False. Lab 4: MIT/Apache permissive (Apache adds patents); GPL requires sharing

changed source under GPL.

GRADE 11 · MODULE 2 · 1 PERIOD · TIERS 1, 2, 3

2. Reading a project: README, CONTRIBUTING, issues

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 1 (clone from bundle), 2–3 · **You need:** a clone of this book's repository (or a teacher-vetted partner project); the treasure-hunt worksheet
>
> **Before this:** Module 1.
>
> **By the end you can:** open an unfamiliar repository and, in 30 minutes, know what it is, how to run it, how to contribute, and what is being worked on.

Hook (5 min)

You join a new club. Before you speak at the first meeting, you read the notice board: rules, who is in charge, what is planned. A repository's notice board is its **README**, **CONTRIBUTING** file, **licence** and **issues**.

Understand it

The standard files

Most healthy open-source projects have the same set of files at the top level. Learn to look for them first:

File	Answers	In this book's repo
`README.md`	What is it? How do I run/build it?	yes
`CONTRIBUTING.md`	How do I help? Branch names, checks, review rules	yes
`LICENSE`	What may I do with it? (Module 1)	`LICENSE`, `LICENSE-CONTENT.md`
`CODE_OF_CONDUCT.md`	How must people behave?	yes
`GOVERNANCE.md`	Who decides? How?	yes
`SECURITY.md`	How to report a security or privacy problem privately	yes
`CHANGELOG.md`	What changed in each version?	yes
`CITATION.cff`	How to cite it in research	yes
`.github/`	issue/PR templates, CI workflows, code owners	yes

A reading strategy (30 minutes)

1. **README** — one sentence: what is this project for?
2. **LICENSE** — may I contribute and reuse?
3. **CONTRIBUTING** — the rules for a first contribution; checks to run.
4. **Folder tree** — where do the main things live? (`ls`, or the server's file view)
5. **Recent history** — `git log --oneline -15`: how do maintainers write messages? What is active?
6. **Issues** — labels such as `good-first-issue`; what is being discussed?
7. **Pull requests** — how are they reviewed? How long do they wait?

8. **Run it** — follow the README's build steps. If they fail, *that* might be your first contribution (improving the README)!

Git commands for orientation

Question	Command
What are the top-level folders?	`ls` / `git ls-files | head`
What changed recently?	`git log --oneline -15`
Who works on what?	`git shortlog -s -n`
Where is a word or function used?	`git grep "word"`
What is the history of one file?	`git log --oneline -- path/to/file`
How big is the project?	`git ls-files | wc -l`

`git grep` searches only tracked files and is much faster than searching the folder by hand.

Is this project healthy?

Signs of a welcoming project: a clear README and CONTRIBUTING, recent commits, issues that get replies, labels for beginners, a code of conduct. Warning signs: no licence, no activity for years, unanswered issues, rude comments. Contribute where you will be welcome.

See it

Orienting in this book's repository:

```
$ git clone https://github.com/rcgiri-physics/git-for-nepal.git
$ cd git-for-nepal
$ ls
ATTRIBUTION.md      CONTRIBUTING.md  README.md     alignment/    offline/
CHANGELOG.md        GOVERNANCE.md    ROADMAP.md    assessments/  practice-repos/
CODE_OF_CONDUCT.md  LICENSE          SECURITY.md   docs/         research/  tools/ ...
$ grep -n "good-first-issue" CONTRIBUTING.md
6:1. Open an issue using a template in `.github/ISSUE_TEMPLATE/`, or pick one labelled `good-first-
issue`.
$ git grep -l "PRIMM" -- docs/en | head -3
docs/en/grade-08/m01-final-final/lesson.md
docs/en/grade-08/m02-snapshots/lesson.md
docs/en/grade-08/m03-first-repo/lesson.md
```

Unplugged: the notice-board hunt (10 min, Tier 0 option)

Print `README.md`, `CONTRIBUTING.md` and the first page of `git log --oneline` of this repository. Pin them on the wall like a notice board. Teams race to answer the worksheet questions using only the printouts.

PRIMM lab (30 min)

1. Open this repo (on the server, or your clone). Answer the 8 questions on the worksheet:
    1. What is the project for, in one sentence?
    2. How do I build the website locally?
    3. Which licence covers the lessons, and which covers the code?
    4. What label marks easy tasks?
    5. Which two checks must pass before a PR?

6. Who must approve a new module?
7. Which label asks for a Nepali reviewer?
8. What was the last thing changed, and by whom?

2. **Predict** where a typo fix should go (branch? issue first?). Check `CONTRIBUTING.md`.
3. Read `git log` for how maintainers write messages. Copy two good examples.
4. **Investigate:** use `git grep` to find every lesson that mentions `bisect`.
5. **Make:** write a 5-line "how to contribute" summary in your own words.

Parsons practice (5 min)

Order the first steps on a new project: *read README* · *read CONTRIBUTING* · *check issues* · *build/run* · *ask*.

Common mistakes

Mistake	Fix
Starting to change code before reading CONTRIBUTING	read it first; it may require an issue first
Ignoring the licence	Module 1: check you may contribute and reuse
Choosing an inactive project	check recent commits and issue replies
Getting lost in a big project	follow the 30-minute strategy; ask in the issue

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Open Source Guides, **How to Contribute** ‹https://opensource.guide/how-to-contribute/› — section "Orienting yourself to a new project". Compare its checklist with ours.	3	20 min	2 differences
2	Apply the 30-minute strategy to First Contributions ‹https://github.com/firstcontributions/first-contributions›. Find its Nepali README.	3	30 min	answers
3	*Pro Git* §7.5 **Searching** ‹https://git-scm.com/book/en/v2/Git-Tools-Searching›: try `git grep -n` and `git log -S "word"`.	1 / 3	15 min	commands + output
4	Pick a Nepali open project (e.g. OKN localboundaries ‹https://github.com/openknowledgenp/localboundaries›). Answer worksheet questions 1, 3 and 8 for it.	3	20 min	answers

Exit ticket (5 min)

1. Which file tells you how to contribute?
2. Where do you look for easy tasks?
3. *Misconception:* "Just send a PR without reading anything."

Key points

- Read the standard files first: README, LICENSE, CONTRIBUTING, CODE_OF_CONDUCT.
- Follow a strategy: purpose → licence → rules → tree → history → issues → PRs → run.
- `git log`, `git shortlog`, `git grep` orient you fast.
- Choose healthy, welcoming projects.

New words

CONTRIBUTING file, **code of conduct**, **governance**, **`git grep`**, **maintainer** — see the glossary.

Extension quest

Draw a map of this repository's folders (like Grade 9 Module 1's tree) with one line describing each folder.

Teacher notes

- **Timing:** hook 5 · understand 10 · lab 30 · Parsons + exit 5. Use the notice-board hunt if computers are limited.
- **Students under 18 should only contribute to projects the teacher has vetted** (this repo and consented partners).
- **Worksheet answers (this repo):** 1 open offline-first Git curriculum for Nepali schools, Grades 8–12; 2 `pip install -r requirements.txt` then `mkdocs serve`; 3 CC BY 4.0 / MIT; 4 `good-first-issue` (and `student-friendly`); 5 `validate_frontmatter.py` and `mkdocs build --strict`; 6 two approvals including one teacher; 7 `needs-nepali-review`; 8 read from `git log -1`.
- **Answer key:** Exit 1: `CONTRIBUTING.md`. 2: issues labelled `good-first-issue` / `student-friendly`. 3: False — read the rules first.

GRADE 11 · MODULE 3 · 1 PERIOD · TIERS 0, 2, 3

3. Triage: finding a good first issue

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 0 (printed issues), 2–3 · **You need:** 10 prepared issues in the curriculum repo (labels `good-first-issue`, `student-friendly`)
>
> **Before this:** Module 2.
>
> **By the end you can:** judge an issue's size and clarity, pick one you can finish, ask a good question, and claim it properly.

Hook (5 min)

A list of 40 issues. Some are one-line typo fixes; some would take a professional a month. Which one can *you* finish in two lessons — and how do you know?

Understand it

What triage means

Triage (from hospitals: deciding who to treat first) means sorting issues: *is it clear? how big? who should do it? how urgent?* Maintainers triage all the time; as a newcomer you triage to **choose well**.

Rate an issue

Question	Signs it's a good first issue
Size — S / M / L?	**S**: one file, a few lines
Clarity — do I know what "done" looks like?	clear steps or a "done when" list
Skill — do I know the tools?	Markdown, text, a small script change
Pointer — does it say *where*?	names the file or line
Free — is anyone else working on it?	no assignee, no recent "I'm on it" comment
Alive — will someone review it?	maintainers replied recently

Labels like `good first issue`, `good-first-issue`, `beginner`, `help wanted` and (in this repo) `student-friendly` are set by maintainers to point newcomers to suitable tasks.

Ask before you start

Always comment before starting work. It avoids two people doing the same thing, and lets the maintainer correct your plan *before* you spend time:

> Hello! I'm a Grade 11 student and would like to work on this. My plan: (1) fix the three spellings in `docs/en/grade-09/m03-branches/lesson.md`, (2) check the glossary for the same words, (3) open a PR that closes this issue. Is that OK?

A good question is **specific** and shows you have read the issue: "*Should the Nepali word go in the glossary table only, or also in the lesson?*" — not "*how do I do this?*".

Wait for a "go ahead" (or a day or two with no objection, if CONTRIBUTING says so).

Quality over quantity

Events like **Hacktoberfest** count pull requests, and some people respond by sending many tiny, useless PRs ("spam PRs"). This wastes maintainers' time and harms the project. Our class rule: **one good, useful contribution is worth more than ten trivial ones** — and we contribute only to listed, vetted projects.

See it

A good claim and a maintainer reply:

```
#23  Fix "Libary" typo and broken link on Grade 10 static-site page   [good-first-issue]

  student-17:  Hi! May I work on this? I'll fix the typo on line 40 and the link
               to cards.md, run check_links.py, and open a PR with "Closes #23".
  maintainer:  Thanks! Go ahead. The link should point to ../../grade-08/cards.md.
               I've assigned it to you.
```

Unplugged: issue rating (15 min)

Print 10 issues (see teacher notes) on cards. Pairs rate each: **size** S/M/L, **clarity** 1–3, **skill needed**, **predicted time**. Then each pair "claims" one issue by writing a claim comment on a sticky note. The teacher plays maintainer and replies.

PRIMM lab (20 min)

1. Teacher prepares 10 issues in the curriculum repo (labels `good-first-issue`, `student-friendly`).
2. Rate each on a sheet; **predict** the time to finish.
3. **Investigate:** filter the issue list by label (most hosts: click the label, or search `label:good-first-issue is:open`).
4. Claim one by commenting. Wait for a maintainer's "go ahead".
5. **Make:** write your plan in 3 steps in the issue.

Parsons practice (5 min)

Order: *read issue* · *ask question* · *comment "I'll work on this"* · *branch* · *PR*.

Common mistakes

Mistake	Fix
Starting work on an issue someone already claimed	read the comments first
Picking the biggest issue to impress	pick one you can finish; then a bigger one
"Can I work on this?" — then silence for weeks	post progress, or say you've stopped so others can take it
Vague questions	quote the line you're unsure about
Opening a PR with no issue in a project that requires one	check CONTRIBUTING

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Open Source Guides, **How to Contribute** ‹https://opensource.guide/how-to-contribute/› — "Finding a project to contribute to" and its checklist. Apply the checklist to one project.	3	20 min	checklist
2	First Contributions ‹https://firstcontributions.github.io/› has a list of projects with beginner-friendly issues. Pick two and rate one issue from each.	3	20 min	ratings
3	Hacktoberfest ‹https://hacktoberfest.com/›: read the current rules about quality and spam. Write our class's 3 rules for events.	3	15 min	3 rules
4	Write a "good first issue" yourself for this book (a typo, a missing glossary word) — with title, what, why, done-when.	2 / 3	15 min	issue

Exit ticket (5 min)

1. What makes an issue "good first"?
2. Why comment before starting?
3. *Misconception:* "Bigger issues are always better."

Key points

- **Triage** = judge size, clarity, skill, pointer, availability, activity.
- Beginner labels point to suitable issues.
- Comment a specific plan before starting; wait for a go-ahead.
- Quality over quantity; no spam PRs.

New words

triage, **good first issue**, **help wanted**, **claim**, **spam PR** — see the glossary.

Extension quest

Find an old, unanswered issue in this repo. Can you add useful information to it (steps to reproduce, a file reference) without solving it? That is triage work too.

Teacher notes

- **Timing:** hook 5 · understand 10 · rating 10 · lab 15 · Parsons + exit 5.
- **Avoid spam:** no one opens PRs for points or events. Hacktoberfest-style events: set rules (quality over quantity, only listed repos).
- **Prepare 10 issues** (mix of sizes): typos in a lesson, a missing glossary word, a broken relative link, a better Nepali-context example, an exercise answer missing from a teacher guide (S); a new Parsons card set (M); a new practice repo (L — to show what *not* to pick first).
- Act as a kind, prompt maintainer during the lab: reply within the lesson.
- **Answer key:** Exit 1: small, clear, says where, labelled, unclaimed, project responsive. 2: avoids duplicate work and gets feedback on the plan. 3: False — finishing a small useful one is better.

GRADE 11 · MODULE 4 · 2 PERIODS · TIERS 2, 3

4. Your first PR — to this curriculum

> **At a glance**
>
> **Time:** 2 periods (+ waiting for review) · **Tiers:** 2 (class copy on the LAN), 3 (the public repository; learners 13+) · **You need:** the issue you claimed in Module 3, Python with `pip install -r requirements.txt`
>
> **Before this:** Modules 1–3; Grade 10 Modules 2–4.
>
> **By the end you can:** contribute a change to a real project from fork to merge, following its rules, and handle review gracefully.

Hook (5 min)

You will improve the very lessons you are learning from. When your PR is merged, your work becomes part of a book that other students in Nepal will read — with your name in the history.

Understand it

The open-source contribution path

You did all of these steps in Grade 10 inside your class. The difference now: **you are a visitor** to someone else's project, so you follow *their* rules.

```
1 fork (server)  ->  2 clone (your PC)  ->  3 branch  ->  4 change
->  5 run checks  ->  6 commit  ->  7 push to your fork
->  8 open PR to upstream  ->  9 respond to review  ->  10 merged!
```

This is the same model used by the well-known First Contributions ‹https://github.com/firstcontributions/first-contributions› practice repository.

Follow the project's rules

For this book, `CONTRIBUTING.md` says:

- branch name `type/short-description` — e.g. `fix/typo-g9-m03`;
- run `python tools/validate_frontmatter.py` and `mkdocs build --strict` before opening the PR;
- use the PR template (what and why; checklist);
- no student names, photos, marks; no text copied from NC/SA sources;
- commit messages: imperative, short first line.

Other projects have other rules — always read theirs (Module 2).

Keep your first PR small

A first PR should be small enough that a maintainer can review it in five minutes: a typo, a broken link, a missing glossary word, a clearer example. Small PRs are merged quickly, and you learn the whole process without getting stuck on a big change.

Review is a conversation

Maintainers may ask for changes. That is normal and good — it is free mentoring from an experienced person. Answer every comment; push fixes to the **same branch**; say thank you. If your PR is closed without merging, ask politely why: maybe the change was already made, or does not fit the project. **A closed PR is not a failure** — you still learned the whole process, and the conversation is public evidence of your work.

Staying in sync

While you wait, `main` upstream may move. If your PR shows a conflict, update your branch exactly as in Grade 10 Module 5:

```
git fetch upstream
git merge upstream/main     # on your PR branch
git push
```

See it

```
$ git clone https://github.com/<you>/git-for-nepal.git
$ cd git-for-nepal
$ git remote add upstream https://github.com/rcgiri-physics/git-for-nepal.git
$ git switch -c fix/typo-g9-m03
  (edit docs/en/grade-09/m03-branches/lesson.md)
$ python tools/validate_frontmatter.py
checked 41 lesson files, 0 error(s)
$ python tools/check_links.py
links checked in ... files, 0 broken
$ git commit -am "Fix spelling of 'separately' in Grade 9 branches lesson"
$ git push -u origin fix/typo-g9-m03
  (open the PR: base rcgiri-physics/git-for-nepal:main  <-  compare <you>:fix/typo-g9-m03)
```

Unplugged rehearsal (10 min)

Before touching a computer, each student writes on paper: the issue number, the branch name, the exact file and line, the commit message, and the PR description (What and why · Type · Checklist). A partner checks it against `CONTRIBUTING.md`.

PRIMM lab (60 min)

1. **Fork** this repo (Tier 3) or the class copy on the LAN (Tier 2).
2. **Clone** your fork; add `upstream`; `git switch -c fix/...` (use the issue's topic).
3. Make **one** change. **Predict:** will the checks pass? Run `python tools/validate_frontmatter.py` and `python tools/check_links.py` (and `mkdocs build --strict` if installed).
4. **Commit** with a clear message; **push**; open a PR using the template, with `Closes #N`.
5. **Investigate:** read the review comments; reply to each; push fixes. Watch the CI checks on the PR.
6. **Make:** after the merge, record it in your `CONTRIBUTIONS.md` portfolio (link, what you changed, what you learned) and add yourself to the contributors list if the project has one.

Parsons practice (5 min)

Order: *fork · clone · branch · change · check · commit · push · PR · respond to review.*

Common mistakes

Symptom	Cause	Fix
PR opened against your own fork's main	wrong base repository	base = upstream `main`, compare = your branch
PR includes unrelated files	committed with `-a` after other edits, or old branch	branch from up-to-date upstream main; review *Files changed*
CI fails on your PR	a check you didn't run	read the CI log, fix, push
No reply for a week	maintainers are busy (often volunteers)	one polite reminder comment; be patient
Arguing with a reviewer	—	ask questions, give reasons, accept the project's decision

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Do the First Contributions ‹https://firstcontributions.github.io/› tutorial (13+, account) — the safe practice PR. Its Nepali README is in the repo's translations folder.	3	45 min	PR link
2	*Pro Git* §6.2 **Contributing to a Project** ‹https://git-scm.com/book/en/v2/GitHub-Contributing-to-a-Project›: read "Advanced Pull Requests" — keeping a PR up to date.	3	20 min	summary
3	Open Source Guides, **How to Contribute** ‹https://opensource.guide/how-to-contribute/› — "What happens after you submit a contribution".	3	15 min	3 possible outcomes
4	Review a classmate's PR to this repo using the Grade 10 checklist.	2 / 3	20 min	review comments

Exit ticket (5 min)

1. What did the review change in your PR?
2. Why keep the first PR small?
3. *Misconception:* "If my PR is closed, I failed."

Key points

- Fork → clone → branch → change → **run checks** → commit → push → PR → review → merge.
- Follow the project's CONTRIBUTING rules exactly.
- Small first PRs; answer every review comment; be patient and polite.
- A closed PR is still learning and evidence.

New words

contribution, **base / compare branch**, **contributor**, **portfolio** — see the glossary.

Extension quest

Make a second contribution that is a little bigger: add an exercise answer to a teacher guide, or improve a Nepali summary (`needs-nepali-review`).

Teacher notes

- **Timing:** P1: hook 5, understand 15, rehearsal 10, lab 1–2 (10). P2: lab 3–6 (35), Parsons + exit 5. Reviews arrive between lessons — check them at the start of Module 5.
- **Maintainers must respond kindly within a week;** add a `student-friendly` label. Agree this with the repository maintainers *before* the lesson.
- CSTA 3B-AP-20 (version control in a group project) is the international precedent for this module — verify wording before citing.
- **Tier 2:** set up a class copy of this repository on the LAN Forgejo with yourself as maintainer; merge good PRs there and forward the best ones upstream yourself.
- **Answer key:** Exit 2: easy to review and merge quickly; you learn the process without getting stuck. Exit 3: False.

GRADE 11 · MODULE 5 · 2 PERIODS · TIERS 0, 1, 2, 3

5. rebase, stash, revert, reset — when to use each

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0 (cards), 1–3 · **You need:** a **throwaway** practice repo (never your real project), commit cards and sticky notes
>
> **Before this:** Grade 8 Module 5 (restore, revert), Grade 9 Modules 3–4, Module 4.
>
> **By the end you can:** park unfinished work, replay a branch on a newer `main`, tidy unpushed commits, choose the safe undo for shared history, and rescue commits you thought were lost.

Hook (5 min)

Your PR is behind `main`, a reviewer asked for a change, you have half-done work in the folder, and your last commit message says `typo fxi`. Which tools fix this safely — and which could destroy a teammate's work?

Understand it

The golden rule

> **Never rewrite history that other people already have.**

Some tools **add** to history (safe anywhere). Others **rewrite** it — they replace commits with new ones. Rewriting is fine on commits that exist only on your computer; on shared, pushed branches it causes confusion and lost work.

Tool	Adds or rewrites?	Use on pushed/shared commits?
`git revert`	adds a new commit	✓ yes — the safe undo
`git stash`	doesn't touch history	✓
`git commit --amend`	rewrites the last commit	only if not pushed
`git reset`	moves the branch label	only if not pushed
`git rebase`	replays commits as new ones	only on your own branch (see below)

`stash` — a drawer for unfinished work

You are halfway through a change and must switch branches (a reviewer needs a quick fix). Commit half-done work? No — **stash** it:

```
git stash            # put uncommitted changes in a drawer; folder is clean again
git switch other-branch ... (do the urgent thing) ... git switch back
git stash pop        # take the changes out of the drawer
git stash list       # see what is in the drawer
```

Use the drawer for minutes or hours, not weeks — stashes are easy to forget.

`revert` — undo by adding (Grade 8 revisited)

Creates a new commit that does the opposite of an old one. History stays true and nobody's copy breaks. **The right undo for anything already pushed.**

`reset` — move the label

`git reset` moves the current branch label to another commit. Three strengths:

Command	Branch label	Staging area	Working files	Use
`git reset --soft HEAD~1`	moves back 1	keeps the changes **staged**	unchanged	redo the last commit (e.g. combine commits)
`git reset HEAD~1` (mixed)	moves back 1	unstages	unchanged	un-commit, keep edits
`git reset --hard HEAD~1`	moves back 1	reset	**reset — edits lost!**	throw away commits and edits

```
before:              c1 -- c2 -- c3 -- c4  [main]
after reset HEAD~2:  c1 -- c2  [main]
                     (c3, c4 still exist, unlabelled; reflog remembers them)
```

`--hard` is the most dangerous everyday command in Git. Check `git status` and `git diff` first.

Small sibling: **`git commit --amend`** replaces the last commit (fix its message or add a forgotten file). Only before pushing.

`rebase` — replay your work on a new base

Your branch started from an old `main`; `main` has moved. In Grade 10 you **merged** `main` into your branch. **Rebase** is the other option: it takes your branch's commits and **replays them one by one on top of the new `main`**, as if you had started today:

```
before:     m1 -- m2 [main]
              \
               B1 -- B2 [feature]

after git rebase main:
             m1 -- m2 [main] -- B1' -- B2' [feature]
```

`B1'` and `B2'` are **new commits** (new ids) with the same changes. Result: a straight, easy-to-read history and no merge commit. Price: history was rewritten. So:

- ✓ rebase **your own** feature branch that nobody else has built on — even if you pushed it for a PR (then push with `git push --force-with-lease`, which refuses if someone else pushed in the meantime);
- ✗ never rebase `main` or any branch others work on.

If a replayed commit conflicts, Git stops: resolve like a merge, `git add`, then `git rebase --continue` (or `git rebase --abort` to give up).

Merge or rebase? Both are fine; teams choose. Merge keeps history exactly as it happened; rebase makes it tidier. Follow the project's CONTRIBUTING rules.

`reflog` — Git's safety net

Commits "lost" by reset or rebase are not deleted at once. **`git reflog`** lists every place `HEAD` has been recently (usually kept 90 days). Find the id and bring it back:

```
git reflog                    # find the commit before the accident
git reset --hard <that-id>    # or: git branch rescue <that-id>
```

Reflog is local — it is your computer's diary, not shared.

Which tool when?

Situation	Tool
Interrupted with half-done work	`stash`
Bad commit already pushed / shared	`revert`
Typo in the last commit message, not pushed	`commit --amend`
Several messy local commits → one clean commit	`reset --soft HEAD~N` then `commit` (or interactive rebase)
My feature branch is behind `main`	`merge main` or `rebase main` (own branch only)
"I reset and lost my commits!"	`reflog`

See it

Real output from a practice repo:

```
$ git status --short
 M b.txt                                      <- half-done work
$ git stash
Saved working directory and index state WIP on feature: 4992393 Extend b
$ git log --oneline --graph main feature
* 4992393 (feature) Extend b
* 24ff680 Add b
| * 547b291 (main) Add m on main
|/
* 5ab9aa8 Add a
$ git rebase main
Successfully rebased and updated refs/heads/feature.
$ git log --oneline --graph main feature
* 3ca532b (HEAD -> feature) Extend b            <- new ids: these are new commits
* 181e7ad Add b
* 547b291 (main) Add m on main
* 5ab9aa8 Add a
$ git stash pop                                <- half-done work is back
$ git reset --hard HEAD~1                      <- oops
HEAD is now at 181e7ad Add b
$ git reflog -3
181e7ad HEAD@{0}: reset: moving to HEAD~1
157ac83 HEAD@{1}: commit: Add b.txt with three lines
181e7ad HEAD@{2}: reset: moving to HEAD~3
$ git reset --hard 157ac83                     <- rescued
```

Unplugged: the card table (20 min)

Lay the commit cards on the table with a `main` sticky note and a `feature` sticky note.

- **revert:** add a new card that undoes an old one. (Nothing moves.)
- **reset:** move the sticky note back two cards. Where do the skipped cards go? (They lie unlabelled — the reflog remembers them.)

- **rebase:** pick up the `feature` cards, *copy* them onto new cards placed after `main`'s newest card, move the `feature` note; the old cards lie unlabelled.
- **stash:** put the working sheet in a drawer.

Now give a second group a copy of the old `feature` cards *before* the rebase, and ask: what happens when they try to add their next card? (Their history no longer matches — that is why we never rebase shared branches.)

PRIMM lab (50 min)

Throwaway repo only: `git init practice-undo`, make 3 commits on `main`, branch `feature` with 2 commits, then 1 more commit on `main`.

1. **Predict** what each command does, and write it down: `git stash`, `git rebase main`, `git reset --soft HEAD~1`, `git revert HEAD`.
2. Run each in turn (stash with some uncommitted edits; then rebase `feature` on `main`; then reset; then revert). After each, run `git log --oneline --graph --all` and compare with your prediction.
3. **Investigate:** compare commit ids before and after the rebase. Why are they different?
4. **Investigate:** `git reset --hard HEAD~2`, then use `git reflog` to find and restore the lost commits.
5. **Modify:** squash your last 3 commits into one with a clean message (`git reset --soft HEAD~3 && git commit -m "…"`). Then try the same with `git rebase -i HEAD~3` (change `pick` to `squash` for the last two lines).
6. **Make:** write your own "which tool when" cheat sheet and keep it in your portfolio.

Parsons practice (5 min)

Match situation → tool: pushed bad commit → revert; unpushed typo commit → reset/amend; branch behind main → rebase or merge; interrupted → stash.

Common mistakes

Symptom	Cause	Fix
`git push` rejected after a rebase	the remote has the old commits	on **your own** PR branch: `git push --force-with-lease`; on shared branches: don't rebase
Teammate's history "goes crazy"	someone rewrote a shared branch	agree a recovery with the teacher; protect `main`
Edits gone after `reset --hard`	`--hard` resets the working files	committed work → `reflog`; uncommitted edits are truly lost
Forgot a stash for weeks	—	`git stash list`; `git stash show -p stash@{0}`
Stuck in the middle of a rebase	conflict	resolve, `git add`, `git rebase --continue`, or `git rebase --abort`
Interactive rebase opened Vim	—	edit, then `:wq`; or set VS Code as editor

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Learn Git Branching: `intro4` Rebase Introduction ‹https://learngitbranching.js.org/?locale=en_US&level=intro4›, `rampup4` Reversing Changes ‹https://learngitbranching.js.org/?locale=en_US&level=rampup4›, `move1` Cherry-pick Intro, `move2` Interactive Rebase Intro.	1 / 3	40 min	levels done
2	*Pro Git* §7.7 **Reset Demystified** ‹https://git-scm.com/book/en/v2/Git-Tools-Reset-Demystified› — the "three trees" explanation. Redraw its main figure with our three places from Grade 8.	3	30 min	drawing
3	*Pro Git* §3.6 **Rebasing** ‹https://git-scm.com/book/en/v2/Git-Branching-Rebasing› — read "The Perils of Rebasing". Explain the golden rule in your own words.	3	20 min	explanation
4	*Pro Git* §7.3 **Stashing and Cleaning** ‹https://git-scm.com/book/en/v2/Git-Tools-Stashing-and-Cleaning›: try `git stash -u` with an untracked file.	1 / 3	15 min	what `-u` does
5	Dangit, Git!? ‹https://dangitgit.com/› — read three rescues; which ones use reflog, reset or amend?	3	15 min	table

Exit ticket (5 min)

1. Why is `reset` risky on a pushed branch?
2. What does `git reflog` do?
3. *Misconception:* "Rebase and merge produce identical history."

Key points

- **Golden rule:** never rewrite history others already have.
- `stash` parks work · `revert` undoes by adding · `reset` moves the label · `rebase` replays commits on a new base · `amend` fixes the last commit.
- `--hard` throws away working edits — check first.
- `reflog` recovers commits lost by reset or rebase.
- Merge vs rebase: both valid; follow the project's rules.

New words

stash, **reset (soft/mixed/hard)**, **amend**, **rebase**, **interactive rebase**, **squash**, **reflog**, **force-with-lease**, **rewrite history** — see the glossary.

Extension quest

Use `git cherry-pick <id>` to copy one commit from another branch onto yours. When would that be better than merging the whole branch?

Teacher notes

- **Timing:** P1: hook 5, understand 20, card table 15. P2: see it 5, lab 30, Parsons + exit 5.
- **Run this lesson on throwaway repos only.** Do not allow `push --force` on shared branches; protect `main` on the class server (Forgejo/GitHub branch protection: block force pushes).
- **Key misconception:** "reset deletes commits." It moves a label; commits remain reachable via reflog until garbage-collected.
- **Fallback Tier 0:** the card table plus the second-group experiment is the whole lesson.

- **Answer key:** Exit 1: others already have those commits; moving the label (and force-pushing) breaks their history and can lose their work. 2: lists where HEAD has been, so you can find and restore "lost" commits. 3: False — rebase creates new commits in a straight line; merge keeps both lines and adds a merge commit. Lab 3: rebased commits are new objects with new parents, so new ids.

GRADE 11 · MODULE 6 · 1 PERIOD · TIERS 0, 1, 2, 3

6. Commit-message and branching conventions

> **At a glance**
>
> **Time:** 1 period · **Tiers:** 0–3 · **You need:** a list of 8 bad commit messages; your repo
>
> **Before this:** Grade 8 Module 2 (message rules), Module 5.
>
> **By the end you can:** write a full commit message (summary, body, references), use Conventional Commits where a project wants them, and agree naming rules with a team.

Hook (5 min)

Which message helps someone in two years: `update` or `Fix wrong date format in report header (#14)`? Imagine searching 3,000 commits for "when did the report date change?".

Understand it

Why conventions?

A **convention** is a shared habit. When everyone writes messages and names branches the same way, the history becomes **searchable, readable and even machine-readable** — tools can build changelogs automatically from it.

Anatomy of a full commit message

```
Fix wrong date format in report header (#14)          <- summary: imperative, ≤ 72 chars
                                                      <- blank line
The header showed dates as MM/DD/YYYY, but Nepali     <- body: WHY, and what was wrong;
school reports use YYYY-MM-DD (and BS dates).            wrap at ~72 characters
Switch to ISO format and add a test for 2083-07-01.

Closes #14                                            <- references (issues, co-authors)
```

Rules of thumb (from Grade 8, grown up):

- **Summary:** imperative mood ("Fix", not "Fixed" or "Fixes") — read it as "*If applied, this commit will … fix wrong date format*".
- **Body:** explain *why* and any trade-offs; the diff already shows *what*.
- **One logical change per commit** — easier to review, revert and bisect (Grade 12).
- Write messages for strangers: the future reader has no memory of today.

Write a multi-line message by running `git commit` without `-m` (an editor opens), or with several `-m` flags: `git commit -m "Summary" -m "Body paragraph"`.

Conventional Commits

Many projects use the Conventional Commits ‹https://www.conventionalcommits.org/en/v1.0.0/› format:

```
<type>(<optional scope>): <summary>

feat(gallery): add photo gallery page
fix(menu): correct link to festival page
docs: explain how to run check.sh
refactor(clean): split cleaning into functions
test: add test for empty student count
chore: update .gitignore
feat!: rename data columns          <- "!" marks a breaking change
```

Type	Means	Version bump (Grade 12)
`feat`	new feature	MINOR
`fix`	bug fix	PATCH
`docs`, `style`, `refactor`, `test`, `chore`	no user-visible change	none
`!` or `BREAKING CHANGE:` in body	breaks compatibility	MAJOR

Use it **if the project uses it**. This book's own repository uses plain imperative messages (see `CONTRIBUTING.md`) — follow each project's rules.

Branch naming

`type/short-topic`, lower-case, hyphens: `feat/photo-gallery`, `fix/menu-link`, `docs/readme-run-steps`. Some teams add the issue number (`fix/14-report-date`) or the author (`sita/report-date`). Pick one style and write it down.

A conventions document

Put the team's choices in `CONVENTIONS.md` (or `CONTRIBUTING.md`): message format, branch names, review rules, merge or rebase. Optionally add a **commit template** so the editor shows the format every time:

```
git config commit.template .gitmessage
```

See it

```
$ git log --oneline -6
a3f9c01 fix(menu): correct link to festival page (#21)
77b2e10 feat(gallery): add photo gallery with alt text (#18)
5c1d9e4 docs: explain how to run check.sh
e02a7b3 feat(contact): add office hours (#12)
9d1e4a7 chore: add .gitignore for editor files
1b8c6f2 feat: first version of the school website
$ git log --oneline --grep="^fix"
a3f9c01 fix(menu): correct link to festival page (#21)
```

Searching history becomes easy when messages follow a convention.

Unplugged: message makeover (15 min)

Pairs rewrite 8 bad messages (with a description of each change) as full messages: summary + body + reference. Then convert each summary to Conventional Commits form. Swap with another pair and check against the rules.

PRIMM lab (25 min)

1. Rewrite 8 bad messages as good ones (teacher list).

2. **Predict** which message style makes `git log --oneline` readable. Compare `git log --oneline` of your Grade 10 team repo with this book's repo.
3. Add a `.gitmessage` template to your repo: write the format as comment lines (`# type(scope): summary`, `# why:`), then `git config commit.template .gitmessage`. Make a commit without `-m` and see it.
4. **Investigate:** use `git log --grep="fix"` and `git log --author="Student 12"` to search history.
5. **Make:** team agreement document `CONVENTIONS.md` (messages, branches, merge vs rebase, review).

Parsons practice (5 min)

Order the parts of a commit message: *summary* · *blank line* · *body (why)* · *issue reference*.

Common mistakes

Mistake	Fix
Summary too long, wraps in `--oneline`	≤ 72 characters; details in the body
Body repeats the diff ("changed line 4")	explain why
Past tense / "this commit fixes…"	imperative: "Fix …"
Mixing conventions in one project	write `CONVENTIONS.md`; follow it
Over-formalising a tiny school repo	readability matters more than rules

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Conventional Commits 1.0.0 ‹https://www.conventionalcommits.org/en/v1.0.0/› — read the summary and the examples; answer its FAQ question about a commit that fits more than one type.	3	20 min	answer
2	*Pro Git* §5.2 **Contributing to a Project** ‹https://git-scm.com/book/en/v2/Distributed-Git-Contributing-to-a-Project› — "Commit Guidelines" (the model message). Compare with ours.	3	15 min	2 differences
3	Software Carpentry **Tracking Changes** ‹https://swcarpentry.github.io/git-novice/04-changes.html› — exercise "Choosing a Commit Message".	1 / 3	10 min	answer
4	Read the last 30 messages of a big open-source project. Which convention does it use?	3	15 min	evidence

Exit ticket (5 min)

1. What is the ideal length of the first line?
2. What should the body explain?
3. *Misconception:* "Messages are only for me."

Key points

- Full message: imperative summary (≤ 72 chars), blank line, body explaining **why**, references.
- Conventional Commits: `type(scope): summary` — readable and machine-usable.
- Branch names: `type/short-topic`.
- Write the team's conventions down; follow each project's own rules.

New words

convention, **imperative mood**, **commit body**, **Conventional Commits**, **commit template**, **breaking change** — see the glossary.

Extension quest

Find a tool that builds a changelog from Conventional Commits messages. What would it produce from the *See it* history?

Teacher notes

- **Timing:** hook 5 · understand 10 · makeover 15 · lab 10 · Parsons + exit 5 (lab steps 3–5 as homework if short).
- **Avoid over-formalising for a school:** the aim is readability, not a rigid standard.
- **Bad messages list:** `update`, `fix bug`, `changes`, `asdf`, `final version`, `did stuff for gallery and fixed menu and readme`, `Fixed the thing Sita said`, `WIP`.
- **Answer key:** Exit 1: about 50, at most 72 characters. 2: why the change was made (and trade-offs). 3: False — they are for every future reader, including tools. Exercise 1: split it into several commits where possible.

GRADE 11 · MODULE 7 · 2 PERIODS · TIERS 0, 1, 2, 3

7. Software process models and Git workflows

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0–3 · **You need:** this book's repository history (or your Grade 10 team repo), chart paper
>
> **Before this:** Grade 10 (issues, PRs, releases), Module 6.
>
> **By the end you can:** describe waterfall, iterative and agile (Scrum, XP) models, and point to where each idea lives in a Git repository.

Hook (5 min)

Your syllabus has "software process models" — usually learned as diagrams for the exam. But you have already *lived* one: the Grade 10 team project had a backlog, weekly demos and a retrospective. Git workflows are the daily practice of process models.

Understand it

What is a process model?

A **software process model** describes the order in which a team does the work of building software: understand requirements, design, build, test, release, maintain. Different models arrange these steps differently.

Waterfall

Each phase finishes before the next starts, like water falling down steps:

```
requirements -> design -> implementation -> testing -> deployment -> maintenance
(months)        (months)   (months)          (months)   (one big release)
```

Good when requirements are fixed and well understood (e.g. building to a government specification), and changes are expensive. Risk: you discover problems very late, and the customer sees the product only at the end.

In Git: long-lived branches, few big merges, one major release tag at the end; changelogs with huge entries.

Iterative and incremental

Build a small working version, then improve it in repeated cycles. Each cycle produces an **increment** — something that works and can be shown.

```
v0.1 -> feedback -> v0.2 -> feedback -> v0.3 -> ... -> v1.0
```

In Git: frequent tags (`v0.1.0`, `v0.2.0` ...), short branches, regular merges into `main`.

Agile: Scrum and XP

Agile is a family of iterative approaches that value working software, collaboration and responding to change (from the *Agile Manifesto*, 2001).

- **Scrum** organises work into **sprints** (1–4 weeks). The **product backlog** lists everything wanted; at **sprint planning** the team picks items; a short **daily stand-up** checks progress; the **sprint review** demonstrates the increment; the **retrospective** improves the process. Roles: product owner, Scrum master, developers.
- **XP (Extreme Programming)** focuses on engineering practices: **pair programming**, **test-driven development**, **continuous integration**, **small releases**, collective code ownership.

Agile does **not** mean "no planning" — it means planning *often*, in small pieces.

Mapping process ideas to Git

Process idea	Git / host artefact
requirement, user story	**issue**
product backlog	open issues, ordered on a **board**
sprint / iteration	**milestone** with a due date
increment	merged PRs + a **release tag**
quality gate / review	**pull request review** + **CI** checks
continuous integration (XP)	merge to `main` often; CI on every push
pair programming (XP)	driver/navigator; `Co-authored-by:` trailer in messages
sprint review / demo	weekly demo of the board and graph (Grade 10)
retrospective	`RETRO.md`
maintenance	issues labelled `bug`, PATCH releases
Gantt chart / schedule	milestones and due dates (some hosts draw roadmaps)

Reading a model from history

A repository's history shows which model a team really follows: many small merged PRs and frequent tags → iterative/agile; long silences then one giant merge → waterfall-like. You will analyse this book's own history in the lab.

See it

```
$ git log --oneline --merges | wc -l              # how many PR merges?
$ git tag --sort=creatordate                      # how often are releases made?
$ git log --format="%ad" --date=format:%Y-%m | sort | uniq -c   # commits per month
```

A team that commits every week and tags every month is working iteratively, whatever its documents say.

Unplugged: two timelines (20 min)

On chart paper draw two timelines of the same 8-week school website project. **Waterfall:** blocks for requirements (2 weeks), design (2), build (2), test (1), release (1). **Agile:** four 2-week sprints, each ending with a demo star and a small release. Mark with a red dot: "*the head teacher changes her mind about the menu in week 5.*" Which timeline copes better? Where is the first moment the head teacher sees something working?

PRIMM lab (50 min)

1. Sort the project's real history (this book's repo, or your Grade 10 team repo: last 20 commits and PRs) onto a timeline. Which model does it resemble? Use the commands in *See it*.

2. **Predict** which model needs more branching; then count branches and merges in the graph (`git log --oneline --graph --all | head -40`).
3. **Investigate:** map each artefact in your Grade 10 team repo to the table above (which issues were "backlog", which tag was the "increment", where the "retrospective" is).
4. **Modify:** re-plan your Grade 10 project as two Scrum sprints: which issues go in sprint 1? Make two milestones on the server and assign issues.
5. **Make:** a one-page comparison table (waterfall vs iterative vs Scrum vs XP) with examples from your Grade 10 team project.

Parsons practice (5 min)

Match term to Git artefact: backlog → issues; increment → tag; review → PR; sprint → milestone; CI → automatic checks.

Common mistakes

Misunderstanding	Better
"Agile means no planning or documents"	planning is frequent and light; documents are kept useful
"Waterfall is always bad"	fits fixed, well-understood, regulated work
"Using GitHub makes us agile"	tools don't decide the model; habits do
"Scrum = daily meetings"	sprints, backlog, review and retrospective matter as much

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Atlassian, **Comparing workflows** ‹https://www.atlassian.com/git/tutorials/comparing-workflows›: match each Git workflow to the process model it suits best.	3	20 min	table
2	*Pro Git* §3.4 **Branching Workflows** ‹https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows›: "long-running branches" vs "topic branches" — which model uses which?	3	15 min	answer
3	**Open Source Guides** ‹https://opensource.guide/› — pick one guide and say how open-source projects plan work without a manager.	3	20 min	5 lines
4	**Paper:** draw the Scrum cycle and label each part with a Git artefact.	0	15 min	diagram

Exit ticket (5 min)

1. Which model uses small frequent releases?
2. What Git feature represents an increment?
3. *Misconception:* "Agile means no planning."

Key points

- **Waterfall:** phases in sequence, one big release. **Iterative:** repeated cycles, increments. **Agile (Scrum, XP):** short sprints, frequent feedback, engineering practices.
- Git artefacts embody process ideas: issues = backlog, milestones = sprints, tags = increments, PR review + CI = quality gates.

- History shows the model a team *really* follows.

New words

process model, **waterfall**, **iterative**, **increment**, **agile**, **Scrum**, **sprint**, **backlog**, **XP**, **stand-up** — see the glossary.

Extension quest

Add a `Co-authored-by: Name <email>` trailer to a commit made in pair programming. How does the host display it?

Teacher notes

- **Timing:** P1: hook 5, understand 25, two timelines 10. P2: lab 35, Parsons + exit 5.
- CDC's Grade 12 syllabus has Unit 7 *Software Development Process* (Waterfall, Agile with Scrum and XP, requirements, Gantt charts, testing). This lesson is a Git-flavoured preview and may be moved to Grade 12. Keep it conceptual; no syllabus text is quoted here.
- Exam link: students can use the mapping table as concrete examples in written answers.
- **Answer key:** Exit 1: iterative/agile. 2: a release tag (with its merged PRs). 3: False — planning happens every sprint.

GRADE 11 · MODULE 8 · 3 PERIODS · TIERS 1, 2, 3

8. Data project: version, clean and document open data

> **At a glance**
>
> **Time:** 3 periods · **Tiers:** 1 (data on USB), 2, 3 · **You need:** `practice-repos/g11-data-thread`, Python; optionally a real open dataset whose licence you have checked
>
> **Before this:** Modules 1 (licences) and 4 (contributing); Grade 9 Modules 6–7.
>
> **By the end you can:** set up a data repository that anyone can trust — raw data untouched, cleaning done by a script, every step in history, and source and licence documented.

Hook (5 min)

Nepal's local-level boundaries were restructured in 2017. If your analysis of ward or municipality data uses an old copy, your answers are wrong — and nobody can tell which copy you used. **Version the data like code.**

Understand it

Why version data?

Data changes: new census rounds, corrections, new boundaries, cleaning fixes. Without history you cannot answer "*which data did this chart use?*", "*who changed this number, and why?*" or "*can I reproduce last year's report?*". Git answers all three — for data files that are text (CSV, JSON, GeoJSON) and of moderate size.

The data rules

1. **Raw stays raw.** Put downloaded data in `raw/` and never edit it by hand. It is your evidence.
2. **Clean by script.** A script (`clean.py`) turns `raw/` into `clean/`. Anyone can re-run it and get the same result — the cleaning is **reproducible**.
3. **Document the source.** `DATA.md` records where the data came from, when, under which licence, and what you changed.
4. **Explain each change.** One cleaning step per commit, with a message saying why; a `CHANGELOG.md` entry per version of the dataset.
5. **No personal data.** Ever. Names, phone numbers, citizenship numbers stay out of repositories.

```
raw/schools.csv  --clean.py-->  clean/schools.csv
  (untouched)      (in Git)       (in Git, or regenerated)
       \                                /
        +---- DATA.md: source, licence, date, changes ----+
```

`DATA.md`: the data card

This repository uses a short format (see `datasets/README.md`):

```
name: Synthetic school table (teaching data)
source_url: written for this project
licence: CC-BY-4.0
redistribution_ok: yes
attribution_text: Git for Nepal contributors, CC BY 4.0
personal_data: none
changes_made: raw/ is never edited; clean/ is produced by clean.py (strip spaces, Title Case, drop rows
without a student count)
```

Add the **date retrieved** and the **version** of the source for real data.

Licences for data

Module 1 applies: **being able to download data does not mean you may redistribute it.** Check the licence on the source page. OKN's `localboundaries` states CC BY 4.0; Open Data Nepal lists a licence per dataset — read it. If a repository has **no stated licence**, do not copy its data; **link** to it instead.

Diffs for data

Because CSV is text, `git diff` shows exactly which rows changed. `git diff --stat` counts them. Useful options: `git diff --word-diff` (changes inside a row), and for large tables, compare with a small script instead. Very large files (hundreds of MB) do not belong in plain Git — see *Version control in any project* for Git LFS and DVC.

See it

```
$ cat raw/schools.csv
school_id,name,district,students
1,  test primary school ,Testpur,310
2,TEST SECONDARY SCHOOL,Testpur,540
3,Test Basic School,Demoganj,
4,test lower secondary school,Demoganj,225
$ python clean.py
3 rows written (1 dropped)
$ cat clean/schools.csv
school_id,name,district,students
1,Test Primary School,Testpur,310
2,Test Secondary School,Testpur,540
4,Test Lower Secondary School,Demoganj,225
$ git add clean/schools.csv
$ git commit -m "Clean school names and drop row without student count" \
             -m "Row 3 has no student count in the source; dropped rather than guessed."
```

Unplugged: the data detective (15 min)

Give pairs a printed "raw" table with problems (extra spaces, mixed capitals, a missing value, a duplicate row) and two different "clean" versions made by two invented students — one documented, one not. Which clean version would you trust for a report to the municipality? Why? What would you need to know?

PRIMM lab (3 periods)

Practice repo: `practice-repos/g11-data-thread` (synthetic starter). **Real data:** choose a dataset from Open Knowledge Nepal `localboundaries` (stated CC BY 4.0) or Open Data Nepal — **verify each licence yourself and record it**; do not copy data whose licence is unclear or missing (e.g. a repo with no stated licence).

1. Fetch the data (Tier 3) or use the teacher's USB copy (Tier 1). Commit it in `raw/` with a message naming the source and date.

2. `raw/` untouched; make `clean/` with a script `clean.py`. **Predict** how many rows the cleaning will change.
3. Run it; check with `git diff --stat` and `git diff` — were you right?
4. Add one cleaning step at a time (e.g. trim spaces → fix capitals → drop empty rows → convert BS dates), **one commit per step**, each message saying why.
5. Fill `DATA.md` (see `datasets/README.md` template). Add a `CHANGELOG.md` entry per dataset version; tag `data-v1.0`.
6. **Investigate:** delete `clean/`, re-run `clean.py`. Is the result identical? (`git status` should show no change — that is reproducibility.)
7. **Make:** open a PR proposing the dataset for the shared `datasets/` folder **only if its licence allows redistribution**.

Parsons practice

Order: *download raw · commit raw · write cleaning script · commit clean · document in DATA.md*.

Rubric

Criterion	3	2	1
Raw data untouched, in `raw/`	yes	edited once, documented	edited
Cleaning by script, reproducible	re-run gives identical output	mostly	manual edits
One step per commit, messages explain why	yes	mostly	no
`DATA.md` complete (source, date, licence, changes, personal data)	all fields	most	missing
Licence checked and respected	yes, with evidence	checked	not checked
Changelog + tag	both	one	neither

Common mistakes

Mistake	Fix
Editing `raw/` "just to fix one typo"	fix it in `clean.py`; raw stays evidence
Cleaning by hand in a spreadsheet	write the step in the script
Excel changes dates/IDs when saving CSV	edit CSV in a text editor or Python
Huge files committed	sample or link; consider Git LFS/DVC
Licence "probably fine"	check and record it, or link only
Personal data in the dataset	remove it before committing; tell the teacher

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	The Turing Way, **Version Control** ‹https://book.the-turing-way.org/reproducible-research/vcs› — read the data versioning part. List two tools for large data.	3	20 min	2 tools

#	Exercise	Tier	Time	Hand in
2	Software Carpentry **Open Science** ‹https://swcarpentry.github.io/git-novice/10-open.html› and **Citation** ‹https://swcarpentry.github.io/git-novice/12-citation.html›: add a `CITATION.cff` to your data repo.	1 / 3	25 min	`CITATION.cff`
3	Perez-Riverol et al., *Ten Simple Rules for Taking Advantage of Git and GitHub* ‹https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004947› — which rules did your project follow?	3	30 min	rule list
4	Browse OKN localboundaries ‹https://github.com/openknowledgenp/localboundaries›: find its licence and how its files are organised.	3	15 min	notes
5	Citation File Format ‹https://citation-file-format.github.io/›: check your `CITATION.cff` with its description of fields.	3	10 min	corrected file

Exit ticket

1. Why keep raw and clean data apart?
2. What goes in `DATA.md`?
3. *Misconception:* "If I can download it, I can redistribute it."

Key points

- Raw stays raw; clean by script; one step per commit; document in `DATA.md`.
- Reproducible = anyone re-runs the script and gets identical output.
- Check and record every licence; link when redistribution is not allowed.
- Never commit personal data.

New words

raw data, **clean data**, **reproducible**, **data card / DATA.md**, **CITATION.cff** — see the glossary.

Chapter complete. You have contributed to a real project and versioned real data. In Grade 12 you will look **inside** Git, design **branching strategies**, build **CI pipelines**, handle **security**, **review AI-written code**, hunt bugs with **blame and bisect**, and run a professional **capstone**.

Teacher notes

- **Teacher checks every licence before class. No personal data. Link-only is always a safe fallback.**
- **Timing:** P1: hook + understand 20, detective 15, lab 1–2. P2: lab 3–5. P3: lab 6–7, peer check with the rubric.
- For Tier 1, prepare the dataset on USB together with its licence page saved as a PDF.
- **Data detective answers:** trust the documented version — it records source, steps and reasons and can be reproduced.
- **Answer key:** Exit 1: raw is evidence; clean can be regenerated and checked against it. 2: name, source, date, licence, redistribution, attribution, personal data, changes made. 3: False.

Part VI. Grade 12 — Engineering with Git

Grade 12 — Engineering with Git

> **Unpiloted draft**
> Core lessons are written but have not been tried in a classroom. Grade placement of topics is unverified against the CDC syllabus (see `research/decisions.md`).

> **CDC fit (verified)**
> CDC Grade 12 Unit 7 *Software Development Process* (Waterfall, Agile: Scrum and XP, requirements, Gantt charts, testing) and Unit 8 *Software Project* (group project, at most four members, 10 practical hours; GitHub named among suggested tools) are the natural home for modules 2, 4 and 8. The capstone here is a longer extension of Unit 8.

Big idea: professional teams ship safely, repeatedly. **Length:** ≈ 20 periods (to calibrate to CDC hours). **Accounts:** Learners 13+; teacher-managed organisations; capstone clients only with written consent. **Tools:** Git internals tools, CI (GitHub Actions / Forgejo Actions), static analysis/check scripts.

How this chapter is built

Grade 12 turns habits into **engineering**: first understanding the machine (internals), then the systems professionals build around it — strategies, automation, releases, security, AI review and debugging — all applied in a six-week capstone.

```
M1  internals             (the machine)
M2  strategies            (team rules)
M3  CI                    (robots)
M4  SemVer                (promises)
M5  security              (safety)
M6  AI diffs              (judgement)
M7  blame/bisect          (detective work)
M8  capstone              (all together)
M9  where next            (beyond school)
```

Modules

#	Module	Periods	Tiers	Key ideas
1	Git from the inside	3	0–3	blob, tree, commit, refs
2	Branching strategies	2	0–3	trunk-based, GitHub flow, Gitflow, protection
3	CI automation	3	1–3	workflows, runners, required checks
4	Releases and SemVer	2	1–3	MAJOR.MINOR.PATCH, changelog
5	Security	2	1–3	secrets, incident checklist, signing, supply chain
6	Reviewing AI-assisted changes	2	0–3	explain every line
7	blame and bisect	2	0–3	binary search, `bisect run`
8	Team capstone	8	1–3	professional process
9	Where Git goes next	3	0–3	pathway map

Chapter outcomes

1. describe Git's object model
2. choose and justify a branching strategy
3. write and read CI workflows
4. version releases with SemVer
5. prevent and respond to secret leaks
6. review AI-assisted diffs responsibly
7. locate a bug with bisect

Free resources used in this chapter

Pro Git ‹https://git-scm.com/book/en/v2› chapters 7 and 10, *Git from the Bottom Up* ‹https://jwiegley.github.io/git-from-the-bottom-up/›, MIT Missing Semester ‹https://missing.csail.mit.edu/2020/version-control/›, Semantic Versioning ‹https://semver.org/›, GitHub/Forgejo Actions docs, pre-commit ‹https://pre-commit.com/›, gitleaks ‹https://github.com/gitleaks/gitleaks›, OpenSSF Scorecard ‹https://scorecard.dev/›, GitHub Skills (Actions, releases, supply chain) and the PLOS *Ten Simple Rules* paper. Full list: free resource library.

Assessment

Capstone rubric, CI green, viva on design decisions. First-draft item bank: `assessments/g12-item-bank.md` (to be reviewed after Pilot 1).

Capstone

A 3–4 person team project for 6+ weeks (CDC's software-project unit caps groups at four). Practice repo: `practice-repos/g12-capstone-starter/`.

After Grade 12

Continue with After Grade 12: the foundation, consolidated and Version control in any project.

GRADE 12 · MODULE 1 · 3 PERIODS · TIERS 0, 1, 2, 3

1. Git from the inside

> **At a glance**
>
> **Time:** 3 periods · **Tiers:** 0 (paper object graph from printed output), 1–3 · **You need:** a terminal, an empty practice folder
>
> **Before this:** Grades 8–11 (you have used every command; now you see how they work).
>
> **By the end you can:** inspect Git's objects with plumbing commands, draw the object graph of a small repository, and explain branches, HEAD and tags as references.

Hook (5 min)

Git feels like magic: it brings back last year's file in a second, knows exactly what changed, and two computers can compare histories without sending everything. Today we open the box. Inside there is no magic — just small files in a folder, named by fingerprints.

Understand it

Content-addressed storage

Git stores everything as **objects** in `.git/objects/`. Each object is named by a **hash** — a 40-character fingerprint (SHA-1; newer Git can use SHA-256) calculated from the object's **content**:

```
"hi\n"  --SHA-1-->  45b983be36b73c0788dc9cbcb76cbb80fc7bb057
```

Three consequences explain much of Git's behaviour:

1. **Same content → same name → stored once.** Ten identical files cost the space of one.
2. **Different content → different name.** Change one letter and the hash changes completely.
3. **Integrity.** If anyone changes an old object, its hash no longer matches its name. And because each commit includes its parent's hash, changing *any* old commit changes every commit after it. History cannot be altered silently — this is why rebase creates *new* commit ids (Grade 11).

The four object types

Object	Stores	Like
blob	the contents of one file (no name, no date)	the text on a page
tree	a folder: list of names → blobs and sub-trees, with file modes	a table of contents
commit	one tree (the snapshot) + parent commit(s) + author, committer, dates + message	a dated, signed photo with a caption
tag (annotated)	a pointer to a commit + tagger, date, message	a label with a note

```
commit e303976 ----------> tree 0d8a474 ----> blob 45b983b  ("hi\n")   named a.txt
  parent: (none)
  "Add a.txt"

commit 7c1f... ----------> tree bee4886 --+--> blob 45b983b   named a.txt
  parent: e303976                         +--> blob 45b983b   named b.txt  <- stored once!
  "Add b.txt with same content"
```

So Grade 8's model was exactly right: **a commit is a whole snapshot** (a tree), and unchanged files are *shared* (the same blob), not copied.

References: branches, HEAD, tags

Objects never change. The things that *move* are **references** — small text files holding a hash:

- a **branch** = `.git/refs/heads/main`, containing one commit hash;
- **HEAD** = `.git/HEAD`, usually `ref: refs/heads/main` (a pointer to a branch); if it holds a commit hash directly, you are in **detached HEAD**;
- a **tag** = `.git/refs/tags/v1.0`;
- **remote-tracking** branches = `.git/refs/remotes/origin/main`.

Committing = write new blobs/trees + a commit object, then update the branch file to the new hash. That is all.

Porcelain and plumbing

Commands you have used (`add`, `commit`, `log`) are **porcelain** — the friendly surface. Underneath are **plumbing** commands that read and write objects directly:

Plumbing	Does
`git hash-object <file>`	compute a file's blob hash
`git cat-file -t <hash>`	object type
`git cat-file -p <hash>`	pretty-print an object
`git ls-tree HEAD`	list a tree
`git rev-parse HEAD`	resolve a name to a hash
`git count-objects -v`	how many objects are stored

Packfiles

Loose objects are compressed (zlib). Later, Git packs many objects into **packfiles** and stores similar versions as **deltas** to save space (`git gc`). This is a storage optimisation; the *model* is still snapshots.

See it

Real output (the blob and tree hashes will be identical on your computer — they depend only on content):

```
$ git init -q demo && cd demo
$ echo hi > a.txt
$ git add a.txt
$ find .git/objects -type f
.git/objects/45/b983be36b73c0788dc9cbcb76cbb80fc7bb057    <- the blob, created by "add"!
$ git cat-file -t 45b983b
blob
$ git cat-file -p 45b983b
hi
$ git commit -qm "Add a.txt"
$ git cat-file -p HEAD
tree 0d8a474fc67971fb3dd7616e26323d3066442555
author Student 07 <s07@example.invalid> 1791692100 +0545
committer Student 07 <s07@example.invalid> 1791692100 +0545

Add a.txt
$ git cat-file -p HEAD^{tree}
100644 blob 45b983be36b73c0788dc9cbcb76cbb80fc7bb057    a.txt
$ cat .git/HEAD
ref: refs/heads/main
$ cat .git/refs/heads/main
e303976a5221b14438483e6b15d11056b060a188
$ echo hi > b.txt && git add b.txt && git commit -qm "Add b.txt with same content"
$ git cat-file -p HEAD^{tree}
100644 blob 45b983be36b73c0788dc9cbcb76cbb80fc7bb057    a.txt
100644 blob 45b983be36b73c0788dc9cbcb76cbb80fc7bb057    b.txt     <- same blob
```

The number `1791692100` is the time in seconds since 1 January 1970 (Unix time); `+0545` is Nepal's time zone.

Unplugged: build the object graph (20 min)

Give pairs a printed version of the *See it* output. Using three colours of cards (blob, tree, commit) and arrows, build the object graph. Then add a third commit on paper that changes `a.txt` to "hello": which cards are new? Which are reused? (New blob, new tree, new commit; `b.txt`'s blob is reused.) Finally, place sticky notes for `main` and `HEAD`.

PRIMM lab (80 min)

1. `git init demo && cd demo`, `echo hi > a.txt`, `git add a.txt`.
2. **Predict:** what is in `.git/objects` now? Look: `find .git/objects -type f`. (Surprise: `add` already wrote the blob.)
3. `git cat-file -t <hash>` and `git cat-file -p <hash>` on the blob. Compare with `git hash-object a.txt`.
4. Commit. Inspect the commit and the tree: `git cat-file -p HEAD`, `git cat-file -p HEAD^{tree}`.
5. **Investigate:** `cat .git/HEAD` and `cat .git/refs/heads/main`. Create a branch and look in `.git/refs/heads/` again. Create an annotated tag and inspect it with `git cat-file -p v0.1`.
6. **Investigate:** add `b.txt` with the same content and commit. How many *new* objects appeared (`git count-objects -v`)? Why?
7. **Investigate:** `git switch --detach HEAD~1` and `cat .git/HEAD`. What changed? Return with `git switch main`.
8. **Make:** draw the object graph for 3 commits and 2 files (one changed, one not), with refs.

Parsons practice (10 min)

Order what `git commit` creates: *blob for each changed file · tree · commit pointing to tree and parent · move branch file*.

Common mistakes

Misunderstanding	Correction
"Git stores differences"	the model stores snapshots; deltas are only a packing optimisation
"A branch contains its commits"	a branch is a file with one hash; commits are reached by following parents
"Deleting a branch deletes the commits"	commits remain until unreachable *and* garbage-collected (reflog helps)
Editing files in `.git` by hand	never in real repos; only read them
"The blob knows its file name"	names live in trees

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	*Pro Git* §10.2 **Git Objects** ‹https://git-scm.com/book/en/v2/Git-Internals-Git-Objects› — build a commit **only with plumbing** (`hash-object -w` , `update-index` , `write-tree` , `commit-tree`).	1 / 3	45 min	your commit's `cat-file -p`
2	*Pro Git* §10.3 **Git References** ‹https://git-scm.com/book/en/v2/Git-Internals-Git-References›: create a branch with `git update-ref` .	1 / 3	15 min	command + result
3	*Git from the Bottom Up* ‹https://jwiegley.github.io/git-from-the-bottom-up/› (CC BY): read "Repository: Directory content tracking" and "Introducing the blob".	3	30 min	summary
4	MIT Missing Semester, **Version Control (Git)** ‹https://missing.csail.mit.edu/2020/version-control/› — watch/read "Data model"; write its pseudocode for a commit in your notebook.	3	40 min	pseudocode
5	*Pro Git* §10.1 **Plumbing and Porcelain** ‹https://git-scm.com/book/en/v2/Git-Internals-Plumbing-and-Porcelain› and §10.4 **Packfiles** ‹https://git-scm.com/book/en/v2/Git-Internals-Packfiles›: run `git gc` on a repo and look in `.git/objects/pack` .	1 / 3	20 min	what you found

Exit ticket (5 min)

1. What is a blob?
2. What does a branch file contain?
3. *Misconception:* "Git stores differences between versions." True or false? (Conceptually it stores snapshots.)

Key points

- Objects are named by the **hash of their content**: identical content stored once; any change is detectable.
- **blob** (contents) · **tree** (folder) · **commit** (tree + parents + metadata + message) · **tag**.
- **References** (branches, HEAD, tags, remote branches) are small files holding hashes; only they move.
- Porcelain commands are built on plumbing commands you can run yourself.

New words

object, **hash / SHA-1**, **blob**, **tree**, **content-addressed**, **reference (ref)**, **plumbing / porcelain**, **packfile**, **detached HEAD** — see the glossary.

Extension quest

Explain, using objects, why `git clone` of a repo with 10 years of history can still be fast, and what `git clone --depth 1` changes.

Teacher notes

- **Timing:** P1: hook 5, understand 25, unplugged 10. P2: see it (live) 10, lab 1–5 (30). P3: lab 6–8 (30), Parsons 5, exit 5.
- Packfiles compress storage — mention only if asked. Tier 0: draw the object graph on paper from a printed `cat-file` listing.
- On Windows, if `core.autocrlf` is on, a file created by Git Bash with LF is still stored with LF, so the `hi` blob hash matches; if students create the file in Notepad, the hash differs (CRLF) — a nice demonstration that the hash depends on every byte.
- **Answer key:** Exit 1: an object holding one file's contents (no name). 2: one commit hash (40 hex characters). 3: False. Lab 6: two new objects (the tree and the commit); the blob is reused. Lab 7: HEAD holds a commit hash instead of `ref: refs/heads/main`.

GRADE 12 · MODULE 2 · 2 PERIODS · TIERS 0, 1, 2, 3

2. Branching strategies

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0–3 · **You need:** chart paper, `practice-repos/g12-capstone-starter`
>
> **Before this:** Grade 10 (PRs), Grade 11 Modules 5–7.
>
> **By the end you can:** describe the main team branching strategies, match them to project types, and write your capstone's strategy with reasons.

Hook (5 min)

A team of 3 building a school website and a team of 300 building a banking app should not work the same way. Neither should a library that must keep fixing version 1 while version 2 is being built. How do teams decide how to use branches?

Understand it

A strategy answers four questions

1. Which branch holds the **truth** (what we release)?
2. Where does **new work** happen, and for how long?
3. How do we **release**, and how do we **fix** a released version?
4. What **rules** protect the important branches?

Trunk-based development

Everyone integrates into one branch (**trunk** = `main`) at least daily. Branches are tiny (hours, a day) or don't exist at all. Unfinished features are merged but hidden behind **feature flags** (an `if` that switches a feature on or off). Strong CI is essential.

```
main: --o--o--o--o--o--o--o--o--o-->   (many small merges per day)
        \/    \/       \/
```

Good for: teams with good tests and CI, web apps deployed often. Fewest conflicts (nothing lives long enough to drift).

Feature branches + pull requests (GitHub flow)

`main` is always releasable. Each feature or fix gets a short branch and a PR with review and CI; merge, deploy, delete the branch. This is what you did in Grades 10–11.

```
main: --o-----------o--------o------->
         \         /  \      /
          o--o--o-'    o--o-'          (one branch per issue, days long)
```

Good for: most small and medium teams, school projects, open-source projects (with forks).

Gitflow

Two long-lived branches — `main` (released versions only, tagged) and `develop` (integration) — plus `feature/*`, `release/*` (stabilising a version) and `hotfix/*` (urgent fix to a released version) branches.

```
main:    --o-----------------o-------o---->   (v1.0)        (v1.1)   (v1.1.1 hotfix)
            \               /       /|
develop:     o--o--o--o--o-'--o--o-'-+---->
               \     /
feature:        o--o
```

Good for: software with planned, versioned releases (installed apps). Costs: more branches, more merges, slower integration. Its own author later said that teams doing continuous delivery should prefer a simpler flow.

Release branches for long support

Libraries or operating systems that support old versions keep **release branches** (`release/1.x`, `release/2.x`) and **cherry-pick** fixes into them (Grade 11 Module 5 extension).

Comparison

	Trunk-based	Feature branches (GitHub flow)	Gitflow
Long-lived branches	`main` only	`main` only	`main` + `develop`
Branch lifetime	hours	days	days–weeks
Releases	continuous	continuous / tagged	planned versions
Conflict risk	lowest	low	higher
Needs strong CI	essential	important	helpful
Team size	any (with discipline)	small–large	medium–large

Rules that protect branches

Hosts let you **protect** important branches: no direct pushes, no force pushes, require a PR with *N* approvals, require CI to pass, require branches to be up to date. A **CODEOWNERS** file names who must review which folders (this book has `.github/CODEOWNERS`). A strategy is only real when these rules enforce it.

See it

Simulating trunk-based with feature flags in the capstone starter:

```
# settings.py
SHOW_DENSITY_REPORT = False      # flag: merged to main, but hidden until ready

# report.py
if SHOW_DENSITY_REPORT:
    print_density_report()
```

```
$ git log --oneline --graph -8
* 4a1e2f0 Turn on density report (flag = True)
* 91b3c77 Add density report behind flag
* 2c8d1a5 Add tests for density calculation
* ...
```

The feature arrived in small, safe merges; switching it on was a one-line change that is easy to revert.

Unplugged: strategy posters (25 min)

Teams of three each get one strategy and chart paper. Draw its history graph for a 4-week project with two features, one release and one urgent fix to the released version. Present in 2 minutes. The class asks: *Where do conflicts happen? How long does a feature wait? How is the urgent fix made?*

PRIMM lab (40 min)

1. Given 3 project descriptions — (a) the school website, updated weekly by 4 students; (b) a mobile app released to an app store monthly; (c) a library used by many schools, where version 1 must still get fixes while version 2 is built — **predict** the best strategy for each and write one reason.
2. Simulate one strategy with the practice repo `g12-capstone-starter` for 10 minutes in your team (two features, one release tag, one hotfix).
3. **Investigate:** count branches, merges and conflicts (`git log --oneline --graph --all`, `git log --merges --oneline | wc -l`).
4. **Investigate:** turn on branch protection for `main` on the class server (require PR + 1 approval + CI). Try to push directly — what happens?
5. **Make:** write the strategy for your capstone in `CONTRIBUTING.md`: the four questions answered, plus the protection rules.

Parsons practice (5 min)

Match strategy to its signature: daily merges → trunk-based; release/hotfix branches → Gitflow; one branch per PR, main always releasable → feature branches; support old versions → release branches.

Common mistakes

Mistake	Better
Choosing Gitflow because it "looks professional"	choose by release style and team size
Feature branches that live for weeks	small PRs; merge `main` in often
Feature flags never removed	delete the flag once the feature is on
Strategy written but not enforced	branch protection + CODEOWNERS
Hotfix made only on `main`, forgotten on `develop`	Gitflow: merge hotfixes into both

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Atlassian, **Comparing workflows** ‹https://www.atlassian.com/git/tutorials/comparing-workflows› — read the Gitflow and forking sections. List two costs of Gitflow.	3	20 min	2 costs
2	*Pro Git* §3.4 **Branching Workflows** ‹https://git-scm.com/book/en/v2/Git-Branching-Branching-Workflows› — draw the "progressive-stability branching" figure as a graph.	3	15 min	drawing
3	Learn Git Branching `move1` Cherry-pick Intro ‹https://learngitbranching.js.org/?locale=en_US&level=move1› — used for back-porting fixes to release branches.	1 / 3	10 min	level done
4	GitHub Skills **Release-based workflow** ‹https://github.com/skills/release-based-workflow› (13+, account).	3	45 min	completed course
5	Read this book's `.github/CODEOWNERS` and `CONTRIBUTING.md`. Which strategy does the book use?	1	10 min	answer + evidence

Exit ticket (5 min)

1. Which strategy suits a team of three?
2. What is a feature flag?
3. *Misconception:* "Gitflow is always the professional choice."

Key points

- A strategy answers: where is truth, where is new work, how to release, how to fix, which rules.
- **Trunk-based** (tiny branches + flags + CI) · **feature branches/GitHub flow** (PR per change) · **Gitflow** (develop/release/hotfix) · **release branches** (old versions).
- Choose by team size, release style and CI strength; justify it.
- Enforce with branch protection and CODEOWNERS.

New words

trunk-based development, **feature flag**, **GitHub flow**, **Gitflow**, **release branch**, **hotfix**, **branch protection**, **CODEOWNERS**, **cherry-pick** — see the glossary.

Extension quest

Design a strategy for a Nepali government open-data portal where data files are updated by 10 district offices. Which strategy, and what protection rules?

Teacher notes

- **Timing:** P1: hook 5, understand 20, posters 15. P2: lab 35, Parsons + exit 5.
- **No single correct answer; grade the reasoning.** Suggested answers: (a) feature branches/GitHub flow; (b) feature branches with release tags, or Gitflow; (c) release branches with cherry-picked fixes.
- Branch protection needs maintainer/admin rights on the class server — set it up yourself if students cannot.
- **Answer key:** Exit 1: feature branches (GitHub flow) or trunk-based — with reasons. 2: a switch in the code that turns a feature on/off, so unfinished work can be merged safely. 3: False — it suits planned versioned releases; it adds overhead elsewhere.

GRADE 12 · MODULE 3 · 3 PERIODS · TIERS 1, 2, 3

3. Automation: CI with GitHub Actions / Forgejo Actions

> **At a glance**
>
> **Time:** 3 periods · **Tiers:** 2 (Forgejo Actions with a runner), 3 (GitHub Actions); Tier 1: run the same steps by hand · **You need:** your capstone repo, `practice-repos/g12-capstone-starter`
>
> **Before this:** Grade 10 Module 8 (check scripts), Module 2.
>
> **By the end you can:** write a CI workflow, read its logs, fix a failing run, and make green CI a rule for merging.

Hook (5 min)

A robot runs your checks on every PR, in a clean computer, in under a minute — and refuses the merge if they fail. This book has one: every lesson you are reading was checked by it.

Understand it

From check script to pipeline

In Grade 10 you ran `check.sh` by hand. **CI (Continuous Integration)** runs such checks automatically. A **workflow** (also called a **pipeline**) is a YAML file in the repository describing:

- **triggers** — *when* to run (`on: push`, `on: pull_request`, on a schedule, by hand);
- **jobs** — groups of work, each on a fresh machine (a **runner**);
- **steps** — commands or reusable **actions** (`actions/checkout` gets the code).

```
# .github/workflows/check.yml  (Forgejo: .forgejo/workflows/check.yml)
name: check
on:
  push:
    branches: [main]
  pull_request:
jobs:
  validate:
    runs-on: ubuntu-latest          # Forgejo: the label of your school's runner
    steps:
      - uses: actions/checkout@v4   # pinned version
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: bash check.sh          # exit code 0 = green, otherwise red
      - run: python -m unittest     # more checks = more steps
```

Every push to a PR triggers a run; the PR shows ✓ or ✗ next to each job, with a link to the **log**.

Why a fresh machine matters

CI starts from a clean computer every time. "Works on my machine" problems — a file you forgot to commit, a library installed only on your PC — fail in CI. That is a feature: CI proves the project builds **from the repository alone**.

Reading a red run

1. Open the failed job; find the **first** red step.
2. Read the last lines of its log — the error is usually there.
3. Reproduce locally: run the same command on your computer.
4. Fix, commit, push — the PR re-runs automatically.

CI as a rule

Combine with branch protection (Module 2): "*require status checks to pass before merging*". Now `main` can only receive changes that passed the checks.

Good CI habits

- **Fast:** under ~5 minutes, or people stop waiting for it.
- **Pinned:** use fixed versions (`actions/checkout@v4`, `python-version: "3.12"`) so runs are repeatable.
- **Safe:** never print secrets; PRs from forks don't get secrets (Module 5).
- **Meaningful:** a check that always passes is noise; a flaky check that fails randomly teaches people to ignore red.
- **Beyond tests:** linters, link checks, data validation, building the website, building the PDF of a book.

CD: continuous delivery/deployment

CD goes one step further: after CI passes on `main`, automatically publish (deploy the website, upload a release). This book's `.github/workflows/pages.yml` builds and deploys the site on every push to `main`.

See it

This book's own CI (`.github/workflows/ci.yml`, shortened):

```
on:
  pull_request:
  push:
    branches: [main]
jobs:
  validate:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - name: Front-matter and alignment
        run: python tools/validate_frontmatter.py
      - name: Link check (relative links)
        run: python tools/check_links.py
      - name: Build site
        run: mkdocs build --strict
      - name: Practice-repo smoke test
        run: bash tools/smoke_practice_repos.sh
```

A failing data check in the capstone starter:

```
$ bash check.sh
bad row: 7,bad row without a value
$ echo $?
1
```

In CI this appears as a red ✗ on the step `run: bash check.sh`, with exactly those lines in its log.

Unplugged: the pipeline relay (15 min)

Students stand in a line as a pipeline: **trigger** (receives a "push" card), **checkout** (copies the paper project), **check 1** (every page has a title), **check 2** (every link exists), **report** (holds up ✓ or ✗). Pass three paper projects through, one with a hidden broken link. Then remove the "checkout" student — what goes wrong? (Checks run on nothing.)

PRIMM lab (90 min)

1. Look at `.github/workflows/ci.yml` in this repo. **Predict** what happens on a PR: which steps, in which order, and what makes it red.
2. In your capstone repo (start from `practice-repos/g12-capstone-starter`), add a workflow running `bash check.sh` on push and pull request. Push; watch the first run.
3. Push a broken change (e.g. add a row `7,bad row without a value` to `data/sample.csv`) on a branch and open a PR. **Investigate** the red run; read the log; reproduce it locally.
4. Fix; see it turn green. Protect `main` so a red run blocks merging.
5. **Modify:** add a second job or step — e.g. a Python test, or a link check for the README.
6. **Make:** add a data-validation step for a CSV (header and row count), e.g. "*`data/sample.csv` has at least 10 rows*".

Tier 1: run the steps by hand in a fresh clone (`git clone . /tmp/ci-test && cd /tmp/ci-test && bash check.sh`) — this imitates the clean machine.

Parsons practice (5 min)

Order: *push · workflow triggers · steps run · pass/fail shown on PR · merge allowed or blocked.*

Common mistakes

Symptom	Cause	Fix
Workflow never runs	file in the wrong folder or wrong `on:`	`.github/workflows/*.yml` (GitHub), `.forgejo/workflows/*.yml` (Forgejo); check triggers
YAML error	indentation (tabs, wrong spaces)	spaces only; validate in VS Code
Passes locally, fails in CI	uncommitted file, missing dependency, Windows vs Linux	commit it; install it in a step; check line endings/paths
`Permission denied: ./check.sh`	file not executable in Git	run with `bash check.sh`, or `git update-index --chmod=+x check.sh`
Forgejo job stays "waiting"	no runner with that label	ask the admin; match `runs-on` to the runner label
Red run ignored and merged	no protection	require status checks

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	GitHub Skills **Hello GitHub Actions** ‹https://github.com/skills/hello-github-actions› (13+, account).	3	45 min	completed course
2	GitHub Actions docs ‹https://docs.github.com/en/actions›: find "workflow syntax" and explain `on`, `jobs`, `runs-on`, `steps` in your own words.	3	20 min	glossary of 4 terms
3	Forgejo Actions docs ‹https://forgejo.org/docs/latest/user/actions/›: what is a runner, and how is one registered?	3	20 min	summary
4	pre-commit ‹https://pre-commit.com/›: run checks *before* each commit on your own PC. Compare with CI: why have both?	1 / 3	30 min	answer
5	Read `tools/smoke_practice_repos.sh` in this repo. What does it test, and why is that useful for a curriculum?	1	15 min	answer

Exit ticket (5 min)

1. What triggers a workflow?
2. Why protect `main`?
3. *Misconception:* "Green CI means no bugs."

Key points

- A **workflow** = triggers + jobs + steps, in YAML in the repo.
- CI runs on a **fresh machine**: it proves the project works from the repo alone.
- Read the first red step's log; reproduce locally; fix; push.
- Require green checks to merge; keep CI fast, pinned and meaningful.
- **CD** publishes automatically after CI passes.

New words

workflow / pipeline, **trigger**, **job**, **step**, **runner**, **action**, **status check**, **continuous delivery (CD)**, **flaky test** — see the glossary.

Extension quest

Add a `workflow_dispatch:` trigger so the workflow can be run by hand, and a `schedule:` trigger that checks external links once a week.

Teacher notes

- **Timing:** P1: hook 5, understand 20, relay 15. P2: see it 5, lab 1–3 (35). P3: lab 4–6 (35), Parsons + exit 5.
- **Tier 0/1:** run `check.sh` by hand; the idea is the same. **Pin action versions.**
- Forgejo Actions needs a runner (`forgejo-runner`) registered on the LAN server; test it before class (`offline/forgejo-lan/` is untested on hardware — budget time).
- GitHub Actions minutes are free for public repositories; private repositories have a monthly quota — check the current limits for your organisation.
- **Answer key:** Exit 1: events such as push, pull_request, schedule, manual dispatch. 2: so only reviewed changes with green checks reach it. 3: False — CI only proves the checks you wrote passed.

GRADE 12 · MODULE 4 · 2 PERIODS · TIERS 1, 2, 3

4. Releases and semantic versioning

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 1–3 · **You need:** your capstone repo with CI from Module 3
>
> **Before this:** Grade 10 Module 6, Grade 11 Module 6 (Conventional Commits).
>
> **By the end you can:** choose the correct next version number, write a changelog, tag and publish a release, and explain the promises a version number makes.

Hook (5 min)

You updated a library from 1.9 to 2.0 and your program broke. Annoying — but the author *warned* you: the **2** means "something changed that may break your code". A version number is a promise. Today you learn to make that promise correctly.

Understand it

Semantic Versioning (SemVer)

A version is **MAJOR.MINOR.PATCH**, e.g. `2.4.1`. The project first defines its **public API** — what users rely on (commands, options, function names, file formats, column names in a dataset). Then:

Change	Bump	Example
incompatible change to the public API (users must change their code/use)	**MAJOR** — `2.4.1 → 3.0.0`	rename a command; remove a column
new, **backwards-compatible** functionality	**MINOR** — `2.4.1 → 2.5.0`	add an option; add a column
backwards-compatible **bug fix**	**PATCH** — `2.4.1 → 2.4.2`	fix a wrong total

Rules worth remembering (from semver.org):

- When MINOR goes up, PATCH resets to 0; when MAJOR goes up, both reset (`1.9.3 → 2.0.0`).
- A released version **must never change** — fix it with a new version.
- `0.y.z` is initial development: anything may change; `1.0.0` defines the stable public API.
- **Pre-releases** have a hyphen: `2.0.0-alpha.1`, `2.0.0-rc.1` — they come *before* `2.0.0`.
- **Build metadata** has a plus: `1.0.0+20261011` — ignored when comparing versions.

From commits to versions

With Conventional Commits (Grade 11): any `feat!` / `BREAKING CHANGE` → MAJOR; else any `feat` → MINOR; else `fix` → PATCH. Tools can then compute the next version and write the changelog automatically.

The release routine

1. All PRs for the release merged; CI green on `main`.
2. Decide the version from the changes since the last tag: `git log --oneline v0.2.1..main`.
3. Move the *Unreleased* changelog entries under the new version and date.

4. Commit: `Release v0.3.0`. Tag: `git tag -a v0.3.0 -m "v0.3.0"`. Push commit and tag.
5. Create the release on the host with notes (and files to download, if any).
6. Announce it.

Many teams let CI do steps 4–5 when a tag is pushed.

Other versioning schemes

CalVer uses dates (`2026.10`, Ubuntu's `24.04`) — good for things released on a schedule. Data releases often use dates too (`census-2021-v2`). SemVer is best when users depend on an API. Choose one and document it.

See it

```
$ git log --oneline v0.2.1..main
9f3a2c1 feat(report): add density per ward
41be0d7 fix(clean): keep leading zeros in ward codes
c07d5e2 docs: explain how to run the report
$ # a feat and a fix, no breaking change -> MINOR: v0.2.1 -> v0.3.0
$ git tag -a v0.3.0 -m "v0.3.0"
$ git push origin main v0.3.0
```

```
## [0.3.0] - 2026-11-20
### Added
- Population density per ward in the report (#31).
### Fixed
- Ward codes such as `007` lost their leading zeros (#29).
```

Unplugged: bump or not? (15 min)

Cards with 10 change descriptions (teacher notes). Teams hold up **MAJOR / MINOR / PATCH / none**. Discuss the disagreements — most come from not agreeing what the public API is.

PRIMM lab (50 min)

1. Given 10 change descriptions, **predict** the version bump for each; compare with another team and with the rules on semver.org ‹https://semver.org/›.
2. In the capstone repo tag `v0.1.0`, then add a feature (`v0.2.0`) and a fix (`v0.2.1`) — each through a PR with green CI.
3. Write `CHANGELOG.md` entries (Keep a Changelog format, *Unreleased* on top).
4. **Investigate:** `git describe --tags` on a commit after `v0.2.1`. What does it say?
5. **Make:** generate release notes with `git log v0.1.0..v0.2.1 --oneline`, then rewrite them for users. Publish the release on the host.

Parsons practice (5 min)

Match change → bump: renamed a command (major), new option (minor), typo fix (patch), README improvement (none).

Common mistakes

Mistake	Fix
Re-tagging a published version to "fix" it	release a new PATCH

Mistake	Fix
Bumping MAJOR for a big but compatible feature	size doesn't matter; compatibility does
Breaking change hidden in a PATCH	users' programs break unexpectedly — MAJOR
Changelog = raw `git log`	write for users: Added/Changed/Fixed
No public API defined	write in the README what users can rely on

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Semantic Versioning 2.0.0 ‹https://semver.org/› — read the specification items 1–11 and the FAQ "What do I do if I accidentally release a backwards incompatible change as a minor version?".	3	30 min	answer
2	Keep a Changelog ‹https://keepachangelog.com/en/1.1.0/› — apply all its principles to your capstone changelog.	3	15 min	changelog
3	Learn Git Branching `mixed5` "Git Describe" ‹https://learngitbranching.js.org/?locale=en_US&level=mixed5›.	1 / 3	10 min	level done
4	GitHub Skills **Release-based workflow** ‹https://github.com/skills/release-based-workflow› (13+, account).	3	45 min	completed course
5	Look at the release history of a tool you use (Git itself, Python, VS Code). Which scheme does it use? Find one MAJOR release and what broke.	3	20 min	notes

Exit ticket (5 min)

1. When do you bump MAJOR?
2. Where does the changelog live?
3. *Misconception:* "The version number is just a date."

Key points

- **MAJOR** breaks the public API · **MINOR** adds compatibly · **PATCH** fixes compatibly.
- Published versions never change; `0.x` is unstable; pre-releases use `-alpha`, `-rc`.
- Conventional Commits can compute the bump.
- Routine: merge → decide version → changelog → tag → release → announce.

New words

semantic versioning (SemVer), **public API**, **breaking change**, **pre-release**, **release candidate (rc)**, **CalVer** — see the glossary.

Extension quest

Add a workflow that runs when a tag `v*` is pushed and creates a release automatically.

Teacher notes

- **Timing:** P1: hook 5, understand 20, cards 15. P2: lab 35, Parsons + exit 5.
- Link to semver.org for the exact rules.

- **10 change cards and answers:** rename `--input` to `--file` (MAJOR) · add `--quiet` option (MINOR) · fix wrong average (PATCH) · README typo (none) · remove a CSV column (MAJOR) · add a CSV column at the end (MINOR — unless users read columns by position: discuss!) · speed up the script, same output (PATCH) · change default output folder (MAJOR) · add Nepali translation of messages (MINOR) · update CI only (none).
- **Answer key:** Exit 1: when a change is incompatible with the public API. 2: `CHANGELOG.md` in the repository (and summarised in release notes). 3: False under SemVer — each part has a meaning (CalVer does use dates).

GRADE 12 · MODULE 5 · 2 PERIODS · TIERS 1, 2, 3

5. Security: secrets and safe habits

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 1–3 · **You need:** a *throwaway* practice repo; **fake** secrets only
>
> **Before this:** Grade 9 Module 6 (`.gitignore`), Grade 12 Modules 1 and 3.
>
> **By the end you can:** keep secrets and private data out of repositories, find a leaked secret in history, respond correctly to a leak, and explain signing and supply-chain basics.

> **Classroom rule**
>
> Use only **fake** secrets like `FAKE-123` . Never type a real password, token or key into a practice repository, chat or AI tool.

Hook (5 min)

A student commits an API key to a public repository. Within minutes, automated bots that scan public repositories have copied it, and someone starts using it — on the student's bill. The student deletes the file and commits again. *Is the key safe now?*

Understand it

History remembers everything

From Module 1: every commit stores a full snapshot, and old objects never change. **Deleting a file in a new commit does not remove it from history** — anyone with a clone can run `git log -p` or `git show <old-commit>:config.py` and read it. And every clone, fork and backup has a copy.

> **A leaked secret is a burned secret.** Revoke it and make a new one — *first*. Cleaning history comes after, and never fully un-leaks it.

What counts as a secret or private data?

Passwords, API keys and tokens, SSH private keys, database connection strings, `.env` files, certificates — and in schools: **students' names with marks, phone numbers, addresses, photos, citizenship or exam numbers**. Private data in a repo is a privacy breach, even in a "private" repository (people leave teams; repos get made public; laptops get lost).

Keeping secrets out

1. **Configuration from the environment.** The code reads `os.environ["API_KEY"]` ; the value lives in a `.env` file on your PC (or in the CI's secret store), never in Git.
2. `.gitignore` `.env` , `*.pem` , `*.key` , local databases. Commit an **example** file without real values: `.env.example` with `API_KEY=put-your-key-here` .
3. **Check before committing:** `git diff --staged` , always.
4. **Scanners:** tools like **gitleaks** scan commits and history for things that look like secrets; run them in a **pre-commit hook** and in CI. Many hosts also scan pushes ("push protection").

5. **CI secrets:** stored in the host's settings (*Secrets*), injected at run time, masked in logs, and **not given to PRs from forks**.

Responding to a leak — the incident checklist

1. **Revoke / rotate** the secret immediately (the service's dashboard) and create a new one.
2. **Remove** it from the code; switch to environment variables.
3. **Check** whether it was used (the service's logs).
4. **Rewrite history if needed** (e.g. `git filter-repo`), force-push, and ask everyone to re-clone — a **teacher/maintainer** task, because it rewrites shared history (Grade 11's golden rule — this is the rare justified exception).
5. **Tell the team** what happened and what changed, without blame; add a scanner so it doesn't happen again.

For private data about people: also inform whoever is responsible (the teacher, the school), following the school's policy.

Signed commits: proving who

Anyone can type any `user.name` and `user.email` (you did: "Student 07"!). Git cannot tell whether a commit really came from the person named. **Signing** a commit (with a GPG or SSH key: `git commit -S`) adds a cryptographic signature; hosts then show **Verified**. A signature proves *who made the commit and that it was not changed* — not that the code is good.

Supply chain: trusting what you depend on

Your project depends on other people's code (libraries, actions, Docker images). Attackers sometimes publish look-alike packages (`requets` instead of `requests`) or take over abandoned ones. Defences:

- add dependencies deliberately; check name, maintainers, activity and licence;
- **pin versions** (`requirements.txt` with exact versions; `actions/checkout@v4` or a full commit hash);
- let a tool alert you about known vulnerabilities (Dependabot, Renovate, `pip-audit`);
- check your project's health with **OpenSSF Scorecard**;
- protect `main` and require review (Module 2) — the best defence against a single bad change.

See it

```
$ git log --oneline
b71c0e2 Remove API key from config        <- "fixed"?
4d9a113 Add config with API key
$ git log -p -S "FAKE-123" --oneline       <- search every diff for the string
b71c0e2 Remove API key from config
-API_KEY = "FAKE-123"
4d9a113 Add config with API key
+API_KEY = "FAKE-123"
$ git show 4d9a113:config.py               <- still readable from history
API_KEY = "FAKE-123"
```

The safe version:

```
# config.py — committed
import os
API_KEY = os.environ.get("API_KEY")        # value comes from the environment
```

```
# .gitignore
.env
```

```
# .env.example — committed, no real values
API_KEY=put-your-key-here
```

Unplugged: where is the secret? (15 min)

Hand out printed "repository snapshots": a commit history with diffs, a `.gitignore`, a CI log, a screenshot of a README. Teams find all the leaks (a key in an old diff, a phone number in test data, a token printed in a CI log, a password in a README example) and write the incident checklist for each.

PRIMM lab (50 min)

1. In a *practice* repo commit a fake `config.py` with `API_KEY = "FAKE-123"`; find it in history with `git log -p -S FAKE-123`.
2. **Predict** whether a new commit that removes it hides it. Remove it, commit, then check with `git show <old-id>:config.py`.
3. Switch to the safe version: environment variable, `.env` in `.gitignore`, `.env.example` committed. Run the program with `API_KEY=FAKE-456 python app.py`.
4. **Investigate (Tier 1+ with the tool installed):** run `gitleaks detect` (or `gitleaks git`, depending on version) on the practice repo. Does it find `FAKE-123`? Read its output.
5. Overview (no setup needed): signed commits prove *who* made a commit. On the host, find a commit marked *Verified* and look at what it shows.
6. **Make:** write an incident checklist for your capstone (revoke → rotate → remove → check use → clean history → tell team) and add it to `SECURITY.md`.

Parsons practice (5 min)

Order the response to a leak: *revoke the key · create a new one · remove from the code · rewrite history if needed · tell the team*.

Common mistakes

Mistake	Fix
Deleting the file and thinking it's gone	it is in history: revoke first
Cleaning history before revoking	attackers already have it — revoke first
"It's a private repo, so it's fine"	private ≠ secret; people and settings change
Secrets printed in CI logs (`echo $TOKEN`)	never print; hosts mask known secrets, not derived ones
Real student data used as "test data"	use synthetic data
Installing a package because a blog said so	check it first; pin versions

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	GitHub Skills **Secure your repository's supply chain** ‹https://github.com/skills/secure-repository-supply-chain› (13+, account).	3	60 min	completed course
2	gitleaks ‹https://github.com/gitleaks/gitleaks› README: how would you run it as a pre-commit hook?	3	20 min	config snippet

#	Exercise	Tier	Time	Hand in
3	*Pro Git* §7.4 **Signing Your Work** ‹https://git-scm.com/book/en/v2/Git-Tools-Signing-Your-Work›: what is the difference between signing a tag and signing a commit?	3	20 min	answer
4	OpenSSF Scorecard ‹https://scorecard.dev/›: look up the checks it runs. Which three would matter most for your capstone?	3	20 min	3 checks + reasons
5	pre-commit ‹https://pre-commit.com/›: set up one hook that blocks committing files named `.env`.	1 / 3	30 min	`.pre-commit-config.yaml`

Exit ticket (5 min)

1. Does deleting a file remove it from history?
2. What belongs in `.gitignore`?
3. *Misconception:* "Private repos mean secrets are safe."

Key points

- History keeps every committed byte; **a leaked secret is a burned secret** — revoke first.
- Secrets live in the environment or CI secret stores; commit only `.env.example`.
- Scan with tools (pre-commit, CI, host push protection); always read `git diff --staged`.
- Signed commits prove who and integrity, not quality.
- Supply chain: choose dependencies carefully, pin versions, get alerts, require review.

New words

secret, **API key / token**, **environment variable**, **revoke / rotate**, **secret scanning**, **signed commit**, **supply chain**, **dependency pinning** — see the glossary.

Extension quest

Find out what `git filter-repo --replace-text` does, and why a maintainer must coordinate with everyone who has a clone before using it.

Teacher notes

- **Timing:** P1: hook 5, understand 25, unplugged 10. P2: see it 5, lab 30, Parsons + exit 5.
- **Use only fake secrets in class. Never ask students to use real credentials.** History rewriting tools (filter-repo) are teacher-demo only.
- `gitleaks` command names changed between versions — check `gitleaks --help` on the lab build before class.
- Connect to the school's own data-protection rules for student records.
- **Answer key:** Exit 1: No — it stays in earlier commits and in every clone. 2: `.env`, keys, local databases, generated files — anything secret or regenerable. 3: False.

GRADE 12 · MODULE 6 · 2 PERIODS · TIERS 0, 1, 2, 3

6. Reviewing AI-assisted changes

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0 (printed diffs), 1–3 · **You need:** three prepared diffs "from an assistant" (teacher notes); no AI tool is required
>
> **Before this:** Grade 10 Module 4 (review), Modules 3 and 5.
>
> **By the end you can:** review AI-generated changes with a specific checklist, keep AI-assisted work in small explainable commits, and write a class AI-use policy.

Hook (5 min)

A tool writes 60 lines for you in 10 seconds. They look tidy and confident. The program runs. Who is responsible if the lines delete a check, copy code under an incompatible licence, or send a password to a website? *You are — your name is on the commit.*

Understand it

AI assistants change the speed, not the responsibility

AI coding assistants can suggest code, tests, documentation and even whole pull requests. They are useful — and they make mistakes in particular ways, often while sounding very sure. The more code arrives at once, the more important review becomes. Everything in this book still applies, with one rule above all:

> **Never commit a change you cannot explain, line by line, in your own words.**

Git is the safety net

- **Commit before you ask.** Then any AI change is just a diff against a known good state; `git restore .` or `git revert` undoes it.
- **Ask for small changes.** One function, one fix. Small diffs can be understood.
- **Read the whole diff** (`git diff`), not just the part you asked for.
- **Run the checks and tests** — and read what they *test*.
- **Commit separately**, with a message in your own words that says what and why.
- **Disclose** AI help where the project or school asks (e.g. a line in the PR description).

Typical problems in AI-generated diffs

Problem	What it looks like	How to catch it
Invented API ("hallucination")	calls a function or option that does not exist (`df.clean_nepali_dates()`)	run it; search the library docs
Unrelated edits	reformats 200 lines, renames variables you didn't ask about	`git diff --stat`; ask for a smaller change
Weakened checks	deletes or loosens a test, a validation or an `if` "to make it pass"	read every `-` line; why was it removed?

Problem	What it looks like	How to catch it
Wrong but plausible logic	off-by-one, wrong rounding, BS/AD date mix-up	test with known answers
Security issues	hard-coded keys, disabled certificate checks, SQL built from strings	Module 5 checklist; scanners
Licence problems	long copied passages from code under an incompatible licence	Grade 11 Module 1; ask where it came from
Private data sent away	you pasted student data or secrets into an online AI tool	never paste private data; use synthetic examples
Outdated practice	uses deprecated commands or old versions	check current docs

The AI-diff review checklist

On top of the Grade 10 checklist:

☐ I can explain **every** changed line.
☐ The diff contains **only** what I asked for.
☐ No test, check or validation was removed or weakened without a reason I agree with.
☐ Every function, option and library it uses **exists** (I checked).
☐ I tested it with at least one case where I **know the right answer**.
☐ No secrets, private data or licence-incompatible copied text.
☐ My commit message is in **my own words**.

Policies

Schools, exam boards, companies and open-source projects set rules for AI use — some forbid it for assessed work, some require disclosure, some allow it freely. A clear **AI-use policy** in a repository (in `CONTRIBUTING.md`) tells everyone the rules. Follow your school's and CDC's guidance; this book does not require any AI tool.

See it

A diff "from an assistant" that was asked to "*make check.sh also accept empty value fields*":

```
--- a/check.sh
+++ b/check.sh
@@ -3,6 +3,4 @@
 f=data/sample.csv
-[ "$(head -1 "$f" | tr -d '\r')" = "id,name,value" ] || { echo "bad header"; exit 1; }
-tail -n +2 "$f" | tr -d '\r' | awk -F, 'NF!=3 || $1 !~ /^[0-9]+$/ || $3 !~ /^[0-9]+$/ {print "bad row:
" $0; bad=1} END {exit bad}'
+tail -n +2 "$f" | awk -F, 'NF<2 {print "bad row: " $0}'
 echo "g12 ok"
```

Review: the header check was **deleted**; the row check no longer **fails** (no `exit`), so CI would stay green on bad data; ID and value validation are gone; `tr -d '\r'` was removed (Windows files will break). The tests still pass — because the checks were weakened. **Request changes.**

Unplugged: spot the defect (20 min)

Print the three prepared diffs (teacher notes). Teams of three: one reads aloud, one marks each line ✓/?/✗, one fills the AI-diff checklist. Then each team explains the defect *in one sentence a Grade 9 student would understand*.

PRIMM lab (50 min)

1. Teacher provides 3 diffs "from an assistant" (one correct, one with an invented function, one that deletes a check) — as patch files to apply with `git apply`, or printed.
2. **Predict** the defect type for each before running.
3. Apply each on a branch; run the checks; mark each line in the review checklist.
4. **Investigate:** for the invented function, find the real library documentation and show it does not exist. For the deleted check, show a bad input that now passes.
5. **Modify:** ask the assistant (or a peer playing "assistant") for a *smaller* change; commit it separately with your own message.
6. **Make:** write the class AI-use policy: what is allowed, what must be disclosed, the explain-every-line rule, what must never be pasted into a tool. Add it to your capstone `CONTRIBUTING.md`.

Parsons practice (5 min)

Order: *ask for a small change · read the whole diff · run checks · explain it aloud · commit in your own words*.

Common mistakes

Mistake	Fix
Accepting a large change because it "works"	ask for small changes; read all of it
Trusting the tool's explanation of its own code	verify against docs and tests
Letting AI write the commit message	write it yourself — it shows you understood
Pasting real data or secrets into a tool	synthetic examples only
"Tests pass, so it's fine"	check whether tests were changed

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	GitHub Skills **Review pull requests** ‹https://github.com/skills/review-pull-requests› (13+) — apply the AI-diff checklist to the course PR as well.	3	45 min	review comments
2	Open Source Guides, **How to Contribute** ‹https://opensource.guide/how-to-contribute/› — what does it say about being respectful of maintainers' time? Connect this to large AI-generated PRs.	3	15 min	3 sentences
3	Search the CONTRIBUTING file of two large open-source projects for "AI" or "generated". What do their policies say?	3	20 min	comparison
4	**Paper:** rewrite three AI-written commit messages ("Updated code to improve functionality") in your own words from the diffs.	0	10 min	messages

Exit ticket (5 min)

1. What is the golden rule for AI-written code?
2. Name two typical problems in AI diffs.
3. *Misconception:* "If tests pass, the AI change is fine."

Key points

- You are responsible for every line you commit, whoever or whatever wrote it.
- Commit before asking; ask for small changes; read the whole diff; run and check tests; commit in your own words.
- Watch for invented APIs, unrelated edits, weakened checks, security and licence problems, and private data.
- Follow and write clear AI-use policies; disclose where required.

New words

AI assistant, **hallucination (invented API)**, **AI-use policy**, **disclosure** — see the glossary.

Extension quest

Use `git log -S "functionName"` to find when a function was introduced, and check whether its first version came with a test.

Teacher notes

- **Timing:** P1: hook 5, understand 20, spot the defect 15. P2: see it 5, lab 30, Parsons + exit 5.
- Policy is set by school and CDC guidance; this lesson does not require any specific AI tool. Offline classes can use pre-made diffs.
- **The three prepared diffs** (make them on the capstone starter): (1) correct — adds a row-count check with a clear message; (2) invented function — a Python change calling `csv.read_clean()` (no such function in the standard library); (3) deleted check — the *See it* diff.
- **Answer key:** Exit 1: never commit what you cannot explain. 2: any two from the table. 3: False — tests may have been weakened or may not cover the change.

GRADE 12 · MODULE 7 · 2 PERIODS · TIERS 0, 1, 2, 3

7. Debugging history: blame and bisect

> **At a glance**
>
> **Time:** 2 periods · **Tiers:** 0 (numbered-card binary search), 1–3 · **You need:** `practice-repos/g12-capstone-starter/make-bisect-demo.sh`
>
> **Before this:** Module 1; Grade 9 Module 6 (`git log -p`).
>
> **By the end you can:** use `git blame` to find the commit and reason behind any line, and `git bisect` to find the commit that introduced a bug among hundreds — by hand or automatically.

Hook (5 min)

The data check worked last month. Now it fails. There are 40 commits in between, by four people. Reading all 40 diffs takes an hour. Can Git find the guilty commit in six steps?

Understand it

`git blame`: who can I ask?

`git blame <file>` shows, for **every line**, the commit that last changed it, the author and the date:

```
$ git blame data/sample.csv
^a346ca2 (Teacher 2026-10-11 01:09:37 +0545  1) id,name,value
^a346ca2 (Teacher 2026-10-11 01:09:37 +0545  2) 1,alpha,10
77330af1 (Teacher 2026-10-11 01:09:37 +0545  3) 2,row2,20
...
47496f8b (Teacher 2026-10-11 01:09:38 +0545  8) 7,bad row without a value
```

(`^` marks lines from the very first commit.) Then `git show 47496f8` gives the full commit: its message, its whole diff, and often an issue number — **the reason** for the line.

The name is unfortunate: blame is for finding **context and the person to ask**, not for finding fault. Good teams talk about "annotate" (VS Code and many hosts show the same information as *annotations* or "blame view").

Useful options: `git blame -L 10,20 file` (only some lines), `git blame -w` (ignore whitespace changes), and `git log -L 10,20:file` (the full history of those lines).

`git bisect`: binary search through history

If you know one commit where things were **good** and one where they are **bad**, the first bad commit is somewhere between. **Bisect** tests the commit in the **middle**: if it is bad, the culprit is in the first half; if good, in the second half. Each test halves the remaining commits.

```
good                                    bad
 c1  c2  c3  c4  c5  c6  c7  c8  c9  c10 c11 c12
                     ^ test c6: good  -> culprit in c7..c12
                                 ^ test c9: bad  -> culprit in c7..c9
                         ^ test c8: bad -> c7..c8
                     ^ test c7: bad -> first bad commit = c7
```

For *n* commits you need about $\lceil \log_2 n \rceil$ tests: 12 commits → 4, 40 → 6, 1,000 → 10, 1,000,000 → 20. That is the power of logarithms (Grade 11–12 mathematics) applied to debugging.

The commands:

```
git bisect start
git bisect bad                 # the current commit is bad
git bisect good <old-id>       # this old commit was good
#   Git checks out a middle commit. Test it, then say:
git bisect good                # or: git bisect bad
#   ... repeat until Git prints "<id> is the first bad commit"
git bisect reset               # return to where you started
```

Automatic bisect

If a script can decide good/bad by its **exit code** (0 = good, 1–127 = bad, 125 = "can't test, skip"), Git can do the whole search alone:

```
git bisect run bash check.sh
```

This is why **small commits** and **automatic checks** (Module 3) matter: bisect points to one small commit you can understand at once. A giant commit "Update everything" is found just as fast, but tells you little.

See it

Real output on the 12-commit demo repository (row 7 is the planted bug):

```
$ git bisect start
$ git bisect bad
$ git bisect good a346ca2
Bisecting: 5 revisions left to test after this (roughly 3 steps)
[35e5fc0...] Add row 6
$ git bisect run bash check.sh
running 'bash' 'check.sh'
g12 ok
Bisecting: 2 revisions left to test after this (roughly 2 steps)
[7148b98...] Add row 9
running 'bash' 'check.sh'
bad row: 7,bad row without a value
Bisecting: 0 revisions left to test after this (roughly 1 step)
[a1c604e...] Add row 8
running 'bash' 'check.sh'
bad row: 7,bad row without a value
Bisecting: 0 revisions left to test after this (roughly 0 steps)
[47496f8...] Add row 7
running 'bash' 'check.sh'
bad row: 7,bad row without a value
47496f8b253518e56e84a38519fa910f3c90e509 is the first bad commit
commit 47496f8b253518e56e84a38519fa910f3c90e509
    Add row 7
 data/sample.csv | 1 +
$ git bisect reset
```

Four tests found the culprit among 11 candidates.

Unplugged: guess the number (15 min)

One student thinks of a commit number from 1 to 64 (written secretly: "the first bad commit"). The class may only ask "is commit *k* good or bad?" (answer: good if *k* is before the secret number). Count questions. Compare a strategy that tests commits in order (1, 2, 3 …) with one that always tests the middle. Then: how many questions for 1,000 commits?

PRIMM lab (50 min)

Build the practice repo: `bash practice-repos/g12-capstone-starter/make-bisect-demo.sh bisect-demo && cd bisect-demo`. It has 12 commits and a planted bug.

1. `bash check.sh` — it fails. `git blame data/sample.csv` — read the author, date and commit of the bad line. `git show` that commit.
2. **Predict:** how many tests will bisect need for 12 commits?
3. `git bisect start`, `git bisect bad`, `git bisect good <first-commit>` (find it with `git log --oneline | tail -1`).
4. Run `bash check.sh` at each step, answering `git bisect good` / `git bisect bad`. Count the steps.
5. **Investigate:** which commit is guilty? Does it match blame? Why might blame and bisect give *different* answers in a real project? (Blame shows the last change to a line; a bug can come from a change to *another* line or file.) `git bisect reset`.
6. **Make:** automate with `git bisect run bash check.sh`. Then write a tiny test script for a different bug in your capstone and bisect with it.

Parsons practice (5 min)

Order: *start* · *mark bad* · *mark good* · *test* · *answer* · *repeat* · *reset*.

Common mistakes

Problem	Fix
Forgot `git bisect reset`; now in detached HEAD	`git bisect reset`
Marked a commit wrongly	`git bisect log` to see answers; `git bisect replay` after editing, or start again
A middle commit can't be tested (doesn't build)	`git bisect skip` (or exit code 125 in a script)
The test script is not in old commits	keep the script *outside* the repo, or copy it in before running
Using blame to shame	use it to find context and who to ask

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	*Pro Git* §7.10 **Debugging with Git** ‹https://git-scm.com/book/en/v2/Git-Tools-Debugging-with-Git›: try `git blame -C` and `git bisect run`.	1 / 3	25 min	outputs
2	*Pro Git* §7.5 **Searching** ‹https://git-scm.com/book/en/v2/Git-Tools-Searching›: `git log -S` ("pickaxe") and `git log -L`. Find when `bad row` text first appeared in `check.sh`.	1 / 3	15 min	commit id
3	VS Code: open a file and turn on the blame/annotation view (Timeline or the Source Control *Blame* setting, depending on your version). Compare with `git blame`.	1	10 min	screenshot or notes
4	**Maths link:** complete a table of n vs $\lceil \log_2 n \rceil$ for n = 8, 50, 100, 1,000, 1,000,000.	0	10 min	table

Exit ticket (5 min)

1. How many tests for 64 commits?
2. What does blame show?

3. *Misconception:* "Blame is for punishing people."

Key points

- `git blame` → for every line: last commit, author, date; then `git show` for the reason.
- `git bisect` → binary search between a good and a bad commit; $\lceil \log_2 n \rceil$ tests.
- `git bisect run <script>` automates it using exit codes (0 good, 125 skip, other bad).
- Small commits and automatic checks make both tools powerful.

New words

blame / annotate, **bisect**, **binary search**, **first bad commit**, **pickaxe (`log -S`)** — see the glossary.

Extension quest

Use `git bisect` with the terms `old` / `new` instead of `good` / `bad` (`git bisect start --term-old=without --term-new=with`) to find when a *feature* (not a bug) first appeared.

Teacher notes

- **Timing:** P1: hook 5, understand 20, guess-the-number 15. P2: lab 35, Parsons + exit 5.
- **Frame `blame` as "who can I ask?", not "who is at fault".** Math link: logarithms.
- Build the demo repo on each PC before class, or bundle it.
- **Answer key:** Exit 1: 6 ($\log_2 64 = 6$). 2: for each line, the last commit, its author and date. 3: False — it finds context and the person to ask. Lab 2: about 4 ($\lceil \log_2 11 \rceil$). Exercise 4: 3, 6, 7, 10, 20.

GRADE 12 · MODULE 8 · 8 PERIODS · TIERS 1, 2, 3

8. Team capstone

> **At a glance**
>
> **Time:** 8 periods across 6+ weeks · **Tiers:** 1–3 · **You need:** a team repository with CI on the class server; `practice-repos/g12-capstone-starter`
>
> **Before this:** Grade 12 Modules 1–7 and everything before.
>
> **By the end you can:** run a professional-style project from charter to second release, and explain and defend its design and process.

Hook (5 min)

A client — a local organisation, with written consent — needs a small tool: a validated data repository for their records, a simple web page, a translation of documentation. Your team of 3–4 (CDC's software-project unit allows at most four members) has 6+ weeks. Everything you learned since Grade 8 comes together here.

Understand it

What makes a project "professional"?

Not size — **process**. A small project is professional when anyone can see *what* is being built (issues), *why* each change was made (PRs, messages), *that* it works (CI), *which* version is which (releases), *how* to use and contribute (docs), and *that* it is safe (security review, no private data).

The planning documents (week 1)

Document	Contents	Built on
`CHARTER.md`	goal, client, team roles (rotating), meetings, communication	G10 M9
`CONTRIBUTING.md`	branching strategy, review rule, message convention, AI-use policy	G12 M2, G11 M6, G12 M6
Definition of done	e.g. *merged via reviewed PR · CI green · docs updated · changelog entry · no secrets*	G10 M1
Risk list	what could go wrong and what you'll do (a member ill, power cuts, client unavailable, data licence unclear)	G11 M7
Issues + milestones	≥ 20 issues, milestones per release	G10 M1

Project options

- **(a) Data repository** with a data card (`DATA.md`, `CITATION.cff`), cleaning scripts, and CI that validates the CSV on every PR — builds directly on Grade 11 Module 8.
- **(b) Web or Python app** — links to the CDC OOP/programming syllabus; tests in CI.
- **(c) Documentation or translation** contribution to a partner project — with the maintainers' consent and their review process.

Requirements (all options)

≥ 20 issues · PRs with reviews (every member authors and reviews) · CI green on `main` · branch protection on `main` · ≥ 2 tagged releases following SemVer · `README`, `CONTRIBUTING`, `CHANGELOG`, `LICENSE`, `SECURITY.md` · no secrets, no personal data.

Milestones

Week	Milestone	Evidence
1	plan: charter, contributing, risks, ≥ 20 issues	documents, board
2	skeleton + CI + protection	first green run, protection settings
3–4	features (first release `v0.1.0` at end of week 3)	PRs, tag, release notes
5	hardening + **security review** (Module 5 checklist, gitleaks, dependency check) + **bug bash** (everyone tries to break it; bisect any regression)	review issue, fixes
6	release `v1.0.0` + client demo + viva + retrospective	tag, demo, `RETRO.md`

The viva

Each member explains, in about 5 minutes: one **design decision** (and an alternative you rejected), one **review** you gave (and what changed because of it), one **problem** you solved with Git (conflict, revert, bisect …), and how the **history** shows your work.

See it

A capstone repository at release time:

```
$ git tag --sort=creatordate
v0.1.0
v0.2.0
v1.0.0
$ git log --oneline --merges v0.2.0..v1.0.0 | wc -l
14                                  <- 14 reviewed PRs in the last release
$ git shortlog -s -n v0.1.0..v1.0.0
    23  Student 12
    19  Student 03
    18  Student 21
    16  Student 07
$ ls
CHANGELOG.md  CITATION.cff     DATA.md  README.md  SECURITY.md  clean.py  raw/
CHARTER.md    CONTRIBUTING.md  LICENSE  RETRO.md   check.sh     data/
```

Weekly cycle

```
Monday: plan issues for the week (board)
  -> branches -> PRs -> review -> CI -> merge (every day)
Friday: 5-minute demo to the class/teacher; update risks
```

Lab

1. Week 1: write the planning documents; open ≥ 20 issues; set milestones.
2. Week 2: build the skeleton from `practice-repos/g12-capstone-starter`; add CI (Module 3); protect `main` (Module 2).
3. Weeks 3–4: features via PRs; release `v0.1.0` (Module 4).

4. Week 5: security review and bug bash; use blame/bisect for any regression (Modules 5, 7).
5. Week 6: release `v1.0.0`; demo; viva; retrospective.

Parsons practice

Order the weekly cycle: *plan issues* · *branch* · *PR* · *review* · *CI* · *merge* · *demo*.

Rubric (100 points)

Area	Points	Look for
Repository quality	25	structure, README/CONTRIBUTING/CHANGELOG/LICENSE, clean history, conventions followed
CI and tests	15	meaningful checks, green `main`, protection on
Review quality	15	every member reviews; specific, kind, actionable comments that changed code
Release practice	10	≥ 2 SemVer releases with notes
Security	10	no secrets/PII, `.gitignore`, scanner run, `SECURITY.md`, dependencies pinned
Documentation	10	a stranger can run and contribute using the docs alone
Viva	15	individual explanation of a decision, a review and a Git problem

Common mistakes

Problem	Fix
Plan written, then ignored	update the board every Monday; issues for all work
Everything merged in week 6	release `v0.1.0` in week 3 — it forces integration early
One member is the "Git person"	every member authors and reviews PRs (rubric)
CI added at the end	week 2
Client's real personal data used	synthetic or anonymised data only, agreed in writing

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	Perez-Riverol et al., *Ten Simple Rules for Taking Advantage of Git and GitHub* ‹https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004947›: score your project against the ten rules.	3	30 min	score sheet
2	Open Source Guides ‹https://opensource.guide/› — "Starting an Open Source Project": which items does your repository still miss?	3	20 min	list
3	OpenSSF Scorecard ‹https://scorecard.dev/› checks: apply them by hand to your repo.	3	30 min	checklist
4	Software Carpentry **Citation** ‹https://swcarpentry.github.io/git-novice/12-citation.html› — make sure your `CITATION.cff` is valid.	1 / 3	15 min	file

Exit ticket

1. What was your hardest decision and why?
2. How does the history show each member's work?
3. *Check:* what would you do differently?

Key points

- Professional = visible process: issues, PRs, reviews, CI, releases, docs, security.
- Plan in week 1, CI in week 2, release early (week 3), harden in week 5, release and reflect in week 6.
- Every member authors, reviews and can explain their work.

Teacher notes

- **Rubric:** repo quality (25) · CI and tests (15) · review quality (15) · release practice (10) · security (10) · documentation (10) · viva (15). Fits CDC Grade 12 Unit 8 Software Project (10 practical hours); this capstone is a longer extension. CSTA 3B-AP-20 wording unverified.
- **Clients:** written consent; no personal data; the school owns the decision to publish. A teacher-invented "client" is fine.
- **Periods:** use the 8 periods for kick-off, two review clinics, security review, two demos, vivas and retrospective; teams work between periods.
- **Viva prompts:** "Show me the PR you are proudest of." "What did a reviewer change in your work?" "Walk me through how you would find the commit that broke X."

GRADE 12 · MODULE 9 · 3 PERIODS · TIERS 0, 1, 2, 3

9. Where Git goes next

> **At a glance**
>
> **Time:** 3 periods · **Tiers:** 0–3 · **You need:** the pathway map template; optionally a guest speaker
>
> **Before this:** the whole book.
>
> **By the end you can:** explain where version control is used beyond school, and write a concrete plan for your own next steps.

Hook (5 min)

Most people learn Git as an "extra". You have now studied it for five years. Where does it actually take you — and what do you do on Monday?

Understand it

Where version control appears

Field	How Git is used	What you already know from this book
University (CS/IT/engineering)	assignments and labs submitted as repos; team projects; GitHub Classroom	Grades 9–12
Software and DevOps	PRs and code review, CI/CD, releases, **infrastructure as code** (servers described in files, changed by PR)	G10, G12 M2–M4
Research and science	reproducible analyses: code, data and paper versioned together; citations of software	G11 M8, G12 M8
Writing and documentation	docs-as-code: Markdown in Git, reviewed by PR, published by CI — like this book	G10 M7
Data and machine learning	versioned datasets and experiments (Git + LFS/DVC)	G11 M8
Design and games	asset versioning with Git LFS	— (see *Version control in any project*)
Government and civic tech	open data and open-source tools; transparent changes	G11 M1, M8
Open source	contribution, maintenance, mentoring programmes	G11

Skills that transfer

Even outside software, the habits matter: **small steps with clear explanations**, **keeping history honest**, **reviewing others' work kindly**, **automating checks**, **documenting for strangers**, and **crediting sources**.

Open-source programmes

Mentoring programmes such as **Google Summer of Code** and **Outreachy** pay contributors to work with open-source projects. Both require applicants to be **18 or older** (checked October 2026; GSoC also requires student or open-source-beginner status), so many Grade 12 students become eligible only after school. **Rules change**

every year — check the official pages yourself. Until then: contribute to welcoming projects, join local communities, and keep a portfolio.

Communities in Nepal

Local open-source and open-data communities (for example FOSS Nepal, NOSK, Open Knowledge Nepal, Code for Nepal) run events and projects. *This book has not yet contacted them; check their current activities yourself.*

Keep learning

Part VIII of this book (*Version control in any project*) shows how to bring Git into any project; Part IX lists **books and free courses** for the next level. A good next step for most students: MIT's *Missing Semester* lectures, the *Pro Git* book chapters you haven't read, and one real contribution per term.

See it

A completed pathway map (example):

Stage	Study	Work	Open source	Next concrete step
Now (Grade 12)	finish capstone v1.0.0	—	2 merged PRs to this book	add Nepali glossary words (issue #41) by Poush
Next year	BSc CSIT entrance prep; Git for all assignments	—	translate one First Contributions page	read Pro Git ch. 7
After school	team projects at university	internship using code review	apply to a mentoring programme once eligible (18+)	keep CONTRIBUTIONS.md updated

Activity: talks and interview (40 min)

Interview an adult who uses version control (teacher, alumnus, parent) or watch a recorded talk. **Guest-talk template:** 5 min *what I do* · 5 min *a story where history saved me* · 10 min Q&A. Students prepare three questions in advance, e.g. "What did you wish you had learned at school?", "How does your team review work?", "What happens when something breaks?".

PRIMM lab

1. Interview / talk (above), or read about a field in the table and find one real example online.
2. **Predict** which of your interests connect to Git.
3. Fill the pathway map: *now* → *next year* → *after school*, with one concrete step each.
4. **Investigate:** find one open-source or open-data project related to your interest; apply the Grade 11 Module 2 reading strategy for 20 minutes.
5. **Make:** publish your map in your portfolio repo, alongside `CONTRIBUTIONS.md` and `BADGES.md`.

Parsons practice

Match field → Git use: research → reproducibility; industry → CI/CD; writing → documentation as code; data → versioned datasets.

Common mistakes

Mistake	Better
"Git is only for programmers"	writers, scientists, designers and governments use it
A plan without concrete steps	one action with a date per row
Waiting for a programme to start contributing	contribute now; programmes come later
Trusting old eligibility rules	check official pages every year

Practise more with free resources

#	Exercise	Tier	Time	Hand in
1	MIT **Missing Semester** ‹https://missing.csail.mit.edu/2020/version-control/› — watch one lecture outside Git (e.g. Shell Tools ‹https://missing.csail.mit.edu/2020/shell-tools/›).	3	60 min	5 things learned
2	The Turing Way ‹https://book.the-turing-way.org/reproducible-research/vcs› — how researchers use version control.	3	30 min	summary
3	Open Source Guides ‹https://opensource.guide/› — read one guide you haven't read.	3	20 min	summary
4	Choose one book from Part IX (Books and reading) to read this year; write why.	0	10 min	choice + reason

Exit ticket

1. Name two fields where Git is used.
2. What is your next concrete step?
3. *Misconception:* "Git is only for programmers."

Key points

- Version control is used in study, software, research, writing, data, design, government and open source.
- The habits transfer: small explained steps, honest history, kind review, automation, documentation, credit.
- Programmes have age and eligibility rules — check yearly; contribute now.
- A pathway map turns interest into a dated next step.

Book complete — for the grades. Continue with Part VII (*After Grade 12: the foundation, consolidated*) and Part VIII (*Version control in any project*).

Teacher notes

- **Timing:** P1: hook + understand 30, prepare questions 10. P2: talk/interview 40. P3: lab 30, exit 10.
- **Re-check programme eligibility and dates the same week you teach this. Do not promise admissions or jobs.** The GitHub Student Developer Pack was not checked.
- Invite alumni from Grade 12 capstones in earlier years once the curriculum has run.

Part VII. After Grade 12

Part VII. After Grade 12: the foundation, consolidated

You have finished the five chapters. This part helps you **check** that your foundation is solid, **fix** any gaps, and see **what comes next**. It also contains a four-week **bridge course** for anyone who joins late — a student starting at +2 or university, a learner who missed grades, or a teacher preparing to teach.

1. The foundation in one page

```
objects & refs:  commit = snapshot (tree) + parents + message      G12 M1 explains G8 M2
      |
three places:    working copy -> staging area -> history           G8 M3, G9 M2
      |
undo:            restore, stash | revert (shared)                  G8 M5, G11 M5
                 reset, rebase (local only) | reflog rescues
      |
labels:          branches and tags point at commits;               G9 M3, G10 M6
                 HEAD = you are here
      |
integration:     merge via common ancestor;                        G9 M4, G10 M5
                 conflicts = human decisions
      |
sharing:         every clone is complete: fetch, pull, push,       G9 M5, G10 M2
                 forks, upstream
      |
team process:    issue -> branch -> PR -> review -> CI             G10, G12
                 -> merge -> release
      |
responsibility:  licences, privacy, secrets, AI review, credit     G11 M1, G12 M5–M6
```

2. Competency checklist

Tick what you can do **without help**. Each line says where it was taught, so you can go back.

A. History and snapshots

☐ I can explain a commit as a snapshot with a message, author, date and parent(s). *(G8 M2)*
☐ I can read `git log`, `git show` and a diff, and answer who/when/what/why for any change. *(G8 M4)*
☐ I can explain blobs, trees, commits and refs, and why commit ids change after a rebase. *(G12 M1)*

B. Undo and safety

☐ I can choose between `restore`, `restore --source`, `revert`, `reset`, `commit --amend` and `stash` for a situation. *(G8 M5, G11 M5)*
☐ I follow the golden rule: never rewrite history others already have. *(G11 M5)*
☐ I can recover "lost" commits with `reflog`. *(G11 M5)*

C. Branching and integration

☐ I can create, switch, merge and delete branches, and read `log --graph`. *(G9 M3–M4)*
☐ I can resolve a conflict calmly, test the result, and finish or abort a merge. *(G9 M4, G10 M5)*
☐ I can rebase my own branch and explain when not to. *(G11 M5)*
☐ I can find the commit that broke something with `bisect`. *(G12 M7)*

D. Sharing and remotes

☐ I can clone, fetch, pull and push, and fix a rejected push. *(G9 M5)*

☐ I can fork a project, set `upstream`, and keep my fork in sync. *(G10 M2)*
☐ I can move a repository with no network using `git bundle`. *(G9 M5)*

E. Collaboration and communication

☐ I can write clear issues, PR descriptions and full commit messages. *(G10 M1, M3; G11 M6)*
☐ I can review a PR with a checklist and write specific, kind, actionable comments. *(G10 M4)*
☐ I can contribute to someone else's project following its rules. *(G11 M2–M4)*

F. Automation and quality

☐ I can write `.gitignore`, check scripts and a CI workflow, and fix a red run. *(G9 M6, G10 M8, G12 M3)*
☐ I can release with tags, SemVer and a changelog. *(G10 M6, G12 M4)*
☐ I can make data cleaning reproducible and documented. *(G11 M8)*

G. Ethics, licences, privacy, security

☐ I can credit sources (TASL) and choose a licence. *(G11 M1)*
☐ I keep secrets and personal data out of repositories and know the incident checklist. *(G12 M5)*
☐ I never commit code I cannot explain, including AI-generated code. *(G12 M6)*

H. Beyond code

☐ I can bring version control into a non-software project. *(Part VIII)*

Reading your result: 20–22 ticks — a strong foundation; go to section 5. 14–19 — fix the gaps with the table below. Fewer than 14 — take the bridge course.

3. The foundation test (practical, 60 minutes)

A teacher can use this as a final check, or a learner as a self-test. Do it in a fresh practice repository.

1. Create a repository with a `README.md` and a `.gitignore` that ignores `*.log`; make two commits with good messages.
2. Create a branch `feature/notes`, add `notes.md` in two commits, merge it into `main`. Was it a fast-forward? Why?
3. Create a conflict on purpose between two branches and resolve it so both ideas are kept.
4. Commit a "bad" change and undo it **safely as if it were already pushed**.
5. Make three messy local commits and turn them into one clean commit.
6. Delete a branch with unmerged work, then recover it with `reflog`.
7. Clone your repository into a second folder, commit there, and bring the commit back to the first folder (push/pull between the two, or a bundle).
8. Tag `v1.0.0`; make a backwards-compatible feature; tag the correct next version.
9. Write a check script that fails if `README.md` has no heading, and a CI workflow that runs it.
10. Use `git bisect run` to find the commit where you broke the check on purpose.

Explain aloud after each step what Git did. The explanations matter as much as the commands.

4. Gaps and fixes

If you struggle with…	The missing idea is usually…	Go back to
"My file disappeared after switching"	switching shows the branch's snapshot	G9 M3 (sticky notes)

If you struggle with…	The missing idea is usually…	Go back to
`git status` output	three places	G8 M3, G9 M2
choosing an undo	adding vs rewriting history	G8 M5, G11 M5
conflicts	common ancestor; same lines	G9 M4
rejected pushes	every clone is complete; remote moved	G9 M5
forks vs clones	where each copy lives	G10 M2
detached HEAD	HEAD is a pointer	G9 M3, G12 M1
rebase surprises	rebased commits are new objects	G11 M5, G12 M1
CI fails but works locally	CI starts from a clean machine	G12 M3

5. Bridge course (4 weeks)

For learners who join late or need to rebuild the foundation, and for teachers preparing to teach. About **3 sessions of 60–90 minutes per week**. Each session points to the lessons to use; do the *Understand it*, *See it* and PRIMM parts, and at least one free-resource exercise.

Week	Session 1	Session 2	Session 3	End-of-week check
1 — My own history	G8 M1–M2: why version control; snapshots (paper)	G8 M3 + G9 M1–M2: first repo, terminal, CLI cycle	G8 M4–M5: history, diff, restore, revert	foundation test steps 1 and 4
2 — Parallel and shared	G9 M3: branches (sticky notes + Learn Git Branching `intro2`)	G9 M4: merge and conflicts	G9 M5: remotes (LAN server or bundles)	steps 2, 3 and 7
3 — Team work	G10 M1–M3: issues, forks, PRs	G10 M4–M5: review, PR conflicts	G10 M6, M8: releases, check scripts	a reviewed PR that closes an issue
4 — Professional practice	G11 M5–M6: stash/reset/rebase/reflog, conventions	G12 M1 + M3: internals, CI	G12 M5 + M7: security, bisect	steps 5, 6, 9 and 10

Fast track for teachers (one weekend): the 3-hour training outline, then Week 1 and Week 2 of the bridge course on your own laptop, then Learn Git Branching to the end of the Remote tab.

6. What comes next

Topics this book does not cover in depth, for university, work and self-study:

Topic	What it is	Where to start
Interactive rebase and history editing	reorder, split, squash commits before sharing	*Pro Git* §7.6 Rewriting History
Advanced merging	strategies, `rerere`, resolving large conflicts	*Pro Git* §7.8 Advanced Merging
Hooks	scripts Git runs on commit, push …	*Pro Git* §8.3 Git Hooks; pre-commit
Submodules and subtrees	repositories inside repositories	*Pro Git* §7.11 Submodules
Worktrees	several branches checked out at once	`git help worktree`
Large files	Git LFS; DVC for data and models	Part VIII §7

Topic	What it is	Where to start
Monorepos and scaling	very large repositories, sparse checkout, partial clone	*Software Engineering at Google*, ch. 16
Merge queues and code owners	busy repositories with many PRs	your host's docs
GitOps / infrastructure as code	servers and deployments managed by PR	CI/CD docs of your host
Running a server	Forgejo/GitLab administration, backups	Forgejo docs; `offline/forgejo-lan/`
Maintaining an open-source project	governance, releases, community	*Producing Open Source Software* (Part IX)

Continue with **Part VIII** to put version control into any project you start, and **Part IX** for books and courses.

Pathway map: where Git goes next

Fill in your own copy in Grade 12 Module 9.

Stage	Study	Work	Open source	Next concrete step
Now (school)	school projects, this curriculum	—	first pull request to this repo	
After SEE / +2	CS/IT/engineering courses use Git for labs and assignments	internships, freelance	translate, document, fix	
University	team projects, research code	DevOps, software, data roles	mentoring programmes such as GSoC and Outreachy (both 18+; re-check each year)	

Fields where version control appears

- **Software and DevOps:** pull requests, CI/CD, infrastructure as code.
- **Research:** reproducible analysis — code, data and papers versioned together.
- **Writing and documentation:** Markdown, translation, documentation-as-code.
- **Data and machine learning:** versioned datasets and experiments.
- **Government and civic tech:** open data and open-source tools (e.g. community projects such as Code for Nepal, Open Knowledge Nepal, FOSS Nepal — not yet contacted).

We do not promise admission, jobs or programme acceptance. Check current eligibility and dates yourself.

Part VIII. Version Control in Any Project

Part VIII. Version control in any project

Grades 8–12 taught version control through school projects. This part is a **practical handbook** for bringing it into *any* project you start or join — an essay, a thesis, a research analysis, a website, an app, a dataset, a design, a hardware build, an organisation's documents. Each section ends with a short checklist you can copy.

1. Should this project use version control?

Almost always **yes** if any of these is true:

- it will change over more than a few days;
- more than one person works on it;
- you might want an old version back;
- someone will need to know *why* it looks the way it does;
- it must be reproducible (research, data, reports).

What to track — anything a person writes and needs to rebuild the project:

Track in Git	Don't track (ignore)	Never track
source code, scripts, notebooks (cleaned)	generated output, caches, builds	passwords, keys, tokens, `.env`
text: Markdown, LaTeX, HTML, CSV, JSON, YAML	local databases, logs, temp files	personal data about others
configuration and build files	editor/OS files (`.vscode/`, `.DS_Store`)	files you have no licence to share (in public repos)
small data you may share	very large data (→ LFS, DVC, or a link)	
documentation, licence, changelog	downloaded dependencies (`node_modules/`, `venv/`)	

Text beats binary. Git can diff and merge text line by line. For binary formats (Word, Excel, Photoshop, video) it only knows "changed" — prefer text formats where you can (Markdown instead of Word, CSV instead of Excel), and see §7 for the rest.

2. The 10-step adoption playbook

Use these steps for a new project, or to bring an existing one under control.

1. **Make the folder a repository.** `git init my-project` (or *Initialize Repository* in VS Code).
2. **Add a `.gitignore` first** — before the first commit — from github/gitignore ‹https://github.com/github/gitignore› for your language or tool, plus `.env`.
3. **Write a `README.md`:** what it is, how to run/build/read it, where data comes from, who to contact.
4. **Choose a licence** and add `LICENSE` (code: MIT/Apache-2.0/GPL; content and data: CC BY/CC BY-SA/CC0). Private project? Still note who owns it.
5. **First commit:** `git add . && git status` (check!) `&& git commit -m "Start project"`.
6. **Set up a remote — and a backup.** A host (GitHub, Codeberg, GitLab, a school/office Forgejo) or at least a bundle on a USB drive: `git bundle create backup.bundle --all`.
7. **Decide the branch rules.** Solo: work on short branches or directly on `main`. Team: branches + PRs, protected `main` (Grade 12 M2).

8. **Decide the review rule.** Team: at least one approval. Solo: review your own diff before every commit (`git diff --staged`).
9. **Automate a check.** Even one script (does it build? are links valid? is the CSV well-formed?) run in CI or a pre-commit hook.
10. **Release and record.** Tag meaningful versions (`v1.0.0`, `submitted-2026-11-30`), keep a `CHANGELOG.md`.

3. Where the repository lives: choosing a home

Option	Good for	Watch out
Local only	private drafts, learning	one disk = one failure away from loss — add backups
USB / external drive (`git bundle` or a bare repo)	offline schools, backup	drives get lost; keep two
School or office LAN server (Forgejo, GitLab)	teams without reliable internet; private data stays in the building	someone must maintain and back it up
Public host (GitHub, Codeberg, GitLab.com)	open-source, portfolios, open data	everything public is public for ever
Private repo on a host	team work that must not be public	private ≠ secret (no passwords, no unnecessary personal data)

Backups: the 3-2-1 rule. Keep **3** copies, on **2** different kinds of storage, **1** somewhere else. With Git this is easy: your laptop, a host or LAN server, and a bundle on a USB drive.

4. A sensible project layout

```
my-project/
├── README.md           what, how to run, contact
├── LICENSE             permission
├── CHANGELOG.md        what changed in each version
├── CONTRIBUTING.md     (teams) how to work together
├── .gitignore          what Git should ignore
├── .gitattributes      line endings, binary files, LFS
├── docs/               longer documentation
├── src/  or  scripts/  code
├── data/raw/           untouched inputs (if small and shareable)
├── data/clean/         produced by scripts
├── tests/  or  check.sh
└── .github/workflows/  (or .forgejo/workflows/) CI
```

Not every project needs every file. Start with README, LICENSE and `.gitignore`.

5. Recipes by project type

5.1 School assignments and homework

- One repository per subject or per big assignment.
- Commit at the end of every work session with a message that says what you did.
- Submit a **tag** (`submitted`) or the repository link if the teacher accepts it; the history shows your process (and protects you if someone claims you copied).
- **Checklist:** `.gitignore` · commit per session · tag on submission.

5.2 Essays, reports, theses and books

- Write in **plain text**: Markdown (convert with Pandoc to Word/PDF), or **LaTeX** (Overleaf has a Git integration ‹https://www.overleaf.com/learn/how-to/Git_integration›).
- Put **one sentence per line** (or one paragraph per line): diffs become readable and two people's edits rarely conflict.
- Keep references in a `.bib` file; figures as files in `figures/` with the scripts that make them.
- Tag each version sent to a supervisor (`draft-1-to-supervisor`); their comments become issues.
- **Have to use Word?** Commit the `.docx` anyway (you still get history and backups), and make Git show text diffs with a converter — see *Pro Git* §8.2 **Git Attributes** ‹https://git-scm.com/book/en/v2/Customizing-Git-Git-Attributes› ("Diffing Binary Files").
- **Checklist:** text format · line-per-sentence · tag per submission · bibliography in Git.

5.3 Research analysis and notebooks

- Separate `data/raw/` (never edited), `data/clean/` (made by scripts), `scripts/` or `notebooks/`, `results/` (usually ignored and regenerated).
- Record the software environment (`requirements.txt`, `environment.yml`).
- **Jupyter notebooks** store outputs inside the file, which makes huge, unreadable diffs: clear outputs before committing (or use a tool such as `nbstripout`), and use nbdime ‹https://nbdime.readthedocs.io/› to diff and merge notebooks.
- Add `CITATION.cff` so others can cite your code; archive releases with a DOI service if your field expects it.
- Read: The Turing Way — Version Control ‹https://book.the-turing-way.org/reproducible-research/vcs›, *Ten Simple Rules for Taking Advantage of Git and GitHub*, *Good Enough Practices in Scientific Computing* (Part IX).
- **Checklist:** raw/clean split · environment file · clean notebooks · CITATION.cff · tag the version used in each paper.

5.4 Data projects

- Everything from Grade 11 Module 8: raw stays raw, clean by script, `DATA.md` with source, date, licence and changes, a changelog per data version.
- Validate the data in CI (header, types, row counts, no personal data columns).
- Large data: see §7.
- **Checklist:** DATA.md · licence checked · validation in CI · no personal data.

5.5 Websites and documentation sites

- Static sites (Grade 10 M7) or site generators (MkDocs, Hugo, Jekyll): content in Markdown, published by CI from `main`.
- Preview every PR before merging; run a link checker in CI.
- **Checklist:** `check` in CI · PR review · publish from `main` only · credits for images.

5.6 Programs and apps (any language)

- `.gitignore` for the language (build folders, virtual environments, `node_modules/`).
- Commit the **lock file** (`package-lock.json`, `poetry.lock`) for apps so builds are repeatable.
- Tests in CI from day one; branch protection when the team grows.
- SemVer releases if others depend on it (Grade 12 M4).
- **Checklist:** language `.gitignore` · lock file · tests in CI · releases.

5.7 Mobile apps and games

- Engines (Unity, Godot, Unreal) have ready `.gitignore` templates — use them; never commit the `Library/` or build caches.
- Art, audio and video → **Git LFS** (§7); consider LFS **file locking** so two artists don't edit the same binary at once.
- Keep signing keys and store credentials **out** of the repository.
- **Checklist:** engine `.gitignore` · LFS for assets · locking for binaries · no signing keys.

5.8 Design, images and other binary files

- Binary files can't be merged: agree **one owner per file** at a time, or use LFS locking.
- Keep **exports** (PNG, PDF) only if needed; keep the **source** (SVG, `.kra`, `.fig` export) as the tracked file.
- Prefer text-based formats where possible (SVG instead of PNG for diagrams).
- **Checklist:** source files tracked · owner per file · LFS above a few MB.

5.9 Spreadsheets

- Keep the **data** as **CSV** (diffable) and the calculations as a script; use the spreadsheet only to view.
- If you must keep `.xlsx`, also export the important sheets to CSV in each commit so changes are visible.
- Beware: spreadsheet programs silently change values when saving CSV (leading zeros in ward codes, dates). Check `git diff` after every save.
- **Checklist:** CSV as truth · formulas → scripts · check diffs for silent changes.

5.10 Arduino, Raspberry Pi and hardware

- Track sketches/code, wiring notes, circuit and PCB design files (KiCad files are text), a parts list (`BOM.csv`), photos of the build (LFS if many).
- Tag the version that is flashed onto each device (`device-a-v1.2`).
- **Checklist:** code + schematics + BOM · tag per flashed version · no Wi-Fi passwords in code (use a `secrets.h` that is git-ignored, with a `secrets.example.h`).

5.11 Translation and multilingual documentation

- One file per language (`docs/en/…`, `docs/ne/…`) with the same structure — as this book does.
- Mark which English version a translation follows (a commit id or a "last synced" note), so translators can run `git diff <that-id> -- docs/en/page.md` to see what changed since.
- Labels such as `needs-nepali-review`; native-speaker review on every PR.
- **Checklist:** parallel folders · "translated from" marker · reviewer per language.

5.12 Organisations: NGOs, offices, municipalities, schools

- Policies, procedures, forms, minutes and open data as **Markdown/CSV in a repository**; proposals for change as **PRs**; discussion in **issues**; approved versions as **tags** (`policy-2083-v2`).
- The history is an audit trail: who changed what, when, who approved it, and why.
- Personal data stays in proper record systems, **not** in Git.
- **Checklist:** text formats · PR for every change · approvals required · tagged official versions · no personal data.

5.13 Personal: notes, configuration files, portfolio

- A notes repository (Markdown) synced between devices.
- "Dotfiles" — your tool configurations — in a repository so a new computer is set up in minutes.

- A portfolio repository: `CONTRIBUTIONS.md`, `BADGES.md`, pathway map (Grade 12 M9).

6. Bringing an existing project under version control

A folder full of `final_final` copies

You can turn renamed copies into a real history, oldest first, keeping their dates. Test on a **copy** of the folder:

```
#!/usr/bin/env bash
# Turn a folder of renamed copies into a Git history, oldest first.
# Usage: bash import-versions.sh <folder-with-copies> <target-file-name> <new-repo>
set -euo pipefail
src="$1"; target="$2"; repo="$3"
git init -q -b main "$repo"
ls -tr "$src" | while IFS= read -r f; do            # oldest first, by modification time
  when=$(date -r "$src/$f" +%Y-%m-%dT%H:%M:%S)
  cp "$src/$f" "$repo/$target"
  git -C "$repo" add "$target"
  GIT_AUTHOR_DATE="$when" GIT_COMMITTER_DATE="$when" \
    git -C "$repo" commit -q -m "Import old copy: $f"
done
git -C "$repo" log --date=short --format="%h %ad %s"
```

```
$ bash import-versions.sh old-essays essay.md essay-history
9768672 2026-09-09 Import old copy: essay_final_final.md
21893f4 2026-09-05 Import old copy: essay_final.md
a95e70a 2026-09-01 Import old copy: essay.md
```

Now `git diff` between any two old versions works — the `final_final` problem from Grade 8, solved retroactively.

A ZIP or shared-drive project

1. Copy the latest version into a new folder; `git init`; add `.gitignore`; commit "Import project as of ".
2. Keep the old ZIPs as an archive; don't try to reconstruct every past state unless it matters.
3. From now on, all changes go through commits.

From another version control system

Subversion, Mercurial and others can be converted with dedicated tools (`git svn`, import tools on hosts). See *Pro Git* chapter 9 (*Git and Other Systems*).

7. Large files and binaries

Plain Git keeps every version of every file in every clone. A 200 MB video changed 10 times = 2 GB for everyone.

Situation	Use
a few images, PDFs, small datasets (< a few MB each)	plain Git is fine
many or large binaries (art, audio, video, models)	**Git LFS** ‹https://git-lfs.com/› — Git stores small pointers; the big files live on an LFS server
large datasets and ML models, pipelines	**DVC** ‹https://dvc.org/doc/start› alongside Git
huge public data	don't copy — link to the source and record the version/date in `DATA.md`

Git LFS in three commands:

```
git lfs install
git lfs track "*.psd" "*.mp4"      # writes rules into .gitattributes
git add .gitattributes
```

Check your host's LFS storage limits; a school Forgejo server can host LFS too.

8. Settings for different contexts

	Solo	Team (school, office)	Open source
Branches	optional; short-lived	branch per issue	fork + branch per PR
Review	read your own diff	≥ 1 approval	maintainers review
`main`	yours	protected; no direct pushes	protected; CODEOWNERS
CI	a simple check	required checks	required checks, from forks without secrets
Communication	commit messages	issues, PRs, charter	issues, PRs, CONTRIBUTING, code of conduct
Licence	still useful	agree with the organisation	essential
Releases	tags for milestones	tags + changelog	SemVer + changelog + release notes

9. Introducing Git to a team that doesn't use it

1. **Start with a pain point** ("which version did we send to the ministry?"), not with Git itself.
2. **Start small:** one project, one champion, the GUI first (VS Code, GitHub Desktop) — Grade 8's path works for adults too.
3. **Write the conventions** on one page (branch names, messages, review rule) — Grade 11 M6.
4. **Teach the three ideas** that prevent most trouble: commit = snapshot; pull before push; never rewrite shared history.
5. **Make the safe way the easy way:** protected `main`, PR templates, a check in CI.
6. **Hold a short retrospective** after a month and adjust.

10. Security and backup checklist

☐ `.gitignore` includes `.env`, keys, local databases and generated files.
☐ Secrets come from environment variables or the host's secret store; only `*.example` files are committed.
☐ A secret scanner runs before commit or in CI.
☐ No personal data in the repository — public or private.
☐ `main` is protected (no force pushes; PR + review + checks for teams).
☐ Dependencies are pinned; alerts for known vulnerabilities are on.
☐ 3-2-1 backups exist (e.g. laptop + host/LAN + bundle on USB), and a restore was tested once.
☐ Someone other than you knows how to access the repository if you are away.

11. Templates

Copy and adapt. (All templates in this book are MIT-licensed.)

`README.md`

```
# Project name
One sentence: what this is and who it is for.

## How to run / read
1. ...
2. ...

## Data
Source, date retrieved, licence — see DATA.md.

## Contributing
See CONTRIBUTING.md. Questions: open an issue.

## Licence
Code: MIT. Content: CC BY 4.0. See LICENSE.
```

`CONTRIBUTING.md`

```
# Contributing
- Open or pick an issue first; comment before starting.
- Branch: type/short-topic (feat/, fix/, docs/).
- Commits: imperative summary ≤ 72 chars; body says why.
- Run the checks: `bash check.sh`.
- Open a PR with "Closes #N"; one approval needed; CI must be green.
- No secrets, no personal data. AI-assisted changes: you must be able to explain every line.
```

Pull request template (`.github/pull_request_template.md` or `.forgejo/pull_request_template.md`)

```
## What
## Why
Closes #
## How to check
## Checklist
- [ ] Checks pass locally
- [ ] Docs / changelog updated
- [ ] No secrets or personal data
```

`.gitattributes`

```
* text=auto
*.sh text eol=lf
*.png binary
*.jpg binary
*.pdf binary
```

Minimal CI workflow (`.github/workflows/check.yml`; Forgejo: `.forgejo/workflows/check.yml`)

```
name: check
on: [push, pull_request]
jobs:
  check:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: bash check.sh
```

`CHANGELOG.md`

```
# Changelog
## [Unreleased]
## [1.0.0] - 2026-11-30
### Added
- First public version.
```

`CITATION.cff`

```
cff-version: 1.2.0
message: "If you use this, please cite it as below."
title: "Project name"
authors:
  - family-names: "Surname"
    given-names: "Given"
version: 1.0.0
date-released: 2026-11-30
license: CC-BY-4.0
```

12. Is my project well version-controlled? (final checklist)

☐ It is a repository with a README, LICENSE and `.gitignore`.
☐ Commits are small, with messages that say what and why.
☐ There is a remote **and** a backup.
☐ Branch, review and message rules are written down (teams).
☐ At least one automatic check runs.
☐ Versions that matter are tagged; a changelog exists.
☐ No secrets or personal data; large files handled with LFS/DVC or links.
☐ A new person could start contributing using only the documentation.

Part IX. Books, Reading and Free Resources

Part IX. Books and further reading

A reading list for after this book, grouped by purpose. **Free** items can be read online at no cost; **print** items are commercial books (check your library). Levels: ★ beginner · ★★ intermediate · ★★★ advanced. Links were checked on 11 October 2026; publisher pages that block automatic checks are cited without a link.

1. Git itself

Book	Level	Free?	Why read it	Extends
Scott Chacon and Ben Straub, *Pro Git* ‹https://git-scm.com/book/en/v2›, 2nd ed. (Apress, 2014)	★–★★★	free (CC BY-NC-SA)	the standard reference; chapters 1–3 for everyone, 7 and 10 for the curious	every grade
John Wiegley, *Git from the Bottom Up* ‹https://jwiegley.github.io/git-from-the-bottom-up/›	★★	free (CC BY)	short; builds Git up from blobs and trees	G12 M1
Think Like (a) Git ‹https://think-like-a-git.net/›	★★	free	"Git is a graph; branches are labels" — the best cure for branch confusion	G9 M3, G11 M5
Raju Gandhi, *Head First Git* (O'Reilly, 2022)	★	print	visual, puzzle-filled introduction; good for teachers new to Git	Grades 8–10
Prem Kumar Ponuthorai and Jon Loeliger, *Version Control with Git*, 3rd ed. (O'Reilly, 2022)	★★–★★★	print	thorough, careful explanations of every major feature	Grades 11–12
Rick Umali, *Learn Git in a Month of Lunches* ‹https://www.manning.com/books/learn-git-in-a-month-of-lunches› (Manning, 2015)	★	print	one short lesson a day; good self-study after Grade 9	Grades 9–10
James Coglan, *Building Git* ‹https://shop.jcoglan.com/building-git/› (2019)	★★★	print/e-book	you write your own Git in Ruby — the deepest understanding of internals	G12 M1 and beyond

Also good, in zine form: Julia Evans's illustrated zines about Git (see Wizard Zines ‹https://wizardzines.com/comics/›) — short, friendly explanations of confusing parts.

2. Working in teams and in open source

Book	Level	Free?	Why read it	Extends
Karl Fogel, *Producing Open Source Software* ‹https://producingoss.com/›, 2nd ed.	★★	free (CC BY-SA)	how open-source projects are run: governance, communication, releases	Grade 11
VM (Vicky) Brasseur, *Forge Your Future with Open Source* ‹https://www.pragprog.com/titles/vbopens/forge-your-future-with-open-source/› (Pragmatic Bookshelf, 2018)	★	print	a practical guide to making your first contributions	G11 M2–M4
Nadia Eghbal, *Working in Public: The Making and Maintenance of Open Source Software* ‹https://press.stripe.com/working-in-public› (Stripe Press, 2020)	★★	print	who maintains the software the world runs on, and why it is hard	Grade 11, G12 M9
Eric S. Raymond, *The Cathedral and the Bazaar* ‹http://www.catb.org/esr/writings/cathedral-bazaar/› (1999)	★★	free essay	the classic essay on open development; read critically — it is a historical view	G11 M7
Open Source Guides ‹https://opensource.guide/› (GitHub and contributors)	★	free (CC BY)	short guides on contributing, starting and maintaining projects	Grade 11

3. Software engineering practice

Book	Level	Free?	Why read it	Extends
Titus Winters, Tom Manshreck and Hyrum Wright, *Software Engineering at Google* ‹https://abseil.io/resources/swe-book› (O'Reilly, 2020)	★★–★★★	free online	chapters on code review ‹https://abseil.io/resources/swe-book/html/ch09.html› and version control and branch management ‹https://abseil.io/resources/swe-book/html/ch16.html› at enormous scale	G10 M4, G12 M2
David Thomas and Andrew Hunt, *The Pragmatic Programmer*, 20th anniversary ed. (Addison-Wesley, 2019)	★★	print	habits of good programmers, including "always use version control"	after school
Jez Humble and David Farley, *Continuous Delivery* (Addison-Wesley, 2010)	★★★	print	the foundations of CI/CD and releasing safely	G12 M3–M4
Nicole Forsgren, Jez Humble and Gene Kim, *Accelerate* (IT Revolution, 2018)	★★	print	research evidence on which practices (trunk-based development, CI) help teams	G12 M2–M3
Trunk Based Development ‹https://trunkbaseddevelopment.com/› (website)	★★	free	everything about the trunk-based strategy	G12 M2
Vincent Driessen, *A successful Git branching model* ‹https://nvie.com/posts/a-successful-git-branching-model/› (2010, with a 2020 note)	★★	free	the original Gitflow post — read the author's own note at the top	G12 M2
The Agile Manifesto ‹https://agilemanifesto.org/› (2001)	★	free	one page; the source of "agile"	G11 M7

4. Research, data and writing

Reading	Level	Free?	Why read it	Extends
The Turing Way Community, *The Turing Way* ‹https://book.the-turing-way.org/reproducible-research/vcs›	★–★★	free (CC BY)	reproducible research, version control, data management	G11 M8, Part VIII
Jenny Bryan, *Happy Git and GitHub for the useR* ‹https://happygitwithr.com/›	★	free (CC BY-NC)	Git for data analysis; most chapters apply beyond R	G11 M8
Blischak, Davenport and Wilson (2016), *A Quick Introduction to Version Control with Git and GitHub* ‹https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004668›, PLOS Computational Biology	★	free (CC BY)	ten pages; ideal first reading for science teachers	Part I
Perez-Riverol et al. (2016), *Ten Simple Rules for Taking Advantage of Git and GitHub* ‹https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004947›, PLOS Computational Biology	★	free (CC BY)	practical rules for any project	G12 M8, Part VIII
Wilson et al. (2017), *Good enough practices in scientific computing* ‹https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1005510›, PLOS Computational Biology	★	free (CC BY)	minimal practices every researcher should adopt, including version control	Part VIII
Wilson et al. (2014), *Best Practices for Scientific Computing* ‹https://journals.plos.org/plosbiology/article?id=10.1371/journal.pbio.1001745›, PLOS Biology	★★	free (CC BY)	the fuller version of the above	after school
Jennifer Bryan (2018), *Excuse Me, Do You Have a Moment to Talk About Version Control?*, The American Statistician 72(1). doi:10.1080/00031305.2017.1399928	★	preprint free	the case for Git in statistics and data science	G11 M8

5. For teachers: how people learn to program

Reading	Free?	Why read it
Greg Wilson, *Teaching Tech Together* ‹https://teachtogether.tech/› (CRC Press, 2019)	free (CC BY-NC)	lesson design, cognitive load, live coding, building a teaching community
Felienne Hermans, *The Programmer's Brain* ‹https://www.manning.com/books/the-programmers-brain› (Manning, 2021)	print	how memory and reading work when learning code — useful for reading diffs and histories
Sentance, Waite and Kallia (2019), *Teaching computer programming with PRIMM: a sociocultural perspective*, Computer Science Education 29(2–3). doi:10.1080/08993408.2019.1608781	publisher	the research behind this book's lesson skeleton
Wagner and Thurner (2025), *Rethinking How We Teach Git* ‹https://jise.org/Volume36/n1/JISE2025v36n1pp1-12.html›, Journal of Information Systems Education 36(1)	free	recommendations on sequencing Git teaching
Raspberry Pi Foundation, research and pedagogy resources ‹https://www.raspberrypi.org/research›	free	short, evidence-based summaries for computing teachers
CS Unplugged ‹https://csunplugged.org/en/›	free	ideas for teaching computing without computers

6. Suggested reading paths

You are…	Read in this order
a Grade 12 finisher going to university	*Pro Git* ch. 7 → *Think Like (a) Git* → *Software Engineering at Google* ch. 9 and 16 → *The Pragmatic Programmer*
a future researcher or data scientist	*A Quick Introduction…* → *Good enough practices…* → *The Turing Way* → *Happy Git…*
an open-source enthusiast	Open Source Guides → *Forge Your Future…* → *Producing Open Source Software* → *Working in Public*
a teacher new to Git	*Head First Git* or *Pro Git* ch. 1–3 → *A Quick Introduction…* → *Teaching Tech Together* → Part I of this book
curious about internals	*Git from the Bottom Up* → *Pro Git* ch. 10 → *Building Git*
interested in DevOps	*Accelerate* → *Continuous Delivery* → trunkbaseddevelopment.com

Licences: free does not always mean "free to copy". Read and share links freely; copy text only from CC BY or similarly permissive sources, with credit (Grade 11 Module 1).

Free resource library

Every lesson in this book has a **"Practise more with free resources"** section. This page is the full catalogue those exercises come from. Every link was opened and returned a working page on **11 October 2026**; licences were read from each project's own licence file where one exists.

> **Link, don't copy**
>
> This book is CC BY 4.0. Many excellent resources are **NonCommercial (NC)** or **ShareAlike (SA)**. You may *use* them freely in class and send students to them, but their text must **not be pasted into this book or into printed copies of it**. The "Copy into our book?" column says which ones are safe to adapt with credit.

How to read the tables

- **Grades** — where the resource fits best in this book.
- **Tier** — the delivery tiers from the Teacher hub: 0 no computer · 1 offline computers · 2 school LAN · 3 internet.
- **Account** — whether a learner must sign up. Learners under 13 (most of Grade 8) must never need an account.

1. Visual sandboxes (see the commit graph move)

Resource	Good for	Grades	Tier	Account	Licence · Copy into our book?
Learn Git Branching ‹https://learngitbranching.js.org/› (source ‹https://github.com/pcottle/learnGitBranching›)	Commits, branches, merge, rebase, remotes as an animated graph; levels with goals; a free "sandbox"	8–12	1 (self-hosted, see `offline/usb-kit/`) or 3	none	MIT · yes
Visualizing Git ‹https://git-school.github.io/visualizing-git/› (source ‹https://github.com/git-school/visualizing-git›)	Type real commands, watch the graph; good for teacher demos on a projector	9–12	3 (or saved copy)	none	MIT · yes
Oh My Git! ‹https://ohmygit.org/› (source ‹https://github.com/git-learning-game/oh-my-git›)	Card game on top of *real* Git repositories; Windows/Linux/macOS download	9–10	1 after download	none	Blue Oak Model 1.0.0 (permissive) · yes for code; check art

Learn Git Branching level map

Levels are opened with `?level=<id>` (online: `https://learngitbranching.js.org/?locale=en_US&level=intro1`). The same ids work in the self-hosted copy.

Level id	Name (Main / Remote tab)	Used in
`intro1`	Introduction to Git Commits	G8 M2, G8 M3
`intro2`	Branching in Git	G9 M3
`intro3`	Merging in Git	G9 M4
`intro4`	Rebase Introduction	G11 M5

Level id	Name (Main / Remote tab)	Used in
`rampup1` – `rampup4`	Detach yo' HEAD · Relative Refs (^) · Relative Refs #2 (~) · Reversing Changes in Git	G9 M3 (`rampup1`), G11 M5 (`rampup4`), G12 M1
`move1`, `move2`	Cherry-pick Intro · Interactive Rebase Intro	G11 M5, G12 M2
`mixed4`, `mixed5`	Git Tags · Git Describe	G10 M6, G12 M4
`remote1` – `remote8`	Clone Intro · Remote Branches · Git Fetchin' · Git Pullin' · Faking Teamwork · Git Pushin' · Diverged History · Locked Main	G9 M5, G10 M2–M3

Type `levels` inside the app to see the full list, and `sandbox` to leave a level and experiment freely.

2. Structured free courses

Resource	Good for	Grades	Tier	Account	Licence · Copy?
Software Carpentry: Version Control with Git ‹https://swcarpentry.github.io/git-novice/›	13 short episodes with exercises and solutions; written for adults but very clear	9–12, teachers	1 (download the site) / 3	none (GitHub episode needs one)	CC BY 4.0 · **yes, with credit**
Software Carpentry: The Unix Shell ‹https://swcarpentry.github.io/shell-novice/›	Terminal basics (`cd`, `ls`, files, pipes, scripts)	9	1 / 3	none	CC BY 4.0 · **yes, with credit**
GitHub Skills ‹https://skills.github.com/›	Self-checking courses that run inside a GitHub repo: Introduction to GitHub ‹https://github.com/skills/introduction-to-github›, Communicate using Markdown ‹https://github.com/skills/communicate-using-markdown›, GitHub Pages ‹https://github.com/skills/github-pages›, Review pull requests ‹https://github.com/skills/review-pull-requests›, Resolve merge conflicts ‹https://github.com/skills/resolve-merge-conflicts›, Hello GitHub Actions ‹https://github.com/skills/hello-github-actions›, Release-based workflow ‹https://github.com/skills/release-based-workflow›, Secure your repository's supply chain ‹https://github.com/skills/secure-repository-supply-chain›	9–12 (13+)	3	**GitHub, 13+**	MIT · yes for course text
MIT "Missing Semester": Version Control ‹https://missing.csail.mit.edu/2020/version-control/› and Shell Tools ‹https://missing.csail.mit.edu/2020/shell-tools/›	University-level lecture + exercises; explains the data model (blobs, trees, commits) beautifully	11–12, teachers	3 (videos)	none	CC BY-NC-SA 4.0 · **link only**
The Odin Project: Git Basics ‹https://www.theodinproject.com/lessons/foundations-git-basics›	Self-paced web-developer track with a Git lesson and project	10–12	3	optional	open curriculum · link only

Resource	Good for	Grades	Tier	Account	Licence · Copy?
Atlassian Git tutorials ‹https://www.atlassian.com/git/tutorials› and workflow comparison ‹https://www.atlassian.com/git/tutorials/comparing-workflows›	Clear diagrams of workflows (feature branch, Gitflow, forking)	10–12	3	none	Atlassian site terms · link only
Git Immersion ‹https://gitimmersion.com/›	Step-by-step labs, one command at a time	9–10	3	none	free online · link only
W3Schools Git tutorial ‹https://www.w3schools.com/git/›	Short pages with "try it" exercises; good for revision	9–10	3	none (optional)	site terms · link only
CS50 Web, Lecture 1: Git ‹https://cs50.harvard.edu/web/2020/weeks/1/›	Free university lecture video + notes	11–12	3	none	Harvard CS50 terms · link only

Software Carpentry episode map

Episode	Topic	Used in
1 Automated Version Control ‹https://swcarpentry.github.io/git-novice/01-basics.html›	why version control	G8 M1
2 Setting Up Git ‹https://swcarpentry.github.io/git-novice/02-setup.html›	`git config`	G8 M3, G9 M2
3 Creating a Repository ‹https://swcarpentry.github.io/git-novice/03-create.html›	`git init`	G8 M3, G9 M2
4 Tracking Changes ‹https://swcarpentry.github.io/git-novice/04-changes.html›	add, commit, staging area	G8 M3, G9 M2
5 Exploring History ‹https://swcarpentry.github.io/git-novice/05-history.html›	`log`, `diff`, `restore`	G8 M4, G8 M5
6 Ignoring Things ‹https://swcarpentry.github.io/git-novice/06-ignore.html›	`.gitignore`	G9 M6
7 Remotes in GitHub ‹https://swcarpentry.github.io/git-novice/07-github.html›	push, pull	G9 M5
8 Collaborating ‹https://swcarpentry.github.io/git-novice/08-collab.html›	two people, one repo	G9 M8, G10 M2
9 Conflicts ‹https://swcarpentry.github.io/git-novice/09-conflict.html›	merge conflicts	G9 M4, G10 M5
10 Open Science ‹https://swcarpentry.github.io/git-novice/10-open.html›	research with Git	G11 M8
11 Licensing ‹https://swcarpentry.github.io/git-novice/11-licensing.html›	choosing a licence	G11 M1
12 Citation ‹https://swcarpentry.github.io/git-novice/12-citation.html›	`CITATION.cff`	G11 M8, G12 M4
13 Hosting ‹https://swcarpentry.github.io/git-novice/13-hosting.html›	where to host	G9 M5, G12 M9

3. Reference books and documentation

Resource	Good for	Grades	Tier	Licence · Copy?
Pro Git, 2nd ed. ‹https://git-scm.com/book/en/v2› (Chacon & Straub)	The standard free Git book; downloadable as PDF/EPUB for offline labs	9–12, teachers	1 (PDF on USB) / 3	CC BY-NC-SA 3.0 · **link only**

Resource	Good for	Grades	Tier	Licence · Copy?
Git reference manual ‹https://git-scm.com/docs› and cheat sheet ‹https://git-scm.com/cheat-sheet›	Exact meaning of every command and option (also offline: `git help <command>`)	9–12	1 / 3	GPL-2.0 docs · link only
Git from the Bottom Up ‹https://jwiegley.github.io/git-from-the-bottom-up/› (Wiegley)	Internals explained from blobs upward	12	3	CC BY 4.0 · yes, with credit
Think Like (a) Git ‹https://think-like-a-git.net/›	Graph thinking: "Git is a graph, branches are labels"	11–12	3	free online · link only
GitHub Docs ‹https://docs.github.com/en/get-started/start-your-journey/hello-world›	Accounts, pull requests, Actions, Pages	9–12	3	CC BY 4.0 (licence ‹https://github.com/github/docs/blob/main/LICENSE›) · yes, with credit
GitHub Desktop docs ‹https://docs.github.com/en/desktop›	GUI path for Grade 8–9	8–9	1 / 3	CC BY 4.0 · yes
VS Code: Source Control ‹https://code.visualstudio.com/docs/sourcecontrol/overview›	Commit, diff and branch from inside the editor	8–10	1 / 3	CC BY 3.0 US · yes, with credit
Codeberg docs ‹https://docs.codeberg.org/›, Forgejo docs ‹https://forgejo.org/docs/latest/›, GitLab Git docs ‹https://docs.gitlab.com/topics/git/›	Same ideas on non-GitHub hosts (Forgejo = our school-LAN server)	9–12	2 / 3	various open licences · link
Markdown Guide ‹https://www.markdownguide.org/› and CommonMark 10-minute tutorial ‹https://commonmark.org/help/tutorial/›	Writing READMEs and documentation	9–10	3	open · link

4. Rescue guides ("I broke it!")

Resource	Good for	Grades	Licence · Copy?
Oh Shit, Git!? ‹https://ohshitgit.com/› and its polite twin Dangit, Git!? ‹https://dangitgit.com/›	Short fixes for the most common mistakes. **Use the "Dangit" version with school students.**	10–12	no stated licence · link only
Git Flight Rules ‹https://github.com/k88hudson/git-flight-rules›	Huge "what to do when…" manual	11–12, teachers	CC BY-SA 4.0 · link only (SA)
Julia Evans: confusing Git terminology ‹https://jvns.ca/blog/2023/11/01/confusing-git-terminology/›	Why words like *HEAD*, *origin* and *reset* confuse people	11–12, teachers	blog · link only
explainshell ‹https://explainshell.com/›	Paste a shell command, see what every part means	9–12	open source · link

Our own rescue guide is Appendix B: Troubleshooting.

5. Open-source contribution and project hygiene

Resource	Good for	Grades	Account	Licence · Copy?
First Contributions ‹https://firstcontributions.github.io/› (repo ‹https://github.com/firstcontributions/first-contributions›)	A safe practice repo for the first fork → PR, in many languages (including Nepali)	11 (13+)	GitHub	MIT · yes

Resource	Good for	Grades	Account	Licence · Copy?
Open Source Guides ‹https://opensource.guide/› — esp. How to Contribute ‹https://opensource.guide/how-to-contribute/›	Etiquette, finding issues, being a good contributor	11–12	none	CC BY 4.0 · yes, with credit
choosealicense.com ‹https://choosealicense.com/›	Picking a licence in plain words	11	none	MIT · yes
SPDX licence list ‹https://spdx.org/licenses/›, REUSE ‹https://reuse.software/›, Creative Commons BY 4.0 ‹https://creativecommons.org/licenses/by/4.0/›	Exact licence identifiers; marking every file's licence	11–12	none	open · link
Semantic Versioning ‹https://semver.org/›	MAJOR.MINOR.PATCH rules	10, 12	none	CC BY 3.0 · yes, with credit
Keep a Changelog ‹https://keepachangelog.com/en/1.1.0/›	Changelog format	10–12	none	MIT · yes
Conventional Commits ‹https://www.conventionalcommits.org/en/v1.0.0/›	`feat:`, `fix:` message convention	11–12	none	MIT (site repo) · yes
Citation File Format ‹https://citation-file-format.github.io/›	`CITATION.cff` so others can cite your work	11–12	none	open · link
github/gitignore ‹https://github.com/github/gitignore›	Ready-made `.gitignore` templates for Python, Node, Java, LaTeX…	9–12	none	CC0 · yes
Hacktoberfest ‹https://hacktoberfest.com/›	October contribution event (check the current year's age rules)	11–12	yes	event · link

6. Automation, quality and security

Resource	Good for	Grades	Tier	Licence
GitHub Actions docs ‹https://docs.github.com/en/actions›	CI workflows	10–12	3	CC BY 4.0
Forgejo Actions ‹https://forgejo.org/docs/latest/user/actions/›	CI on a school LAN server	10–12	2	open
pre-commit ‹https://pre-commit.com/›	Run checks before every commit	12	1 after install	MIT
gitleaks ‹https://github.com/gitleaks/gitleaks›	Find passwords and keys committed by mistake	12	1 after install	MIT
OpenSSF Scorecard ‹https://scorecard.dev/›	Security health score of an open-source project	12	3	Apache-2.0
Git LFS ‹https://git-lfs.com/›	Large binary files (images, video, models)	12, any project	1 / 3	MIT

7. Research, data and writing

Resource	Good for	Grades	Licence · Copy?
The Turing Way: Version Control ‹https://book.the-turing-way.org/reproducible-research/vcs›	Version control for research and data, with clear illustrations	11–12, after school	CC BY 4.0 · yes, with credit

Resource	Good for	Grades	Licence · Copy?
Blischak, Davenport & Wilson (2016), *A Quick Introduction to Version Control with Git and GitHub* ‹https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004668›, PLOS Comp. Biol.	Short paper; ideal first reading for science teachers	teachers, 12	CC BY 4.0 · yes, with credit
Perez-Riverol et al. (2016), *Ten Simple Rules for Taking Advantage of Git and GitHub* ‹https://journals.plos.org/ploscompbiol/article?id=10.1371/journal.pcbi.1004947›	Practical rules for projects	12, after school	CC BY 4.0 · yes, with credit
Happy Git and GitHub for the useR ‹https://happygitwithr.com/› (Bryan)	Git for data analysis (R users, but most chapters are general)	after school	CC BY-NC 4.0 · link only
Our Coding Club: Git for reproducible research ‹https://ourcodingclub.github.io/tutorials/git/›	Step-by-step tutorial for scientists	11–12	see site · link only
DVC: Get started ‹https://dvc.org/doc/start›	Versioning large datasets and ML models next to Git	after school	Apache-2.0 docs
Overleaf Git integration ‹https://www.overleaf.com/learn/how-to/Git_integration›	Version-controlled LaTeX papers and theses	after school	site terms
nbdime ‹https://nbdime.readthedocs.io/›	Readable diffs of Jupyter notebooks	11–12	BSD
Open Data Nepal ‹https://opendatanepal.com/›, OKN localboundaries ‹https://github.com/openknowledgenp/localboundaries›	Nepali open data for the data thread (check each dataset's licence)	9–12	per dataset

8. For teachers: pedagogy

Resource	Good for	Licence
Wilson, *Teaching Tech Together* ‹https://teachtogether.tech/›	How people learn programming; lesson design; live coding	CC BY-NC 4.0 · link only
CS Unplugged ‹https://csunplugged.org/en/›	Computing without computers; ideas for Tier 0	open licence (check per activity) · link only
Parsons problem tool ‹https://parsons.problemsolving.io/›	Make drag-and-drop Parsons puzzles	open source
Wizard Zines comics ‹https://wizardzines.com/comics/›	Hand-drawn one-page explanations (several on Git) — good poster inspiration	free to view · link only
GitHub Education ‹https://docs.github.com/en/education› and GitHub Classroom ‹https://classroom.github.com/›	Free teacher accounts, assignment repos with autograding	service terms

9. Offline packing list (Tier 1 USB kit)

Put these on the school USB stick alongside the kit in `offline/usb-kit/`:

1. Git for Windows (portable) and VS Code (portable).
2. The self-hosted **Learn Git Branching** build (`build-learngitbranching.sh`).
3. **Pro Git** PDF (download from the book page; NC licence allows sharing for non-commercial school use — do not sell copies).
4. A saved copy of the **Software Carpentry** git-novice and shell-novice sites (CC BY 4.0).
5. **Oh My Git!** installers for the lab's operating systems.
6. This book's PDF and the practice-repo `.bundle` files.

Check before every school year: links change, courses move, and age rules for events and programmes are updated.

Appendices

Appendix A. Command cheat sheet

Commands grouped by the grade where they are taught. `<file>`, `<branch>`, `<id>` are placeholders — replace them, without the `< >`.

Set up once per computer (teacher, Grade 8–9)

```
git config --global user.name "Student 07"            # pseudonym in school
git config --global user.email "s07@example.invalid"
git config --global init.defaultBranch main
git config --global pull.rebase false                 # pull = fetch + merge
git config --global core.editor "code --wait"         # VS Code instead of Vim
```

Grade 8 — my own history

I want to…	Command	Button (VS Code)
make a folder a repository	`git init`	*Initialize Repository*
see where my changes are	`git status`	Source Control panel
stage a file	`git add <file>`	+
commit	`git commit -m "Add …"`	✓ Commit
see the history	`git log --oneline`	Timeline / Graph
see one commit's change	`git show <id>`	click the commit
see my uncommitted changes	`git diff`	click the file
throw away uncommitted edits ⚠	`git restore <file>`	*Discard Changes*
bring back an old version	`git restore --source=<id> <file>`	—
undo a commit safely	`git revert <id>`	*Revert*
dates for statistics	`git log --date=short --format=%ad`	—

Grade 9 — branch and share

I want to…	Command
where am I / what's here / move / make folder	`pwd` · `ls` · `cd <path>` · `mkdir <name>`
see staged changes	`git diff --staged`
list / create+switch / switch branch	`git branch` · `git switch -c <branch>` · `git switch <branch>`
draw the graph	`git log --oneline --graph --all`
merge a branch into the current one	`git merge <branch>`
give up a merge	`git merge --abort`
copy a repository	`git clone <url>`
send / receive commits	`git push origin <branch>` · `git pull`
download without merging	`git fetch`
list remotes	`git remote -v`

I want to…	Command
move a repo by USB	`git bundle create x.bundle --all` · `git clone x.bundle` · `git pull x.bundle main`
commits with their diffs	`git log -p`
tag a version	`git tag v1.0` (push: `git push origin v1.0`)
commits per person	`git shortlog -s -n`

Grade 10 — team workflows

I want to…	Command
connect the original project of my fork	`git remote add upstream <url>`
update my fork	`git fetch upstream` · `git merge upstream/main` · `git push origin main`
push a new branch and track it	`git push -u origin <branch>`
fix a conflicted PR	`git fetch origin` · `git merge origin/main` · resolve · `git add` · `git commit` · `git push`
annotated tag	`git tag -a v1.0.0 -m "…"`
what changed since a release	`git log --oneline v1.0.0..main` · `git diff v1.0.0 main --stat`
patches without a server	`git format-patch main` · `git am *.patch`
close an issue from a commit	message contains `Closes #12`

Grade 11 — contributor tools

I want to…	Command
search tracked files	`git grep "word"`
park unfinished work	`git stash` · `git stash list` · `git stash pop`
fix the last commit (not pushed)	`git commit --amend`
undo commits, keep changes staged (not pushed)	`git reset --soft HEAD~1`
undo commits and changes ⚠ (not pushed)	`git reset --hard HEAD~1`
replay my branch on new main	`git rebase main` (then `git push --force-with-lease` on **your own** branch only)
tidy last 3 commits	`git rebase -i HEAD~3`
find "lost" commits	`git reflog`
copy one commit here	`git cherry-pick <id>`
search history	`git log --grep="fix"` · `git log --author="…"` · `git log -S "text"`

Grade 12 — engineering

I want to…	Command
inspect objects	`git cat-file -t <id>` · `git cat-file -p <id>` · `git ls-tree HEAD`
hash a file	`git hash-object <file>`
who last changed each line	`git blame <file>` · `git blame -L 10,20 <file>`
find the bad commit	`git bisect start` · `git bisect bad` · `git bisect good <id>` · … · `git bisect reset`
automatic bisect	`git bisect run bash check.sh`
describe a commit by nearest tag	`git describe --tags`
signed commit	`git commit -S -m "…"`
large files	`git lfs install` · `git lfs track "*.mp4"`

Which undo do I need?

Situation	Pushed / shared?	Use
messy edits, not committed	—	`git restore <file>` ⚠ edits lost
staged something by mistake	—	`git restore --staged <file>`
want an old version of a file	—	`git restore --source=<id> <file>` then commit
bad commit	**yes**	`git revert <id>`
bad last commit message / forgot a file	no	`git commit --amend`
several messy commits	no	`git reset --soft HEAD~N` then commit
need to stop for a moment	—	`git stash`
lost commits after reset/rebase	—	`git reflog`, then `git reset --hard <id>` or `git branch rescue <id>`
secret committed	either	**revoke the secret first**, then see Grade 12 Module 5

Names for commits

Write	Means
`HEAD`	the commit you are on
`HEAD~1`, `HEAD~2`	one / two commits before HEAD (first parents)
`HEAD^2`	the second parent of a merge commit
`main`, `origin/main`, `v1.0`	the commit a branch, remote branch or tag points to
`a..b`	commits reachable from `b` but not from `a` ("what's new in b")
`<id>:<path>`	a file as it was in a commit (`git show v1.0:README.md`)

Reading `git status -s`

Code	Means
`??`	untracked
`A`	new file, staged
`M`	modified, staged
`M`	modified, not staged
`MM`	staged, then modified again
`D`	deleted, not staged
`UU`	conflict (both modified)
`!!`	ignored (with `--ignored`)

Offline help: `git help <command>` or `git <command> -h`.

Appendix B. Troubleshooting: "Oops, Git!"

Real situations from classrooms and projects, with **safe** fixes. Before any fix: **stop, run `git status`, and read what Git says** — its hints are usually right. If you are unsure, copy the whole folder (including `.git`) as a backup first.

> **Two lifelines**
> - **Committed work is almost never lost:** `git reflog` remembers where you have been.
> - **Uncommitted work is fragile:** commit (or `git stash`) before trying anything risky.

Setup and everyday messages

1. `Please tell me who you are`

Git needs a name and email for commits.

```
git config --global user.name "Student 07"
git config --global user.email "s07@example.invalid"
```

2. `fatal: not a git repository (or any of the parent directories)`

You are not inside a repository. Run `pwd`, then `cd` into the project folder (the one containing `.git`). If you never ran `git init` or `git clone`, do that.

3. A black screen with `~` signs (Vim) opened

Git wants a message. Type `:wq` and Enter to save and continue, or `:q!` and Enter to cancel. Avoid it next time: `git commit -m "…"`, or `git config --global core.editor "code --wait"`.

4. `git log` "froze" and I can't type

It opened a pager. **Space** = next page, **q** = quit.

5. `warning: LF will be replaced by CRLF`

Windows and Linux end lines differently. Harmless. For projects with shell scripts, add a `.gitattributes` with `* text=auto` and `*.sh text eol=lf`.

6. `error: pathspec 'Add' did not match any file(s)`

The message was not in quotes: `git commit -m Add README` → `git commit -m "Add README"`.

7. I committed in the wrong folder / my whole home folder is a repository

Someone ran `git init` in the wrong place. Find it: `git rev-parse --show-toplevel`. If it is your home folder, remove **only that `.git` folder** (ask the teacher; check the path twice).

Commits

8. Wrong message in the last commit (not pushed)

`git commit --amend -m "Better message"`. Already pushed? Leave it; write a better message next time.

9. Forgot a file in the last commit (not pushed)

`git add forgotten.txt && git commit --amend --no-edit`.

10. Committed to `main` instead of a new branch (not pushed)

```
git branch feature/my-work        # label the commits where they are
git reset --hard origin/main      # move main back to the server's version
git switch feature/my-work        # continue on the branch
```

Check `git status` first: uncommitted changes would be lost by `--hard`.

11. I need to undo a commit that is already pushed

`git revert <id>` — adds an opposite commit; safe for everyone.

12. I discarded changes with `restore` and want them back

If they were never committed or stashed, Git cannot recover them. Check your editor's undo history or local history (VS Code: *Timeline → Local History*). Lesson: commit often.

13. I ran `git reset --hard` and lost commits

```
git reflog                        # find the line before the reset
git reset --hard <id-from-reflog>
```

14. I deleted a branch with work on it

`git reflog` shows the last commit of that branch; `git branch rescued <id>`.

Branches

15. "My files disappeared!" after switching branches

They belong to the other branch's snapshot. `git switch <other-branch>` and they are back.

16. `Your local changes to the following files would be overwritten by checkout`

Git protects uncommitted work. Commit it, `git stash` it, or (if you don't need it) `git restore` it — then switch.

17. `You are in 'detached HEAD' state`

You checked out a commit or tag, not a branch. To look around: fine. To return: `git switch main`. To keep work made here: `git switch -c new-branch` **before** leaving.

Merging and conflicts

18. `CONFLICT (content): Merge conflict in …`

Open the file; find `<<<<<<<`, `=======`, `>>>>>>>`; edit to the correct final text; delete all markers; `git add <file>`; `git commit`. To start over: `git merge --abort`. (Rebase: `git rebase --continue` / `--abort`.)

19. I committed conflict markers by mistake

Search: `git grep -n "<<<<<<<"`. Fix the files and commit again. Add a check to CI that fails on markers.

20. `fatal: You have not concluded your merge (MERGE_HEAD exists)`

A merge is half done. Finish (`git add …` + `git commit`) or `git merge --abort`.

21. My PR says "This branch has conflicts"

On your branch: `git fetch origin`, `git merge origin/main`, resolve, test, commit, `git push`.

Remotes

22. `! [rejected] main -> main (fetch first)` / `non-fast-forward`

The remote has commits you don't have. `git pull`, resolve any conflict, `git push`. **Do not** force-push a shared branch.

23. `fatal: Need to specify how to reconcile divergent branches`

Choose how `pull` combines work, once: `git config --global pull.rebase false` (merge). Then `git pull` again.

24. `Authentication failed` / `Permission denied`

Check the account and URL (`git remote -v`). GitHub does not accept account passwords for HTTPS pushes — use a personal access token, a credential manager, or SSH keys. Pushing to someone else's repository? Push to **your fork** instead.

25. `repository not found`

URL typo, private repository, or no access. Copy the URL from the web page again; ask the owner for access.

26. I pushed to the wrong branch

Tell your team. If the wrong branch is shared, `git revert` the commits there and push them to the right branch (`git cherry-pick`). Ask the maintainer before any force push.

27. My push was rejected after a rebase

Expected: rebase made new commits. On **your own** PR branch only: `git push --force-with-lease`.

Files and ignoring

28. A file is still tracked after adding it to `.gitignore`

`.gitignore` affects only untracked files. `git rm --cached <file>`, commit.

29. `.gitignore` doesn't work at all

On Windows it may have been saved as `.gitignore.txt`. Rename it (turn on "show file extensions"). Check patterns with `git check-ignore -v <file>`.

30. The repository is huge / clone is slow

Large binaries were committed. For the future: Git LFS or DVC (Part VIII §7). Removing them from history requires rewriting it — a maintainer task with `git filter-repo`, coordinated with everyone who has a clone.

Security and privacy

31. I committed a password, key or token

1. **Revoke/rotate it now** (the service's settings) — assume it is stolen.
2. Remove it from the code; use environment variables and `.gitignore`.
3. Tell the teacher/maintainer; they decide about rewriting history. See Grade 12 Module 5.

32. I committed someone's personal data (names with marks, phone numbers, photos)

Stop pushing; tell the teacher immediately. Remove it in a new commit; the teacher/maintainer decides about history rewriting and who must be informed, following the school's policy.

CI

33. CI is red but it works on my computer

Read the first failing step's log. Common causes: a file you forgot to commit (`git status`), a dependency installed only on your PC, Windows line endings in scripts, upper/lower case in file names (Linux is case-sensitive).

34. The workflow never runs

Wrong folder (`.github/workflows/` or `.forgejo/workflows/`), wrong file extension (`.yml`), wrong trigger, YAML indentation error, or (Forgejo) no runner with the label in `runs-on`.

Still stuck?

- `git help <command>` (works offline).
- Software Carpentry's git-novice ‹https://swcarpentry.github.io/git-novice/› episodes and *Pro Git* §2.4 Undoing Things ‹https://git-scm.com/book/en/v2/Git-Basics-Undoing-Things›.
- Dangit, Git!? ‹https://dangitgit.com/› for quick rescues; Git Flight Rules ‹https://github.com/k88hudson/git-flight-rules› for everything else.
- Ask: describe what you ran, what you expected, and paste the exact message — never paste secrets.

Appendix C. Glossary

Terms used in this book, alphabetical. The grade column says where a term is first taught (8–12; "VIII" = Part VIII).

Nepali column: first draft, needs native review (`needs-nepali-review`). Blank = not yet translated; the English term is commonly used.

Word	नेपाली	Meaning	Grade
absolute path	पूर्ण मार्ग	a path from the top of the folder tree, e.g. `/c/Users/s07/school`	9
AI-use policy		written rules for using AI assistants in a project	12
amend		replace the last commit (`git commit --amend`); only before pushing	11
annotated tag		a tag that stores tagger, date and message (`git tag -a`)	10
API key / token		a secret string that lets a program use a service; never commit it	12
approve	स्वीकृत गर्नु	a reviewer's decision that a PR may be merged	10
assignee		the one person responsible for an issue	10
attribution (TASL)	श्रेय	credit with Title, Author, Source, Licence	11
backlog		the ordered list of work not yet done (issues)	11
binary file		a non-text file (image, Word, database); Git cannot show line diffs	9
bisect		binary search through history to find the first bad commit	12
blame / annotate		show the last commit, author and date for every line of a file	12
blob		Git object holding one file's contents	12
branch	शाखा	a movable label pointing at a commit	9
branch protection		host rules that stop direct or force pushes to a branch and require reviews/checks	12
breaking change		a change that makes users' old use stop working → MAJOR version	11
bundle		a single file containing a repository, for moving it by USB (`git bundle`)	9
changelog	परिवर्तन सूची	a human-written list of what changed in each version (`CHANGELOG.md`)	9
check script		a program that answers pass/fail about the project (exit code 0 = pass)	10
cherry-pick		copy one commit onto the current branch	11
CI (continuous integration)		running checks automatically on every push or pull request	10
CD (continuous delivery/deployment)		publishing automatically after CI passes	12
claim (an issue)		comment that you will work on an issue, with a plan	11
clone	क्लोन	copy a whole repo with its history	9

Word	नेपाली	Meaning	Grade
closing keyword		`Closes #N` / `Fixes #N` in a merged commit or PR closes issue N	10
code of conduct	आचारसंहिता	rules for how people behave in a project	11
code review		a teammate reads and comments on a change before it is merged	10
CODEOWNERS		a file naming who must review changes to which folders	12
commit	कमिट / स्न्यापसट	a saved snapshot of the whole project with a message, author, date and parent	8
commit id / hash		the unique name of a commit (e.g. `78a873e…`), computed from its content	8
common ancestor	साझा पुर्खा	the last commit two branches share; merges compare against it	9
conflict	द्वन्द्व	two changes to the same lines that Git cannot combine alone	9
conflict markers		`<<<<<<<`, `=======`, `>>>>>>>` lines showing both sides of a conflict	9
content-addressed		stored under a name computed from the content (Git objects)	12
context line		an unchanged line shown around changes in a diff	8
Conventional Commits		message format `type(scope): summary`, e.g. `fix(menu): …`	11
CONTRIBUTING file		a file explaining how to contribute to a project	11
copyright	प्रतिलिपि अधिकार	the creator's automatic right over their work	11
detached HEAD		HEAD points directly at a commit, not at a branch	9
diff	फरक	the difference between two versions, line by line (`-` removed, `+` added)	8
dump		a database exported as SQL statements that recreate it	9
environment variable		a setting given to a program from outside its code (used for secrets)	12
exit code		a number a program returns: 0 = success, other = failure	10
fast-forward		a merge where the branch label simply moves forward; no merge commit	9
feature flag		a switch in the code that turns a feature on or off	12
fetch		download new commits from a remote without merging	9
fork	फोर्क	your own copy of someone else's repo on a server	10
`.git` folder		the hidden folder where a repository keeps its history	8
`.gitignore`		a file listing files Git should not track	9
Gitflow		a branching strategy with `main`, `develop`, release and hotfix branches	12
GitHub flow		strategy: `main` always releasable, one short branch + PR per change	12
HEAD		the pointer to "where you are" — usually the current branch	8
history / log	इतिहास	the list of commits	8

Word	नेपाली	Meaning	Grade
host		a service that stores Git repositories (GitHub, Codeberg, GitLab, Forgejo)	9
hotfix		an urgent fix to a released version	12
increment		a working, shown piece of progress at the end of an iteration	11
issue	मुद्दा / काम	a task, bug or idea written down on the server, with a number	10
kanban board		a board with columns To do · Doing · Done	10
label	लेबल	a tag on an issue such as `bug` or `good-first-issue`	10
licence	इजाजतपत्र	written permission to use, change or share a work	11
maintainer		a person who reviews and merges contributions to a project	11
merge	मिलाउनु	combine the work of two branches	9
merge commit		a commit with two parents, joining two lines of work	9
merge request (MR)		GitLab's name for a pull request	10
message	सन्देश	the text of a commit explaining what and why	8
milestone		a target (date or version) grouping issues	10
modified	परिवर्तित	a tracked file that changed since the last commit	8
object		a stored item in Git: blob, tree, commit or tag	12
origin		the usual name of the remote you cloned from	9
pager		a page viewer for long output; Space = next page, q = quit	8
parent		the commit that came just before a commit	8
patch		a text file containing a diff (`git format-patch`)	10
path	मार्ग	the address of a file or folder in the tree	9
plumbing / porcelain		low-level Git commands / everyday friendly commands	12
pre-release		a test version before a release, e.g. `2.0.0-rc.1`	12
process model		the order in which a team does the work of building software	11
prompt		the text before the cursor in a terminal, showing where you are	9
public API		what users of a project rely on; changes to it decide the version bump	12
pull	ल्याउनु	fetch + merge commits from a remote	9
pull request (PR)		a request to merge your branch, with diff, checks and discussion	10
push	पठाउनु	send commits to a remote	9
README		the front page of a repository: what, how to run, who	9
rebase		replay a branch's commits on a new base; rewrites history	11
reference (ref)		a name pointing at a commit: branch, tag, HEAD, remote branch	12
reflog		a local record of where HEAD has been; recovers "lost" commits	11
release	रिलिज	a tag plus notes (and files) for users	10
relative path	सापेक्ष मार्ग	a path from where you are now, e.g. `class9/ward-facts`	9

Word	नेपाली	Meaning	Grade
remote	टाढाको भण्डार	a copy of the repo on another computer or server	9
repository (repo)	भण्डार	a project folder that remembers its history	8
reproducible	पुनरुत्पादनीय	anyone can re-run the steps and get the same result	11
reset		move the current branch label (`--soft`, mixed, `--hard`); rewrites if pushed	11
restore	पुनर्स्थापना	bring a file back to a committed version (`git restore`)	8
retrospective	पुनरावलोकन	a team meeting about what went well and what to change	10
revert		undo a commit by adding a new opposite commit	8
reviewer	समीक्षक	the person checking a pull request	10
root commit		the first commit; it has no parent	9
runner		the machine that runs CI jobs	12
schema		the structure of a database (`CREATE TABLE …`)	9
secret	गोप्य	a password, key or token; never commit it	9
semantic versioning (SemVer)		MAJOR.MINOR.PATCH: break · add · fix	10
share alike (SA)		a licence condition: changed versions must use the same licence	11
signed commit		a commit with a cryptographic signature proving who made it	12
snapshot	स्न्यापसट	a complete picture of the project at one moment	8
spam PR		a useless pull request made only to collect points	11
SPDX identifier		the official short name of a licence, e.g. `MIT`, `CC-BY-4.0`	11
sprint		a short fixed period of work in Scrum (1–4 weeks)	11
squash		combine several commits into one	11
staging area (index)		the "next snapshot" box you are preparing (`git add`)	8
stash		a drawer for unfinished, uncommitted work (`git stash`)	11
static site		a website made only of files, served as they are	10
switch		change to another branch (`git switch`)	9
tag	ट्याग	a permanent name on one commit, e.g. `v1.0.0`	9
team charter		a team's written roles, rules and privacy promises	10
terminal / shell	टर्मिनल	a window where you type commands	9
tree		Git object for a folder: names → blobs and trees	12
triage		sorting issues by clarity, size and priority	11
trunk-based development		everyone merges small changes into `main` at least daily	12
unstage		take a change out of the staging area (`git restore --staged`)	8
untracked		a file Git has never been told to track	8
upstream		the original repository a fork came from	10
version	संस्करण	the state of a work at one moment	8

Word	नेपाली	Meaning	Grade
version control system	संस्करण नियन्त्रण प्रणाली	a program that keeps the history of your files	8
workflow / pipeline		a CI configuration: triggers, jobs and steps	12
working copy	काम गर्ने प्रति	the files you see and edit	8

Appendix D. Module template (for contributors)

Every module lives in `docs/en/grade-NN/mNN-slug/` and has a `lesson.md` (front-matter below). Grade 8 also has a separate `teacher-guide.md`; Grades 9–12 keep their teacher notes at the end of the lesson.

```
---
id: g9-m03-branches        # gN-mNN-slug, must match the folder
grade: 9
hours: 2
prerequisites: [ g9-m02-cli-basics ]
outcomes: [ "Create and switch branches", "Explain a branch as a movable label" ]
tier_support: [0, 1, 2, 3]  # which delivery tiers the module works in
license: CC-BY-4.0
---
```

Sections of a lesson (v0.2 "book" structure)

#	Section	Purpose	Typical time
—	**At a glance** box	time, materials, tiers, prerequisites, outcomes	—
1	**Hook**	a real problem in a Nepali context	5 min
2	**Understand it**	the full explanation a student can read alone: analogy → precise idea → text diagram → what Git really does	read 10–15 min (before or after class)
3	**See it**	a worked example: exact commands *and expected output*, annotated	5–10 min demo
4	**Unplugged / concept activity**	paper or visual model first	15 min
5	**PRIMM lab**	Predict, Run, Investigate, Modify, Make, with checkpoints	30–40 min
6	**Parsons practice**	put jumbled steps in order	10 min
7	**Common mistakes**	symptom → cause → fix table	—
8	**Practise more with free resources**	3–6 exercises from the resource library, each with tier and what to hand in	homework / lab
9	**Exit ticket**	3 questions, one checks a misconception	5 min
10	**Key points**	4–6 revision bullets	—
11	**New words**	terms used, all in the glossary	—
12	**Extension quest**	for fast finishers	—
13	**Teacher notes** (Grades 9–12)	timing, misconceptions, fallbacks, answer key	—

Teacher guide sections (Grade 8)

Timing · materials · common errors · fallback if no internet / no computer · differentiation · evidence to collect · answer key (including the free-resource exercises) · extension quest.

Rules

- Concept before commands. Commands and code stay in English.
- Diagrams are plain text inside code blocks, so they print on any printer and work offline.

- Free resources are **linked, not copied**, unless their licence allows it (see the library).
- Every external link is opened and checked before it is added.
- Practice data is real-and-open or clearly synthetic; never student PII.
- Pair work (driver/navigator) is the default.